
Efficient Data Structures and Algorithms for Dynamic Analysis of Concurrent Programs

Hünkar Can Tunç

PhD Dissertation



Department of Computer Science
Aarhus University
Denmark

Efficient Data Structures and Algorithms for Dynamic Analysis of Concurrent Programs

A Dissertation
Presented to the Faculty of Natural Sciences
of Aarhus University
in Partial Fulfillment of the Requirements
for the PhD Degree

by
Hünkar Can Tunç
March 1, 2025

Abstract

Software systems are indispensable to modern society, powering everything from critical infrastructure to everyday applications. Nevertheless, defects in software are prevalent and can lead to far-reaching effects on society. Meanwhile, software is constantly evolving to meet the ever-changing demands of users and technological advancements. This renders software development a continuous process and dealing with software defects a recurring challenge. A key part of this continuous software development process is concerned with enhancing efficiency, a need that has become particularly pronounced due to the growing demands from society. In this context, concurrent programming serves as a fundamental paradigm for enhancing the efficiency of software. Concurrency enables the simultaneous execution of multiple tasks and facilitates the effective utilization of multiple processor cores. However, writing concurrent programs and reasoning about their behavior is notoriously challenging due to their inherent non-deterministic characteristics. This complexity often leads to the emergence of defects in concurrent programs, with concurrency bugs such as data races, deadlocks, and atomicity violations commonly surfacing in practice.

Dynamic analysis techniques have emerged as a prominent technique for detecting defects in concurrent programs. Dynamic analysis involves examining the characteristics of a running program and validating its runtime properties. This concept has proven invaluable for identifying concurrency bugs. Consequently, dynamic analysis has become a widely adopted tool by developers in ensuring the reliability of concurrent systems.

In this thesis, we push the boundaries of dynamic analysis techniques for detecting concurrency bugs by introducing novel data structures and algorithms. In particular, we present two novel data structures, *Tree Clocks* and *Collective Sparse Segment Trees*, that serve as drop-in replacements for the widely used vector clocks data structure in the literature, with the goal of enhancing its efficiency. These data structures are versatile and can be applied across a wide range of dynamic analyses, often resulting in significant improvements to overall performance. On the algorithms front, we investigate two fundamental problems: dynamic deadlock detection and consistency checking in weak memory models. We study these problems from both theoretical and practical perspectives, establish algorithmic complexity results, and present efficient algorithms that demonstrate high performance in real-world settings. Our findings are validated through extensive evaluations on large real-world benchmark suites, demonstrating their applicability and effectiveness in practical scenarios.

Resumé

Softwaresystemer er uundværlige for det moderne samfund og driver alt fra kritisk infrastruktur til hverdagsbrug. Ikke desto mindre, er defekter i software udbredte og kan lede til vidtrækkende konsekvenser for samfundet. Samtidig er software i konstant udvikling for at imødekomme de stadigt skiftende krav fra brugere og teknologiske fremskridt. Dette gør softwareudvikling til en kontinuerlig proces og håndtering af softwarefejl til en tilbagevendende udfordring. En vigtig del af denne kontinuerlige softwareudviklingsproces handler om at forbedre effektiviteten, et behov der er blevet særligt udtalt på grund af de stigende krav fra samfundet. I denne sammenhæng fungerer samtidig programmering som et grundlæggende paradigme for at forbedre softwareeffektiviteten. Samtidig muliggør simultan udførelse af flere opgaver og faciliterer en effektiv udnyttelse af flere processor-kerner. Det er imidlertid notorisk udfordrende at skrive konkurrente programmer og ræsonnere om deres adfærd på grund af deres iboende ikke-deterministiske karakteristika. Denne kompleksitet fører ofte til fremkomsten af fejl i samtidige programmer, hvor samtidighedsfejl såsom datakapløb, deadlocks og atomaritetskrænkelser hyppigt opstår i praksis.

Dynamiske analyseteknikker er kommet frem som en fremtrædende metode til at opdage fejl i samtidige programmer. Dynamisk analyse indebærer at undersøge karakteristikaene for et kørende program og validere dets runtime-egenskaber. Dette koncept har vist sig uvurderligt til at identificere samtidighedsfejl. Som følge heraf er dynamisk analyse blevet et bredt anvendt værktøj blandt udviklere til at sikre pålideligheden af samtidige systemer.

I denne afhandling udvider vi grænserne for dynamiske analyseteknikker til at opdage samtidighedsfejl ved at introducere nye datastrukturer og algoritmer. Specifikt præsenterer vi to nye datastrukturer, *Tree Clocks* og *Collective Sparse Segment Trees*, der fungerer som direkte erstatninger for den bredt anvendte vektorklokke-datastruktur i litteraturen, med det formål at forbedre dens effektivitet. Disse datastrukturer er alsidige og kan anvendes på tværs af en bred vifte af dynamiske analyser, hvilket ofte resulterer i betydelige forbedringer af den samlede ydeevne. På algoritmefronten undersøger vi to grundlæggende problemer: dynamisk deadlock-detektion og konsistenskontrol i svage hukommelsesmodeller. Vi undersøger disse problemer både fra et teoretisk og praktisk perspektiv, fastlægger algoritmisk kompleksitet og præsenterer effektive algoritmer, der demonstrerer høj ydeevne i praksis. Vores resultater er blevet valideret gennem omfattende evalueringer på store benchmarksæt fra den virkelige verden, hvilket demonstrerer deres anvendelighed og effektivitet i praktiske scenarier.

Acknowledgments

First and foremost, I would like to thank my supervisor Andreas Pavlogiannis for his constant support, guidance, and invaluable feedback throughout my PhD. Andreas has taught me how to approach problems with rigor, gave me freedom to follow my curiosity and develop my own ideas, and consistently provided valuable insights, turning my PhD into a truly enriching journey.

I would like to give a special thanks to Umang Mathur, with whom I collaborated from the very beginning of my PhD and who in ways acted as a second supervisor to me. I am thankful to Mahesh Viswanathan, Parosh Aziz Abdulla, Soham Chakraborty, Krishna Shankaranarayanan, Ameya Prashant Deshmukh, Berk Çirisci and Constantin Enea for the fruitful collaborations. Thanks to all members of Programming Languages and Logic and Semantics groups for creating an inspiring work environment. I am grateful to Anders Møller who has provided valuable feedback throughout my PhD and guided me in shaping my career path. I would like to thank to Charles Zhang and Heqing Huang for hosting my stay at the Hong Kong University of Science and Technology, and Milind Chabbi and Zhizhou Zhang for hosting my stay at Uber. Thanks to Mikael Bisgaard Dahlsen-Jensen, who has helped me translate the abstract of this dissertation into Danish.

I was fortunate to have had mentors Michael Schmuker, Cemal Okan Şakar, Tatjana Petrov and Georgiana Caltais during my bachelor's and master's studies, whose guidance sparked my interest in research, helped me find my research direction, and ultimately led me to pursue a PhD. I will always be grateful for their support.

Finally, I would like to express my deepest gratitude to my parents, Nurgül Tunç and Dursun Tunç, and to my partner, Pan Qu, for their unwavering love, support and encouragement.

*Hüncar Can Tunç,
Aarhus, March 1, 2025.*

Contents

Abstract	i
Resumé	iii
Acknowledgments	v
Contents	vii
I Overview	1
1 Introduction	3
1.1 Summary of Contributions	7
1.2 Published Papers	10
1.3 Outline	10
2 Background	13
2.1 Concurrency	13
2.2 Memory Consistency	18
2.3 Dynamic Program Analysis	20
3 Dynamic Concurrency Analysis	25
3.1 Concurrent Execution Model	25
3.2 Partial Orders	29
3.3 Concurrency Bugs	33
3.4 Predictive Analysis	38
4 Conclusions	43
4.1 Discussion	43
4.2 Scope	48
4.3 Future Work	49

II Publications	53
5 A Tree Clock Data Structure for Causal Orderings in Concurrent Executions	55
5.1 Introduction	56
5.2 Preliminaries	59
5.3 The Tree Clock Data Structure	62
5.4 Tree Clocks for Happens-Before	68
5.5 Tree Clocks in Other Partial Orders	70
5.6 Experiments	71
5.7 Related Work	77
5.8 Conclusion	77
6 CSSTs: A Dynamic Data Structure for Partial Orders in Concurrent Execution Analysis	79
6.1 Introduction	80
6.2 Preliminaries	83
6.3 Collective Sparse Segment Trees	85
6.4 Incremental CSSTs	95
6.5 Experimental Evaluation	97
6.6 Related Work	105
6.7 Conclusion	105
7 Sound Dynamic Deadlock Prediction in Linear Time	107
7.1 Introduction	108
7.2 Preliminaries	112
7.3 The Complexity of Dynamic Deadlock Prediction	114
7.4 Synchronization-Preserving Deadlocks and their Prediction	118
7.5 On-the-fly Deadlock Prediction	126
7.6 Experimental Evaluation	128
7.7 Related Work	134
7.8 Conclusion	135
8 Optimal Reads-From Consistency Checking for C11-Style Memory Models	137
8.1 Introduction	138
8.2 Axiomatic Concurrency Semantics	142
8.3 Auxiliary Functions, Data Structures and Observations	146
8.4 Consistency Checking	148
8.5 Experimental Evaluation	159
8.6 Conclusion	162
Bibliography	165

Part I

Overview

Chapter 1

Introduction

Software systems have become an indispensable part of the modern society. The integration of software systems spans essential aspects of everyday life, including healthcare, financial services, and transportation. This integration is expected to increase rapidly in the coming decades, with virtually all aspects of our lives being moderated by software [139]. Meanwhile, it is well known that defects in software are ubiquitous [24, 63, 74, 174, 181, 189, 231, 287]. Such defects hinder the proper functioning of software systems and may result in severe consequences, including disruptions in crucial services [215, 220], unauthorized access to sensitive data [93, 141], and substantial financial loss for businesses [83, 207]. The level of disastrous consequences of defects in software systems reach to the point of causing loss of human lives [178]. Due to the constantly evolving nature of software, defects are continuously being introduced [278], rendering the challenge of maintaining software reliability virtually an intractable problem. Furthermore, as software evolves, the complexity of software systems grows substantially, increasing the likelihood of manifesting such defects [283]. Consequently, it is well-established that existing software development practices are inadequate and there is a critical need to further develop methodologies that enhance the reliability of software systems.

Numerous techniques and engineering practices have been developed to address the issue of software defects, including code review, testing, programming language design, and program analysis. These methodologies are often used in a complementary fashion, collectively enhancing the reliability of software systems. Among these, techniques based on program analysis play a central role in tackling software defects. The process of program analysis involves an automated procedure that analyzes the behaviors of software and extracts properties to ensure that the system functions as intended. However, as established by Turing [299] and Rice [245], it is generally impossible to ascertain any non-trivial property concerning the behavior of a program. The inherent undecidability of the underlying problem presents a fundamental challenge that necessitates the pursuit of various abstractions and approximations. The effectiveness of these approaches can vary depending on the context, leading to the development of diverse automated reasoning techniques tailored to different

settings. Program analysis techniques have been widely and successfully employed, including in large-scale industrial settings, in the task of enhancing program reliability [45, 55, 58, 84, 96, 102, 179, 254, 255, 269].

A fundamental aspect of software development is the pursuit for higher performance, fueled by an ever-growing demand for greater efficiency and responsiveness. A prominent technique for achieving improved performance is concurrent programming [261], which enables simultaneous execution of multiple tasks and facilitates the effective utilization of multiple processor cores. Despite the widespread use of concurrent programming, writing concurrent programs is a notoriously difficult task [125, 189]. This difficulty follows naturally from the challenges of writing sequential programs, where a common source of bugs arises from unexpected inputs [212]. While developers typically design software to handle a range of inputs, unforeseen edge cases can lead to erroneous behavior, making it challenging to create software that is robust in all potential scenarios. The incorporation of concurrency introduces an additional layer of complexity to the already difficult task. The primary challenge introduced by concurrency arises from the inherent non-determinism of inter-thread communication, resulting in an explosion of possible program behaviors, especially when considered together with all potential inputs [288]. Such a complexity often exceeds the developers' mental model of the possible program behaviors, leading to concurrency related issues such as data races, deadlocks, and atomicity violations commonly arising in practice [189, 289]. Concurrency bugs are generally viewed to be among the most challenging issues to address, and they can potentially lead to catastrophic consequences for software security and reliability [41, 127, 216, 322].

It has been reported that program tools that target analyzing concurrent programs can be of significant assistance to developers in creating concurrent software [45, 125, 189]. There have thus been many proposed program analysis techniques and tools that target concurrent programs [23, 111, 112, 131, 138, 150, 179, 267, 302]. However, automatically analyzing concurrent programs has also proven to be an intricate task. This again follows naturally from the aforementioned underlying problem of automated program analysis being inherently complex—both in theory and practice, even for sequential programs—and further compounded by the large space of new program behaviors introduced by non-deterministic inter-thread communication. Due to this inherent complexity of the problem, existing techniques for automatically analyzing concurrent programs come with various trade-offs, with prominent methods such as model checking [115, 131, 216], static analysis [38, 46, 99], and dynamic analysis [111, 188, 259] each balancing factors such as precision, scalability, and computational cost of the analysis differently.

Model checking is a verification technique that, given an abstract model of the underlying software and a formal property, exhaustively checks whether that property is satisfied within the state model [24]. The main benefit of model checking is its ability to provide strong guarantees about system correctness. However, establishing a suitable abstract model and property, along with performing the exhaustive check in a scalable manner for large models are the primary challenges in model checking.

Static analysis is a technique that analyzes the behavior of programs without

executing them [214]. It is often regarded as a more practical and scalable solution compared to techniques such as model checking and has been widely used for bug finding in large software environments. Such techniques can reason about all possible behaviors of programs and provide guarantees regarding the properties of programs. However, to facilitate practical analysis, they often approximate the behaviors of the programs. This approximation can lead to a loss of precision in the analysis results. Consequently, a major drawback of static analysis techniques is the presence of issues such as false positive bug reports, which can complicate the software development process and diminish developers' confidence in the analysis outcomes.

Dynamic analysis involves running a program and monitoring its behavior during execution [27]. Such techniques have the more modest goal of discovering faults by analyzing observed program executions instead of whole programs. However, they provide high precision in identifying real bugs and are among the most widely used analysis tools in practice due to their ease of use and the precise information they offer. Although this approach cannot guarantee the absence of bugs, it is typically more scalable than static analysis and generates more reliable analysis reports.

Overall, each of the aforementioned techniques has its strengths and weaknesses. They are also often considered complementary and can be utilized together to improve bug detection capabilities [100, 101, 175, 228, 265]. The main focus of this thesis is on dynamic analysis techniques for concurrency bug detection. This area of research has been very active in the recent years [23, 54, 122, 157, 199, 202, 236, 247, 248, 285], including industrial adaptations of the proposed techniques [23, 137, 251, 267]. Below we outline the key challenges in the domain and how this thesis addresses these challenges.

- **Specification.** Determining what software *should* do is challenging. While executing a program and monitoring its runtime behavior has proven to be an invaluable concept, this approach is often insufficient by itself without being coupled with a process that defines the expected behavior. This coupling is typically essential for validating the correctness of the observed behavior. However, a clear specification of intended behavior of software often does not exist [319]. An effective strategy to address this issue has been to analyze software based on certain universal specifications that are widely applicable, such as memory safety. In this respect, although concurrency often introduces additional challenges to program analysis, it also encompasses certain universal specifications such as data race freedom and deadlock freedom. Violations of universal specifications can often lead to critical bugs and thus has been an active area of research. Additionally, detecting violations of different specifications typically requires or benefits from dedicated analysis techniques. In this thesis, we focus on checking such universal specifications in concurrent programs and develop specialized analyses tailored to specific specifications.
- **Soundness.** We refer to an analysis technique as sound if it does not report any spurious bugs (i.e., no false positives). Soundness has been identified as one

of the most critical features of program analysis techniques [76, 90, 255]. If program analysis tools report too many spurious issues, it can lead developers to lose trust in the tool and ultimately abandon its use. Soundness is particularly crucial in concurrent programming, where debugging and reproducing bugs is inherently challenging and time-consuming due to the non-deterministic nature of the program executions [216]. In this thesis, we focus on developing analysis techniques that do not compromise soundness.

- **Completeness.** We refer to an analysis technique as complete if all bugs violating the specification are reported by the technique (i.e., no false negatives). Soundness and completeness are interconnected properties; a technique can attain soundness by reporting no bugs, while it can typically achieve completeness by reporting all potential issues as bugs. In the context of dynamic analysis, completeness is typically assessed based on a specific program execution rather than the entire program. That is, completeness pertains to the ability of the technique to identify all bugs present during a particular execution path, rather than the detection of all bugs throughout all possible executions. Throughout this thesis, we take this view when assessing the completeness of a dynamic analysis technique. There are also additional intricacies related to completeness in this context. Certain properties of a running program may not be directly observable but can be inferred from the information obtained during program execution, which can help uncover more bugs. Completeness is then evaluated based on the extent of the space of properties that can be inferred from a program's execution [266]. In this thesis, we design techniques that strive for high completeness, taking into account such "inferred" program properties, while maintaining soundness.
- **Scalability.** Scalability is another essential aspect of analysis techniques. As software systems grow in size and complexity, the ability of analysis techniques to scale effectively for such large systems becomes increasingly important. Balancing scalability with completeness, while also maintaining soundness, is particularly challenging. For instance, many fundamental algorithmic problems that commonly surface in program analysis are computationally intractable [35, 123, 135, 173, 180, 200], rendering achieving full completeness nontrivial. This often implies that achieving scalable techniques with high completeness requires extracting insights from observations made in practice to eliminate common bottlenecks in the analysis process. In this thesis, we focus on developing scalable analysis techniques designed for large scale real-world software, striving for an effective balance that ensures both high coverage and soundness.
- **Concurrency Models.** Another challenge in designing concurrent program analysis techniques is handling ever richer concurrency models and communication mechanisms such as shared memory, message passing, transactional memory, weak memory and futures. Each of these typically requires specialized

analysis techniques capable of handling the primitives and requirements of that setting. In this thesis, we focus on various such models and develop tailored analysis methods for particular settings, as well as techniques that are generally applicable across such models.

The difficulty in designing dynamic analysis techniques stems from the challenge of achieving all the above mentioned desirable properties. In this thesis, we aim to push the boundaries of addressing these challenges.

1.1 Summary of Contributions

In summary, this thesis focuses on improving the reliability of concurrent software. It achieves this goal by developing novel algorithms and data structures within the context of dynamic program analysis. These advancements address key challenges that often hinder the application of dynamic program analysis techniques in real-world software systems. The thesis is centered around the following themes:

- (i) Advancing the state-of-the-art dynamic program analysis techniques in terms of efficiency while preserving soundness and coverage.
- (ii) Developing efficient general purpose data structures that can be utilized in numerous dynamic analysis techniques.
- (iii) Developing dynamic program analysis techniques tailored for emerging concurrency paradigms.

Below, we present an overview of each individual contribution presented in this thesis.

Data Structures

Dynamic analysis techniques observe program executions denoted σ and analyze them to detect bugs. While each analysis employs reasoning tailored to the specific problem at hand, typically a core component of the analysis is establishing a causal ordering among the events observed in σ . In this context, events corresponds to the actions performed during the execution such as individual memory accesses, and synchronization operations. Although the particular notion of causality depends on the application setting, the transitive nature of such relationships make it amenable to be expressed as a partial order P over the events. Following are two of the most common operations performed on P during the analysis: (i) *inserting* new orderings $e_1 \rightarrow e_2$ in P , and (ii) *querying* if $e_1 \rightarrow e_2$ in P . These operations play a crucial role in establishing certain properties of σ (e.g., data race freedom, deadlock freedom) and are performed repeatedly—often proportional to the number of events, which can easily reach to billions in real-world settings. Hence, the overall scalability of the analysis largely depends on handling these operations efficiently.

A simple data structure known as vector clocks [205] has been the primary driver of scalability of dynamic concurrency analyses. Vector clocks are commonly used to represent partial orders in this context, and many algorithms are built around these representations to enhance efficiency [109, 111, 164, 198, 202, 247, 284]. However, the use of vector clocks can still result in performance bottlenecks. To address this limitation, we propose two new data structures, *Tree clocks* and *Collective Sparse Segment Trees*, that improve performance of vector clocks in certain important settings.

- *Tree Clocks* [203]: The two fundamental operations of vector clocks are join and copy, which are repeatedly applied when maintaining partial orders. These operations require $\Theta(k)$ time, where k typically corresponds to the number of threads. This time complexity can become a bottleneck, especially in large-scale systems with many threads. To address this, we introduce the tree clocks data structure, which can act as a drop-in replacement for vector clocks. The join and copy operations for tree clocks take time that is proportional to the number of modified entries, thereby avoiding the inherent $\Theta(k)$ cost per operation. We demonstrate that replacing vector clocks with tree clocks leads to significant speedups in various dynamic analyses.
- *Collective Sparse Segment Trees* [292]: Dynamic analysis techniques can be broadly categorized into two types: *streaming* analysis and *non-streaming* analysis. In streaming analysis, events are processed as they are observed, and for each new ordering $e_1 \rightarrow e_2$ that is inserted, e_2 corresponds to the current event being processed. In such settings, a single edge insertion can be handled in $\Theta(k)$ time by vector clocks. In contrast, certain analyses follow a non-streaming fashion, typically while aiming to assert more complex properties of σ . In such settings, the analysis can add orderings between arbitrary events within σ . These new orderings might have to be propagated across n events, where n is the size of σ which is typically very large in practice. Thus, vector clocks can incur a significant slowdown while maintaining P for such analyses, costing $O(nk)$ time for inserting (and propagating) each new ordering in P . Additionally, certain non-streaming analyses may require deleting orderings from P , a feature not supported by vector clocks. To address these limitations, we introduce Collective Sparse Segment Trees (CSSTs), a data structure for maintaining a partial order P in the context of non-streaming dynamic analyses. CSSTs thrive when the width k of P is significantly smaller than n , which is the case in most dynamic analysis settings. This enables insertion, deletion, and querying for orderings in P to operate in roughly $O(\log n)$ time. Our experiments show that CSSTs are currently the best data structure for handling a variety of dynamic analyses.

Algorithms

- *Deadlock Prediction* [291]: A key limitation of dynamic analysis is its reliance on specific execution traces while attempting to assert certain properties of the

analyzed program. A recent trend in addressing this challenge advocates for dynamic predictive analysis [117, 122, 137, 138, 154, 157, 276] where the goal is to enhance coverage by additionally reasoning about alternative reorderings of the observed execution trace that could have taken place. Such reorderings of the observed execution trace can reveal bugs that would otherwise remain hidden. Predictive techniques have gained significant popularity as they can significantly enhance the coverage of dynamic analyses. These analyses target various concurrency bugs, including data races [199, 236, 247], atomicity violations [54, 275], memory safety issues [137, 317], and deadlocks [54, 154]. Deadlocks are widely regarded as one of the most notorious concurrency bugs, and extensive research has been dedicated to developing efficient deadlock detection methods. However, dynamic deadlock prediction remains a challenging task, as existing analyses techniques cannot simultaneously satisfy the criteria of soundness, high coverage, and efficiency. In this work, we address this problem. We explore the deadlock prediction problem first from a theoretical perspective and establish that a tradeoff is inevitable. In particular, we show that in general sound and complete deadlock prediction is intractable. Furthermore, we demonstrate that even the seemingly simpler task of detecting potential deadlocks is intractable. Then, we introduce a new class of predictable deadlocks, accompanied by an efficient algorithm designed to predict this category of deadlocks in a sound and complete manner. Our findings demonstrate that the proposed notion is highly effective, as it (i) captures the vast majority of deadlocks, and (ii) can be detected through an online, sound, complete, and highly efficient algorithm.

- *Consistency Checking* [290]: Memory models define the rules that govern the order in which memory operations (reads and writes) appear to execute in concurrent systems. These models are critical for ensuring the correctness of concurrent programs by specifying how memory operations from different threads are observed by each other. A concept that has gained increasing attention in recent years is weak memory models. Unlike traditional memory models that enforce strict ordering of operations, weak memory models allow for various degrees of reorderings. This flexibility enables better performance optimizations by compilers and hardware. One notable example of a weak memory model is the C11 memory model, which captures the concurrency semantics of C and C++ programs. One of the most fundamental problems associated with a memory model $\mathcal{M}$ is consistency checking. The input to this problem is an execution X consisting of a set of events, their program order (i.e., thread-local executions) and reads-from relation (i.e., a mapping that associates each read operation with the write from which it acquires its value). The task is to decide if X is consistent with $\mathcal{M}$. This problem lies at the heart of numerous applications in dynamic analysis and model checking. Motivated by such applications, we study the problem of consistency checking in the context of popular variants of the C11 memory model. We characterize the complexity of the problem of consistency checking of executions in such

weak memory models and develop novel algorithms. We show that replacing the existing methods for solving the consistency checking with our proposed techniques leads to significant speedups in the state-of-the-art dynamic analysis and model checking tools.

1.2 Published Papers

The results presented in the Part II of this thesis are based on the publications listed below. The papers included in the thesis are identical to the published versions, with differences limited to formatting changes, and minor textual revisions.

- Umang Mathur, Andreas Pavlogiannis, Hünkar Can Tunç, Mahesh Viswanathan. A Tree Clock Data Structure for Causal Orderings in Concurrent Executions. Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), February 2022 [203]. Included in Chapter 5. This work has received ASPLOS best paper award.
- Hünkar Can Tunç, Ameya Prashant Deshmukh, Berk Çirisci, Constantin Enea, Andreas Pavlogiannis. Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), April 2024 [292]. Included in Chapter 6. This work has received ASPLOS best paper award.
- Hünkar Can Tunç, Umang Mathur, Andreas Pavlogiannis, Mahesh Viswanathan. Sound Dynamic Deadlock Prediction in Linear Time. Proceedings of the ACM on Programming Languages, Volume 7, Issue PLDI, June, 2023 [291]. Included in Chapter 7.
- Hünkar Can Tunç, Parosh Aziz Abdulla, Soham Chakraborty, Shankaranarayanan Krishna, Umang Mathur, Andreas Pavlogiannis. Optimal Reads-From Consistency Checking for C11-Style Memory Models. Proceedings of the ACM on Programming Languages, Volume 7, Issue PLDI, June, 2023 [290]. Included in Chapter 8.

The author of this thesis has made significant contributions to both the writing and research phases of all the associated research projects. Additionally, all the artifacts associated with each of the published papers were primarily developed by the author [201, 293, 295, 297].

1.3 Outline

This thesis is divided into two parts. The first part explores key concepts on dynamic analysis and concurrency, introduces formalisms, and provides discussions around the

contributions of this thesis, their implications, as well as future avenues of research. In particular, Chapter 2 begins the first part by presenting background information on dynamic analysis and concurrency, with discussions primarily remaining at an intuitive level. Chapter 3 introduces the formalisms related to dynamic analysis and concurrency. Finally, Chapter 4 presents a concluding discussion and future work. The second part of the thesis consists of the research papers. In particular, Chapter 5 and Chapter 6 present the tree clocks and CSSTs data structures, and Chapter 7 and Chapter 8 present the algorithmic results on deadlock prediction and consistency checking, respectively.

Chapter 2

Background

The main focus of this thesis is on detection of concurrency bugs via dynamic analysis. This chapter serves the purpose of providing an overview of the key concepts in concurrency and dynamic analysis. In the context of concurrency (Section 2.1), we discuss key concepts such as concurrency models, communication mechanisms, nondeterminism, and common concurrency bugs. In the context of dynamic analysis (Section 2.3), we discuss main aspects of dynamic analysis, including instrumentation, trace generation, monitoring, and also explore the main dynamic analysis techniques used in concurrency bug detection. Additionally, we contrast dynamic and static analysis, highlighting their strengths, limitations, and the synergy between them. We remark that the discussions in this chapter provide an intuitive perspective of the concepts without delving into precise definitions. We relegate Chapter 3 to presenting the precise formal definitions that underpin the background information discussed in this chapter.

2.1 Concurrency

Concurrency is a fundamental aspect of modern computing systems, enabling multiple threads to execute simultaneously [249]. As systems grow in complexity and scale, concurrency becomes essential for improving performance, resource utilization, and responsiveness. However, concurrency also introduces challenges, particularly in ensuring correct and reliable behavior in the presence of shared resources. In this section, we provide an overview of concurrency, focusing particularly on how concurrency issues manifest and the methods used to detect and mitigate them. This section is organized as follows. We begin by exploring the key aspects that govern concurrent systems. Next, we discuss the concept of nondeterminism in concurrent programming and its implications. Finally, we explore the common concurrency bugs and the strategies for mitigating them.

Inter-Thread Communication

In concurrent systems, communication between threads plays a central role. The two primary models used to enable communication between processes are *shared memory* [10] and *message passing* [213]. These models offer different approaches to managing communication in multi-threaded environments, each with distinct advantages and challenges. Shared memory allows processes to communicate by directly accessing and modifying shared data while message passing involves explicit exchanges of messages between processes. In this thesis, we focus on shared memory concurrency. A precise notion of shared memory semantics is needed to develop correct and efficient shared memory programs. We relegate Section 2.2 for a discussion on memory consistency models, which define this underlying semantics.

Synchronization Primitives

We consider two primitives as the synchronization mechanism to manage access to shared resources: locks and atomic variables. Locks are arguably the most commonly used synchronization mechanism, enforcing mutual exclusion by allowing only one thread to access a critical section at a time. On the other hand, atomic variables enable certain operations to complete in a single observable step, ensuring that no other thread can observe an intermediate value during the operation. Atomic variables provide lightweight synchronization without the overhead typically associated with locks. Atomics have been gaining increasing interest as many mainstream programming languages such as C, C++, Java, and Rust have introduced direct language support for atomics. However, atomics are generally less commonly used than locks [314] and are primarily intended for experts use in settings where performance is crucial [282]. One domain where atomics are widely utilized is in highly efficient, lock-free data structure libraries [7, 39, 210, 226, 246]. These libraries are widely used in real-world applications, making their correctness critically important. In this thesis, we consider both locks and atomic variables as synchronization primitives.

Nondeterminism

A central characteristic of concurrent programs is *nondeterminism*, arising from the unpredictable order in which threads execute. This unpredictability is often influenced by the thread scheduler, which dynamically determines the order and timing of thread execution based on factors such as available system resources and thread priorities. As a result, the execution of a concurrent program can vary between runs, leading to different interleavings of operations even with identical inputs. This nondeterministic behavior can introduce subtle concurrency bugs such as data races, deadlocks, and livelocks, which may only surface under certain conditions [127, 216]. Nondeterminism has profound implications, necessitating a careful design of programs to ensure that they remain correct regardless of how the scheduler determines execution paths. We demonstrate this concept with an example.

Example 1. Consider the program P given in Figure 2.1. The program consists of two threads, a shared variable x , and a lock l used for synchronization. The threads perform a write operation to x at ② and a read operation at ⑤ within a critical section protected by l , with the initial value of x set to 0. Additionally, the program contains an assertion at ⑦, which checks that the value of the thread-local variable a , determined by the value of x read within the critical section, is equal to 2. If the instructions are executed in such an order that the write on x at ② happens before the read at ⑤, the assertion will pass. However, if the read at ⑤ occurs before the write at ②, the assertion will fail. Thus, the outcome of the assertion is dictated by the scheduling decision of which thread acquires the lock l first.

	$x := 0;$	
① $lock(l);$		④ $lock(l);$
② $x := 1;$		⑤ $a := x + 1;$
③ $unlock(l);$		⑥ $unlock(l);$
		⑦ $assert(a = 2);$

Figure 2.1: Program P

Example 1 presents a simplified example of how thread scheduling affect the outcome of concurrent executions. Further observe that in the example, while all accesses to shared resources are synchronized, this alone is often insufficient to prevent concurrency related issues that arise due to nondeterminism. Dealing with the nondeterminism inherent in thread scheduling has proven to be an exceptionally challenging task, leading to decades of research in the field [41, 70, 132, 274, 304]. A particularly notorious consequence of this nondeterminism is the phenomenon of Heisenbugs [127], which are elusive errors that may remain undetected because they manifest unpredictably [216, 234]. Such bugs make debugging exceedingly difficult, as they often only appear under specific conditions and can lead to inconsistent results when the program is executed multiple times.

The ramifications of nondeterminism are further compounded by various factors that influence thread scheduling decisions. For instance, the conditions during in-house testing—such as the specific architecture or compiler used—might favor certain interleavings, reducing the likelihood of encountering problematic executions. However, once the program is deployed on different architectures or compiled with other compilers, new, untested interleavings may arise, potentially triggering failures that were not previously observed [30]. The impact of nondeterminism becomes particularly pronounced when considering the input space, as the variability of inputs can significantly influence thread interleavings. Making software reliable in the presence of a wide range of inputs has been a challenge for decades, as specific input values can lead to divergent execution paths. This complexity is compounded when these inputs are combined with various thread interleavings, creating a rich landscape of potential execution scenarios.

Concurrency Bugs

A concurrency bug is an error in a concurrent program that occurs due to incorrect interactions between threads, often arising from mismanaged synchronization. Most notorious concurrency bugs include data races, deadlocks, livelocks and atomicity violations. These bugs can lead to unpredictable behavior where the timing and order of thread execution cause unexpected outcomes. Additionally, these bugs can be silent, corrupting data without causing immediate crashes or obvious errors, making them difficult to diagnose and debug. For these reasons, a large body of research has focused on concurrency bugs. Research on concurrency bugs spans several areas, including bug detection [43, 85, 154], debugging techniques [192, 209, 300, 307], and automated fixing [147, 184, 186]. This thesis primarily focuses on the detection of concurrency bugs, which is typically a critical first step in all these areas. Static and dynamic program analysis techniques are the prominent techniques used for detecting concurrency bugs. A detailed discussion of these techniques is provided in Section 2.3. Nevertheless, it is worth noting that certain techniques aim to prevent concurrency bugs entirely, avoiding the need for detection techniques. Next, we provide an overview of these strategies.

Correctness by Construction

Several techniques utilize system or language-level support to prevent concurrency bugs. These approaches typically ensure correctness by construction or enforce it at runtime. In theory, these techniques could eliminate the need for concurrency bug detection entirely. However, in practice, they come with inherent limitations. A general shortcoming of these techniques is that they are often not directly applicable to existing software. In particular, they often require manual effort, such as introducing code annotations or altering the runtime environment. Implementing these changes can be exceptionally complex in large-scale codebases, posing a significant barrier to the widespread adoption of such techniques. In this section, we examine these approaches and highlight their strengths and shortcomings.

Model checking. Model checking is an automated verification technique that, given an abstract model of the underlying software and a formal property, exhaustively checks whether that property is satisfied within the state model. The main benefit of model checking is its ability to provide strong guarantees about system correctness. However, despite the availability of well-developed model checking tools such as SPIN [134] and TLA+ [172], and successful case studies in industrial settings [62, 223, 281], integrating model checking into professional software development processes on a large scale remains a challenge. The challenge is inherent in the model checking process, which involves creating an abstract representation of a program and defining a specification. The abstraction corresponds to a simplification of the program into an analyzable model that captures its essential features, while the specification outlines the desired properties to which the program must adhere. Traditionally, both the abstraction and specification are manually defined by the user,

which is a challenging task that requires ensuring the model accurately reflects the program's features and that the specifications represent the desired properties. Although significant advancements have been made in addressing the challenges on both fronts, i.e., automatically extracting models from programs [79, 133] and utilizing universal specifications, there are still limitations. In particular, techniques that automatically extract models may not produce precise representations of the program's behavior, leading to potential inaccuracies in the verification process. As for specifications, while universally desirable properties, such as data race freedom, can be provided as specifications, richer context-specific properties often require manual specification. Another key challenge in model checking is the state space explosion problem [80], where the number of possible program states grows exponentially as the complexity of the underlying abstraction and specification increases. As a result, utilizing model checking effectively at scale often demands significant expertise and a considerable upfront investment, which hinders widespread adoption. In practice, model checking is typically used to verify key components of a larger system, rather than the entire software stack [223].

Type systems. Various type systems have been developed to ensure freedom from concurrency bugs, including for data races [48, 110, 211], deadlocks [48, 158], and atomicity violations [116]. Such systems have also been implemented in mainstream programming languages. For example, Rust [5] and Swift [6] (starting from Swift 6) provide guarantees for race freedom supported by their type systems. The primary limitations of these type systems include the potential need for manual type annotations and the need for developers to adopt a different programming mindset. This shift can be cumbersome, resulting in an unergonomic burden and creating a steep learning curve for developers accustomed to traditional programming paradigms [119]. Furthermore, the expressivity of programs may become limited. For example, Rust's type system presents fundamental challenges when implementing certain fundamental data structures, such as doubly linked lists. This often leads developers to resort to versions of the systems that bypass these restrictions (e.g., unsafe Rust), which lack the aforementioned guarantees and can inevitably lead to bugs [239].

Program logics. Program logics are formal methods that provide a structured framework for reasoning about the correctness of programs. Such techniques work by utilizing assertions to establish the conditions that must hold before and after the execution of program components. These assertions serve as formal statements that specify the expected behavior of the program. Over the years, various program logics have been developed that target reasoning about concurrent programs [114, 149, 230]. However, a significant drawback of this approach is that it requires significant expertise and manual effort. As a result, such techniques are primarily applied in critical components of a system where correctness is paramount, but it remains impractical for large-scale software due to its high cost and complexity.

Transactional memory. Transactional memory presents an alternative approach to traditional concurrency control by enabling atomic execution of shared memory accesses through transactions [129]. A transaction is a sequence of operations executed by a single thread, ensuring that all changes are applied as a single unit. This

means that a transaction can either fully commit its changes, making them visible to other threads, or abort, in which case all modifications are discarded. This model helps eliminate certain concurrency bugs (e.g., deadlocks), and has been shown to result in fewer errors overall compared to traditional lock based approaches [232, 250]. However, transactional memory relies on hardware or software support, which may require substantial effort for integration into existing systems. Additionally, it can introduce additional overhead [56], and bugs such as data races and atomicity violations can still occur in this setting. These factors constrain the widespread adoption of transactional memory.

Program synthesis. Program synthesis is the process of automatically generating code from high-level specifications [128]. This technique can also be utilized to generate correct-by-design concurrent programs [14, 57, 140]. However, several limitations hinder its widespread adoption. These include constraints on the expressiveness of specifications, the expertise required from developers to craft accurate specifications, and challenges in generating code that is both correct and efficient. Consequently, program synthesis faces significant challenges when it comes to widespread applicability in large-scale concurrent systems.

2.2 Memory Consistency

Memory consistency is a crucial aspect of shared memory computer systems, defining the underlying semantics of how threads interact with each other. In this chapter, we discuss various memory consistency models as well as certain fundamental algorithmic problems associated with these models.

Consistency Models

A precise notion of shared memory semantics is needed to develop correct and efficient shared memory programs. To this end, memory consistency models, often referred to simply as memory models, define the rules that govern the order in which memory operations (reads and writes) appear to execute in shared memory systems. These models are critical for ensuring the correctness of concurrent programs by specifying how memory operations from different threads are observed by each other. Both programming languages (i.e., software memory models) and computer architectures (i.e., hardware memory models) have their own associated memory models. Arguably the most intuitive memory model is Lamport’s *sequential consistency* (SC) [170], which guarantees that the results of execution are the same as if the operations of all threads were interleaved in some sequential order. Intuitively, this corresponds to the requirement that any update performed on a shared resource becomes immediately visible to all other threads, which makes reasoning about concurrency easier for developers.

While SC provides simplicity, it is often considered impractical to implement due to performance concerns [279]. Weak memory models were primarily proposed to address the limitations [21, 142, 143, 195, 270]. These models allow a broader

range of behaviors compared to SC, enabling optimizations that can lead to increased performance. However, this comes at the cost of greater hardware complexity and a more intricate programming model. Examples of widely adopted memory models include those defined by modern hardware architectures such as x86-TSO, Power, and Arm, as well as software models like C11 and the Java Memory Model.

Consistency Checking

Consistency checking is one of the fundamental challenges associated with a memory model. Intuitively, consistency checking involves determining whether a given execution adheres to the rules defined by the memory model. We demonstrate this concept with a concrete example.

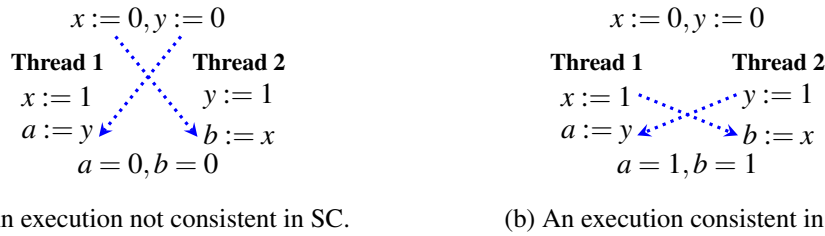


Figure 2.2: Two program executions. Blue edges represent the write that a load observes.

Example 2. Consider the program execution in Figure 2.2a and assume sequential consistency as the underlying memory model. This execution contains two threads and has two shared variables x and y . Initially, both of these variables are set to 0. In the execution, thread 1 first writes 1 to x and loads y and thread 2 writes 1 to y and loads x . The consistency checking problem on this execution corresponds to checking if the load operations on x and y can both read from the initialization writes. Observe that this is not possible under sequential consistency semantics, as either thread 1 or thread 2 should execute its first instruction and after that point, at least one of these load operations can no longer observe the initialization write as there will be a later write operation on that location. Hence, this execution is not consistent under sequential consistency. Consider now the execution in Figure 2.2b. The consistency checking problem on this execution corresponds to checking if the load operations on x and y can read from write operation performed in the other thread. Observe that this behavior is possible under sequential consistency, where the condition is satisfied if both write operations are performed first, followed by the load operations. Hence, this execution is consistent under sequential consistency.

The example above illustrates how loads observing different write operations can influence whether an execution is consistent in a memory model. This problem is formally known as consistency checking [123]. In general, the problem takes a memory model and a program execution as input and outputs whether the execution

is consistent within the framework of that model. Consistency checking is a critical problem due to its significant applications in areas such as model checking and testing [161, 191]. For example, in a testing context, a technique may speculate on potential program behaviors to expand coverage and then perform consistency checks to ensure that these behaviors align with the given memory model, thereby maintaining soundness.

The problem of consistency checking encompasses various variations, each making different assumptions about the available information regarding the program execution. The general setting, sometimes called the reads-value-from consistency [12], assumes that the execution simply contains the thread-local executions and the values observed by load operations. In contrast, reads-from consistency [9] depends on the presence of a "reads-from map" in conjunction with the program execution. This map links each load operation to the write operation from which it retrieves its value. Furthermore, consistency checking on execution graphs [160, 166] assumes that, in addition to the reads-from map, a coherence relation among the write operations is provided, which is typically not directly observable from an execution itself. These consistency checking problems are fundamental challenges that repeatedly arise when verifying the correctness of concurrent programs.

2.3 Dynamic Program Analysis

Program analysis is the process of examining software to reveal its underlying properties and behaviors [225]. Program analysis plays a vital role in enhancing software quality and security by supporting tasks such as bug detection, vulnerability identification, and code optimization for improved performance. It has been established by Turing and Rice that it is generally impossible to ascertain any non-trivial property concerning the behavior of a program. To work around the undecidability of determining precise properties in programs, program analysis techniques in general rely on approximations. These approximations aim to capture useful insights about program behavior without needing full accuracy. However, finding the right level of approximation is challenging, as it depends heavily on the specific problem, goals of the analysis, and acceptable trade-offs between precision and performance. As a result, program analysis has been the focus of extensive research, yielding successful outcomes and leading to the development of widely adopted tools that are a vital part of modern software engineering practices. Program analysis can generally be divided into static analysis and dynamic analysis. Static analysis examines the software without executing it, whereas dynamic analysis involves observing program execution at run-time. Both static and dynamic analyses come with distinct trade-offs, each offering unique advantages and limitations. In this thesis, we focus on dynamic program analysis techniques. While the primary emphasis of this section is on providing an overview of dynamic analysis, we also explore how static analysis compares with dynamic analysis, as well as their potential synergy. Additionally, recall that the primary goal of this thesis is the detection of concurrency bugs. This section will also examine

various aspects of dynamic program analysis from this perspective.

Overview of Dynamic Analysis

Dynamic analysis involves examining the characteristics of a running program [27]. Inspecting the running program allows dynamic analysis to identify properties that hold during the observed executions. Although in general this approach cannot conclusively prove that a program adheres to a specific property, it can be used to detect violations of such properties and offer valuable insights to developers regarding the behavior of the program. This concept has gained significant popularity and has been utilized across various settings to address different problems including bug finding, profiling, and debugging [15, 26, 137, 190, 202, 247, 267, 268]. The primary factor driving the popularity of dynamic analysis are its ability to provide precise information about the program behavior. Additionally, factors such as ease of use, which allows developers to adopt the techniques without extensive expertise, and scalability, allowing them to be effectively applied to large software systems. The main limitations of dynamic analysis stem from its reliance on the observed program executions, which can limit coverage and lead to missed issues that do not manifest during analyzed execution instances. Nevertheless, even this limitation of dynamic analysis is sometimes regarded as a powerful feature. In particular, dynamic analysis inherently enables a straightforward correlation between the changes in program inputs and their subsequent effects on internal behavior of the program and its outputs, as all elements are directly observable during program execution [243]. In the following, we describe the main components of dynamic analysis.

Property. The main goal of dynamic analysis is to uncover specific runtime properties. Dynamic analysis procedures continuously observe program behavior, performing checks to validate or uncover the property of interest. For example, in the context of concurrency, a property of interest can be data races, which dynamic analysis techniques can identify by monitoring thread interactions and data access patterns during program execution.

Traces. Uncovering the property of interest necessitates that dynamic analysis collects data while monitoring the program's execution. We consider the concept of a *trace* [81, 242] to be the collection of all information obtained during program execution. A trace consists of a series of events, which are individual actions that occur during program execution. Each trace captures information relevant to a specific analysis, and different analyses may record varying types of event data. For instance, some analyses may focus on coarse control flow information, while others may capture more fine grained data, such as precise memory access patterns.

Instrumentation. The generation of traces relies on instrumentation, which dictates how the data is collected. Instrumentation is a fundamental component of dynamic analysis and can have profound impacts on performance [313]. Instrumentation can be categorized into two main types: binary instrumentation and source-level instrumentation. Binary instrumentation involves modifying a program's binary to insert monitoring capabilities. Prominent tools for binary instrumentation include

Intel Pin [190] and Valgrind [221], which are widely utilized in practice. Binary instrumentation is regarded as highly versatile and easy to use, as it does not require source code, thereby effectively addressing practical challenges such as instrumenting third-party library code. However, the main drawback of binary instrumentation is that it can introduce significant performance overhead. On the other hand, source-level instrumentation modifies the source code to include additional logging mechanisms that capture relevant information. While it can be faster than binary instrumentation, this approach has its challenges, including difficulties in instrumenting library code and heavy reliance on specific programming languages and compilers. In order to amplify instrumentation, certain dynamic analysis techniques have also been proposed to leverage hardware support which yields enhanced performance [17, 269].

Input. A requirement of dynamic analysis is an input to run the program. The input provided to a dynamic analysis can significantly influence the properties observed during execution, as different inputs can lead to varying program behaviors. Extensive research has focused on understanding the criteria that define adequate test inputs [72, 118, 325]. Moreover, to enhance the adequacy of existing test inputs, a significant amount of research has also focused on automated input generation techniques [75, 98, 303].

Analysis. After defining the property of interest, instrumenting the program, finding the appropriate input, and executing the program to obtain the trace, the final step involves conducting the analysis to extract the desired property from the execution. Typically, an analysis operates by undertaking certain actions in response to the events observed during execution, ultimately producing a result regarding the property of interest. We refer to an analysis as *offline* if it logs the traces during execution and performs a postmortem analysis, whereas an analysis is said to be *online* if it monitors and analyzes events during the program's execution.

Static Analysis vs. Dynamic Analysis

Static techniques analyze source code and build an abstract representation of the analyzed program [214]. This representation often constitutes an approximation of the underlying program in order to facilitate a practical analysis. The abstract representation of the program is then analyzed to extract properties of the program. In contrast to dynamic analysis, static analysis aims to reason about all behaviors of the program instead of focusing on a particular execution. In this regard, dynamic and static analyses can be characterized as "input centric" and "program centric", respectively [27]. A major drawback of static analysis is its imprecision; it may yield conclusions about properties that do not hold true in the actual program. Imprecision can stem from several sources, including the approximate abstraction of the program itself, the potential approximations employed within the analysis algorithm—often implemented to enhance scalability—or a combination of these factors. This challenge is exacerbated in the context of concurrency, where static analyses simultaneously face two dimensions of non-determinism—inputs and scheduling—often leading to even

further degradation in precision and complicating the extraction of reliable program properties.

Dynamic and static analyses serve as complementary techniques [27, 101]. In terms of completeness, dynamic analysis generates properties that hold for the observed executions. Static analysis can aid in verifying whether these dynamic properties extend to all possible program executions, thus addressing gaps in dynamic analysis coverage. In terms of scope, dynamic analysis follows a long execution path, allowing it to reveal semantic dependencies between program components that may occur far apart in time or code. Conversely, static analysis is typically limited in the scope it can handle efficiently, making it less adept at uncovering such distant dependencies. Yet static analysis, by reasoning over all possible behaviors within defined scopes, often achieves precise analysis for dependencies confined within boundaries.

In concurrent program analysis, dynamic and static techniques have been used in complementary ways to enhance precision and efficiency. For instance, hybrid analysis approaches are employed to detect concurrency bugs more effectively by combining dynamic observations with static reasoning [156, 235, 316]. Another example is the use of static analysis to streamline the instrumentation phase [136]. Static techniques, such as escape analysis, can identify shared resources, allowing instrumentation to be applied only to relevant sections. This selective instrumentation avoids instrumenting unnecessary memory accesses and thus reducing overhead for the dynamic analysis. Utilizing dynamic and static techniques in this complementary manner remains an active area of research.

Chapter 3

Dynamic Concurrency Analysis

This chapter introduces the formal foundations of dynamic analysis for detecting concurrency bugs. We begin by formalizing program executions in Section 3.1. Section 3.2 explores the concept of partial orders, a critical concept for dynamic analysis of concurrent programs and one of the central topics studied in this thesis. In Section 3.3, we present formal definitions of common concurrency bugs and review literature that focuses on dynamic analysis methods for detecting these bugs. Finally, in Section 3.4 we review predictive analysis, another key concept in concurrency bug detection and a central topic studied in this thesis.

3.1 Concurrent Execution Model

A dynamic analysis monitors the events generated by a concurrent program and examines the relationships between these events to validate certain program properties. In this section, we formalize the representation of such executions under the assumption of sequential consistency.

Events. An event observed during a concurrent program's execution is represented by a tuple $e = \langle t, \text{id}, \text{op}, m \rangle$. Consider `Threads`, `Vars`, `Locks`, which represent fixed sets of thread, memory location and lock identifiers, respectively. The field $t \in \text{Threads}$ is a unique identifier of the thread that performed e , and $\text{id} \in \mathbb{N}$ is the sequence number of e which captures the order of events that occur within each thread. The sequence number is unique within each thread. The field op corresponds to the operation performed by e , which can be one of the following:

- $\text{r}(x, v)$ denotes reading a memory location $x \in \text{Vars}$ and observing the value v
- $\text{w}(x, v)$ denotes writing the value v on a memory location $x \in \text{Vars}$
- $\text{acq}(l)$ denotes the acquisition of a lock object $l \in \text{Locks}$
- $\text{rel}(l)$ denotes the release of a lock object $l \in \text{Locks}$
- $\text{req}(l)$ denotes a request to acquire of a lock object $l \in \text{Locks}$

- $\text{fork}(t')$ denotes the fork of a new thread $t' \in \text{Threads}$
- $\text{join}(t')$ denotes the join of a thread $t' \in \text{Threads}$

The field m corresponds to the stored meta information. The stored meta information is specific to a given dynamic analysis. When necessary, we will explicitly state the stored meta information; otherwise, we assume that m is empty. Given $e = \langle t, \text{id}, \text{op} \rangle$, we write $\text{tid}(e)$, $\text{id}(e)$, $\text{op}(e)$, $\text{meta}(e)$ to refer to t , id , op , and m respectively. For a read/write event, we use $\text{obj}(e)$ to refer to the memory location that e accesses and $\text{value}(e)$ to refer to the value read/written by e . For acquire/release/request and fork/join events, $\text{obj}(e)$ refers to the acquired/released/requested lock object and forked/joined thread by e , respectively. When denoting an event we often omit certain fields in $\langle t, \text{id}, \text{op} \rangle$, if the information is not relevant or apparent from context. For example, we may represent an event as $\langle t, \text{id} \rangle$ or $\langle t, \text{op} \rangle$. We may also simply represent an event by its operation (e.g., $w(x, v)$). For events $w(x, v)$ and $r(x, v)$, we may omit the value v if it is not relevant.

Execution Traces. We write σ to represent an execution trace, which constitutes the history of all the events observed during a program's execution. For an event $e = \langle t, \text{id}, \text{op} \rangle$, the pair $\langle t, \text{id} \rangle$ uniquely identifies e in a given σ . We use the following notations when referring to the components of σ .

- Events_σ : The set of all events in σ .
- MainThread_σ : The thread identifier reserved for the main thread of the execution.
- Threads_σ : The set of thread identifiers in σ such that $t \in \text{Threads}_\sigma$ iff $\exists e \in \text{Events}_\sigma$ where $\text{tid}(e) = t \vee \text{op}(e) = \text{fork}(t) \vee t = \text{MainThread}_\sigma$.
- Vars_σ : The set of memory location identifiers in σ such that $x \in \text{Vars}_\sigma$ iff $\exists e \in \text{Events}_\sigma$ such that $\text{op}(e) \in \{r(x), w(x)\}$.
- Locks_σ : The set of lock object identifiers in σ such that $l \in \text{Locks}_\sigma$ iff $\exists e \in \text{Events}_\sigma$ such that $\text{op}(e) \in \{\text{acq}(l), \text{rel}(l)\}$.
- $\text{Reads}_\sigma(x)$: The set of read operations in σ such that $e \in \text{Reads}_\sigma$ iff $e \in \text{Events}_\sigma$ and $\text{op}(e) = r(x)$. We let $\text{Reads}_\sigma = \bigcup_{x \in \text{Vars}_\sigma} \text{Reads}_\sigma(x)$.
- $\text{Writes}_\sigma(x)$: The set of write operations in σ such that $e \in \text{Writes}_\sigma$ iff $e \in \text{Events}_\sigma$ and $\text{op}(e) = w(x)$. We let $\text{Writes}_\sigma = \bigcup_{x \in \text{Vars}_\sigma} \text{Writes}_\sigma(x)$.
- $\text{Accesses}_\sigma(x)$: The set of memory operations in σ such that $\text{Accesses}_\sigma(x) = \text{Reads}_\sigma(x) \cup \text{Writes}_\sigma(x)$. We let $\text{Accesses}_\sigma = \text{Reads}_\sigma \cup \text{Writes}_\sigma$.
- $\text{Acquires}_\sigma(l)$: The set of acquire operations in σ such that $e \in \text{Acquires}_\sigma$ iff $e \in \text{Events}_\sigma$ and $\text{op}(e) = \text{acq}(l)$. We let $\text{Acquires}_\sigma = \bigcup_{l \in \text{Locks}_\sigma} \text{Acquires}_\sigma(l)$.

- $\text{Releases}_\sigma(l)$: The set of release operations in σ such that $e \in \text{Releases}_\sigma$ iff $e \in \text{Events}_\sigma$ and $\text{op}(e) = \text{rel}(l)$. We let $\text{Releases}_\sigma = \bigcup_{l \in \text{Locks}_\sigma} \text{Releases}_\sigma(l)$.
- $\text{Synchs}_\sigma(l)$: The set of synchronization operations in σ such that $\text{Synchs}_\sigma(l) = \text{Acquires}_\sigma(l) \cup \text{Releases}_\sigma(l)$. We let $\text{Synchs}_\sigma = \text{Acquires}_\sigma \cup \text{Releases}_\sigma$.
- Forks_σ : The set of fork operations in σ such that $e \in \text{Forks}_\sigma$ iff $e \in \text{Events}_\sigma$ and $\text{op}(e) = \text{fork}(x)$.
- Joins_σ : The set of join operations in σ such that $e \in \text{Joins}_\sigma$ iff $e \in \text{Events}_\sigma$ and $\text{op}(e) = \text{join}(x)$.

Relations and Functions on Execution Traces

A partial order on a set S is a reflexive, transitive and anti-symmetric binary relation on the elements of S . Partial orders form the basis of the analysis of concurrent executions. The core idea behind these techniques is to establish a partial order $\leq_P^\sigma$ on the set of events of the execution trace σ , which represents a certain notion of causality among the events. Although the particular notion of causality depends on the application setting, the transitive nature of such relationships make it amenable to be expressed as a partial order. Intuitively, if two events in σ are causally related, then this is captured by $\leq_P^\sigma$ via ordering these events in the partial order.

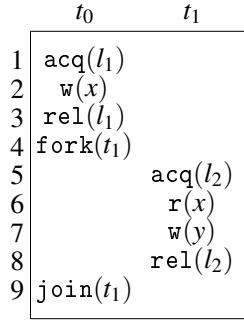
We denote by $\leq_P^\sigma \subseteq \text{Events}_\sigma \times \text{Events}_\sigma$ a partial order P which orders the events in σ . We write $\leq_P^S$ where $S \subseteq \text{Events}_\sigma$ to restrict the partial order to certain events of σ . For a given $e_i, e_j \in \text{Events}_\sigma$, we denote by $e_i \leq_P^\sigma e_j$ to represent $(e_i, e_j) \in \leq_P^\sigma$. We denote by $e_i <_P^\sigma e_j$ to indicate that $e_i \leq_P^\sigma e_j$ and $e_i \neq e_j$. We write $e_i \not\leq_P^\sigma e_j$ (resp. $e_i \not<_P^\sigma e_j$) to represent $(e_i, e_j) \notin \leq_P^\sigma$ (resp. $(e_i, e_j) \notin <_P^\sigma$). We use $e_i \parallel_P^\sigma e_j$ to represent that both $e_i \not\leq_P^\sigma e_j$ and $e_j \not\leq_P^\sigma e_i$ holds. In such a case, e_i and e_j are said to be unordered by P . We say that a set $S \subseteq \text{Events}_\sigma$ is *downward-closed* with respect to $\leq_P^\sigma$, if for all $e, e' \in \text{Events}_\sigma$ with $e \leq_P^\sigma e'$ and $e' \in S$ we have that $e \in S$.

The *program order*, denoted by $\leq_{\text{po}}^\sigma$, is the smallest partial order such that $e_i \leq_{\text{po}}^\sigma e_j$ iff $\text{tid}(e_i) = \text{tid}(e_j)$ and $\text{id}(e_i) \leq \text{id}(e_j)$. In sequentially consistent systems, Events_σ is totally ordered. Accordingly, we define *trace order*, denoted by $\leq_{\text{tr}}^\sigma$, which orders the events in σ based on their order of occurrence. We have that $e \leq_{\text{tr}}^\sigma e'$ iff $e = e'$ or e occurs before e' in σ . When clear from context, we sometimes write e_i to refer to the i^{th} event of σ .

The *reads-from* from function, denoted by rf_σ , maps each read event to the corresponding write event from which it takes its value. In sequentially consistent systems, this correspond to the last write operation that occurred in the same memory location. Formally, for $e \in \text{Reads}_\sigma$ and the mapping $\text{rf}_\sigma(e) = e'$, we have that $e' \leq_{\text{tr}}^\sigma e$ where $e' \in \text{Writes}_\sigma(\text{obj}(e))$ and there exists no $e'' \in \text{Writes}_\sigma(\text{obj}(e))$ such that $e' \leq_{\text{tr}}^\sigma e'' \leq_{\text{tr}}^\sigma e$.

For $e_i, e_j \in \text{Accesses}_\sigma$, we say that e_i and e_j are conflicting, denoted by $e_i \succ e_j$, if $\text{tid}(e_i) \neq \text{tid}(e_j)$, $\text{obj}(e_i) = \text{obj}(e_j)$, and $(\text{op}(e_i), \text{op}(e_j)) \in \{(\text{r}, \text{w}), (\text{w}, \text{r}), (\text{w}, \text{w})\}$. For $l \in \text{Locks}_\sigma$ and $e_i \in \text{Acquires}_\sigma(l)$, $e_j \in \text{Releases}_\sigma(l)$, we say that e_i and e_j are

matching acquire/release events, denoted by $\text{match}_\sigma(e_i) = e_j$ or $\text{match}_\sigma(e_j) = e_i$, if $e_i \leq_{\text{po}}^\sigma e_j$ and there exists no $e \in \text{Synchs}_\sigma(\text{obj}(e_i))$ such that $e_i \leq_{\text{po}}^\sigma e \leq_{\text{po}}^\sigma e_j$. For $e \in \text{Events}_\sigma$, we denote by $\text{LocksHeld}_\sigma(e)$ the set of locks held before executing e . If $l \in \text{LocksHeld}_\sigma(e)$, then there exists $e' \in \text{Acquires}_\sigma(l)$ such that $e' <_{\text{po}}^\sigma e$ and $\text{match}_\sigma(e') \not\leq_{\text{po}}^\sigma e$. For $e \in \text{Acquires}_\sigma$, we denote the set of events contained in the critical section formed by e by $\text{CS}_\sigma(e) = \{e' \mid e \leq_{\text{po}}^\sigma e' \leq_{\text{po}}^\sigma \text{match}_\sigma(e)\}$. If $\text{match}_\sigma(e)$ is not defined, then we let $\text{CS}_\sigma(e) = \{e' \mid e \leq_{\text{po}}^\sigma e'\}$. For $e \in \text{Releases}_\sigma$, we let $\text{CS}_\sigma(e) = \text{CS}_\sigma(\text{match}_\sigma(e))$.

Figure 3.1: Execution trace σ .

Example 3. We illustrate the definitions on the execution trace σ displayed in Figure 3.1. We have the set of events, threads, and locks in σ as $\text{Events}_\sigma = \{e_i\}_{i=1}^9$, $\text{Threads}_\sigma = \{t_0, t_1\}$, $\text{Locks}_\sigma = \{l_1, l_2\}$, respectively. Some examples of trace and program ordered events are $e_1 \leq_{\text{tr}}^\sigma e_2$, $e_4 \leq_{\text{tr}}^\sigma e_5$, and $e_1 \leq_{\text{po}}^\sigma e_2$, $e_5 \leq_{\text{po}}^\sigma e_6$. The reads-from map is $\text{rf}_\sigma(e_5) = e_2$. The set $S_1 = \{e_1, e_2\}$ is downward-closed with respect to $\leq_{\text{tr}}^\sigma$, whereas the set $S_2 = \{e_1, e_6\}$ does not satisfy this property. Some examples of locks held sets are $\text{LocksHeld}_\sigma(e_1) = \text{LocksHeld}_\sigma(e_9) = \emptyset$, whereas $\text{LocksHeld}_\sigma(e_2) = \text{LocksHeld}_\sigma(e_3) = \{l_1\}$ and $\text{LocksHeld}_\sigma(e_6) = \text{LocksHeld}_\sigma(e_7) = \{l_2\}$. We have that $\text{CS}_\sigma(e_1) = \text{CS}_\sigma(e_3) = \{e_1, e_2, e_3\}$ and $\text{CS}_\sigma(e_5) = \text{CS}_\sigma(e_8) = \{e_5, e_6, e_7, e_8\}$.

Well-Formed Execution Traces

An execution trace is said to be well-formed if it satisfies the following rules.

1. **Reads.** All reads have a write event on the same memory location from which they can take their value. In particular, for all $e \in \text{Reads}_\sigma$, we have that $\text{rf}_\sigma(e)$ is defined and $\text{obj}(\text{rf}_\sigma(e)) = \text{obj}(e)$ and $\text{value}(\text{rf}_\sigma(e)) = \text{value}(e)$.
2. **Forks.** For all $t \in \text{Threads}_\sigma \setminus \{\text{MainThread}_\sigma\}$, there exists a unique event $e \in \text{Events}_\sigma$ such that $\text{op}(e) = \text{fork}(t)$. We require that a thread cannot perform any events before it is forked.
3. **Joins.** For all $t \in \text{Threads}_\sigma \setminus \{\text{MainThread}_\sigma\}$, there exists a unique event $e \in \text{Events}_\sigma$ such that $\text{op}(e) = \text{join}(t)$. We require that a thread cannot perform any events after it is joined.

4. **Lock Semantics.** Critical sections induced by the same lock object must not overlap. In particular, for any pair of events $e, e' \in \text{Events}_\sigma$ such that $e = \langle t, \text{acq}(l) \rangle$, $e' = \langle t', \text{acq}(l) \rangle$, and $e \leq_{\text{tr}}^\sigma e'$, there must exist an $e'' = \langle t, \text{rel}(l) \rangle$ such that $e \leq_{\text{tr}}^\sigma e'' \leq_{\text{tr}}^\sigma e'$.

This is a standard definition that aligns with the widely adopted definitions in the literature [202, 236, 276]. In this thesis, we restrict ourselves to only well-formed execution traces. Nevertheless, in order to avoid clutter, we sometimes omit these requirements when discussing example execution traces. For instance, when certain details are not crucial, we may present example execution traces involving threads without forks or joins, or reads without an initial write.

3.2 Partial Orders

As described in Section 3.1, partial orders are a standard mathematical tool in dynamic concurrency analysis. While each analysis employs reasoning tailored to the specific problem at hand, typically a core component of the analysis is establishing a causal ordering among the events observed in σ , which is typically represented as a partial order. In this section, we discuss the key aspects of maintaining such partial orders in the context of dynamic concurrency analysis and highlight the associated efficiency concerns.

A straightforward approach to constructing and maintaining such a partial order is to represent it explicitly as an acyclic directed graph over the events in σ . When the analysis requires determining the ordering between events, a graph search is performed to verify their relationship. However, this approach quickly becomes impractical for dynamic analysis in real-world scenarios. The graph can become extremely large, and the number of queries needed can increase rapidly, resulting in significant inefficiencies. In contrast, vector clocks [205] provide a more efficient way of representing partial orders by exploiting the inherent properties of partial orders in this context. Thus, vector clocks have become the primary data structure utilized within numerous techniques in dynamic concurrency analysis [109, 111, 164, 198, 202, 247, 284]. Next, we describe the details of the vector clock data structure.

Vector Clocks

A common approach for dynamic analyses which aim to capture the causality among the events of execution is to associate a *vector timestamp* with each observed event. A vector timestamp, denoted by V , corresponds to a vector of counters, one for each thread in the execution. For $t \in \text{Threads}_\sigma$, we write $V(t)$ to refer to the t^{th} component of V . Let V be the vector timestamp of a given event $e \in \text{Events}_\sigma$. Intuitively, each index $V(t)$ reflects the causal relationship of e with the thread t . Formally, a vector timestamp is defined as a mapping $V: \text{Threads}_\sigma \rightarrow \mathbb{N}$. The supported operations on vector timestamps are as follows.

- **Comparison.** Determines the causal relationship between two vector timestamps. A vector timestamp V_1 is ordered before another vector timestamp V_2 if each of its components has a smaller value than the corresponding components in V_2 . That is, $V_1 \sqsubseteq V_2 \iff \forall t: V_1(t) \leq V_2(t)$.
- **Join:** Merges two vector timestamps by taking the maximum value of each corresponding component. That is, $V_1 \sqcup V_2 = \lambda t: \max(V_1(t), V_2(t))$.
- **Increment:** Updates the vector timestamp by incrementing the counter for the corresponding thread. That is, $V[t \mapsto +i] = \lambda t': \begin{cases} V(t') + i, & \text{if } t = t' \\ V(t'), & \text{otherwise} \end{cases}$

The underlying partial order can be precisely captured by vector timestamps. In particular, to determine whether two events are ordered according to a partial order $\leq_p^\sigma$, it is sufficient to compare their corresponding vector timestamps. Vector clocks implement the vector timestamps abstraction. Vector clocks are typically represented as integer arrays, indexed by thread identifiers, and support all the operations on vector timestamps. In particular, for a given vector clock $\mathbb{C}$ and a vector timestamp V , we write $\mathbb{C}.\text{Join}(V)$ for the operation that stores the value $\mathbb{C} \sqcup V$ in $\mathbb{C}$. Similarly, we write $\mathbb{C}.\text{Increment}(t, i)$ for the operation that stores the value $\mathbb{C}[t \mapsto +i]$ in $\mathbb{C}$. Additionally, vector clocks support a copy operation $\mathbb{C}.\text{Copy}(V)$ which copies the timestamps in V to $\mathbb{C}$.

Partial Order Based Analyses

Dynamic analysis based on partial orders involves a series of interleaved operations: (i) *inserting* new orderings $e_1 \rightarrow e_2$ in P , and (ii) *querying* if $e_1 \rightarrow e_2$ in P (iii) *deleting* certain ordering from P . These operations play a crucial role in establishing certain properties of σ (e.g., data race freedom) and are performed repeatedly—often proportional to the number of events which can easily reach to billions in real-world settings. Hence, the overall scalability of the analysis largely depends on handling these operations as efficiently as possible. We generally classify analyses into two categories: *streaming analyses* and *non-streaming analyses*.

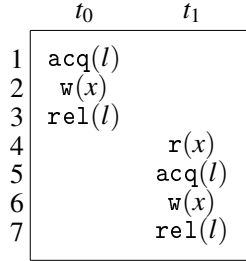
In streaming analysis, events are processed as they observed, and for each new ordering $e_1 \rightarrow e_2$ that is inserted, e_2 corresponds to the current event being processed and deletion of orderings is not supported. In such settings, a single edge insertion can be handled in $\Theta(k)$ time by vector clocks. We present a concrete instance of an analysis that follows a streaming fashion based on Lamport's *happens-before* partial order. This partial order is one of the most widely used in the analysis of concurrent executions, particularly in dynamic race detection. For a given a trace σ , the Lamport's *happens-before* partial order $\leq_{\text{hb}}^\sigma$ is the smallest partial order over Events_σ that fulfills the following conditions.

1. $\leq_{\text{TO}}^\sigma \subseteq \leq_{\text{hb}}^\sigma$.
2. For all $e \in \text{Releases}_\sigma(l)$ and $e' \in \text{Acquires}_\sigma(l)$ such that $e <_{\text{tr}}^\sigma e'$, we have that $e \leq_{\text{hb}}^\sigma e'$.

Algorithm 1: Computing the Lamport's happens-before partial order.

1 procedure <code>acquire</code> (t, ℓ) 2 $\mathbb{C}_t.\text{Join}(\mathbb{C}_\ell)$	3 procedure <code>release</code> (t, ℓ) 4 $\mathbb{C}_\ell.\text{Copy}(\mathbb{C}_t)$
---	---

Algorithm 1 presents a simplified procedure for computing the happens-before partial order. This algorithm tracks a vector clock $\mathbb{C}_t$ for each $t \in \text{Threads}_\sigma$, and a vector clock $\mathbb{C}_l$ for each $l \in \text{Locks}_\sigma$. For an event $e = \langle t, \text{acq} \rangle$, the algorithm joins $\mathbb{C}_{\text{obj}(e)}$ into $\mathbb{C}_t$. For an event $e = \langle t, \text{rel} \rangle$, the algorithm copies $\mathbb{C}_t$ into $\mathbb{C}_{\text{obj}(e)}$. Additionally, while processing an event e , the algorithm implicitly increments the timestamp of $\text{tid}(e)$ in $\mathbb{C}_{\text{tid}(e)}$. The happens-before timestamp of an event e is simply the value in $\mathbb{C}_{\text{tid}(e)}$ after the event has been processed. We show how vector clocks are used to capture the happens-before partial order with an example.

Figure 3.2: Execution trace σ .

Example 4. Consider the execution trace σ in Line 4. We demonstrate how the happens-before partial order is computed in σ using vector clocks. Since there are two threads t_0 and t_1 , the analysis will maintain a set of vector clocks which are arrays of length two. We write $\mathbb{C}_e$ to refer to the vector timestamp associated with an event e . We assume that the i -th index of the array is associated with the i -th thread t_i . Initially, all the vector clocks maintained in the analysis are equal to $[0, 0]$. Observe that after e_3 is processed, we have that $\mathbb{C}_{t_0} = \mathbb{C}_{e_3} = \mathbb{C}_l = [3, 0]$ and $\mathbb{C}_{t_1} = [0, 0]$. The processing of e_4 results in $\mathbb{C}_{t_1} = \mathbb{C}_{e_4} = [0, 1]$. While processing e_5 , the clock $\mathbb{C}_l$ is joined into $\mathbb{C}_{t_1}$, and hence we have that $\mathbb{C}_{e_5} = \mathbb{C}_{t_1} = [3, 2]$. Accordingly, we have that $\mathbb{C}_{e_6} = [3, 3]$. The happens-before ordering relations are captured via vector clocks in the following way. Observe that we have $e_2 \leq_{\text{hb}}^\sigma e_6$ and it is the case that $\mathbb{C}_{e_2} \sqsubseteq \mathbb{C}_{e_6}$. Moreover, we have that $e_2 \parallel_{\text{hb}}^\sigma e_4$ and it is the case that $\mathbb{C}_{e_2} \not\sqsubseteq \mathbb{C}_{e_4}$ and $\mathbb{C}_{e_4} \not\sqsubseteq \mathbb{C}_{e_2}$.

We remark that sometimes happens-before partial order is used as a general term for capturing causal relationships in an execution and do not refer to the Lamport's happens-before relation. Throughout this thesis, unless otherwise stated, we refer to Lamport's definition when discussing the happens-before relation.

There are also dynamic analyses that follow a non-streaming fashion. Typically, such analyses aim to assert more complex properties of σ and they are more

computationally heavy than streaming-style analyses. In such settings, the analysis can add orderings between arbitrary events within σ . These new orderings might have to be propagated across all of Events_σ . While maintaining a partial order for such analyses using vector clocks, a single edge insertion (and propagation) costs $O(|\text{Threads}_\sigma| \cdot |\text{Events}_\sigma|)$. Additionally, certain non-streaming analyses may require deleting orderings from the partial order. This feature is not directly supported by vector clocks. Hence, in such cases analyses may resort to using traditional graphs for maintaining the underlying partial order.

We present a concrete instance of an analysis that follows a non-streaming fashion based on the setting of consistency checking.

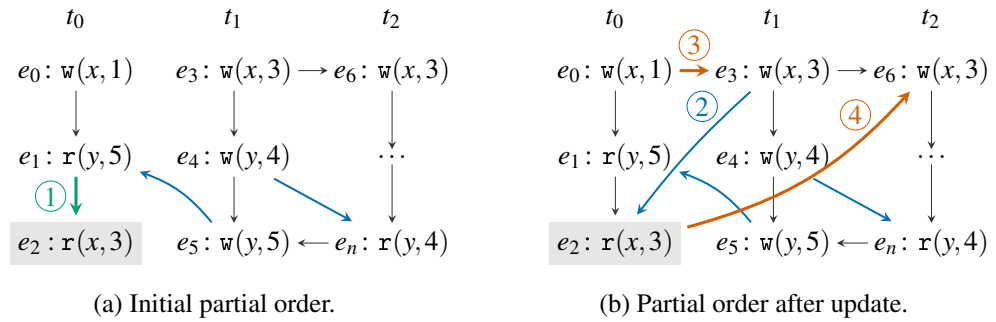


Figure 3.3: A consistency analysis. Blue edges represent reads-from information. Numbered edges represent new orderings inserted by the analysis. Adapted from [292].

Example 5. Consider the execution trace in Figure 3.3. We consider the setting of consistency checking which has numerous applications in dynamic concurrency analyses [12, 49, 60, 61, 97, 137, 154, 191, 252, 291]. In essence, the consistency checking problem seeks to determine if there exists an interleaving of an execution that is consistent with the values observed by each read [123]. In this setting, during the analysis of an execution trace σ , a partial order $\leq_P^\sigma$ is maintained that captures the order induced by the underlying rf_σ map, along with any additional indirect orderings that are imposed.

When a new mapping is added to rf_σ due to the observation of a new event, it may necessitate inferring additional orderings in the partial order. This process is known as saturation and widely used in dynamic concurrency analyses [9, 191, 236, 247, 252, 320, 321]). Formally, consider $r \in \text{Reads}_\sigma(x)$, $\text{rf}_\sigma(r) = w$, $w' \succ r$ and the following cases.

1. $w' \leq_P^\sigma r$ implies adding the ordering $w' \leq_P^\sigma w$.
2. $w \leq_P^\sigma w'$ implies adding the ordering $r \leq_P^\sigma w'$.

The orderings captured through these cases are illustrated in Figure 3.4.

Assume that the consistency analysis has established a partial order P depicted in Figure 6.1a and is now processing the read event $e_2: r(x, 3)$. The analysis first inserts the edge ① in P due to $e_1 \leq_{\text{po}}^\sigma e_2$. This edge insertion can be handled in a similar

approach used in streaming-style analysis, as e_2 has no successor in P , meaning that this insertion does not trigger any propagation.

Observe that e_2 reads the value 3 from the variable x . There are two writes, e_3 and e_6 , which both write the value 3 on x . At this stage, the analysis must determine which of the available writes can be observed by e_2 . In general, no efficient procedure exists for making this decision [123], so the consistency checking algorithm proceeds by testing each possibility.

In Figure 6.1b, the analysis tries the case $\text{rf}_\sigma(e_2) = e_3$. As a result, the analysis first inserts the edge ② in P . Furthermore, since $e_0 \leq_P^\sigma e_2$ and $e_0 \prec e_2$, the analysis inserts ③ in P due to the first saturation rule. Similarly, since $e_3 \leq_P^\sigma e_6$ and $e_6 \prec e_2$, the analysis also inserts ④ in P due to the second saturation rule.

Observe that both e_3 and e_6 have many successors in P . Thus, these edges must be propagated along many events. When vector clocks are used to represent P , each event is associated with a corresponding vector clock. As a result, each step in the propagation process involves performing a vector clock join operation. Consequently, the cost of inserting a single edge using vector clocks is $O(nk)$, where n represents the number of events and k denotes the number of threads in σ . This creates scalability challenges, as both n and k can be large in real-world scenarios.

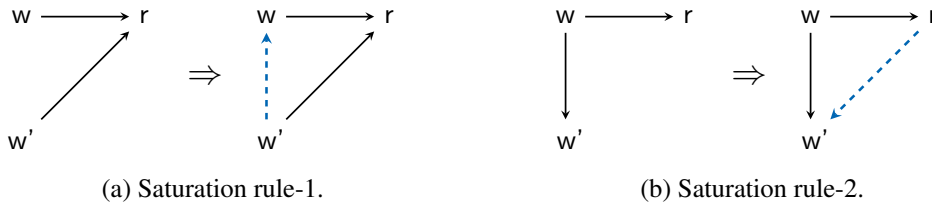


Figure 3.4: Saturation rules where $\text{rf}(r) = w$ and $r \prec w'$. Dashed blue edges represent the inferred orderings.

Example 5 highlights the stark contrast in algorithmic complexity between maintaining partial orders in streaming versus non-streaming analysis. This distinction highlights the crucial role that partial orders play in dynamic analysis, significantly influencing both the efficiency and feasibility of the process.

3.3 Concurrency Bugs

In this section, we examine the two of the most commonly studied concurrency bugs: data races and deadlocks. While different types of concurrency bugs will also be discussed throughout the thesis, these two are particularly important due to their prevalence and impact in concurrent programs and will be revisited repeatedly throughout the thesis.

Data Races

Data races are one of the most common concurrency bugs. A data race occurs when two events are conflicting and they are not causally ordered. That is, a pair of events $e_1, e_2 \in \text{Events}_\sigma$ is said to be in a data race if $e_1 \succ e_2$ and $e_1 \parallel_{\text{hb}}^\sigma e_2$. Note that this is a general definition of data race, which aligns with the definitions of a data race found in mainstream programming languages [42, 194]. The precise definition of a data race can vary depending on the specific concurrency model in use. We demonstrate the concept of data races with an example.

Example 6. Consider the program P given in Figure 3.5a. Observe that the instructions ② and ⑤ perform conflicting operations on the variable x . These instructions are synchronized properly as they are protected by the common lock l . On the other hand, the instructions ② and ⑦ also perform conflicting operations on x . However, observe that these instructions are not properly synchronized. The execution trace σ displayed in Figure 3.5b provides an execution of P in which a data race occurs. In particular, we have that $e_2 \leq_{\text{hb}}^\sigma e_6$ (due to the synchronization induced by e_3 and e_5), hence this pair of events does not form a data race. On the other hand, we have that $e_4 \parallel_{\text{hb}}^\sigma e_6$, hence, this pair of events forms a data race.

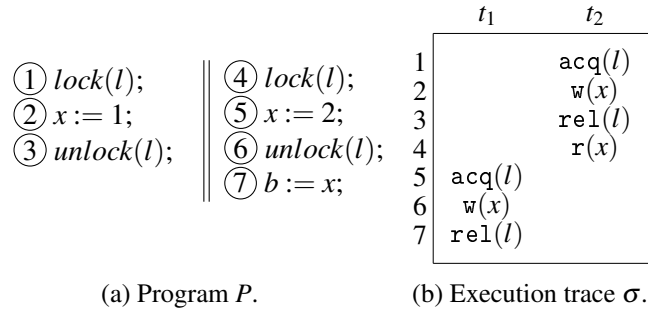


Figure 3.5: An execution trace of a program containing a data race.

There exists a rich literature on dynamic data race detection. In this domain, two of the most popular approaches are partial order based and lockset based techniques. Among the partial order-based techniques, the Lamport's happens-before (HB) is arguably the most prominent method. The key advantages of the HB partial order is that it is a sound approach and it can be computed in linear time. Due to these factors, it has gained traction in practice and forms the basis of widely adopted industrial data race detectors [267]. However, it has been shown that techniques based on HB can miss many data races in practice. This situation occurs because Lamport's HB definition was originally formulated for the context of distributed systems, where an hb ordering represents explicit communication by processes and typically enforces a strict ordering. In contrast, lock synchronization does not impose the same strict ordering [276]. In response to this limitation, numerous techniques have been introduced that employ more sophisticated partial order reasoning to catch

additional data races, albeit at the expense of increased computational cost [138, 199, 202, 236, 248, 276].

On the other hand, lockset based techniques [89, 259] rely on tracking a set of locks that guard accesses to each memory location. These techniques report data races when they encounter an access to a location without sharing a common lock with prior accesses. Although these methods can potentially detect all data races in an execution, they are unsound, often producing false positives. To mitigate this issue, various approaches have been proposed. Some methods utilize an hybrid approach that combine HB and lockset techniques to improve accuracy [95, 318]. There also exists techniques take as input the potentially false data race reports produced by lockset approaches and verify whether these data races are genuine by attempting to realize an execution in which the accesses can execute simultaneously [263]

While data races can lead to subtle errors, it is also argued in the literature that certain data races are benign, having no impact on program correctness. In this thesis, we do not make such a distinction and take the view that there are no benign data races. This aligns with the specifications of languages such as C and C++, where programs with data races have undefined behavior [42]. It has been shown that having "benign" races in such languages, as regarded in the literature, can lead to miscompilations [40].

In summary, data races are arguably the most well-studied concurrency bugs. Nevertheless, the challenges associated with detecting, debugging, and fixing these issues remain far from resolved. New academic studies continue to emerge, exploring innovative approaches to tackle these challenges [17, 54, 191, 192, 248], while tooling support is continuously being developed by the community [38, 267]. For example, tools such as LLVM's ThreadSanitizer are continuously being engineered to scale effectively for large software projects and have been widely adopted to detect data races in industrial settings. Despite these efforts, one reason why the issues persist is that avoiding data races is inherently difficult. While a general strategy for mitigating data races involves introducing synchronization mechanisms, doing so can inadvertently lead to other complications, such as a deadlock or performance degradation due to unnecessary synchronization. Consequently, eliminating data races is in general a non-trivial task that requires careful consideration.

Race conditions. Data races and race conditions are related concepts. However, we are not aware of a widely accepted definition that distinguishes between data races and race conditions. Here, we clarify the terminology used in this thesis. Two events are said to form a race condition if the program exhibits observably different behavior depending on the order of their execution. For instance, in Example 1, the difference in the execution order of events ② and ⑤ leads to an observable difference in the program behavior. However, there are no data races present in that program, as all accesses to shared variables are properly synchronized. Hence, race conditions do not necessarily imply data races. On the other hand, the question of if data races imply race conditions is more intricate and depends on the precise definition of data races. In particular, if for instance the definition of a data race requires that the conflicting events must be able to execute *simultaneously*, then it follows that a data race implies a race condition. This is because varying the execution order of these conflicting

events will result in different behaviors of the program. However, not all widely adopted definitions of data races require the simultaneous execution condition (e.g., C11 memory model [28]). That is, two conflicting events can form a data race even if they can only execute in a specific order rather than simultaneously. This intricacy is not relevant to the results presented in this thesis, so we will not discuss it further.

Deadlocks

Deadlocks represent another significant category of concurrency bugs observed in real-world software [189, 289]. Intuitively, a deadlock occurs among a set of events, each trying to acquire a lock object simultaneously while holding the locks that another event is attempting to acquire. We demonstrate this concept with an example.

Example 7. Consider the program P in Figure 3.6a and two of its execution traces σ_1 and σ_2 in Figure 3.6b and Figure 3.6c, respectively. There exist two locks in the program l_1 , l_2 , and each thread acquires and releases these locks in reversed order. Observe that if the threads execute their instructions serially without interleaving then no deadlock will occur, as shown in σ_1 . Whereas if both threads execute their first instruction and acquires the corresponding lock then a deadlock occurs, as demonstrated in σ_2 . In particular, none of the lock request events e_5 , e_6 in σ_2 will be granted as the lock that is requested is held by another thread that is blocked.

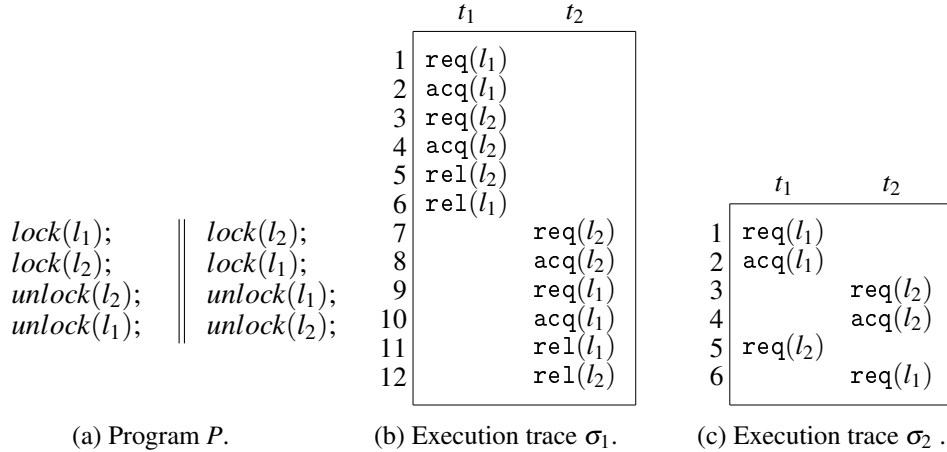


Figure 3.6: A program containing a deadlock and two of its execution traces.

An important concept in topic is the notion of *deadlock patterns*. Intuitively, deadlock patterns arise when different threads acquire locks in an inverse order, as shown in the above example. Formally, a deadlock pattern of size k is a sequence of events $e_0 \dots e_{k-1}$, such that for each event e_i , we have that $\text{op}(e_i) = \text{acq}$ and the $\text{tid}(e_i)$ and $\text{obj}(e_i)$ are distinct in the sequence. Additionally, we have that $\text{obj}(e_i) \in \text{LocksHeld}_\sigma(e_{(i+1)\%k})$ and $\text{LocksHeld}_\sigma(e_i) \cap \text{LocksHeld}_\sigma(e_j) = \emptyset$ for every pair of distinct i and j in the sequence. For example, in Figure 3.6b the events e_4 and e_{10}

form a deadlock pattern. There exists a rich literature in dynamic deadlock detection techniques which aim to detect such deadlock patterns [51, 53, 131]. Nevertheless, a deadlock pattern represents a necessary yet insufficient condition for the occurrence of a real deadlock. We demonstrate this with an example.

Example 8. Consider the program P in Figure 3.7a, which corresponds to a slight modification of the program discussed in Example 7. In particular, in the revised program the threads perform a read and a write operation on a shared variable x . Crucially, whether a lock acquire operation will be performed now depends on the value of this variable. One of the execution traces of P is displayed in Figure 3.7b. Observe that events e_4 and e_{12} form a deadlock pattern. However, this deadlock pattern cannot be realized to an actual deadlock. In particular, executing e_{12} is conditioned on executing e_5 first as otherwise the read in e_{10} will observe a different value and the branch that leads to executing e_{12} will not be taken. However, after e_5 is executed, t_2 cannot execute e_9 until l_2 is released by t_1 . Hence, a deadlock cannot form in P .

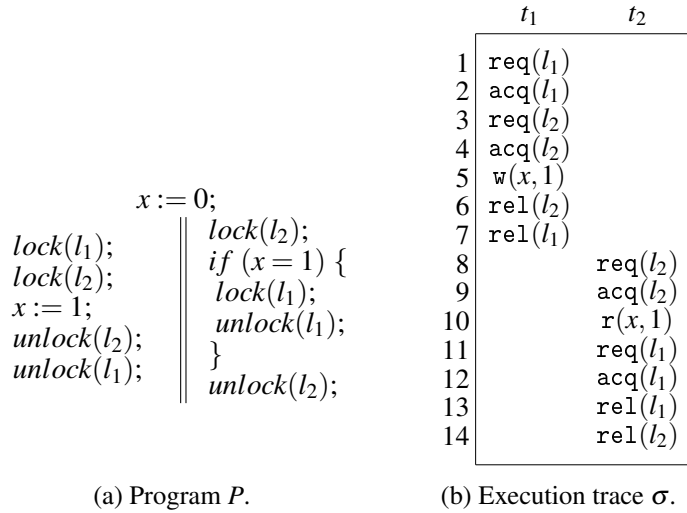


Figure 3.7: A program containing a deadlock.

Due to the this imprecision of deadlock patterns, subsequent works aimed to refine the notion of deadlock patterns to reduce the false positive rate [11, 32]. Moreover, similar to data race detection, some approaches aim to identify all deadlock patterns and then validate them as real deadlocks through re-execution of the program [151]. However, one of the fundamental aspects that distinguishes deadlocks from data races is that a data race involves two events, whereas a deadlock can occur among multiple events. Hence, even identifying all potential deadlocks can also be a challenging task. Predictive techniques have also been applied in this context. A predictive technique based on SMT solving achieves sound and complete deadlock detection, although it is computationally heavy [154]. Additionally, predictive techniques based on partial

orders have been applied in this context, aiming to provide both soundness and more efficient reasoning [54].

3.4 Predictive Analysis

As emphasized previously, dynamic analysis is often limited by its reliance on examining only a specific execution trace, potentially missing bugs that can occur under different conditions. In sequential programs, this limitation is typically mitigated by testing the programs with different inputs to explore a wider range of execution paths. However, even with varied inputs, bugs may still remain undetected in concurrent programs if they only manifest under specific interleavings. Predictive analysis is proposed to address this problem by enhancing the bug finding capability of dynamic analysis in concurrent programs. This approach goes beyond analyzing a single execution by considering alternative interleavings that can occur in the same program which can be inferred from the observed execution. Numerous predictive analysis techniques have been developed that target various concurrency bugs, including data races [202, 236, 276], deadlocks [54, 154], atomicity violations [54, 275], and memory safety issues in concurrent settings [137, 317]. These methods have proven highly effective, often uncovering significantly more bugs than traditional approaches. We demonstrate the concept of predictive analysis with an example.

Example 9. *We revisit the program P discussed in Example 1, which is provided again in Figure 3.8a for convenience. Figure 3.8b–3.8c display two possible execution traces, σ_1 , and σ_2 , respectively, that can be observed from P . Observe that in σ_1 , no assertion violation occurs, whereas the thread interleaving in σ_2 leads to the assertion violation being observed. The primary goal of predictive analysis is to reason about the presence of such an assertion violation without requiring the direct observation of a trace that exhibits the violation. In principle, a predictive analysis can predict the assertion violation given only P and σ_1 , without directly observing σ_2 . In this example, this entails that the underlying analysis must be able to reason that the critical sections $CS_{\sigma_1}(e_1)$ and $CS_{\sigma_1}(e_4)$ can commute.*

Example 9 illustrates again that even in simple concurrent programs bugs can remain hidden. Overall, the process of predictive analysis involves, given a program and an observed execution, uncovering the hidden bugs of the program by reasoning about potential reorderings of the events in the observed execution. In this respect, predictive analysis is regarded as a blend of testing and model checking [276]. However, like model checking, fully complete predictive analysis that aims to explore all possible reorderings faces the challenge of state space explosion, as in general the number of possible reorderings grows exponentially. A critical task in predictive analysis is to develop strategies that can effectively explore and reduce the search space. In practice, this challenge is often addressed by either focusing on a small window of the observed execution for reorderings [137, 154] or restricting the range of reorderings that are explored [54, 202, 236]. Another fundamental challenge in predictive analysis lies in

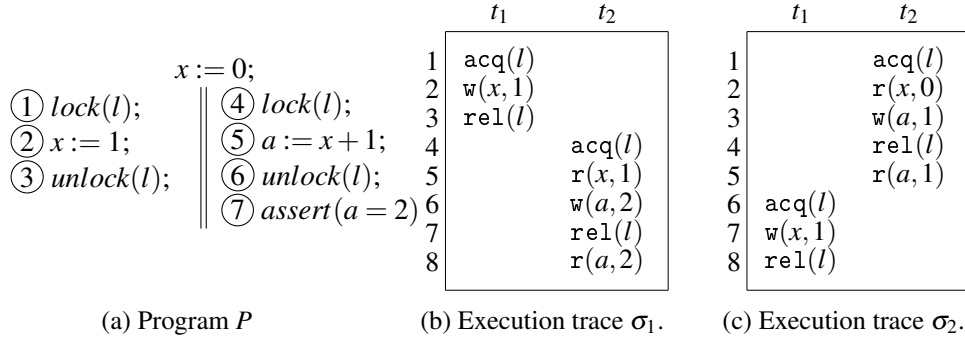


Figure 3.8: Two execution traces of a program.

determining which reorderings are feasible. Next, we discuss the notion of correct reorderings that addresses this challenge.

Correct Reorderings

A core machinery required by predictive analysis techniques is the ability to reason about alternate thread interleavings of a given execution without actually re-executing the program. The notion of *correct reorderings* forms the basis of this machinery. Intuitively, this concept aims to capture, for a given program P and an observed execution σ of P , possible reorderings of the events in σ such that the resulting reordered execution trace σ' could also occur in some run of P . This is crucial for both soundness and completeness of predictive analysis. In particular, producing a reordered trace that cannot occur in any execution of P risks reporting spurious bugs. Conversely, failing to consider certain reorderings that are feasible in different runs of P and can be deduced from the given observed execution may result in missing bugs. We demonstrate this concept with an example.

Example 10. Example 9 demonstrated that critical section can potentially commute and such reasoning can lead to finding bugs which are otherwise not possible. However, here we demonstrate that such a reasoning is not always possible without compromising soundness. Consider the program P in Figure 3.9a and an observed execution trace σ_1 of P is displayed in Figure 3.9b. Observe that the events e_1 and e_8 are potentially in a data race as $e_1 \asymp e_8$, and $\text{LocksHeld}_{\sigma_1}(e_1) \cap \text{LocksHeld}_{\sigma_1}(e_8) = \emptyset$. Nevertheless, it is the case that $e_1 \leq_{\text{hb}}^{\sigma_1} e_8$, hence these events do not constitute a data race in σ_1 . A predictive analysis may aim to perform certain reorderings on σ_1 to arrive at an execution which realizes this potential data race. A possible such reordering σ_2 is displayed in Figure 3.9c, in which we have that $e_4 \parallel_{\text{hb}}^{\sigma_2} e_5$ (e_4 and e_5 in σ_2 are the corresponding events of e_1 and e_8 in σ_1), and hence the data race is realized. However, a careful examination of P reveals that the reordered trace σ_2 is not possible in any execution of P . Observe that the event e_4 of σ_2 will not actually be executed; the branch ⑧ will not be taken since the value 0 is observed for the variable y at the read event in e_2 in σ_2 .

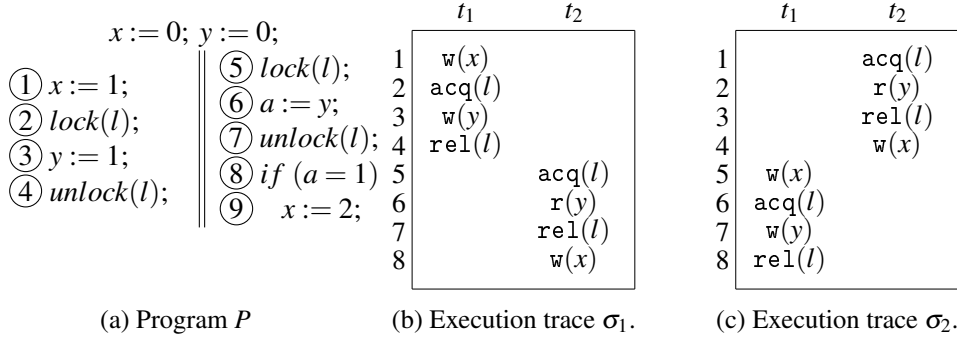


Figure 3.9: Two execution traces of the same program which are not correct reorderings of each other.

Example 10 emphasizes the importance considering only feasible reorderings to preserve the soundness of analysis. Below, we present the definition of correct reordering that we adopt that guarantees feasibility, which is widely recognized in the literature [202, 236, 276].

Definition 1. We say that a trace ρ is a correct reordering of a trace σ if the following conditions are satisfied:

1. $\text{Events}_\rho \subseteq \text{Events}_\sigma$.
2. Events_ρ is downward-closed with respect to $\leq_{\text{po}}^\sigma$.
3. For all $e, e' \in \text{Events}_\sigma$ with $e \leq_{\text{po}}^\sigma e'$ and $e' \in \text{Events}_\rho$, we have that $e \leq_{\text{po}}^\rho e'$.
4. For all $e \in \text{Reads}_\rho$, we have that $\text{rf}_\rho(e) \in \text{Events}_\rho$ and $\text{rf}_\rho(e) = \text{rf}_\sigma(e)$.

Intuitively, the above definition stipulates that the reordered trace must not introduce any new events not observed in the original trace, thread local executions must be maintained, and all read events must observe the same write as in the original trace. The above imposed conditions are to ensure that any correct reordering of a given trace σ can also be generated by the same program which generated σ , potentially under a different thread interleaving. Nevertheless, we remark that this definition is rather conservative. We demonstrate this conservativeness in the following example.

Example 11. Consider the program demonstrated in Figure 3.10a and two of its execution traces σ_1, σ_2 in Figure 3.10b–3.10c, respectively. Observe that both σ_1 and σ_2 can originate from the same program, as well as all read events observe the same value and the final state of the memory is the same. However, σ_1 and σ_2 are not correct reorderings of each other. This is due to the fact that $\text{rf}_{\sigma_1}(e_3) \neq \text{rf}_{\sigma_2}(e_3)$, which violates the condition (4) of Definition 1.

One of the core limitations of Definition 1 is that it is oblivious to the actual values read/written by the events and it enforces that all the read events observe the exact

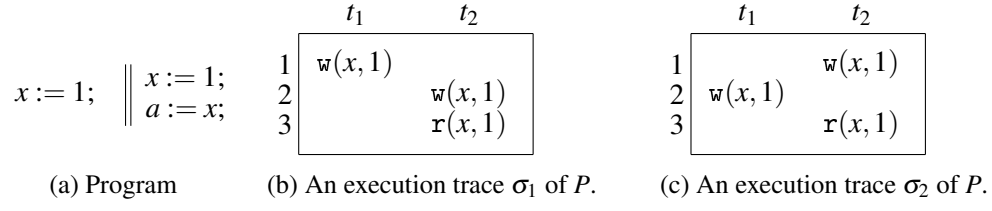


Figure 3.10: Two execution traces of the same program which are not correct reorderings of each other.

same write event as in the original trace. While there are more general definitions of correct reorderings [266], we refrain from adopting them as they typically rely on heavyweight reasoning mechanisms (e.g., SMT solving).

Chapter 4

Conclusions

In this chapter, we present the concluding discussions. Chapters 2–3 presented the foundational background information that supports the technical aspects of this thesis. Building on this foundation, we begin this chapter by presenting a more detailed discussion on the main insights underlying each contribution of this thesis. Next, we provide a discussion outlining the main assumptions that form the basis of this thesis and argue its broad scope. Finally, we explore potential avenues for future research.

4.1 Discussion

In this section, we provide a critical discussion on each individual contribution of this thesis, outline their limitations and compare them with the relevant literature.

Tree Clocks

A central focus of this thesis is improving the data structures used for maintaining partial orders in dynamic concurrency analysis. Partial order-based techniques are widely used in dynamic concurrency analysis, but their performance largely relies on the data structures used to maintain the underlying partial order. As noted in Section 3.2, while vector clocks [205] are the primary data structure used in this field, they have several limitations.

One of the major performance bottlenecks in vector clock based dynamic analysis is the linear dependence of vector timestamp size on the number of threads. Prior research has attempted to address this issue by exploiting special characteristics of the settings under consideration [16, 71, 88, 107, 111, 240, 241, 286, 306] and developing tailored data structures [11, 77, 179]. A notable example is the FastTrack optimization, which replaces the vector clocks used for memory access histories in dynamic data race detection with a lightweight representation. While these optimizations improve efficiency in certain scenarios, they are not universally applicable. In happens-before based data race detection, for instance, vector clocks maintained per thread and lock (as discussed in Section 3.2) are not amenable to such optimizations, as they require

maintaining the timestamps for all threads. In this context, a vector clock join or copy operation takes $\Theta(k)$ time, where k is the number of threads, as each invocation of these operations involve iterating over all threads. Thus, these operations become a bottleneck for large k . The difficulty in addressing this challenge lies in the fact that these vector clock operations already operate in linear time. To improve efficiency, any solution would need to operate in sub-linear time, meaning that it should perform the operation without even accessing certain entries in the vector clock.

We propose the tree clocks data structure to address this challenge. Join and copy operations in tree clocks take time roughly proportional to the number of entries being modified, meaning these operations do not incur the a priori $\Theta(k)$ cost per invocation. The fundamental insight behind tree clocks is that certain vector timestamps are learned transitively through intermediate threads and this can be exploited to gain efficiency. For instance, when performing a join operation, if both the target and the joined vector clocks are aware of the same timestamp from an intermediate thread, there is no need to compare the timestamps learned through that intermediate thread during the join operation, as the target clock would already be aware of them. However, the vector clocks data structure is not suitable for exploiting such properties. Hence, tree clocks turns the flat representation of vector clocks into a hierarchical representation, and stores slightly more additional data alongside the traditional timestamps that are stored in vector clocks. This representation enables tree clocks to exploit such transitivity properties and gain efficiency.

We evaluate tree clocks using Lamport’s happens-before (HB) partial order, as well as the Mazurkiewicz (MAZ) partial order [208] and the schedulable happens-before (SHB) partial order [199]. MAZ serves as a canonical representation for concurrent execution traces and is widely used in areas such as stateless model checking. SHB, on the other hand, is a strengthened version of HB and is used in data race detection. We demonstrate that tree clocks can serve as a drop-in replacement for vector clocks in computing HB, SHB, and MAZ, highlighting their versatility. Additionally, we show that tree clocks are asymptotically the optimal data structure for computing HB. We conduct an extensive evaluation using a large set of benchmarks for dynamic analyses based on HB, SHB, and MAZ. Our experimental results confirm that replacing vector clocks with tree clocks often leads to significant performance improvements in these analyses.

Subsequent studies have applied tree clocks in other settings, such as testing isolation levels of databases [185], where a significant number of sessions exists during the analysis (analogous to threads in our context) and the concept of tree clocks provide significant speedups. However, tree clocks also have certain limitations. For instance, vector clocks benefit from certain properties of modern hardware, such as vectorized instructions, data locality, and prefetching, which can diminish some of the theoretical benefits of tree clocks in practice. Additionally, certain other partial orders proposed in the context of dynamic concurrency analysis, such as WCP [157], DC [247] and SDP [122], cannot be directly represented using tree clocks. Interesting directions for future work include exploring more efficient implementation aspects of tree clocks and extending them to other partial orders.

Collective Sparse Segment Trees

Another limitation of vector clocks arises in non-streaming dynamic concurrency analysis (as discussed in Section 3.2). In this context, inserting a single edge into a partial order maintained by vector clocks has a time complexity of $O(nk)$, where n is the size of the execution and k is the number of threads. This bound becomes prohibitively large for typical values of n in practice ranging from 10^4 to 10^{10} .

To address the challenge of maintaining partial orders in the setting of non-streaming analysis, we introduce a data structure called Collective Sparse Segment Trees (CSSTs). Similar to vector clocks, CSSTs exploit the fact that $k \ll n$ and provide edge insertion operations to run in roughly $O(\log n)$ time, reducing the linear dependency on n to logarithmic. This, however, comes at the cost of handling query operations also in $O(\log n)$ time, which is a task that is handled in constant time in vector clocks. Additionally, we introduce a version of CSSTs that supports edge deletions, a feature not supported by vector clocks.

CSSTs are based on the insight that maintaining partial orders with small width (i.e., small number of threads) can be reduced to the problem of dynamic suffix minima, which is efficiently solvable using a classic data structure known as segment trees. The novelty of CSSTs lies in adapting segment trees for the setting of dynamic concurrency analysis. A key observation behind CSSTs is that in many cases, the underlying graph of the partial orders is sparse, leading us to propose a sparse representation of segment trees to efficiently exploit this structure. This optimization results in a tighter complexity bound than $O(\log n)$ for both edge insertion and query operations and leads to significant performance improvements. Furthermore, incorporating certain additional optimizations allows many query operations to run in constant time, making CSSTs highly efficient in practice.

We evaluate CSSTs across a variety of dynamic concurrency analyses, such as data race detection [191, 236], deadlock detection [54], memory bug prediction [317], consistency checking, use-after-free prediction [137], and root cause analysis [78]. In our experiments, we compare the performance of CSSTs against vector clocks, segment trees, and standard graph representations. The results demonstrate that CSSTs are the most suitable data structure for maintaining partial orders in non-streaming analyses.

Nevertheless, there are also certain limitations of CSSTs. A fundamental aspect of CSSTs is that they are designed for settings with a relatively small number of threads. Alternative data structures are needed for settings which do not satisfy this requirement. In addition, in certain settings edge insertions in vector clocks are performed in a controlled manner and this limits the propagation of edges (e.g., [191]). In such cases, CSSTs may not offer any performance advantages over vector clocks. Also, CSSTs are generally not amenable to fine-grained locking in multi-threaded analysis, requiring the entire data structure to be locked at every operation. Promising directions for future research involve enhancing these aspects of CSSTs.

Deadlock Prediction

Predictive analysis techniques (see Section 3.4) have predominantly targeted finding data races [122, 202, 236, 247, 276], and relatively few approaches have addressed other types of concurrency bugs. In this work, we study predictive techniques in the context of deadlocks.

We first establish the theoretical groundwork for dynamic deadlock prediction. We demonstrate that the task of predicting deadlocks is NP-hard. While analogous complexity results exist for other concurrency bugs (e.g., data races [200] and atomicity violation [106]), this intractability had not been established for deadlocks. Perhaps more surprisingly, we also show that identifying potential deadlocks, i.e., deadlock patterns (see Section 3.3), is NP-hard. This comes, for instance, as a stark contrast to the setting of data races where potential data races can be identified in polynomial-time.

Given these hardness results, we introduce approximations that narrow the scope of the problem to make it tractable. We propose two central novel concepts.

1. First, to address the challenge of finding potential deadlocks, we introduce the notion of *abstract deadlock patterns*. An abstract deadlock pattern is a novel notion that serves as a concise representation for a class of deadlock patterns sharing the same signature. This concept enables the development of efficient algorithms for deadlock prediction.
2. To tackle the problem of deadlock prediction, we propose the notion of synchronization-preserving deadlocks. A central challenge in predictive analysis is identifying constraints on the allowed reorderings that facilitate efficient analysis while maintaining high coverage. The concept of synchronization-preserving deadlocks is tailored for the context of deadlock detection, effectively narrowing the scope of exploration while still capable of capturing the majority of deadlocks encountered in practice. This idea is inspired by synchronization-preserving reorderings [202], initially developed for data race detection. Intuitively, this approach ensures that any reordering preserves the sequence of synchronization operations as they appear in the original execution trace. With this restriction in place, we develop an efficient algorithm to identify all synchronization-preserving deadlocks. A critical component of our algorithm is that its runtime complexity is linear in the length of the execution trace, which can reach billions of events in practical settings and is typically the primary factor in computational complexity. This efficiency is essential for making deadlock prediction feasible in real-world applications.

A central question is the impact of the notion of synchronization-preserving deadlocks on coverage. To address this, we conduct an extensive experimental evaluation and compare our approach with state-of-the-art offline predictive techniques Dirk [154], and SeqCheck [54]. Dirk is a theoretically complete technique, i.e., it can find all deadlocks in an execution trace pertaining to the notion of correct reorderings (see Section 3.4). However, Dirk relies on heavyweight SMT-solving and hence it is computationally intensive. In the case of SeqCheck, its complexity has a high polynomial dependence of n^4 , where n is the length of the execution trace, and this is typically the dominant factor. This is in contrast with the linear dependence on n

in our proposed algorithm. In terms of coverage, SeqCheck and our algorithm are theoretically incomparable, as each can detect certain deadlocks that the other cannot. The results of our evaluation show that our algorithm is significantly more efficient than the state-of-the-art techniques, while maintaining a comparable or better level of coverage.

Furthermore, we develop an online version of our deadlock prediction technique and conduct a runtime analysis. We compare this method with the state-of-the-art runtime tool DeadlockFuzzer [150], which is a testing method based on randomized scheduling. Our online approach incorporates an additional component that biases the scheduler, enabling it to effectively explore various race conditions. To the best of our knowledge, this is the first technique to combine predictive methods with schedule fuzzing techniques. Our evaluation results demonstrate that our online approach is more effective than DeadlockFuzzer.

There are several promising future directions for this work. Our current integration of online predictive analysis with randomized scheduling relies on basic heuristics. Exploring a more tightly coupled approach between these techniques could be an interesting avenue to enhance their overall effectiveness. Additionally, in some cases, although rarely observed in practice, our proposed algorithm misses deadlocks detected by other predictive techniques. A natural question is whether we can extend the coverage of synchronization-preserving deadlocks while maintaining efficiency. Additionally, some of the missed deadlocks do not fall under the notion of correct reorderings. Therefore, it is worth exploring predictive analyses techniques that rely on more relaxed definitions of correct reorderings.

Consistency Checking

Reads-from consistency checking is a fundamental problem associated with a memory model and plays a crucial role in applications involving dynamic analysis and model checking (as discussed in Section 2.2). Motivated by these applications, we investigate this problem within the context of the C11 memory model, which underpins the memory semantics of the C and C++ programming languages. Notably, the influence of the C11 model extends beyond these languages, influencing the memory models of popular languages such as Rust, Swift, and JavaScript.

We study the problem of reads-from consistency checking for popular variants of the C11 memory model and uncover a rich landscape of results. We characterize the complexity of the problem under these models, develop efficient algorithms, and establish NP-hardness results for certain variants. When designing a consistency checking algorithm in this context, a common approach to proving consistency is to establish a (partial) coherence relation that serves as a witness for consistency. This approach has been widely used in prior work, with numerous witness coherence relations proposed for different variants of the C11 memory model [8, 167, 191]. The central concept introduced in this work is the notion of *minimal coherence* witness relation. Compared to prior approaches, this notion establishes a weaker witness rela-

tion and it is more efficient to compute. This leads to consistency checking algorithms with tighter theoretical bounds and significant practical performance improvements.

We implemented our algorithms within the state-of-the-art model checker GenMC and the testing tool C11Tester, replacing only the consistency checking component. Our evaluation on standard benchmarks demonstrated significant performance speedups in practice.

A limitation of this work is that the consistency algorithms do not support sequentially consistent (SC) accesses. Performing consistency checking in the presence of SC accesses presents a fundamental challenge, as the problem becomes NP-hard in general. Nevertheless, previous research has shown practical success in handling SC consistency through certain heuristics (e.g., techniques based on saturation [9, 236]). A promising future direction would be to integrate these techniques with the concept of minimal coherence.

A notable remark is that a subsequent study by Chakraborty et al. [60] showed that consistency checking based on coarser abstractions, i.e., reads-value-from consistency (see Section 2.2), turns out to be NP-complete for many weak memory models even with bounded input parameters (e.g., number of threads, and memory locations). This includes models for which efficient algorithms have been developed in our work. We believe this highlights the significance of the practical and efficient algorithms developed in this work for reads-from consistency, and in applications such as model checking an exploration strategy based on reads-from equivalence may be preferable in practice.

4.2 Scope

Here, we reiterate the main assumptions underlying this thesis and define the scope.

- **Programming language.** The techniques developed in this thesis are generally language agnostic and do not depend on features unique to a specific programming language. Nevertheless, we remark that the experimental evaluations of the techniques proposed in this thesis were conducted on programs written in mainstream languages such as C, C++, and Java. Therefore, the evaluation results may carry a bias toward programs written in these languages.
- **Communication mechanism.** We assume shared memory as the underlying communication mechanism, which is the widely used mechanism in mainstream programming languages. Additionally, we consider two synchronization primitives, locks and atomic variables, which are commonly used to coordinate accesses to shared resources in shared-memory environments. These assumptions aligns with current programming practices.
- **Concurrency model.** We consider both sequential consistency and weak consistency models. In particular, the proposed data structures, tree clocks and

CSSTs, do not rely on any specific concurrency model and are broadly applicable. On the other hand, our contribution to deadlock prediction explicitly assumes sequential consistency, while our contribution to consistency checking assumes a weak consistency setting. Overall, our findings are not limited to a single concurrency model and we present results that demonstrate applicability across various models.

- **Specifications.** We assume that the specifications evaluated by dynamic analyses are readily available and accurate. In our evaluations, we utilize universal specifications such as data race freedom and deadlock freedom. While the development of accurate specifications is a vital component of dynamic analysis, we view it as orthogonal to the techniques presented in this thesis.
- **Inputs.** We assume that the inputs for the dynamic analysis are readily available. In our experimental evaluations, we utilize the inputs available in the benchmark suites. While inputs are a fundamental component of dynamic analysis with many associated aspects—such as input generation and prioritization—we consider these elements to be orthogonal to the focus of our study.
- **Soundness.** We restrict our attention to only sound dynamic analysis (i.e., no false positives). We recognize that, for instance, it may be possible to improve the scalability or coverage of a dynamic analysis technique by forgoing soundness. However, such approaches are not explored in this thesis, as soundness has been repeatedly reported as a critical feature of program analysis tools [76, 90, 255].
- **Hardware support.** We do not rely on any hardware support. While certain features of specialized hardware could potentially enhance the effectiveness of a proposed technique [17], relying on such hardware may limit the technique’s direct applicability in broader contexts. This assumption allows us to maintain a more general framework that can be applied across a variety of environments without dependency on specific hardware configurations.

Overall, we believe that the results obtained in this thesis have broad applicability.

4.3 Future Work

In the following, we discuss avenues for future work.

- **Specifications.** Finding the right specifications that precisely match program correctness requirements remains a critical challenge. In the context of concurrency, universal specifications such as data race freedom and deadlock freedom are the most commonly utilized. However previous research has shown these specifications can be inadequate. For instance, data race freedom alone does not prevent certain critical concurrency issues, as well as data races sometimes

being regarded as false positives [155], which has led to research on developing alternative specifications [105, 188, 233, 234]. Despite advances in this field, challenges still remain. An ideal specification must accurately capture the developer's intentions regarding the program's behavior, eliminate the need for manual annotations (which are typically impractical), and be efficiently verifiable. Current techniques fail to satisfy these requirements simultaneously, and further development of novel specifications could lead to increased reliability in concurrent systems.

- **Inputs.**

A common source of bugs in software systems is unexpected inputs. Traditional fuzz testing has proven to be an effective method for generating a wide range of inputs to test programs. However, this process becomes more complex in the context of concurrency. Fuzzing for concurrency bugs requires simultaneous exploration of two dimensions, thread interleavings and input generation, making it inherently more challenging. A fundamental challenge in concurrency fuzzing is devising a feedback mechanism that guides the fuzzing process effectively through the vast search space of possible thread interleavings combined with the input space. Additionally, there is a need for techniques that can accurately and efficiently mutate the thread interleavings. Developing such feedback and mutation mechanisms is crucial for improving the effectiveness of fuzzing in analyzing concurrent programs. While prior work has partially addressed these issues [66, 145, 146], the combination of thread scheduling and input generation for effective fuzzing remains a largely unexplored area.

- **Predictive analysis in weak memory.**

Predictive analysis techniques have proven to be highly effective in identifying concurrency bugs in sequential consistency settings. However, these approaches have not been explored in the context of weak memory models, which present additional complexities. A promising direction for future research is to study predictive analysis within this context. This, however, may require developing efficient algorithms and heuristics to address certain intermediate problems. In this respect, our work discussed in Chapter 8 introduces novel algorithms for efficiently performing reads-from consistency checking in a weak memory setting. We believe this addresses one of the key intermediate problems that needs to be solved, and it may serve as a crucial first step toward enabling effective predictive analysis techniques for weak memory models.

- **Stateful and hybrid dynamic analysis.**

Most dynamic analysis techniques focus on a single execution of the program, limiting their ability to detect bugs to that specific execution. Two directions can address this limitation. One direction is stateful dynamic analysis, where properties learned from one execution are retained and used to guide the analysis in subsequent runs. The second direction is hybrid analysis, which incorporates

static program properties to enhance the effectiveness of dynamic analysis. While prior work has partly explored these directions [156, 235, 305, 316], combining them effectively to achieve stateful and hybrid analysis remains a challenging task. Developing novel techniques that can extract and combine complex static properties, as well as dynamic properties across executions, could provide invaluable information. This concept has the potential to utilize this information in a way that significantly enhances both the coverage and efficiency of dynamic analysis.

Part II

Publications

Chapter 5

A Tree Clock Data Structure for Causal Orderings in Concurrent Executions

By Umang Mathur, Andreas Pavlogiannis, Hünkar Can Tunç, Mahesh Viswanathan.
Proceedings of the 27th ACM International Conference on Architectural Support for
Programming Languages and Operating Systems (ASPLOS), February 2022.

Abstract

Dynamic techniques are a scalable and effective way to analyze concurrent programs. Instead of analyzing all behaviors of a program, these techniques detect errors by focusing on a single program execution. Often a crucial step in these techniques is to define a causal ordering between events in the execution, which is then computed using *vector clocks*, a simple data structure that stores logical times of threads. The two basic operations of vector clocks, namely join and copy, require $\Theta(k)$ time, where k is the number of threads. Thus they are a computational bottleneck when k is large.

In this work, we introduce *tree clocks*, a new data structure that replaces vector clocks for computing causal orderings in program executions. Joining and copying tree clocks takes time that is roughly proportional to the number of entries being modified, and hence the two operations do not suffer the a-priori $\Theta(k)$ cost per application. We show that when used to compute the classic happens-before (HB) partial order, tree clocks are *optimal*, in the sense that no other data structure can lead to smaller asymptotic running time. Moreover, we demonstrate that tree clocks can be used to compute other partial orders, such as schedulable-happens-before (SHB) and the standard Mazurkiewicz (MAZ) partial order, and thus are a versatile data structure. Our experiments show that just by replacing vector clocks with tree clocks, the computation becomes from $2.02\times$ faster (MAZ) to $2.66\times$ (SHB) and $2.97\times$ (HB) on average per benchmark. These results illustrate that tree clocks have the potential to become a standard data structure with wide applications in concurrent analyses.

5.1 Introduction

The analysis of concurrent programs is one of the major challenges in formal methods, due to the non-determinism of inter-thread communication. The large space of communication interleavings poses a significant challenge to the programmer, as intended invariants can be broken by unexpected communication patterns. The subtlety of these patterns also makes verification a demanding task, as exposing a bug requires searching an exponentially large space [217]. Consequently, significant efforts are made towards understanding and detecting concurrency bugs efficiently [40, 104, 189, 272, 289, 322].

Dynamic analyses and partial orders. One popular approach to the scalability problem of concurrent program verification is dynamic analysis [111, 200, 206, 237]. Such techniques have the more modest goal of discovering faults by analyzing program executions instead of whole programs. Although this approach cannot prove the absence of bugs, it is far more scalable than static analysis and typically makes sound reports of errors. These advantages have rendered dynamic analyses a very effective and widely used approach to error detection in concurrent programs.

The first step in virtually all techniques that analyze concurrent executions is to establish a causal ordering between the events of the execution. Although the notion of causality varies with the application, its transitive nature makes it naturally expressible as a *partial order* between these events. One prominent example is the Mazurkiewicz partial order (MAZ), which often serves as the canonical way to represent concurrent traces [34, 208] (aka Shasha-Snir traces [271]). Another vastly common partial order is Lamport’s *happens-before* (HB) [171], initially proposed in the context of distributed systems [262]. In the context of testing multi-threaded programs, partial orders play a crucial role in dynamic race detection techniques, and have been thoroughly exploited to explore trade-offs between soundness, completeness, and running time of the underlying analysis. Prominent examples include the widespread use of HB [95, 111, 144, 237, 267], schedulably-happens-before (SHB) [199], causally-precedes (CP) [276], weak-causally-precedes (WCP) [157], doesn’t-commute (DC) [247], and strong/weak-dependently-precedes (SDP/WDP) [122], M2 [236] and SyncP [202]. Beyond race detection, partial orders are often employed to detect and reproduce other concurrency bugs such as atomicity violations [36, 117, 198], deadlocks [257, 284], and other concurrency vulnerabilities [317].

Vector clocks in dynamic analyses. Often, the computational task of determining the partial ordering between events of an execution is achieved using a simple data structure called *vector clock*. Informally, a vector clock $\mathbb{C}$ is an integer array indexed by the processes/threads in the execution, and succinctly encodes the knowledge of a process about the whole system. For vector clock $\mathbb{C}_{t_1}$ associated with thread t_1 , if $\mathbb{C}_{t_1}(t_2) = i$ then it means that the latest event of t_1 is ordered after the first i events of thread t_2 in the partial order. Vector clocks, thus seamlessly capture a partial order, with the point-wise ordering of the vector timestamps of two events capturing the ordering between the events with respect to the partial order of interest. For this reason, vector clocks are instrumental in computing the HB partial order efficiently [108, 109, 206],

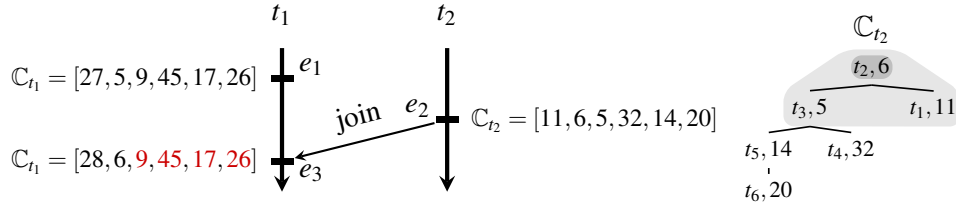


Figure 5.1: (Left) Illustration of the effect of a join operation $\mathbb{C}_{t_1} \leftarrow \mathbb{C}_{t_1} \sqcup \mathbb{C}_{t_2}$ on the clocks of the two threads. The j -th entry in timestamps correspond to thread t_j . Red entries remain unchanged, as t_1 already knows of a later time. (Right) A tree representation of the clocks $\mathbb{C}_{t_2}$ that encodes transitivity. Dark gray marks the threads whose clock has processed in $\mathbb{C}_{t_2}$ compared to $\mathbb{C}_{t_1}$ (i.e., just t_2). Light gray marks the nodes that we need to examine when performing the join operation.

and are ubiquitous in the efficient implementation of analyses based on partial orders even beyond HB [111, 157, 164, 198, 199, 247, 257, 284].

The fundamental operation on vector clocks is the pointwise *join* $\mathbb{C}_{t_1} \leftarrow \mathbb{C}_{t_1} \sqcup \mathbb{C}_{t_2}$. This occurs whenever there is a causal ordering from thread t_2 to t_1 . Operationally, a join is performed by updating $\mathbb{C}_{t_1}(t) \leftarrow \max(\mathbb{C}_{t_1}(t), \mathbb{C}_{t_2}(t))$ for every thread t , and captures the transitivity of causal orderings: as t_1 learns about t_2 , it also learns about other threads t that t_2 knows about. Note that if t_1 is aware of a later event of t , this operation is vacuous. With k threads, a vector clock join takes $\Theta(k)$ time, and can quickly become a bottleneck even in systems with moderate k . This motivates the following question: is it possible to speed up join operations by proactively avoiding vacuous updates? The challenge in such a task comes from the efficiency of the join operation itself—since it only requires linear time in the size of the vector, any improvement must operate in sub-linear time, i.e., not even touch certain entries of the vector clock. We illustrate this idea on a concrete example, and present the key insight in this work.

Motivating example. Consider the example in Figure 5.1. It shows a partial trace from a concurrent system with 6 threads, along with the vector timestamps at each event. When event e_2 is ordered before event e_3 due to synchronization, the vector clock $\mathbb{C}_{t_2}$ of t_2 is joined with that of $\mathbb{C}_{t_1}$, i.e., the t_j -th entry of $\mathbb{C}_{t_1}$ is updated to the maximum of $\mathbb{C}_{t_1}(t_j)$ and $\mathbb{C}_{t_2}(t_j)^*$. Now assume that thread t_2 has learned of the current times of threads t_3, t_4, t_5 and t_6 via thread t_3 . Since the t_3 -th component of the vector timestamp of event e_1 is larger than the corresponding component of event e_2 , t_1 cannot possibly learn any *new* information about threads t_4, t_5 , and t_6 through the join performed at event e_3 . Hence the naive pointwise updates will be redundant for the indices $j = \{3, 4, 5, 6\}$. Unfortunately, the flat structure of vector clocks is not amenable to such reasoning and cannot avoid these redundant operations.

*As with many presentations of dynamic analyses using vector clocks [144], we assume that the *local* entry of a thread's clock increments by 1 after each event it performs. Hence, in Figure 5.1, the t_1 -th entry of $\mathbb{C}_{t_1}$ increases from 27 to 28 after e_1 is performed.

To alleviate this problem, we introduce a new hierarchical tree-like data structure for maintaining vector times called a *tree clock*. The nodes of the tree encode local clocks, just like entries in a vector clock. In addition, the structure of the tree naturally captures which clocks have been learned transitively via intermediate threads. Figure 5.1 (right) depicts a (simplified) tree clock encoding the vector times of $\mathbb{C}_{t_2}$. The subtree rooted at thread t_3 encodes the fact that t_2 has learned about the current times of t_4 , t_5 and t_6 *transitively*, via t_3 . To perform the join operation $\mathbb{C}_{t_1} \leftarrow \mathbb{C}_{t_1} \sqcup \mathbb{C}_{t_2}$, we start from the root of $\mathbb{C}_{t_2}$, and traverse the tree as follows. Given a current node u , we proceed to the children of u *if and only if* u represents the time of a thread that is not known to t_1 . Hence, in the example, the join operation will now access only the light-gray area of the tree, and thus compute the join without accessing the whole tree, resulting in a *sublinear running time* of the join operation.

The above principle, which we call *direct monotonicity* is one of two key ideas exploited by tree clocks; the other being *indirect monotonicity*. The key technical challenge in developing the tree clock data structure lies in (i) using direct and indirect monotonicity to perform efficient updates, and (ii) perform these updates such that direct and indirect monotonicity are preserved for future operations. Section 5.3 illustrates the intuition behind these two principles in depth.

Contributions. Our contributions are as follows.

1. We introduce *tree clock*, a new data structure for maintaining logical times in concurrent executions. In contrast to the flat structure of the traditional vector clocks, the dynamic hierarchical structure of tree clocks naturally captures ad-hoc communication patterns between processes. In turn, this allows for join and copy operations that run in *sublinear time*. As a data structure, tree clocks offer high versatility as they can be used in computing many different ordering relations.
2. We prove that tree clocks are an *optimal data structure* for computing HB, in the sense that, *for every input trace*, the total computation time cannot be improved (asymptotically) by replacing tree clocks with any other data structure. On the other hand, vector clocks do not enjoy this property.
3. We illustrate the versatility of tree clocks by presenting tree clock-based algorithms for the MAZ and SHB partial orders.
4. We perform a large-scale experimental evaluation of the tree clock data structure for computing the MAZ, SHB and HB partial orders, and compare its performance against the standard vector clock data structure. Our results show that just by replacing vector clocks with tree clocks, the computation becomes up to $2.97\times$ faster on average. Given our experimental results, we believe that replacing vector clocks by tree clocks in partial order-based algorithms can lead to significant improvements on many applications. We provide the proofs for the theorems and lemmas presented in the paper in our technical report [204].

5.2 Preliminaries

In this section we develop relevant notation and present standard concepts regarding concurrent executions, partial orders and vector clocks.

Concurrent Model and Traces

We start with our main notation on traces. The exposition is standard and follows related work (e.g., [111, 157, 276]).

Events and traces. We consider execution traces of concurrent programs represented as a sequence of events performed by different threads. Each event is a tuple $e = \langle i, t, \text{op} \rangle$, where i is the unique event identifier of e , t is the identifier of the thread that performs e , and op is the operation performed by e , which can be one of the following types [†].

1. $\text{op} = \text{r}(x)$, denoting that e reads global variable x .
2. $\text{op} = \text{w}(x)$, denoting that e writes to global variable x .
3. $\text{op} = \text{acq}(\ell)$, denoting that e acquires the lock ℓ .
4. $\text{op} = \text{rel}(\ell)$, denoting that e releases the lock ℓ .

We write $\text{tid}(e)$ and $\text{op}(e)$ to denote the thread identifier and the operation of e , respectively. For a read/write event e , we denote by $\text{Variable}(e)$ the (unique) variable that e accesses. We often ignore the identifier i and represent e as $\langle t, \text{op} \rangle$. In addition, we are often not interested in the thread of e , in which case we simply denote e by its operation, e.g., we refer to event $\text{r}(x)$. When the variable of e is not relevant, it is also omitted (e.g., we may refer to a read event r).

A (concrete) *trace* is a sequence of events $\sigma = e_1, \dots, e_n$. The trace σ naturally defines a total order $\leq_{\text{tr}}^\sigma$ (pronounced *trace order*) over the set of events appearing in σ , i.e., we have $e \leq_{\text{tr}}^\sigma e'$ iff either $e = e'$ or e appears before e' in σ ; when $e \neq e'$, then we say $e <_{\text{tr}}^\sigma e'$. We require that σ respects the semantics of locks. That is, for every lock ℓ and every two acquire events $\text{acq}_1(\ell)$, $\text{acq}_2(\ell)$ on the lock ℓ such that $\text{acq}_1(\ell) <_{\text{tr}}^\sigma \text{acq}_2(\ell)$, there exists a lock release event $\text{rel}_1(\ell)$ in σ with $\text{tid}(\text{acq}_1(\ell)) = \text{tid}(\text{rel}_1(\ell))$ and $\text{acq}_1(\ell) <_{\text{tr}}^\sigma \text{rel}_1(\ell) <_{\text{tr}}^\sigma \text{acq}_2(\ell)$. Finally, we denote by Thrs_σ the set of thread identifiers appearing in σ .

Thread order. Given a trace σ , the *thread order* $\leq_{\text{TO}}^\sigma$ is the smallest partial order such that $e_1 \leq_{\text{TO}}^\sigma e_2$ iff $\text{tid}(e_1) = \text{tid}(e_2)$ and $e_1 \leq_{\text{tr}}^\sigma e_2$. For an event e in a trace σ , the local time $\text{lTime}^\sigma(e)$ of e is the number of events that appear before e in the trace σ that are also performed by $\text{tid}(e)$, i.e., $\text{lTime}^\sigma(e) = |\{e' \mid e' \leq_{\text{TO}}^\sigma e\}|$. We remark that the pair $(\text{tid}(e), \text{lTime}^\sigma(e))$ uniquely identifies the event e in the trace σ .

Conflicting events. Two events e_1, e_2 of σ are called *conflicting*, denoted by $e_1 \asymp e_2$, if (i) $\text{Variable}(e_1) = \text{Variable}(e_2)$, (ii) $\text{tid}(e_1) \neq \text{tid}(e_2)$, and (iii) at least one of e_1, e_2 is a write event. The standard approach in concurrent analyses is to detect conflicting events that are causally independent, according to some pre-defined notion of causality, and can thus be executed concurrently.

[†]Fork and join events are ignored for ease of presentation. Handling such events is straightforward.

Partial Orders, Vector Times and Vector Clocks

A partial order on a set S is a reflexive, transitive and anti-symmetric binary relation on the elements of S . Partial orders are the standard mathematical object for analyzing concurrent executions. The main idea behind such techniques is to define a partial order $\leq_P^\sigma$ on the set of events of the trace σ being analyzed. The intuition is that $\leq_P^\sigma$ captures *causality* — the relative order of two events of σ must be maintained if they are ordered by $\leq_P^\sigma$. More importantly, when two events e_1 and e_2 are unordered by $\leq_P^\sigma$ (denoted $e_1 \parallel_P^\sigma e_2$), then they can be deemed *concurrent*. This principle forms the backbone of all partial-order based concurrent analyses.

A naïve approach for constructing such a partial order is to explicitly represent it as an acyclic directed graph over the events of σ , and then perform a graph search whenever needed to determine whether two events are ordered. Vector clocks, on the other hand, provide a more efficient method to represent partial orders and therefore are the key data structure in most partial order-based algorithms. The use of vector clocks enables designing streaming algorithms, which are also suitable for monitoring the system. These algorithms associate *vector timestamps* [108, 109, 206] with events so that the point-wise ordering between timestamps reflects the underlying partial order. Let us formalize these notions now.

Vector Timestamps. Let us fix the set of threads Thrds in the trace. A *vector timestamp* (or simply vector time) is a mapping $V: \text{Thrds} \rightarrow \mathbb{N}$. It supports the following operations.

$$\begin{aligned} V_1 \sqsubseteq V_2 & \quad \text{iff } \forall t: V_1(t) \leq V_2(t) & \text{(Comparison)} \\ V_1 \sqcup V_2 & = \lambda t: \max(V_1(t), V_2(t)) & \text{(Join)} \\ V[t' \rightarrow +i] & = \lambda t: \begin{cases} V(t) + i, & \text{if } t = t' \\ V(t), & \text{otherwise} \end{cases} & \text{(Increment)} \end{aligned}$$

We write $V_1 = V_2$ to denote that $V_1 \sqsubseteq V_2$ and $V_2 \sqsubseteq V_1$. Let us see how vector timestamps provide an efficient implicit representation of partial orders.

Timestamping for a partial order. Consider a partial order $\leq_P^\sigma$ defined on the set of events of σ such that $\leq_{TO}^\sigma \subseteq \leq_P^\sigma$. In this case, we can define the P-timestamp of an event e as the following vector timestamp:

$$C_e^{\leq_P^\sigma} = \lambda u: \max\{\text{LTime}^\sigma(f) \mid f \leq_P^\sigma e, \text{tid}(f) = u\}$$

In words, $C_e^{\leq_P^\sigma}$ contains the timestamps of the events that appear the latest in their respective threads such that they are ordered before e in the partial order $\leq_P^\sigma$. We remark that $C_e^{\leq_P^\sigma}(\text{tid}(e)) = \text{LTime}^\sigma(e)$. The following observation then shows that the timestamps defined above precisely capture the order $\leq_P^\sigma$.

Lemma 1. *Let $\leq_P^\sigma$ be a partial order defined on the set of events of trace σ such that $\leq_{TO}^\sigma \subseteq \leq_P^\sigma$. Then for any two events e_1, e_2 of σ , we have, $C_{e_1}^{\leq_P^\sigma} \sqsubseteq C_{e_2}^{\leq_P^\sigma} \iff e_1 \leq_P^\sigma e_2$.*

In words, Lemma 1 implies that, in order to check whether two events are ordered according to $\leq_P^\sigma$, it suffices to compare their vector timestamps.

Algorithm 2: Computing the HB partial order.

1 procedure acquire(t, ℓ) 2 $\mathbb{C}_t.\text{Join}(\mathbb{C}_\ell)$	3 procedure release(t, ℓ) 4 $\mathbb{C}_\ell.\text{Copy}(\mathbb{C}_t)$
---	---

The vector clock data structure. When establishing a causal order over the events of a trace, the timestamps of an event is computed using timestamps of other events in the trace. Instead of explicitly storing timestamps of each event, it is often sufficient to store only the timestamps of a few events, as the algorithm is running. Typically a data-structure called *vector clocks* is used to store vector times. Vector clocks are implemented as a simple integer array indexed by thread identifiers, and they support all the operations on vector timestamps. A useful feature of this data-structure is the ability to perform in-place operations. In particular, there are methods such as $\text{Join}(\cdot)$, $\text{Copy}(\cdot)$ or $\text{Increment}(\cdot, \cdot)$ that store the result of the corresponding vector time operation in the original instance of the data-structure. For example, for a vector clock $\mathbb{C}$ and a vector time V , a function call $\mathbb{C}.\text{Join}(V)$ stores the value $\mathbb{C} \sqcup V$ back in $\mathbb{C}$. Each of these operations iterates over all the thread identifiers (indices of the array representation) and compares the corresponding components in $\mathbb{C}$ and V . The running time of the join operation for the vector clock data structure is thus $\Theta(k)$, where k is the number of threads. Similarly, copy and comparison operations take $\Theta(k)$ time.

The Happens-Before Partial Order

Lamport's Happens-Before (HB) [171] is one of the most frequently used partial orders for the analysis of concurrent executions, with wide applications in domains such as dynamic race detection. Here we use HB to illustrate the disadvantages of vector clocks and form the basis for the tree clock data structure. In later sections we show how tree clocks also apply to other partial orders, such as Schedulably-Happens-Before and the Mazurkiewicz partial order.

Happens-before. Given a trace σ , the *happens-before* (HB) partial order $\leq_{\text{HB}}^\sigma$ of σ is the smallest partial order over the events of σ that satisfies the following conditions.

1. $\leq_{\text{TO}}^\sigma \subseteq \leq_{\text{HB}}^\sigma$.
2. For every release event $\text{rel}(\ell)$ and acquire event $\text{acq}(\ell)$ on the same lock ℓ with $\text{rel}(\ell) <_{\text{tr}}^\sigma \text{acq}(\ell)$, we have $\text{rel}(\ell) \leq_{\text{HB}}^\sigma \text{acq}(\ell)$.

For two events e_1, e_2 in trace σ , we use $e_1 \parallel_{\text{HB}}^\sigma e_2$ to denote that neither $e_1 \leq_{\text{HB}}^\sigma e_2$, nor $e_2 \leq_{\text{HB}}^\sigma e_1$. We say $e_1 <_{\text{HB}}^\sigma e_2$ when $e_1 \neq e_2$ and $e_1 \leq_{\text{HB}}^\sigma e_2$. Given a trace σ , two events e_1, e_2 of σ are said to be in a *happens-before (data) race* if (i) $e_1 \asymp e_2$ and (ii) $e_1 \parallel_{\text{HB}}^\sigma e_2$.

The happens-before algorithm. In light of Lemma 1, race detection based on HB constructs the $\leq_{\text{HB}}^\sigma$ partial order in terms of vector timestamps and detects races using these. The core algorithm for constructing $\leq_{\text{HB}}^\sigma$ is shown in Algorithm 2. The algorithm maintains a vector clock $\mathbb{C}_t$ for every thread $t \in \text{Thrds}$, and a similar one $\mathbb{C}_\ell$ for every lock ℓ . When processing an event $e = \langle t, \text{op} \rangle$, it performs an update $\mathbb{C}_t.\text{Increment}(t, 1)$, which is implicit and not shown in Algorithm 2. Moreover, if

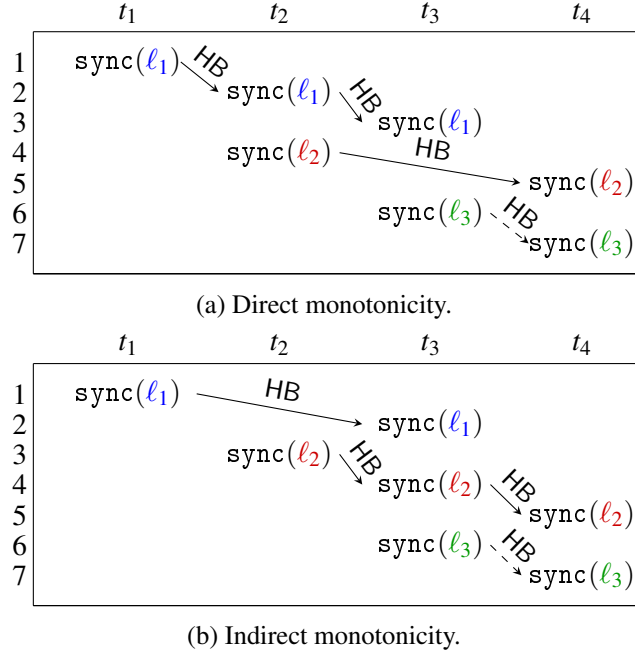


Figure 5.2: Illustration of the two insights behind tree clocks. An event $\text{sync}(\ell)$ represents two events $\text{acq}(\ell), \text{rel}(\ell)$.

$\text{op} = \text{acq}(\ell)$ or $\text{op} = \text{rel}(\ell)$, the algorithm executes the corresponding procedure. The HB-timestamp of e is then simply the value stored in $\mathbb{C}_{\text{tid}(e)}$ right after e has been processed.

Running time using vector clocks. If a trace σ has n events and k threads, computing the HB partial order with Algorithm 2 and using vector clocks takes $O(n \cdot k)$ time. The quadratic bound occurs because every vector clock join and copy operation iterates over all k threads.

5.3 The Tree Clock Data Structure

In this section we introduce tree clocks, a new data structure for representing logical times in concurrent and distributed systems. We first illustrate the intuition behind tree clocks, and then develop the data structure in detail.

Intuition

Like vector clocks, tree clocks represent vector timestamps that record a thread's knowledge of events in other threads. Thus, for each thread t , a tree clock records the last known local time of t . However, unlike a vector clock which is flat, a tree clock maintains this information hierarchically — nodes store local times of a thread, while the tree structure records how this information has been obtained transitively

through intermediate threads. In the following examples we use the operation $\text{sync}(\ell)$ to denote the sequence $\text{acq}(\ell), \text{rel}(\ell)$.

1. Direct monotonicity. Recall that a vector clock-based algorithm like Algorithm 2 maintains a vector clock $\mathbb{C}_t$ which intuitively captures thread t 's knowledge about all threads. However, it does not maintain *how* this information was acquired. Knowledge of how such information was acquired can be exploited in join operations, as we show through an example. Consider a computation of the HB partial order for the trace σ shown in Figure 5.2a. At event e_7 , thread t_4 transitively learns information about events in the trace through thread t_3 because $e_6 <_{\text{HB}}^\sigma e_7$ (dashed edge in Figure 5.2a). This is accomplished by joining with clock $\mathbb{C}_{t_3}$ of thread t_3 . Such a join using vector clocks will take 4 steps because we need to take the pointwise maximum of two vectors of length 4.

Suppose in addition to these timestamps, we maintain how these timestamps were updated in each clock. This would allow one to make the following observations.

1. Thread t_3 knows of event e_1 of t_1 transitively, through event e_2 of thread t_2 .
2. Thread t_4 (before the join at e_7) knows of event e_1 through e_4 of thread t_2 .

Before the join, since t_4 has a more recent view of t_2 when compared to t_3 , it is aware of all the information that thread t_3 knows about the world via thread t_2 . Thus, when performing the join, we need not examine the component corresponding to thread t_1 in the two clocks. Tree clocks, by maintaining such additional information, can avoid examining some components of a vector timestamp and yield sublinear updates.

2. Indirect monotonicity. We now illustrate that if in addition to information about “how a view of a thread was updated”, we also maintained “when the view of a thread was updated”, the cost of join operations can be further reduced. Consider the trace σ of Figure 5.2b. At each of the events of thread t_4 , it learns about events in the trace transitively through thread t_3 by performing two join operations. At the first join (event e_5), thread t_4 learns about events e_1, e_2, e_3 transitively through event e_4 . At event e_7 , thread t_4 finds out about new events in thread t_3 (namely, e_6). However, it does not need to update its knowledge about threads t_1 and t_2 — thread t_3 's information about threads t_1 and t_2 were acquired by the time of event e_4 about which thread t_4 is aware. Thus, if information about when knowledge was acquired is also kept, this form of “indirect monotonicity” can be exploited to avoid examining all components of a vector timestamp.

The flat structure of vector clocks misses the transitivity of information sharing, and thus arguments based on monotonicity are lost, resulting in vacuous operations. On the other hand, tree clocks maintain transitivity in their hierarchical structure. This enables reasoning about direct and indirect monotonicity, and thus avoid redundant operations.

Tree Clocks

We now present the tree clock data structure in detail.

Tree clocks. A tree clock TC consists of the following.



Figure 5.3: The tree clock of t_4 after processing the event e_7 in the traces of Figure 5.2a (left) and Figure 5.2b (right).

1. $T = (\mathcal{V}, \mathcal{E})$ is a *rooted tree* of nodes of the form $(\text{tid}, \text{clk}, \text{aclk}) \in \text{Thrds} \times \mathbb{N}^2$. Every node u stores its children in an ordered list $\text{Chld}(u)$ of descending aclk order. We also store a pointer $\text{Prnt}(u)$ of u to its parent in T .
2. $\text{ThrMap}: \text{Thrds} \rightarrow \mathcal{V}$ is a *thread map*, with the property that if $\text{ThrMap}(t) = (\text{tid}, \text{clk}, \text{aclk})$, then $t = \text{tid}$.

We denote by $T.\text{root}$ the root of T , and for a tree clock TC we refer by $\text{TC}.T$ and $\text{TC}.\text{ThrMap}$ to the rooted tree and thread map of TC , respectively. For a node $u = (\text{tid}, \text{clk}, \text{aclk})$ of T , we let $u.\text{tid} = \text{tid}$, $u.\text{clk} = \text{clk}$ and $u.\text{aclk} = \text{aclk}$, and say that u *points to* the unique event e with $\text{tid}(e) = \text{tid}$ and $\text{lTime}(e) = \text{clk}$. Intuitively, if $v = \text{Prnt}(u)$, then u represents the following information.

1. TC has the *local time* $u.\text{clk}$ for thread $u.\text{tid}$.
2. $u.\text{aclk}$ is the *attachment time* of $v.\text{tid}$, which is the local time of v when v learned about $u.\text{clk}$ of $u.\text{tid}$ (this will be the time that v had when u was attached to v).

Naturally, if $u = T.\text{root}$ then $u.\text{aclk} = \perp$. See Figure 5.3.

Tree clock operations. Just like vector clocks, tree clocks provide functions for initialization, update and comparison. There are two main operations worth noting. The first is **Join** — $\text{TC}_1.\text{Join}(\text{TC}_2)$ joins the tree clock TC_2 to TC_1 . In contrast to vector clocks, this operation takes advantage of the direct and indirect monotonicity outlined in Section 5.3 to perform the join in sublinear time in the size of TC_1 and TC_2 (when possible). The second is **MonotoneCopy**. We use $\text{TC}_1.\text{MonotoneCopy}(\text{TC}_2)$ to copy TC_2 to TC_1 when we know that $\text{TC}_1 \sqsubseteq \text{TC}_2$. The idea is that when this holds, the copy operation has the same semantics as the join, and hence the principles that make **Join** run in sublinear time also apply to **MonotoneCopy**.

Algorithm 3 gives a pseudocode description of this functionality. The functions on the left column present operations that can be performed on tree clocks, while the right column lists helper routines for the more involved functions **Join** and **MonotoneCopy**. In the following we give an intuitive description of each function.

1. **Init**(t). This function initializes a tree clock TC_t that belongs to thread t , by creating a node $u = (t, 0, \perp)$. Node u will always be the root of TC_t . This initialization function is only used for tree clocks that represent the clocks of threads. Auxiliary tree clocks for storing vector times of release events do not execute this initialization.
2. **Get**(t). This function simply returns the time of thread t stored in TC , while it returns 0 if t is not present in TC .
3. **Increment**(i). This function increments the time of the root node of TC . It is only used on tree clocks that have been initialized using **Init**, i.e., the tree clock belongs

Algorithm 3: The tree clock data structure.

```

// Initialize a tree clock for thread  $t$ 
1 function Init( $t$ )
2   Let  $u \leftarrow (t, 0, \perp)$ 
3   Make  $u$  the root of  $T$ 
4   Let ThrMap( $t$ )  $\leftarrow u$ 

// Get the clock for thread  $t$ 
5 function Get( $t$ )
6   if TC.ThrMap( $t$ )  $\neq \perp$  then
7     Let  $u \leftarrow$  ThrMap( $t$ )
8     return  $u$ .clk
9   return 0

// Increment the clock of the root thread
10 function Increment( $i$ )
11   Let  $z \leftarrow T$ .root
12    $z$ .clk  $\leftarrow z$ .clk +  $i$ 

// True iff  $\sqsubseteq TC'$ 
13 function LessThan( $TC'$ )
14   Let  $z \leftarrow T$ .root
15   return  $z$ .clk  $\leq TC'$ .Get( $z$ .tid)

// Update with  $\sqsubseteq TC'$ 
16 function Join( $TC'$ )
17   Let  $z' \leftarrow TC'$ .T.root
18   if  $z'$ .clk  $\leq$  Get( $z'$ .tid) then
19     return
20   Let  $\mathcal{S} \leftarrow$  an empty stack
21   getUpdatedNodesJoin( $\mathcal{S}, z'$ )
22   detachNodes( $\mathcal{S}$ )
23   attachNodes( $\mathcal{S}$ )
24   // Place the updated subtree under the
25   // root of  $T$ 
26   Let  $w \leftarrow$  ThrMap( $z'$ .tid)
27   Let  $z \leftarrow T$ .root
28   Assign  $w$ .aclk  $\leftarrow z$ .clk
29   pushChild( $w, z$ )

// Monotone copy, assumes that this  $\sqsubseteq TC'$ 
28 function MonotoneCopy( $TC'$ )
29   Let  $z' \leftarrow TC'$ .T.root
30   Let  $z \leftarrow T$ .root
31   Let  $\mathcal{S} \leftarrow$  an empty stack
32   getUpdatedNodesCopy( $\mathcal{S}, z', z$ )
33   detachNodes( $\mathcal{S}$ )
34   attachNodes( $\mathcal{S}$ )
35   // New root has the same tid as the root
36   // of  $TC'$ .T
37   Assign  $T$ .root  $\leftarrow$  ThrMap( $z'$ .tid)

// Populate  $\mathcal{S}$  with a pre-order traversal
// of the subtree rooted at  $u'$  with nodes
// whose clock has progressed
36 routine getUpdatedNodesJoin( $\mathcal{S}, u'$ )
37   foreach  $v'$  in Chld( $u'$ ) do
38     if Get( $v'$ .tid)  $< v'$ .clk then
39       getUpdatedNodesJoin( $\mathcal{S}, v'$ )
40     else if  $v'$ .aclk  $\leq$  Get( $u'$ .tid) then break
41   Push  $u'$  in  $\mathcal{S}$ 

// Detach from  $T$  the nodes with tid that
// appears in  $\mathcal{S}$ 
41 routine detachNodes( $\mathcal{S}$ )
42   foreach  $v'$  in  $\mathcal{S}$  do
43     if ThrMap( $v'$ .tid)  $\neq \perp$  then
44       Let  $v \leftarrow$  ThrMap( $v'$ .tid)
45       if  $v \neq T$ .root then
46         Let  $x \leftarrow$  Prnt( $v$ )
47         Remove  $v$  from Chld( $x$ )

// Re-attach the nodes of  $T$  with tid that
// appears in  $\mathcal{S}$  to obtain the shape
// corresponding to  $TC'$ .T
48 routine attachNodes( $\mathcal{S}$ )
49   while  $\mathcal{S}$  is not empty do
50     Let  $u' \leftarrow$  pop  $\mathcal{S}$ 
51     if ThrMap( $u'$ .tid)  $\neq \perp$  then
52       Let  $u \leftarrow$  ThrMap( $u'$ .tid)
53     else
54       Let  $u \leftarrow (u'.tid, 0, \perp)$ 
55       Let ThrMap( $u$ .tid)  $\leftarrow u$ 
56     Assign  $u$ .clk  $\leftarrow u'$ .clk
57     Let  $y' \leftarrow$  Prnt( $u'$ )
58     if  $y' \neq \perp$  then
59       Assign  $u$ .aclk  $\leftarrow u'$ .aclk
60       Let  $y \leftarrow$  ThrMap( $y'$ .tid)
61       pushChild( $u, y$ )

// Similar to getUpdatedNodesJoin
62 routine getUpdatedNodesCopy( $\mathcal{S}, u', z$ )
63   foreach  $v'$  in Chld( $u'$ ) do
64     if Get( $v'$ .tid)  $< v'$ .clk then
65       getUpdatedNodesCopy( $\mathcal{S}, v', z$ )
66     else
67       if  $z \neq \perp$  and  $v'$ .tid =  $z$ .tid then Push  $v'$  in  $\mathcal{S}$ 
68       if  $v'$ .aclk  $\leq$  Get( $u'$ .tid) then break
69   Push  $u'$  in  $\mathcal{S}$ 

70 routine pushChild( $u, v$ )
71   Assign Prnt( $u$ )  $\leftarrow v$ 
72   Push  $u$  to the front of Chld( $v$ )

```

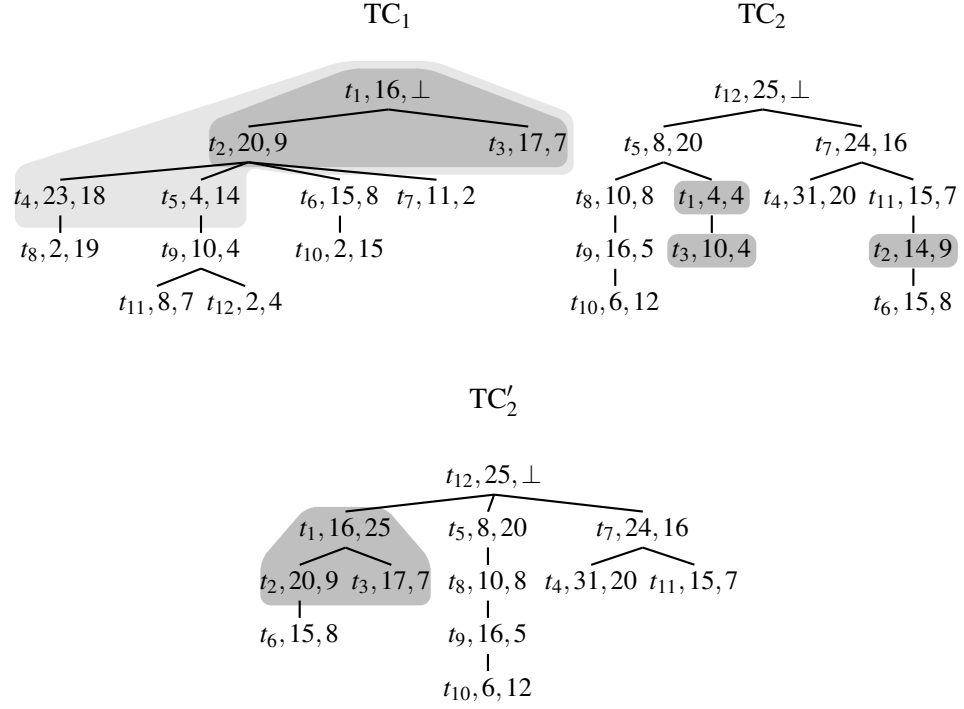


Figure 5.4: Illustration of $TC_2.Join(TC_1)$. Light gray marks the nodes of TC_1 whose time is compared to the time of the respective thread in TC_2 (i.e., the total iterations in Line 37). Dark gray marks the nodes that are updating/being updated (i.e., the size of $\mathcal{S}$). TC'_2 is the result of the join, where dark gray marks the sub-tree updated by Join.

to a thread that is always stored in the root of the tree.

4. $LessThan(TC')$. This function compares the vector time of TC to the vector time of TC' , i.e., it returns True iff $TC \sqsubseteq TC'$.

5. $Join(TC')$. This function implements the join operation with TC' , i.e., updating $TC \leftarrow TC \sqcup TC'$. At a high level, the function performs the following steps.

1. Routine `getUpdatedNodesJoin` performs a pre-order traversal of TC' , and gathers in a stack $\mathcal{S}$ the nodes of TC' that have progressed in TC' compared to TC . The traversal may stop early due to direct or indirect monotonicity, hence, this routine generally takes sub-linear time.
2. Routine `detachNodes` detaches from TC the nodes whose tid appears in $\mathcal{S}$, as these will be repositioned in the tree.
3. Routine `attachNodes` updates the nodes of TC that were detached in the previous step, and repositions them in the tree. This step effectively creates a subtree of nodes of TC that is identical to the subtree of TC' that contains the progressed nodes computed by `getUpdatedNodesJoin`.
4. Finally, the last 4 lines of `Join` attach the subtree constructed in the previous step under the root z of TC , at the front of the $Chld(z)$ list.

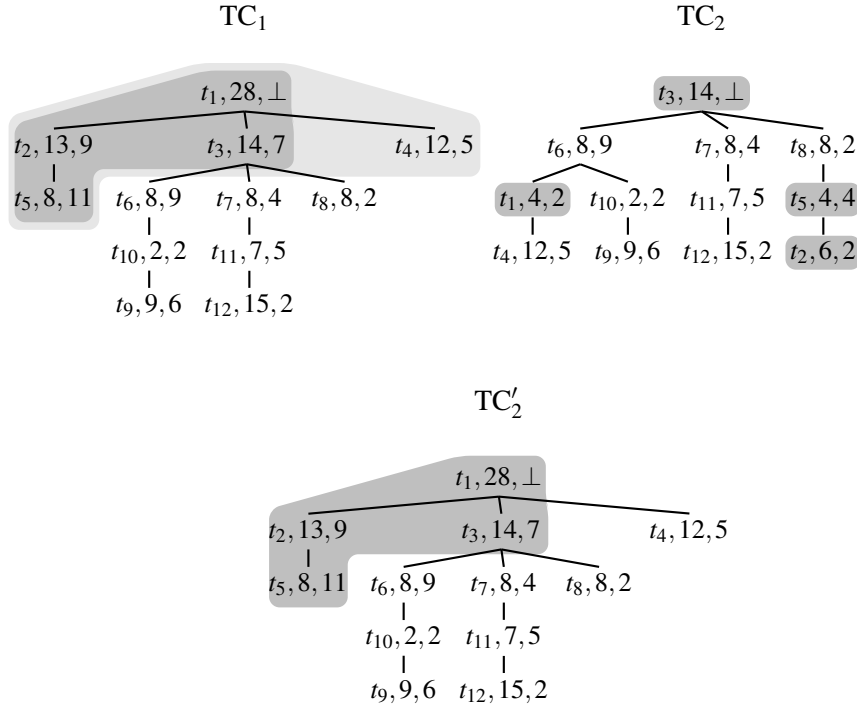


Figure 5.5: Illustration of $TC_2.\text{MonotoneCopy}(TC_1)$. Light gray marks the nodes of TC_1 whose time is compared to the time of the respective thread in TC_2 (i.e., the total iterations in Line 63). Dark gray marks the nodes that are updating/being updated (i.e., the size of $\mathcal{S}$). TC'_2 is the result of the copy, where dark gray marks the sub-tree updated by MonotoneCopy . Node $(t_3, 14, \perp)$ (i.e., the root) of TC_2 is updated although t_3 has not progressed in TC_1 , as it is placed under the new root $(t_1, 28, \perp)$ in TC'_2 .

Figure 5.4 provides an illustration.

6. $\text{MonotoneCopy}(TC')$. This function implements the copy operation $TC \leftarrow TC'$ assuming that $TC \sqsubseteq TC'$. The function is very similar to Join . The key difference is that this time, the root of TC is always considered to have progressed in TC' , even if the respective times are equal. This is required for changing the root of TC from the current node to one with tid equal to the root of TC' . Figure 5.5 provides an illustration.

The crucial parts of Join and MonotoneCopy that exploit the hierarchical structure of tree clocks are in $\text{getUpdatedNodesJoin}$ and $\text{getUpdatedNodesCopy}$. In each case, we proceed from a parent u' to its children v' only if u' has progressed wrt its time in TC (recall Figure 5.2a), capturing *direct monotonicity*. Moreover, we proceed from a child v' of u' to the next child v'' (in order of appearance in $\text{Chld}(u')$) only if TC is not yet aware of the attachment time of v' on u' (recall Figure 5.2b), capturing *indirect monotonicity*.

Remark 1 (Constant time epoch accesses). *The function $\text{TC.Get}(t)$ returns the time of thread t stored in TC in $O(1)$ time, just like vector clocks. This allows all epoch-related optimizations [111, 248] from vector clocks to apply to tree clocks.*

5.4 Tree Clocks for Happens-Before

Let us see how tree clocks are employed for computing the HB partial order. We start with the following observation.

Lemma 2 (Monotonicity of copies). *Right before Algorithm 2 processes a lock-release event $\langle t, \text{rel}(\ell) \rangle$, we have $\mathbb{C}_\ell \sqsubseteq \mathbb{C}_t$.*

Tree clocks for HB. Algorithm 4 shows the algorithm for computing HB using the tree clock data structure for implementing vector times. When processing a lock-acquire event, the vector-clock join operation has been replaced by a tree-clock join. Moreover, in light of Lemma 2, when processing a lock-release event, the vector-clock copy operation has been replaced by a tree-clock monotone copy.

Algorithm 4: HB with tree clocks.

1 procedure acquire(t, ℓ) 2 $\mathbb{C}_t.\text{Join}(\mathbb{C}_\ell)$	3 procedure release(t, ℓ) 4 $\mathbb{C}_\ell.\text{MonotoneCopy}(\mathbb{C}_t)$
---	---

Correctness. We now state the correctness of Algorithm 4, i.e., we show that the algorithm indeed computes the HB partial order. We start with two monotonicity invariants of tree clocks.

Lemma 3. *Consider any tree clock $\mathbb{C}$ and node u of $\mathbb{C}$. T. For any tree clock $\mathbb{C}'$, the following assertions hold.*

1. Direct monotonicity: *If $u.\text{clk} \leq \mathbb{C}'.\text{Get}(u.\text{tid})$ then for every descendant w of u we have that $w.\text{clk} \leq \mathbb{C}'.\text{Get}(w.\text{tid})$.*
2. Indirect monotonicity: *If $v.\text{aclk} \leq \mathbb{C}'.\text{Get}(u.\text{tid})$ where v is a child of u then for every descendant w of v we have that $w.\text{clk} \leq \mathbb{C}'.\text{Get}(w.\text{tid})$.*

The following lemma follows from the above invariants and establishes that Algorithm 4 with tree clocks computes the correct timestamps on all events, i.e., the correctness of tree clocks for HB.

Lemma 4. *When Algorithm 4 processes an event e , the vector time stored in the tree clock $\mathbb{C}_{\text{tid}(e)}$ is $C_e^{\leq_{HB}^\sigma}$.*

Data structure optimality. Just like vector clocks, computing HB with tree clocks takes $\Theta(n \cdot k)$ time in the worst case, and it is known that this quadratic bound is likely to be tight for common applications such as dynamic race prediction [164]. However, we have seen that tree clocks can take sublinear time on join and copy operations, whereas vector clocks always require time linear in the size of the vector

(i.e., $\Theta(k)$). A natural question arises: is there a more efficient data structure than tree clocks? More generally, what is the most efficient data structure for the HB algorithm to represent vector times? To answer this question, we define *vector-time work*, which gives a lower bound on the number of data structure operations that HB has to perform regardless of the actual data structure used to store vector times. Then, we show that tree clocks match this lower bound, hence achieving optimality for HB.

Vector-time work. Consider the general HB algorithm (Algorithm 2) and let $\mathcal{D} = \{\mathbb{C}_1, \mathbb{C}_2, \dots, \mathbb{C}_m\}$ be the set of the vector-time data structures used. Consider the execution of the algorithm on a trace σ . Given some $1 \leq i \leq |\sigma|$, we let C_j^i denote the vector time represented by $\mathbb{C}_j$ after the algorithm has processed the i -th event of σ . We define the *vector-time work* (or *vt-work*, for short) on σ as

$$\text{VTWork}(\sigma) = \sum_{1 \leq i \leq |\sigma|} \sum_j |t \in \text{Thrds} : C_j^{i-1}(t) \neq C_j^i(t)|.$$

In words, for every processed event, we add the number of vector-time entries that change as a result of processing the event, and $\text{VTWork}(\sigma)$ counts the total number of entry updates in the overall course of the algorithm. Note that vt-work is independent of the data structure used to represent each $\mathbb{C}_j$, and satisfies the inequality

$$n \leq \text{VTWork}(\sigma) \leq n \cdot k.$$

as with every event of σ the algorithm updates one of $\mathbb{C}_j$.

Vector-time optimality. Given an input trace σ , we denote by $\mathcal{T}_{\text{DS}}(\sigma)$ the time taken by the HB algorithm (Algorithm 2) to process σ using the data structure DS to store vector times. Intuitively, $\text{VTWork}(\sigma)$ captures the number of times that instances of DS change state. For data structures that represent vector times explicitly, $\text{VTWork}(\sigma)$ presents a natural lower bound for $\mathcal{T}_{\text{DS}}(\sigma)$. Hence, we say that the data structure DS is *vt-optimal* if $\mathcal{T}_{\text{DS}}(\sigma) = O(\text{VTWork}(\sigma))$. It is not hard to see that vector clocks are not vt-optimal, i.e., taking $\text{DS} = \text{VC}$ to be the vector clock data structure, one can construct simple traces σ where $\text{VTWork}(\sigma) = O(n)$ but $\mathcal{T}_{\text{DS}}(\sigma) = \Omega(n \cdot k)$, and thus the running time is k times more than the vt-work that must be performed on σ . In contrast, the following theorem states that tree clocks are vt-optimal.

Theorem 1 (Tree-clock Optimality). *For any input trace σ , we have $\mathcal{T}_{\text{TC}}(\sigma) = O(\text{VTWork}(\sigma))$.*

The key observation behind Theorem 1 is that, when HB uses tree clocks, the total number of tree-clock entries that are accessed over all join and monotone copy operations (i.e., the sum of the sizes of the light-gray areas in Figure 5.4 and Figure 5.5) is $\leq 3 \cdot \text{VTWork}(\sigma)$.

Remark 2. *Theorem 1 establishes strong optimality for tree clocks, in the sense that they are vt-optimal on every input. This is in contrast to usual notions of optimality that is guaranteed on only some inputs.*

Algorithm 5: SHB with tree clocks.

1 procedure <code>acquire</code> (t, ℓ) 2 $\mathbb{C}_t.$ Join($\mathbb{L}_\ell$) 3 procedure <code>read</code> (t, x) 4 $\mathbb{C}_t.$ Join($\mathbb{LW}_x$)	5 procedure <code>release</code> (t, ℓ) 6 $\mathbb{L}_\ell.$ MonotoneCopy($\mathbb{C}_t$) 7 procedure <code>write</code> (t, x) 8 $\mathbb{LW}_x.$ CopyCheckMonotone($\mathbb{C}_t$)
--	--

5.5 Tree Clocks in Other Partial Orders

Schedulable-Happens-Before

SHB is a strengthening of HB, introduced recently [199] in the context of race detection. Given a trace σ and a read event r let $\text{lw}_\sigma(r)$ be the last write event of σ before r with $\text{Variable}(w) = \text{Variable}(r)$. SHB is the smallest partial order that satisfies the following.

1. $\leq_{\text{HB}}^\sigma \subseteq \leq_{\text{SHB}}^\sigma$.
2. for every read event r , we have $\text{lw}_\sigma(r) \leq_{\text{SHB}}^\sigma r$.

Algorithm for SHB. Similarly to HB, the SHB partial order is computed by a single pass of the input trace σ using vector-times [199]. The SHB algorithm processes synchronization events (i.e., `acq`(ℓ) and `rel`(ℓ)) similarly to HB. In addition, for each variable x , the algorithm maintains a data structure $\mathbb{LW}_x$ that stores the vector time of the latest write event on x . When a write event $w(x)$ is encountered, the vector time $\mathbb{C}_{\text{tid}(w)}$ is copied to $\mathbb{LW}_x$. In turn, when a read event $r(x)$ is encountered the algorithm joins $\mathbb{LW}_x$ to $\mathbb{C}_{\text{tid}(r)}$.

SHB with tree clocks. Tree clocks can directly be used as the data structure to store vector times in the SHB algorithm. We refer to Algorithm 5 for the pseudocode. The important new component is the function `CopyCheckMonotone` in Line 8 that copies the vector time of $\mathbb{C}_t$ to $\mathbb{LW}_x$. In contrast to `MonotoneCopy`, this copy is not guaranteed to be monotone, i.e., we might have $\mathbb{LW}_x \not\sqsubseteq \mathbb{C}_t$. Note, however, that using tree clocks, this test requires only constant time. Internally, `CopyCheckMonotone` performs `MonotoneCopy` if $\mathbb{LW}_x \sqsubseteq \mathbb{C}_t$ (running in sublinear time), otherwise it performs a deep copy for the whole tree clock (running in linear time). In practice, we expect that most of the times `CopyCheckMonotone` results in `MonotoneCopy` and thus is very efficient. The key insight is that if `MonotoneCopy` is not used, then $\mathbb{LW}_x \not\sqsubseteq \mathbb{C}_t$ and thus we have a race $(\text{lw}_\sigma(r), r)$. Hence, the number of times a deep copy is performed is bounded by the number of write-read races in σ between a read and its last write.

The Mazurkiewicz Partial Order

The Mazurkiewicz partial order (MAZ) [208] has been the canonical way to represent concurrent executions algebraically using an independence relation that defines which events can be reordered. This algebraic treatment allows to naturally lift language-inclusion problems from the verification of sequential programs to concur-

Algorithm 6: MAZ with tree clocks.

1 procedure <code>acquire</code> (t, ℓ) 2 $\mathbb{C}_t.\text{Join}(\mathbb{L}_\ell)$ 3 procedure <code>read</code> (t, x) 4 $\mathbb{C}_t.\text{Join}(\mathbb{LW}_x)$ 5 $\mathbb{R}_{t,x}.\text{MonotoneCopy}(\mathbb{C}_t)$ 6 $\text{LRDs}_x \leftarrow \text{LRDs}_x \cup \{t\}$	7 procedure <code>release</code> (t, ℓ) 8 $\mathbb{L}_\ell.\text{MonotoneCopy}(\mathbb{C}_t)$ 9 procedure <code>write</code> (t, x) 10 $\mathbb{C}_t.\text{Join}(\mathbb{LW}_x)$ 11 foreach $t' \in \text{LRDs}_x$ do 12 $\mathbb{C}_t.\text{Join}(\mathbb{R}_{t',x})$ 13 $\mathbb{LW}_x.\text{MonotoneCopy}(\mathbb{C}_t)$ 14 $\text{LRDs}_x \leftarrow \emptyset$
---	---

rent programs [34]. As such, it has been the most studied partial order in the context of concurrency, with deep applications in dynamic analyses [117, 198, 222], ensuring consistency [271] and stateless model checking [115]. In shared memory concurrency, the standard independence relation deems two events as dependent if they conflict, and independent otherwise [126]. In particular, MAZ is the smallest partial order that satisfies the following conditions.

1. $\leq_{\text{HB}}^\sigma \subseteq \leq_{\text{MAZ}}^\sigma$.
2. for every two events e_1, e_2 such that $e_1 \leq_{\text{tr}}^\sigma e_2$ and $e_1 \asymp e_2$, we have $e_1 \leq_{\text{MAZ}}^\sigma e_2$.

MAZ with tree clocks. The algorithm for computing MAZ is similar to that for SHB. The main difference is that MAZ includes read-to-write orderings, and thus we need to store additional vector times $\mathbb{R}_{t,x}$ of the last event $r(x)$ of thread t . In addition, we use the set LRDs_x to store the threads that have executed a $r(x)$ event after the latest $w(x)$ event so far. This allows us to only spend computation time in the first read-to-write ordering, as orderings between the read event and later write events follow transitively via intermediate write-to-write orderings. Overall, this approach yields the efficient time complexity $O(n \cdot k)$ for MAZ, similarly to HB and SHB. We refer to Algorithm 6 for the pseudocode.

5.6 Experiments

In this section we report on an implementation and experimental evaluation of the tree clock data structure. The primary goal of these experiments is to evaluate the practical advantage of tree clocks over the vector clocks for keeping track of logical times in a concurrent program executions.

Implementation. Our implementation is in Java and closely follows Algorithm 3. The tree clock data structure is represented as two arrays of length k (number of threads), the first one encoding the shape of the tree and the second one encoding the integer timestamps as in a standard vector clock. For efficiency reasons, recursive routines have been made iterative.

Benchmarks. Our benchmark set consists of standard benchmarks found in benchmark suites and recent literature. In particular, we used the Java benchmarks from the IBM Contest suite [104], Java Grande suite [277], DaCapo [37], and SIR [91].

In addition, we used OpenMP benchmark programs, whose execution length and number of threads can be tuned, from DataRaceOnAccelerator [260], DataRaceBench [182], OmpSCR [92] and the NAS parallel benchmarks [25], as well as large OpenMP applications contained in the following benchmark suites: CORAL [1, 2], ECP proxy applications [3], and Mantevo project [4]. Each benchmark was instrumented and executed in order to log a single concurrent trace, using the tools RV-Predict [251] (for Java programs) and ThreadSanitizer [267] (for OpenMP programs). Overall, this process yielded a large set of 153 benchmark traces that were used in our evaluation. Table 5.1 presents aggregate information about the benchmark traces generated. Information on the individual traces is provided in our technical report [204].

Table 5.1: Trace Statistics

	Min	Max	Mean		Min	Max	Mean
Threads	3	222	31	Events	51	2.1B	227M
Locks	1	60.5k	688	Sync. Events (%)	0.0	44.4	9.5
Variables	18	37.8M	1.8M	R/W Events (%)	55.6	100	90.5

Setup. Each trace was processed for computing each of the MAZ, SHB and HB partial orders using both tree clocks and the standard vector clocks. This allows us to directly measure the speedup conferred by tree clocks in computing the respective partial order, which is the goal of this paper.

As the computation of these partial orders is usually the first component of any analysis, in general, we evaluated the impact of the conferred speedup in an overall analysis as follows. For each pair of conflicting events e_1, e_2 , we computed whether these events are concurrent wrt the corresponding partial order (e.g., whether $e_1 \parallel_{HB}^\sigma e_2$). This test is performed in dynamic race detection (in the cases of HB and SHB) where such pairs constitute data races, as well as in stateless model checking (in the case of MAZ) where the model checker identifies such event pairs and attempts to reverse their order on its way to exhaustively enumerate all Mazurkiewicz traces of the concurrent program. For a fair comparison, in the case of HB we used common epoch optimizations [111] to speed up the analysis for both tree clocks and vector clocks (recall Remark 1). For consistency, every measurement was repeated 3 times and the average time was reported.

Running times. For each partial order, Table 5.2 shows the average speedup over all benchmarks, both with and without the analysis component. We see that tree clocks are very effective in reducing the running time of the computation of all 3 partial orders, with the most significant impact being on SHB where the average speedup is 2.53 times. For the cases of MAZ and SHB, this speedup also leads to a significant speedup in the overall analysis time. On the other hand, although HB with tree clocks is about 2 times faster than with vector clocks, this speedup has a smaller effect on the overall analysis time. The reason behind this observation is straightforward: SHB and MAZ are much more computationally-heavy, as they are defined using all types

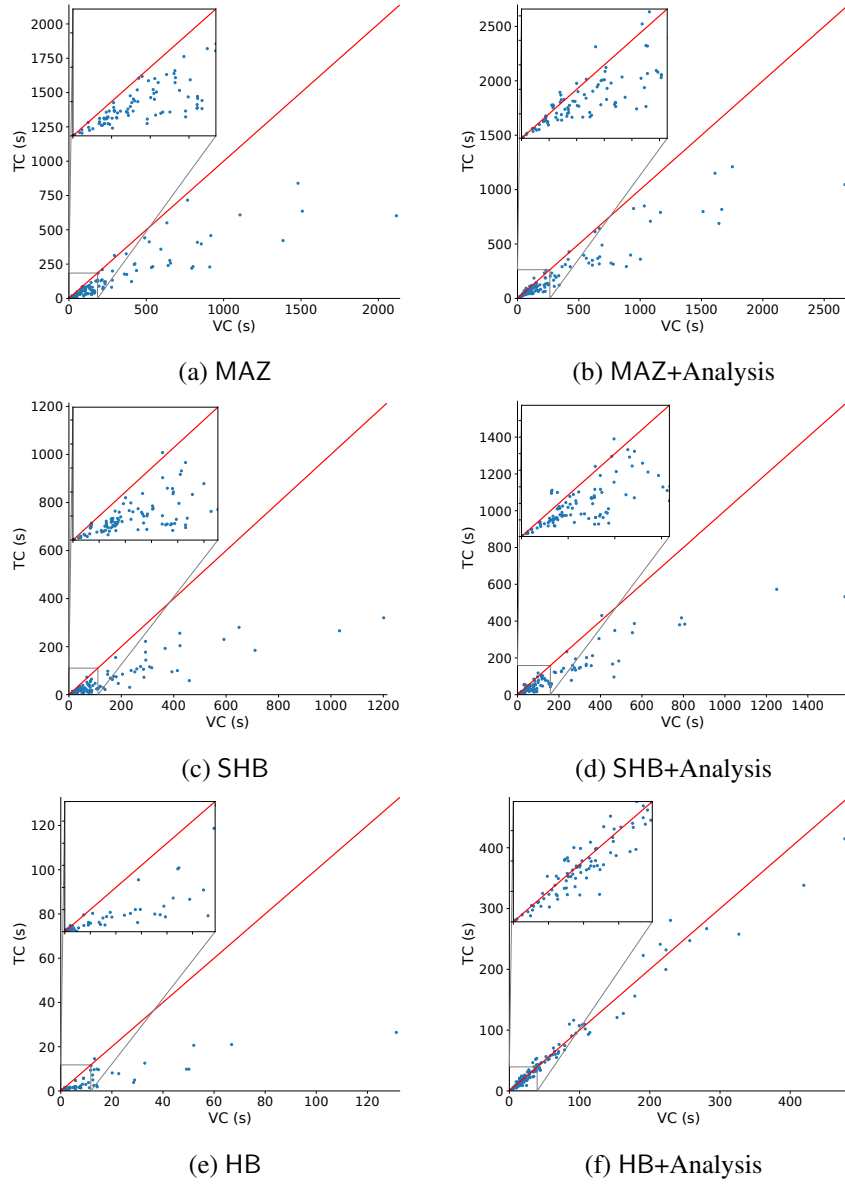


Figure 5.6: Times for processing each benchmark trace using tree clocks (TC) and vector clocks (VC). The left column shows the time for computing the partial order, while the right column shows the time including the analysis component.

Table 5.2: Average speedup for computing the partial order due to tree clocks.

	MAZ	SHB	HB
PO	2.02	2.66	2.97
PO + Analysis	1.49	1.80	1.11

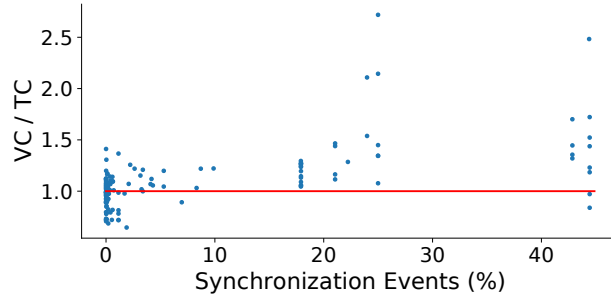


Figure 5.7: Speedup on HB+analysis computation as a function of the percentage of synchronization events, for the traces where the total time is not too small ($\geq 100\text{ms}$).

of events; on the other hand, HB is defined only on synchronization events (`acq` and `rel`) and on average, only $\simeq 9.5\%$ of the events are synchronization events on our benchmark traces. Since our analysis considers all events, the HB-computation component occupies a smaller fraction of the overall analysis time. We remark, however, that for programs that are more synchronization-heavy, or for analyses that are more lightweight (e.g., when checking for data races on a specific variable as opposed to all variables), the speedup of tree clocks will be larger on the whole analysis. Indeed, Figure 5.7 shows the obtained speedup on the total analysis time for HB as a function of synchronization events. We observe a trend for the speedup to increase as the percentage of synchronization events increases in the trace. A further observation is that speedup is prominent when the number of threads are large.

Figure 5.6 gives a more detailed view of the tree clocks vs vector clocks times across all benchmarks. We see that tree clocks almost always outperform vector clocks on all partial orders, and in some cases by large margins. Interestingly, the speedup tends to be larger on more demanding benchmarks (i.e., on those that take more time). In the very few cases tree clocks are slower, this is only by a small factor. These are traces where the sub-linear updates of tree clocks only yield a small potential for improvement, which does not justify the overhead of maintaining the more complex tree data structure (as opposed to a vector). Nevertheless, overall tree clocks consistently deliver a generous speedup to each of MAZ, HB and SHB. Finally, we remark that all these speedups come directly from just replacing the underlying data structure, without any attempt to optimize the algorithm that computes the respective partial order, or its interaction with the data structure.

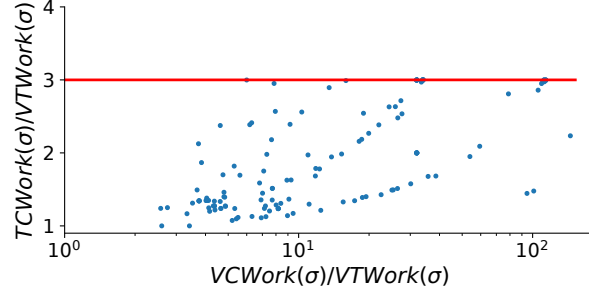


Figure 5.8: Comparison of the ratios $\text{TCWork}(\sigma)/\text{VTWork}(\sigma)$ and $\text{VCWork}(\sigma)/\text{VTWork}(\sigma)$ across all benchmarks.

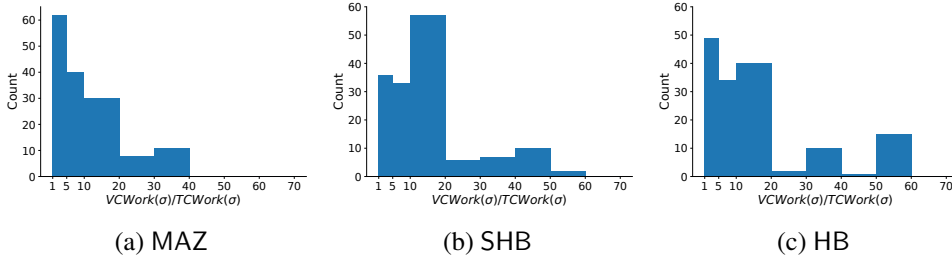


Figure 5.9: Histogram of the ratios $\text{VCWork}(\sigma)/\text{TCWork}(\sigma)$ across all benchmarks in our dataset.

Comparison with vt-work. We also investigate the total number of entries updated using each of the data structures. Recall that the metric $\text{VTWork}(\sigma)$ (Section 5.4) measures the minimum amount of updates that any implementation of the vector time must perform when computing the HB partial order. We can likewise define the metrics $\text{TCWork}(\sigma)$ and $\text{VCWork}(\sigma)$ corresponding to the number of entries updated when processing each event when using respectively the data structures tree clocks and vector clocks. These metrics are visualized in Figure 5.8 for computing the HB partial order in our benchmark suite. The figure shows that the $\text{VCWork}(\sigma)/\text{VTWork}(\sigma)$ ratio is often considerably large. In contrast, the ratio $\text{TCWork}(\sigma)/\text{VTWork}(\sigma)$ is typically significantly smaller. The differences in running times between vector and tree clocks reflect the discrepancies between $\text{TCWork}(\cdot)$ and $\text{VCWork}(\cdot)$. Next, recall the intuition behind the optimality of tree clocks (Theorem 1), namely that $\text{TCWork}(\sigma) \leq 3 \cdot \text{VTWork}(\sigma)$. Figure 5.8 confirms this theoretical bound, as the $\text{TCWork}(\sigma)/\text{VTWork}(\sigma)$ ratio stays nicely upper-bounded by 3 while the $\text{VCWork}(\sigma)/\text{VTWork}(\sigma)$ ratio grows to nearly 100. Interestingly, for some benchmarks we have $\text{TCWork}(\sigma) \simeq 2.99 \cdot \text{VTWork}(\sigma)$, i.e., these benchmarks push tree clocks to their vt-work upper-bound. Going one step further, Figure 5.9 shows the ratio $\text{VCWork}(\sigma)/\text{TCWork}(\sigma)$ for each partial order in our dataset. The results indicate that vector clocks perform a lot of unnecessary work compared to tree clocks, and experimentally demonstrate the source of speedup on tree clocks. Although Figure 5.9

indicates that the potential for speedup can be large (reaching $60\times$), the actual speedup in our experiments (Figure 5.6) is smaller, as a single tree clock operation is more computationally heavy than a single vector clock operation.

Scalability. To get a better insight on the scalability of tree clocks, we performed a set of controlled experiments on custom benchmarks, by controlling the number of threads and the number of locks parameters while keeping the communication patterns constant. Each trace consists of 10M events, while the number of threads varies between 10 and 360. The traces are generated in a way such that a randomly chosen thread performs two consecutive operations, $acq(l)$ followed by a $rel(l)$, on a randomly (when applicable) chosen lock l . We have considered four cases: (a) all threads communicate over a single common lock (single lock); (b) similar to (a), but there are 50 locks, and 20% of the threads are 5 times more likely to perform an operation compared to the rest of the threads (50 locks, skewed); (c) $k - 1$ client threads communicate with a single server thread via a dedicated lock per thread (star topology); (d) similar to (a), but every pair of threads communicates over a dedicated lock (pairwise communication). Figure 5.10 shows the obtained results. Scenario

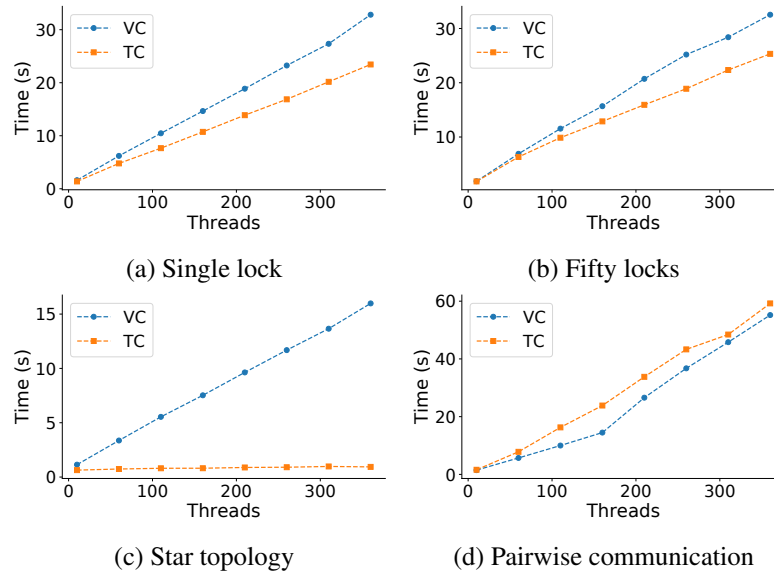


Figure 5.10: Comparison of tree clocks (TC) and vector clocks (VC) on four different benchmarks with increasing number of threads.

(a) shows how tree clocks have a constant-factor speedup over vector clocks in this setting. As we move to more locks in scenario (b), thread communication becomes more independent and the benefit of tree clocks may slightly diminish. As a subset of the threads is more active than the rest, timestamps are frequently exchanged through them, making tree clocks faster in this setting as well. Scenario (c) represents a case in which tree clocks thrive: while the time taken by vector clocks increases

with the number of threads, it stays constant for tree clocks. This is because the star topology implies that, on average, every tree clock join and copy operation only affects a constant number of tree clock entries, despite the fact that every thread is aware of the state of every other thread. Intuitively, the star communication topology *naturally* affects the shape of the tree to (almost) a star, which leads to this effect. Finally, scenario (d) represents the worst case for tree clocks as all pairs of threads can communicate with each other and the communication is conducted via a unique lock per thread pair. This pattern nullifies the benefit of tree clocks, while their increased complexity results in a general slowdown. However, even in this worst-case scenario, the difference between tree clocks and vector clocks remains relatively small.

5.7 Related Work

Other partial orders and tree clocks. As we have mentioned in the introduction, besides HB and SHB, many other partial orders perform dynamic analysis using vector clocks. In such cases, tree clocks can replace vector clocks either partially or completely, sometimes requiring small extensions to the data structure as presented here. In particular, we foresee interesting applications of tree clocks for the WCP [157], DC [247] and SDP [122] partial orders.

Speeding up dynamic analyses. Vector-clock based dynamic race detection is known to be slow [253], which many prior works have aimed to mitigate. One of the most prominent performance bottlenecks is the linear dependence of the size of vector timestamps on the number of threads. Despite theoretical limits [64], prior research exploits special structures in traces [16, 71, 88, 107, 286] that enable succinct vector time representations. The Goldilocks [95] algorithm infers HB-orderings using locksets instead of vector timestamps but incurs severe slowdown [111]. The FASTTRACK [111] optimization uses epochs for maintaining succinct access histories and our work complements this optimization — tree clocks offer optimizations for other clocks (thread and lock clocks). Other optimizations in clock representations are catered towards dynamic thread creation [240, 241, 306]. Another major source of slowdown is program instrumentation and expensive metadata synchronization. Several approaches have attempted to minimize this slowdown, including hardware assistance [87, 324], hybrid race detection [229, 318], static analysis [113, 244], and sophisticated ownership protocols [44, 248, 313].

5.8 Conclusion

We have introduced tree clocks, a new data structure for maintaining logical times in concurrent executions. In contrast to standard vector clocks, tree clocks can dynamically capture communication patterns in their structure and perform join and copy operations in sublinear time, thereby avoiding the traditional overhead of these operations when possible. Moreover, we have shown that tree clocks are vector-time optimal for computing the HB partial order, performing at most a constant

factor work compared to what is absolutely necessary, in contrast to vector clocks. Finally, our experiments show that tree clocks effectively reduce the running time for computing the MAZ, SHB and HB partial orders significantly, and thus offer a promising alternative over vector clocks.

Interesting future work includes incorporating tree clocks in an online analysis such as ThreadSanitizer [267]. Any use of additional synchronization to maintain analysis atomicity in this online setting is identical and of the same granularity to both vector clocks and tree clocks. However, the faster joins performed by tree clocks may lead to less congestion compared to vector clocks, especially for partial orders such as SHB and MAZ where synchronization occurs on all events (i.e., synchronization, as well as access events). We leave this evaluation for future work. Finally, since tree clocks are a drop-in replacement of vector clocks, most of the existing techniques that minimize the slowdown due to metadata synchronization (Section 5.7) are directly applicable to tree clocks.

Chapter 6

CSSTs: A Dynamic Data Structure for Partial Orders in Concurrent Execution Analysis

By Hünkar Can Tunç, Ameya Prashant Deshmukh, Berk Çirisci, Constantin Enea, Andreas Pavlogiannis. Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), April 2024.

Abstract

Dynamic analyses are a standard approach to analyzing and testing concurrent programs. Such techniques observe program traces σ and analyze them to infer the presence or absence of bugs. At its core, each analysis maintains a partial order P that represents order dependencies between the events of σ . Naturally, the scalability of the analysis largely depends on maintaining P efficiently. The standard data structure for this task has thus far been Vector Clocks. These, however, are slow for analyses that follow a non-streaming style, costing $O(n)$ time for inserting (and propagating) each new ordering in P , where n is the size of σ , while they cannot handle the deletion of existing orderings.

In this paper we develop *Collective Sparse Segment Trees (CSSTs)*, a simple but elegant data structure for maintaining a partial order P . CSSTs thrive when the width k of P is much smaller than the size n of its domain, allowing inserting, deleting, and querying for orderings in P to run in $O(\log n)$ time. For a concurrent trace, k normally equals the number of its threads, and is orders of magnitude smaller than its size n , making CSSTs fitting for this setting. Our experiments confirm that CSSTs are the best data structure currently to handle a range of dynamic analyses from existing literature.

6.1 Introduction

Concurrent programming is notoriously difficult due to the inherent nondeterminism in interprocess communication [218]. As such, considerable efforts go towards effective techniques for analyzing concurrent programs and testing them for correctness. Dynamic analyses are one of the most widespread types of such techniques. Their main working principle is to discover software faults by analyzing concrete program executions. Prominent examples include concurrency bugs such as data races [94, 111, 121, 191, 199, 200, 236, 247, 248], deadlocks [54, 154, 291], linearizability [78] and atomicity violations [36, 54, 117, 199], thread-safety [179], and memory violations [137, 317], to name a few.

Although each analysis employs reasoning that is specific to the problem at hand, one of its core components is nearly always the construction of a partial order P (often termed generically as “happens-before” [171]) that captures order dependencies between events of the analyzed trace σ . The analysis undergoes a sequence of interleaved operations of (i) *inserting* new orderings $e_1 \rightarrow e_2$ in P , (ii) *querying* whether $e_1 \rightarrow e_2$ in P , and even (iii) *deleting* existing orderings $e_1 \rightarrow e_2$ from P , as it attempts to prove that σ has certain properties (e.g., that it is sequentially-consistent, or it contains a data race). To benefit scalability, each operation must take as little time as possible, as both the number of traces and the size of each trace normally span several orders of magnitude.

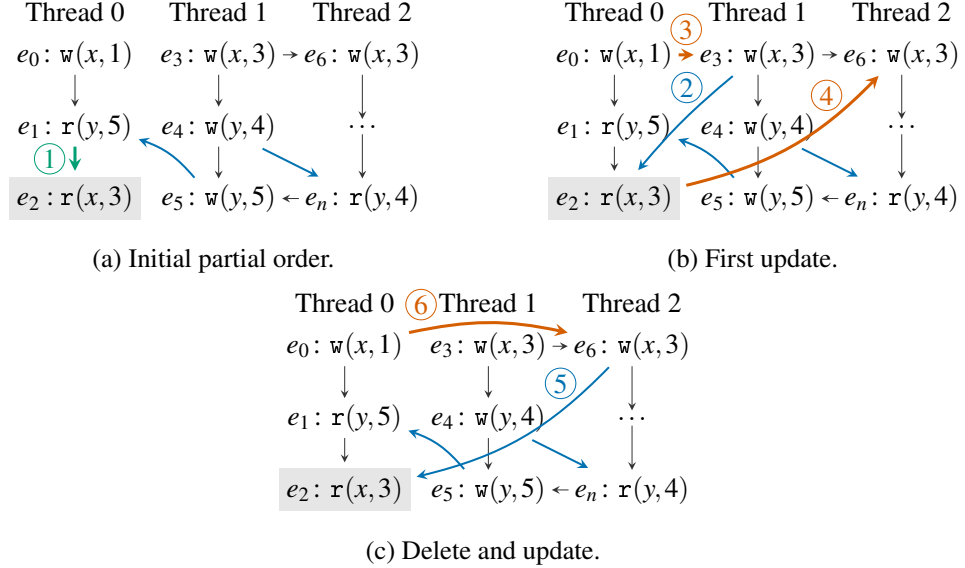


Figure 6.1: A consistency analysis. Blue edges represent reads-from information. Numbered edges represent new orderings inserted by the analysis.

Consider analyzing a trace σ of size n . Representing P as a graph G , the above setting is colloquially known as *dynamic graph reachability*, with dynamically inserting/deleting edges and performing reachability queries. Such queries are handled in

$O(1)$ time, at the cost of $O(n^2)$ for keeping G transitively closed after each update. This bound is prohibitively large for typical values of n in the range 10^4 - 10^{10} .

The scalability of dynamic analyses has been driven primarily by a simple data structure known as *Vector Clocks* [205], based on the realization that (i) σ consists of a small number of threads k , and (ii) normally, P contains k totally ordered chains (or a number proportional to k). Consequently, the whole backward set of an event e can be compactly summarized as an integer array, i.e., a Vector Clock. This allows to replace one factor n in the update complexity by the much smaller k , i.e., a single edge insertion now takes $O(nk)$ time.

In many analyses that have a *streaming nature*, the insertion cost of Vector Clocks is further reduced to $O(k)$. Here the events of σ are processed one by one, and new orderings $e_1 \rightarrow e_2$ are always inserted where e_2 is the current event being processed. Popular examples include analyses for detecting data-races [111, 199, 247], deadlocks [291], and atomicity violations [198], as well as consistency checking in some weak-memory models [290]. Thus in the streaming setting, Vector Clocks are arguably the best data structure to represent a partial order. On the other hand, many analyses are *non-streaming*, i.e., as the analysis progresses, it might insert new orderings between arbitrary events of σ . These new orderings might have to be propagated across n events, recovering the $O(nk)$ bound, which can lead to a significant slowdown. A natural question thus arises: *can the $O(nk)$ cost be reduced further?* In general, *what is the best data structure for non-streaming concurrent execution analysis?*

Here we develop a simple but elegant data structure for this general setting, called *Collective Sparse Segment Trees* (CSSTs). Like Vector Clocks, CSSTs exploit the fact that $k \ll n$, and offer update and querying operations that run roughly in $O(\log n)$ time, thereby reducing the prohibitive linear dependency on n to logarithmic. We argue that, as Vector Clocks are the most fitting data structure for handling *streaming* dynamic analyses, CSSTs are the most fitting data structure for handling non-streaming analyses.

Motivating Example

Consider the analysis of an abstract trace shown in Figure 6.1, consisting of three threads and write/read events $w(x, v)/r(x, v)$, where x is a variable and v is a value. The consistency-testing problem asks whether there is an interleaving that is consistent with the values observed by each read [123], and has numerous applications in dynamic concurrency analyses [12, 49, 60, 61, 97, 137, 154, 191, 252, 291]. The analysis maintains (i) a reads-from map $w(x, v) \mapsto r(x, v)$, denoting that $r(x, v)$ obtains its value from $w(x, v)$, and (ii) a partial order P that is induced by this reads-from map and any additional indirect orderings this might impose.

Assume that the consistency analysis has, so far, established the partial order P in Figure 6.1a, with blue edges marking the current reads-from map. The analysis proceeds on the read event $e_2: r(x, 3)$. It first inserts in P the program order edge ①. As e_2 has no successors in P , this edge need not be propagated further. Now the

analysis must decide which of the two writes e_3 and e_6 can be observed by e_2 . In general, there is no efficient procedure for deciding this [123], and the consistency algorithm proceeds by trying each option.

In Figure 6.1b, the analysis tries to map $e_3 \mapsto e_2$, leading to inserting ② in P . Moreover, since $e_0 \rightarrow e_2$ in P , the analysis also inserts ③ and ④ in P . These are necessary orderings to respect the fact that e_2 reads from e_3 — the process of inferring such orderings is known as saturation, and is used widely in dynamic analyses (e.g., [9, 191, 236, 247, 252, 320, 321]). Crucially, both e_3 and e_6 have many successors in P , thus these edges must be propagated along many events, costing $O(n)$ time, even if P is represented using Vector Clocks. In contrast, CSSTs achieve a much smaller $O(\log n)$ bound.

The reads-from choice $e_3 \mapsto e_2$ is inconsistent due to the cycle $e_2 \rightarrow e_6 \rightarrow \dots \rightarrow e_n \rightarrow e_5 \rightarrow e_1 \rightarrow e_2$. The analysis must delete the orderings ②, ③ and ④, before proceeding with an alternative writer for e_2 . When P is represented via Vector Clocks, there is no efficient method for deletion, and essentially it must be recomputed from scratch, taking $O(n^2)$ time. In contrast, CSSTs only need $O(\log n)$ time per edge deletion, thereby completing this task efficiently.

Finally, in Figure 6.1c, the analysis tries to map $e_6 \mapsto e_2$, leading to inserting ⑤ in P . It also infers ordering ⑥, which again must be propagated further, a task that costs $O(n)$ time for Vector Clocks but only $O(\log n)$ time for CSSTs.

Contributions

We develop CSSTs, a new data structure for maintaining and querying partial orders of small width. Concretely, consider the maintenance of a DAG G , representing a partial order P over n events and k totally ordered chains (normally k equals the number of threads).

1. CSSTs support fully dynamic updates (inserting and deleting edges) in $O(\max(\log \delta, \min(\log n, d)))$ time, where δ is the maximum degree in G and d is the *cross-chain density* of G , a novel notion we introduce here that captures a type of sparsity prevalent in practice. Normally, δ is constant and the above expression results in $O(d)$. Reachability (ordering) queries take $O(k^3 \min(\log n, d))$ time.
2. We further optimize CSSTs for the purely incremental setting, supporting the insertion, but not deletion, of edges. Incremental CSSTs run updates and queries in time $O(k^2 \min(\log n, d))$ and $O(\min(\log n, d))$, respectively.
3. We make an extensive experimental evaluation of CSSTs in concurrency analyses from the literature, spanning data-race detection [236], deadlock detection [54], memory bugs [317], consistency checking for X86-TSO [252], use-after-free bugs [137], data-races in the C11 memory model [191], and root cause analysis of linearizability violations [78]. Our results show that CSSTs speed up the corresponding analysis in most cases, indicating that they are the most fitting data structure in this ubiquitous setting.

CSSTs are based on the insight that dynamic reachability on partial orders of small width can be reduced to a basic algorithmic problem known as *dynamic suffix*

minima. This idea was partly explored in a data structure for incremental reachability underpinning the M2 data-race detector [236]. However, CSSTs are more advanced, as (i) they employ techniques making them faster in all cases, (ii) they achieve tighter time and space bounds on sparse instances, and (iii) they support fully dynamic updates (i.e., both incremental and decremental). Our experimental results confirm that CSSTs are indeed more efficient in both time and memory, besides being broader (as they handle edge deletions). In our technical report [298], we provide the proofs for the theorems and lemmas presented in the paper.

6.2 Preliminaries

In this section, we develop some general notation and introduce standard concepts on concurrent executions and partial orders that will be used throughout the paper.

Concurrent Execution Model

Events and traces. We broadly consider traces σ of concurrent programs, representing the history of operations in an execution. We write Events_σ to denote the set of events occurring in σ . Each event of σ is a tuple $e = \langle t, i, m \rangle$, where t is an identifier of the thread that performed e and i is the sequence id of e . The pair $\langle t, i \rangle$ serves as a unique identifier for e . The field m represents meta information for e that might be relevant to a dynamic analysis, but is immaterial for CSSTs. E.g., m may record the operation performed by e (read, write, etc), as well as the variable it accesses and the value it acquires. CSSTs operate only on the identifier $\langle t, i \rangle$, while the meta information m will be used only in our examples. We write $\text{tid}(e)$, $\text{id}(e)$ for the thread identifier and sequence id of e , respectively. Depending on the application, the events in σ may or may not be totally ordered.

Relations and functions on execution traces. For a binary relation X , we denote by X^* its reflexive transitive closure. A partial order is a reflexive, transitive, and anti-symmetric binary relation defined on the elements of a given set S . We denote by $\leq_P^\sigma$ a partial order P over the set Events_σ . Given $e_i, e_j \in \text{Events}_\sigma$, we write $e_i \leq_P^\sigma e_j$ to represent $(e_i, e_j) \in \leq_P^\sigma$. We write $e_i <_P^\sigma e_j$ to represent $e_i \leq_P^\sigma e_j$ and $e_i \neq e_j$. The *program order* $\leq_{\text{TO}}^\sigma$ represents a partial order where $e_i \leq_{\text{TO}}^\sigma e_j$ iff $\text{id}(e_i) \leq \text{id}(e_j)$ and $\text{tid}(e_i) = \text{tid}(e_j)$. We write $\text{width}(P)$ to represent the largest size of a set $S \subseteq \text{Events}_\sigma$ such that for all distinct pairs $e_i, e_j \in S$, we have that $e_i \not\leq_P^\sigma e_j$ and $e_j \not\leq_P^\sigma e_i$.

In concurrency analyses, $\text{width}(P)$ is almost always bounded by some constant, a fact that is exploited by Vector Clocks, as well as the CSSTs we introduce in this work. E.g., in many analyses $\leq_{\text{TO}}^\sigma \subseteq \leq_P^\sigma$, implying that $\text{width}(P) \leq k$, where k is the number of threads. Although in other settings (e.g., those involving weak-memory concurrency) $\text{width}(P)$ might be larger than k , it still remains bounded by some small factor of k . For example, analyses involving TSO normally satisfy $\text{width}(P) \leq 2k$ (we touch on this later in Section 6.5).

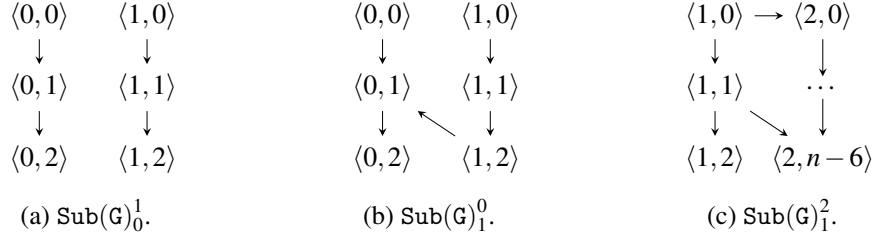


Figure 6.2: Three subgraphs of the chain DAG G formed from the execution in Figure 6.1a. The cross-chain density of G is 2, as chain 1 (i.e., thread 1 in Figure 6.1a) has two outgoing edges to chain 2 (from events e_3 and e_4).

Dynamic Reachability on DAGs

Chain directed acyclic graphs. We consider directed acyclic graphs (DAGs) $G = (V, E)$, where V is the set of nodes and E is the set of edges. Dynamic analyses use such DAGs to represent partial orders on n events of a concurrent execution, where k is normally the number of threads, or a value proportional to that. For this purpose, a node in V is a pair $u = \langle t, i \rangle \in [k] \times [n]$, where $[j] = \{0, 1, \dots, j-1\}$. The set of edges normally captures the program order, which is reflected in the fact that for any two nodes $\langle t, i \rangle, \langle t, i+1 \rangle \in V$, we have $(\langle t, i \rangle, \langle t, i+1 \rangle) \in E$. We can thus represent G as k totally-ordered chains with additional edges across chains, and call such graphs *chain DAGs*. A node $\langle t, i \rangle$ is said to be *earlier* than another node in the same chain $\langle t, j \rangle$ if $i \leq j$, and *later* otherwise. Then, G can be written as the composition of $k(k-1)$ subgraphs $\text{Sub}(G)_{t_1}^{t_2}$ induced by the chains t_1 and t_2 , as well as the cross-chain edges from t_1 to t_2 (see Figure 6.2). The *cross-chain density* d of G is the maximum, among all chains t_1 , number of nodes in chain t_1 that have an outgoing edge to some other chain.

Dynamic reachability. The dynamic reachability problem is defined with respect to a given chain DAG $G = (V, E)$ and online sequence of operations of the following types.

1. An `insertEdge`(u, v) operation, such that $(u, v) \notin E$, inserts the edge (u, v) in G .
2. An `deleteEdge`(u, v) operation, such that $(u, v) \in E$, deletes the edge (u, v) in G .
3. A `reachable`(u, v) operation returns True iff $u \rightarrow^* v$.
4. A `successor`(u, t) operation returns the earliest successor of u in the t -th chain.
5. A `predecessor`(u, t) operation returns the latest predecessor of u in the t -th chain.

The operations `insertEdge` and `deleteEdge` are updates, while `reachable`, `successor` and `predecessor` are queries. We allow update operations only across nodes in different chains. In this dynamic setting, we define the cross-chain density of G to be the maximum cross-chain density of the graph across all update operations on G .

6.3 Collective Sparse Segment Trees

We now present *Collective Sparse Segment Trees* (CSSTs), a data structure for efficiently solving the dynamic reachability problem in the context of concurrent execution analysis. The properties of CSSTs are stated in the following theorem.

Theorem 2. *Consider a chain DAG G of n nodes, k chains, maximum out-degree δ and cross-chain density d . CSSTs maintain G under dynamic updates, costing $O(\max(\log \delta, \min(\log n, d)))$ time per update and $O(k^3 \min(\log n, d))$ time per query.*

CSSTs are based on the insight that dynamic reachability on DAGs with few chains can be efficiently reduced to repeated instances of another basic problem known as *dynamic suffix minima*. We begin by developing this concept when G consists of only $k = 2$ chains in Section 6.3. Dynamic suffix minima are solved using a data structure known as *Segment Trees*. A key novelty of CSSTs is based on the realization that most direct orderings in dynamic analyses make G have low cross-chain density (i.e., the cross-chain edges are sparse). We exploit this insight in Section 6.3, by developing *Sparse Segment Trees* that handle arbitrary graphs, but become more efficient the lower their cross-chain density becomes. Finally, in Line 31 we handle DAGs of arbitrarily many chains k , by appropriately maintaining *collections* of Sparse Segment Trees, which also explains the name of the data structure.

Dynamic Suffix Minima and Segment Trees

Here we state the algorithmic problem of dynamic suffix minima, and relate it to dynamic reachability on chain DAGs, focusing on the special case of $k = 2$ chains. In later sections we explain how to handle $k > 2$. Some insights are based on observations made in [236] for the purely incremental case.

Dynamic suffix minima. The dynamic suffix minima problem concerns maintaining an array A of n values in $\mathbb{N} \cup \{\infty\}$ under an online sequence of updates, and answering minima queries on ranges of A . We write $A[i]$ and $A[i : j]$ to denote the value of A at index i and the sequence of values $A[i], \dots, A[j]$, respectively, and write $A[i :]$ as shorthand for $A[i : |A| - 1]$. We say that A is empty on index i if $A[i] = \infty$. The density of A is the number of non empty entries of A .

For $i \in [n]$ and $a \in \mathbb{N} \cup \{\infty\}$, an operation on A is one of:

1. $\min(A, i)$, returning $\min_{i \leq j < n} A[j]$, i.e., the minimum value in the suffix $A[i :]$;
2. $\text{argleq}(A, a)$, returning $\max_{i: A[i] \leq a} i$, i.e., the largest index i of A such that $A[i] \leq a$;
and
3. $\text{update}(A, i, a)$, setting $A[i] = a$.

The task is to answer $\min$ and argleq queries, taking into consideration all preceding update operations.

Dynamic suffix minima (in fact, the more general problem of dynamic *range* minima) is solved efficiently using Segment Trees (STs), taking $O(\log n)$ time per operation (see, e.g., [22]). A ST T represents A as a binary tree of nodes $\text{nd}_{i,j}$ each associated with a range $[i, j]$. Each $\text{nd}_{i,j}$ maintains a value $\text{nd}_{i,j}.\text{min} = \min(A[i : j])$.

We write $\text{depth}(\text{nd}_{i,j})$ to denote the depth of $\text{nd}_{i,j}$ in T . We let $T.\text{root}$ be the root node of T . Each query is answered by traversing T from the root to a leaf, the latter containing the value to be returned.

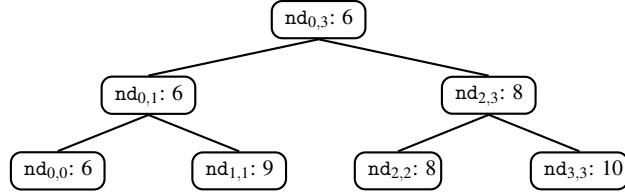


Figure 6.3: A Segment Tree representation of $A = [6, 9, 8, 10]$.

Example 12. Consider the Segment Tree T in Figure 6.3 representing the array $A = [6, 9, 8, 10]$. We have the following.

- $\min(A, 0) = 6$, $\min(A, 1) = \min(A, 2) = 8$, $\min(A, 3) = 10$
- $T.\text{root}.\text{min} = 6$, $\text{nd}_{2,3}.\text{min} = 8$
- $\text{argleq}(A, 7) = 0$, $\text{argleq}(A, 9) = 2$, $\text{argleq}(A, 11) = 3$
- $\text{update}(A, 3, 7)$ sets $A[3] = 7$. In T , this is captured by setting $\text{nd}_{2,3}.\text{min} = \text{nd}_{3,3}.\text{min} = 7$.

Dynamic reachability on $k = 2$ chains. Recall that a chain DAG G of k chains is the composition of $k(k-1)$ subgraphs $\text{Sub}(G)_{t_1}^{t_2}$ (Figure 6.2). Now assume that $k = 2$. We represent each $\text{Sub}(G)_{t_1}^{t_2}$ as an array $A_{t_1}^{t_2}: [n] \rightarrow [n] \cup \infty$. $A_{t_1}^{t_2}[j_1]$ stores a unique neighbor of node $\langle t_1, j_1 \rangle$ in chain t_2 . What if $\langle t_1, j_1 \rangle$ has multiple neighbors to t_2 ? Then, it suffices to store the *earliest neighbor* of $\langle t_1, j_1 \rangle$ in t_2 . Although this is not an exact representation of G , it captures exactly all reachability in G . In particular, we maintain the invariant

$$A_{t_1}^{t_2} = \min(\{j_2: \langle t_1, j_1 \rangle \rightarrow \langle t_2, j_2 \rangle\}) \quad (6.1)$$

under the standard convention that $\min(\emptyset) = \infty$. A query $\text{successor}(\langle t_1, j_1 \rangle, t_2)$ simply returns the suffix minima $x = \min(A_{t_1}^{t_2}, j_2)$, while a query $\text{predecessor}(\langle t_2, j_2 \rangle, t_1)$ returns the suffix arg-minima $\text{argleq}(A_{t_1}^{t_2}, j_2)$. For reachability queries, observe that we have $\langle t_1, j_1 \rangle \rightarrow^* \langle t_2, j_2 \rangle$ in G iff there is an edge $\langle t_1, i_1 \rangle \rightarrow \langle t_2, i_2 \rangle$ such that $i_1 \geq j_1$ and $i_2 \leq j_2$. Hence, a query $\text{reachable}(\langle t_1, j_1 \rangle, \langle t_2, j_2 \rangle)$ reduces to checking whether $\text{successor}(\langle t_1, j_1 \rangle, t_2) \leq j_2$.

How can we maintain the invariant Eq. (6.1) under edge insertions and deletions? We achieve this by storing all current successors of $\langle t_1, j_1 \rangle$ to chain t_2 in a min heap $\text{edgeHeap}_{t_1}^{t_2}[j_1]$. Inserting and deleting edges from $\langle t_1, j_1 \rangle$ is handled by applying the same operation in $\text{edgeHeap}_{t_1}^{t_2}[j_1]$. This ensures that the earliest neighbor of $\langle t_1, j_1 \rangle$ in t_2 appears in the root of $\text{edgeHeap}_{t_1}^{t_2}[j_1]$, which is also stored in $A_{t_1}^{t_2}[j_1]$.

Example 13. Consider again the partial order $\leq_p^\sigma$ in Figure 6.1a. Some of the arrays $A_{t_1}^{t_2}$ are shown in Figure 6.4. Observe that $\text{successor}(e_3, 2) = e_6$ can be determined by the suffix minima query $\min(A_1^2, 0)$. Similarly, $\text{predecessor}(e_2, 1) = e_5$ can be determined by the suffix arg-minima query $\text{argleq}(A_1^0, 1)$.

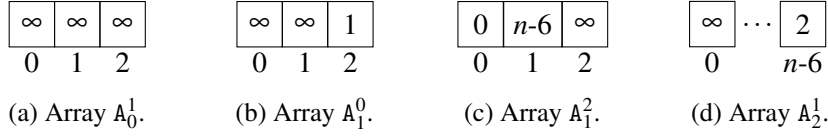


Figure 6.4: Representation of the partial order in Figure 6.1a.

Handling $k > 2$ chains. Naturally, when $k > 2$, we can have a suffix minima array $A_{t_1}^{t_2}$ for every pair of threads $t_1, t_2 \in [k]$ with $t_1 \neq t_2$. However, a path $\langle t_1, j_1 \rangle \rightarrow^* \langle t_2, j_2 \rangle$ might go through intermediate nodes $\langle t_3, j_1 \rangle$, i.e., reachability is formed transitively across chains (e.g., the path $e_6 \rightarrow^* e_2$ in Figure 6.1a). We address this challenge depending on the setting (fully dynamic or purely incremental).

On a high level, in the fully dynamic setting (Line 31) in each $A_{t_1}^{t_2}$ we store direct edges, i.e., each edge insertion is handled similarly to what was already described for $k = 2$. During queries, we perform a transitive closure operation across chains. Taking advantage of the properties of our data structure, this transitive closure takes $O(k^3)$ time, as opposed to $O(n^2)$ time, and thus remains efficient.

In the purely incremental setting (Section 6.4), on the other hand, $A_{t_1}^{t_2}$ stores transitive reachability across chains, which is computed every time a new edge inserted. Again, taking advantage of the properties of our data structure, this transitive closure takes $O(k^2)$ time, as opposed to $O(n)$ time, and is thus very efficient. Now, queries can be performed by directly querying the respective suffix minima array, similarly to what was already described for $k = 2$.

Sparse Segment Trees

Using STs T to solve suffix minima, each operation completes in time proportional to the height of T , i.e., $O(\log n)$. In practice n can be very large and thus hinder performance. Here we develop Sparse Segment Trees (SSTs), that optimize STs for our setting. These are based on two main ideas, namely (i) *minima indexing*, and (ii) *sparse tree representation*.

Intuitively, STs solve the more general problem of *range minima queries*, where every query asks for the minima in a range $A[i : j]$, as opposed to the suffix $A[i : \cdot]$. Minima indexing exploits the fact that our queries always concern a suffix of A , and allows them to often complete without traversing a full path from the root to a leaf.

Sparse tree representation is an optimization of the tree representation, which becomes more condensed the lower the density of A is (i.e., the fewer non- ∞ entries it has), stemming from the observation that ∞ entries do not contribute to the queries. In a dynamic analysis setting, G normally has low cross-chain density d , which means that the suffix-minima arrays storing reachability across chains (as described in Section 6.3) are sparse. This reduces the height of the tree, allowing both queries and updates to run faster.

Minima indexing. Each node $\text{nd}_{i,j}$ of a Sparse Segment Tree T maintains a field $\text{pos} \in [i, j]$, that is defined recursively: it points to the largest index of A that contains

the minimum element in the range $A[i : j]$, after excluding all indexes that are stored in the pos fields of all ancestors of $\text{nd}_{i,j}$. Formally, let $\text{nd}^1, \dots, \text{nd}^m$ be the ancestors of $\text{nd}_{i,j}$ in T . Then

$$\text{nd}_{i,j}.\text{pos} = \max \left(\arg \min_{\ell \in [i,j] \setminus \{\text{nd}^1.\text{pos}, \dots, \text{nd}^m.\text{pos}\}} A[\ell] \right) \quad (6.2)$$

We also set $\text{nd}_{i,j}.\text{min} = A[\text{nd}_{i,j}.\text{pos}]$. Observe that now $\text{nd}_{i,j}.\text{min}$ might not be the minimum value in $A[i : j]$, in contrast to the standard Segment Trees.

The intuition behind $\text{nd}_{i,j}.\text{pos}$ is as follows. Each query $\min(A, i)$ starts a traversal from $T.\text{root}$ to a leaf. If we encounter a node $\text{nd}_{i,j}$ with $\text{nd}_{i,j}.\text{pos} \geq i$, the traversal can stop early and return that $\min(A, i) = \text{nd}_{i,j}.\text{min}$. Since we always query on a suffix of A , storing the *largest* index pointing to a minima increases the chances of stopping early. Moreover, we exclude the pos fields of the ancestors of $\text{nd}_{i,j}$ as these are irrelevant for $\text{nd}_{i,j}$, in the sense that this index would have already stopped the traversal in an ancestor of $\text{nd}_{i,j}$.

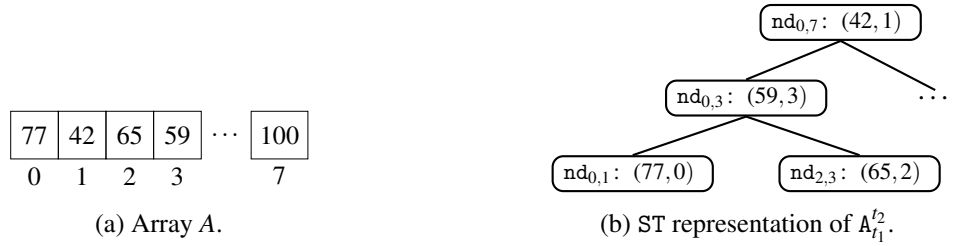


Figure 6.5: ST representation with minima indexing. The entries $A[4 : 6]$ are > 59 . The notation $\text{nd}_{i,j} : (x, y)$ denotes $\text{nd}_{i,j}.\text{min} = x$, $\text{nd}_{i,j}.\text{pos} = y$.

Example 14. Consider the array A in Figure 6.5a, and part of the corresponding ST in Figure 6.5b. We have $\text{nd}_{0,7}.\text{pos} = 1 = \arg \min_i A[i]$. A query $\min(A, 0)$ is now directly answered at the root, as $\text{nd}_{0,7}.\text{pos} \in [0 : 7]$. On the other hand, the query $\min(A, 2)$ is not answered at the root as $\text{nd}_{0,7}.\text{pos} \notin [2 : 7]$. The procedure then traverses its child $\text{nd}_{0,3}$. We have $\text{nd}_{0,3}.\text{pos} = 3$, which indexes the smallest element in $A[0 : 3]$ after excluding $\text{nd}_{0,7}.\text{pos}$. The traversal stops here, as $\text{nd}_{0,3}.\text{pos} \in [2 : 7]$.

Sparse representation. Sparse Segment Trees are tailored towards representing sparse arrays. This allows the height of the tree to shrink, which consequently results in more efficient updates and queries. A crucial aspect of SSTs is that they are self-adapting and do not require any a priori information about the sparsity of the input.

Intuitively, we achieve this as follows. Empty array indexes are never explicitly represented in the intermediate nodes of the tree. Every intermediate node nd is such that $\text{nd}.\text{pos}$ points to a unique, non-empty entry of A . The opposite is also true: every non-empty entry of A is indexed by the pos field of a node in the earliest possible level satisfying Eq. (6.2).

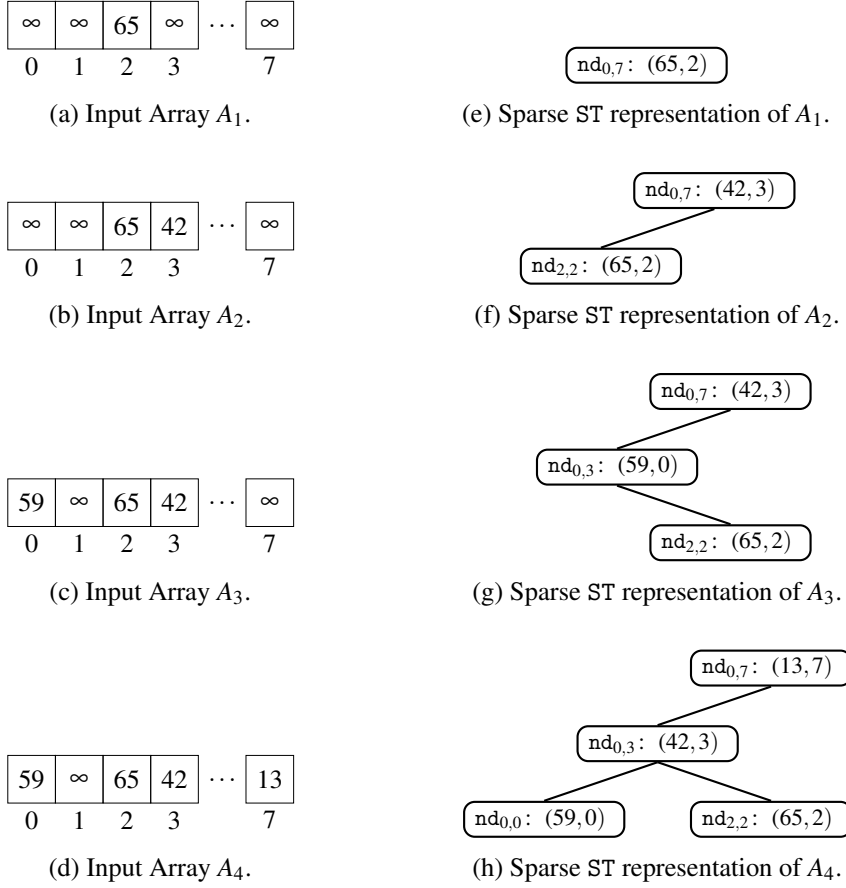


Figure 6.6: Sparse ST representation. The notation $\text{nd}_{i,j}: (x, y)$ denotes $\text{nd}_{i,j}.\text{min} = x$, $\text{nd}_{i,j}.\text{pos} = y$.

Example 15. Consider an array A of length 8, initially empty. Figure 6.6a–6.6d sequentially update A with new entries, and Figure 6.6e–6.6h show the evolution of the corresponding SST T . In Figure 6.6e, the unique node of T represents that A has only one non-empty entry, at index 2 with value 65. In contrast, a standard (non-sparse) Segment Tree would contain 15 nodes and have height 3. Figure 6.6f shows the update of T after setting $A[3] = 42$. Since 42 is now the smallest value of A , it is stored in the root, while a new node $\text{nd}_{2,2}$ becomes the left child of the root, with its pos and min fields set according to Eq. (6.2). Again, in the standard ST representation, $\text{nd}_{2,2}$ would have nodes $\text{nd}_{0,3}$ and $\text{nd}_{2,3}$ as ancestors, but these nodes are never created in our SST. Figure 6.6g, shows another update of T after setting $A[0] = 59$. As $59 \leq \text{nd}_{0,7}.\text{min}$, the root remains intact. The index 0 belongs to the left subtree of the root and $59 < \text{nd}_{2,2}.\text{min}$. This causes the new entry to be inserted to a fresh node $\text{nd}_{0,3}$ and assigned as the left child of the root, taking the existing node $\text{nd}_{2,2}$ as its right child. Figure 6.6h shows the final update of T after setting $A[7] = 13$. Since 13 is now the smallest value of A , it is stored in the root, while the previous

value stored in the root is recursively pushed downwards.

One last optimization step is to flatten subtrees of small size at the leaves to array blocks. For a node $nd_{i,j}$, if $\text{depth}(nd_{i,j})$ is larger than a value deeming that the node represents a small sub-array of A , then it becomes a leaf in T , storing the subarray $A[i:j]$. We call such a node $nd_{i,j}$ a *block node*. This representation is valuable when the array contains segments which are densely populated but also far apart from each other.

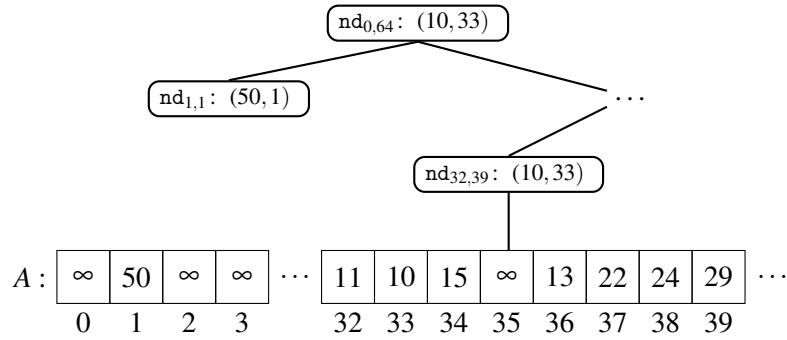


Figure 6.7: Flattening small subtrees to block nodes.

Example 16. Consider the array A and its corresponding SST in Figure 6.7. A contains only a single entry in the range $0, \dots, 31$. This entry is sparsely represented in the left subtree of the root. On the other hand, A is dense in the range $32, \dots, 39$, which would require a full subtree. Instead, when the range of the represented subarray is smaller than a threshold b , the corresponding subtree is flattened to an array block.

Operations on Sparse Segment Trees. Algorithm 7 implements the operations `update`, `min`, `argleq` on a SST.

Function `update(nd, pos, v)`. This function inserts a node nd' into the subtree of nd , with $nd'.pos = pos$ and $nd'.min = v$. It first checks if nd is initialized; if not, it creates a new node (Line 2). If nd is the root, the function checks and deletes if a node nd'' exists in the subtree of nd where $nd''.pos = pos$ (Line 3). Then, it checks if $2^{\text{depth}(nd)} \geq n/b$, where b is the block-size threshold. If so, nd becomes a block node and the update is performed directly on $nd.block$ (Line 4). Otherwise, it checks if $v \leq nd.min$. If so, it swaps the values of $nd.pos$ and $nd.min$ with pos and v (Line 7). This captures that the new entry will be stored in nd and the previous entry in nd will be relocated to a child node. Next, it identifies which subtree the corresponding entry belongs to, and makes a call to `updateHelper` (Line 10, Line 12). If the subtree is not present, `updateHelper` creates a new node (Line 14). Otherwise, it checks if pos is within the range of the subtree and if so it recursively calls `update`. If the subtree is present but pos is not within the range, `createLowestCommonAncestor` creates

Algorithm 7: The Sparse Segment Tree.

```

1 function update(nd, pos, v)
2   if nd = NIL then nd ← createNode(pos, v)
3   else if nd = T.root then nd ← deleteNode(nd, pos)
4   else if  $2^{\text{depth}(\text{nd})} \geq n/b$  then nd.block[pos] = v;
5   else
6     if  $v < \text{nd.min} \vee (v = \text{nd.min} \wedge \text{pos} > \text{nd.pos})$  then
7       nd.min, nd.pos, v, pos ← v, pos, nd.min, nd.pos
8     Let mid ← nd.start +  $\lfloor (\text{nd.end} - \text{nd.start})/2 \rfloor$ 
9     if pos ≤ mid then
10      nd.left ← updateHelper(nd.left, pos, v)
11    else
12      nd.right ← updateHelper(nd.right, pos, v)

13 function updateHelper(nd, pos, v)
14   if nd = NIL then nd ← createNode(pos, v)
15   else if pos ∈ [nd.start, nd.end] then update(nd, pos, v)
16   else nd ← createLowestCommonAncestor(nd, pos, v)
17   return nd

18 function min(nd, i)
19   if nd = NIL  $\vee i > \text{nd.end}$  then return ∞
20   if nd.block ≠ NIL then return min(nd.block, i);
21   if nd.pos ≥ i then return nd.min;
22   else
23     l ← min(nd.left, i)
24     r ← min(nd.right, i)
25     if l < r then return l else return r

26 function argleq(nd, v)
27   if nd = NIL  $\vee \text{nd.min} > v$  then return −∞
28   if nd.block ≠ NIL then return argleq(nd.block, v);
29   if nd.pos ≥ nd.left.end  $\wedge$  nd.pos ≥ nd.right.end then return nd.pos
30   else if nd.right.min ≤ v then return
31     max(nd.pos, argleq(nd.right, v))
32   else return max(nd.pos, argleq(nd.left, v))

```

a new node whose range corresponds to the lowest common ancestor of the existing subtree and pos.

Function min(nd, i). This recursive function returns ∞ if nd is NIL or $i > \text{nd.end}$ (Line 19), which captures the end of the recursion. Otherwise, it returns the smallest min value in the subtree of nd whose corresponding pos is greater than or equal

to i , as follows. First, it checks if nd is a block node, in which case it returns the minimum of the block array $nd.block$ (Line 20). Otherwise, if the minima indexing optimization succeeds, the traversal stops here, returning $nd.min$ (Line 21). If both conditions fail, the function proceeds recursively on the left and right child of nd (Lines 22–25).

Function $argleq(nd, v)$. This function returns $-\infty$ if nd is NIL or $nd.min > v$. Otherwise, it returns the largest pos in the subtree of nd whose corresponding min value is smaller than or equal to v , as follows. It first checks if nd is a block node, in which case it simply performs an array traversal on $nd.block$ (Line 28). Otherwise, the function uses the minima indexing information in $nd.pos$ to check if it can return early (Line 29). If the check fails, it makes a recursive call on the right subtree if $nd.right.min \leq v$ (Line 30). If all conditions fail, it makes a recursive call on the left subtree (Line 31).

Using SSTs. SSTs handle operations on the represented suffix-minima array A as follows. Queries $\min(A, i)$ and $argleq(A, a)$ are handled as $\min(T.root, i)$ and $argleq(T.root, a)$, respectively. On the other hand, $update(A, i, a)$ is handled as $update(T.root, i, a)$.

Complexity and the height of SSTs. The theoretical advantage of SSTs over STs is the fact that their height is bounded not only by $\log n$, but also by the density of the represented array. This is captured in the following lemma.

Lemma 5. *Consider an array A of size n with d non-empty (i.e., non- ∞) entries, and let T be its corresponding SST. Then the height of T is bounded by $\min(\log n, d)$.*

Thus, we can use Lemma 5 to bound the time taken to perform any update and query on the represented array A . In practice, however, the running time can be even smaller, as, in contrast to standard STs, the update and query operations can stop before traversing a full path from the root to a leaf.

Collective Sparse Segment Trees

We are now ready to describe CSSTS, leading to Theorem 2.

CSSTS. For every two distinct chains $t_1, t_2 \in [k]$, CSSTS maintain a suffix-minima array $A_{t_1}^{t_2}$ storing direct edges from t_1 to t_2 , together with a min-heap $edgeHeap_{t_1}^{t_2}[j_1]$, for each $j_1 \in [n]$, as described in Section 6.3. Suffix-minima queries on $A_{t_1}^{t_2}$ are handled using a SST, as described in Section 6.3. The pseudocode is shown in Algorithm 8. The functions `insertEdge` and `deleteEdge` follow directly the description in Section 6.3, i.e., they manipulate the arrays $A_{t_1}^{t_2}$ and min-heaps $edgeHeap_{t_1}^{t_2}[j_1]$.

The function `reachable` is implemented via a successor query, which is more involved. As we have $k > 2$ chains, `successor` runs a forward traversal from $\langle t_1, j_1 \rangle$ to discover transitive reachability across chains. This is done by repeated `min` queries in the suffix minima arrays $A_{t_2'}^{t_1'}$ (Line 9), which find the earliest node of t_1' that has an edge from the currently earliest node of t_2' found to be reachable from $\langle t_1, j_1 \rangle$. Crucially, this computation completes in $O(k^3)$ steps, as opposed to $O(n^2)$ steps normally required for a graph traversal.

Algorithm 8: Fully dynamic CSSTs.

```

1 function successor( $\langle t_1, j_1 \rangle, t_2$ )
2   foreach  $t'_1 \in [k] \setminus \{t_1\}$  do
3      $\text{closure}[t'_1] \leftarrow \min(A_{t'_1}^{t'_1}, j_1)$ 
4   do
5      $\text{changed} \leftarrow \text{false}$ 
6     foreach  $t'_1 \in [k] \setminus \{t_1\}$  do
7       foreach  $t'_2 \in [k] \setminus \{t_1\}$  do
8         if  $t'_1 \neq t'_2$  then
9            $\text{Let } v \leftarrow \min(A_{t'_2}^{t'_1}, \text{closure}[t'_2])$ 
10          if  $v < \text{closure}[t'_1]$  then
11             $\text{closure}[t'_1] \leftarrow v$ 
12             $\text{changed} \leftarrow \text{true}$ 
13   while  $\text{changed}$ 
14   return  $\text{closure}[t_2]$ 

15 function predecessor( $\langle t_1, j_1 \rangle, t_2$ )
    // Similar to successor

16 function reachable( $\langle t_1, j_1 \rangle, \langle t_2, j_2 \rangle$ )
17   if successor( $\langle t_1, j_1 \rangle, t_2$ )  $\leq j_2$  then return True else return False

18 function insertEdge( $\langle t_1, j_1 \rangle, \langle t_2, j_2 \rangle$ )
19   if  $j_2 < \text{edgeHeap}_{t_1}^{t_2}[j_1].\text{min}$  then update( $A_{t_1}^{t_2}, j_1, j_2$ )
20    $\text{edgeHeap}_{t_1}^{t_2}[j_1].\text{insert}(j_2)$ 

21 function deleteEdge( $\langle t_1, j_1 \rangle, \langle t_2, j_2 \rangle$ )
22   if  $j_2 = \text{edgeHeap}_{t_1}^{t_2}[j_1].\text{min}$  then
23      $\text{edgeHeap}_{t_1}^{t_2}[j_1].\text{delete}(j_2)$ 
24     update( $A_{t_1}^{t_2}, j_1, \text{edgeHeap}_{t_1}^{t_2}[j_1].\text{min}$ )
25   else  $\text{edgeHeap}_{t_1}^{t_2}[j_1].\text{delete}(j_2)$ 

```

Example 17. Consider the chain DAG in Figure 6.8 and the operation $\text{successor}(\langle 0, 0 \rangle, 3)$. The first step is identifying the earliest successors of $\langle 0, 0 \rangle$ reachable via direct edges from $\langle 0, 0 \rangle$ and its thread-local successors, in all other threads. This discovers $\langle 1, 0 \rangle$ via ① and $\langle 3, 2 \rangle$ via ②. While $\langle 3, 2 \rangle$ is indeed in chain 3, it is not necessarily the earliest successor of $\langle 0, 0 \rangle$ (indeed, the earliest successor is $\langle 3, 1 \rangle$). Hence the function proceeds iteratively for each newly discovered node until reaching a fixed point. This leads to the discovery of $\langle 2, 1 \rangle$ via ③ and eventually the discovery of $\langle 3, 1 \rangle$ via ④.

Analysis. We now state the properties of fully dynamic CSSTs. First, since the arrays $A_{t_1}^{t_2}$ only store direct edges, the following sparsity property is straightforward.

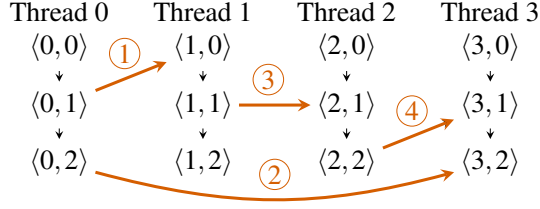


Figure 6.8: Query $\text{successor}(\langle 0,0 \rangle, 3)$.

Lemma 6 (Sparsity). *Let d be the cross-chain density of G . Then the density of each array $A_{t_1}^{t_2}$ is bounded by d .*

The use of min-heaps guarantees the following invariant.

Lemma 7 (Invariant). *For every distinct $t_1, t_2 \in [k]$, we have $A_{t_1}^{t_2}[j_1] = \min(\{j_2 \in [n]. \langle t_1, j_1 \rangle \rightarrow \langle t_2, j_2 \rangle\})$.*

A *crossing path* is a sequence of nodes $\pi: \langle t_0, j_0 \rangle, \dots, \langle t_{m-1}, j_{m-1} \rangle$ such that for each $\ell \in [m-2]$ we have $t_\ell \neq t_{\ell+1}$ and there are $i_1, i_2 \in [n]$ such that (i) $i_1 \geq j_\ell$, (ii) $i_2 \leq j_{\ell+1}$, and (iii) $\langle t_\ell, i_1 \rangle \rightarrow \langle t_{\ell+1}, i_2 \rangle$. Clearly, for any two nodes $\langle t_1, j_1 \rangle$ and $\langle t_2, j_2 \rangle$, if $\langle t_1, j_1 \rangle \rightarrow^* \langle t_2, j_2 \rangle$, then there exists a crossing path between them of length at most k . The following lemma implies the correctness of queries.

Lemma 8. *For every $i \in [k]$, after the i -th iteration of the do-while loop in Line 4, $\text{closure}[t'_1]$ points to the earliest node in chain t'_1 that is reachable from $\langle t_1, j_1 \rangle$ via a crossing path of length $\leq i+1$.*

Regarding the time complexity, due to Lemma 6 and Lemma 5, each operation in a suffix minima array $A_{t_1}^{t_2}$ takes $O(\min(\log n, d))$ time. Since the maximum out-degree is δ , each operation on a min heap $\text{edgeHeap}_{t_1}^{t_2}[j_1]$ runs in $O(\log \delta)$ time. Thus, updates take $O(\max(\log \delta, \min(\log n, d)))$ time, while, due to Lemma 8, queries take $O(k^3 \min(\log n, d))$ time, arriving at Theorem 2.

Space usage. We now discuss the space usage of CSSTs. Consider a chain DAG G of n nodes, k chains, and cross-chain density d . In this fully dynamic setting, CSSTs store all edges in the min heaps $\text{edgeHeap}_{t_1}^{t_2}[j_1]$, as edge deletions requires to remember all edges added so far. The suffix minima arrays $A_{t_1}^{t_2}$ store in total $O(dk)$ entries, as (i) each chain has at most d entries that have at least one successor to another chain, and (ii) there are $< k$ other chains. Naturally, $d \leq n$, thus in the worst case the above expression becomes $O(nk)$. This is the same space complexity as Vector Clocks. However, Vector Clocks do not exploit the sparsity of G . As we will see in Section 6.5, in practice typically $d \ll n$, which makes CSSTs more space-efficient than Vector Clocks.

6.4 Incremental CSSTs

In practice, many dynamic analyses maintain their partial order incrementally, i.e., they only insert and never delete orderings. To optimize for this incremental setting, in this section we specialize CSSTs to only handle incremental updates, with guarantees stated in the following theorem.

Theorem 3. *Consider a chain DAG G of n nodes, k chains, and cross-chain density d . Incremental CSSTs maintain G under incremental updates, costing $O(k^2 \min(\log n, d))$ time per update and $O(\min(\log n, d))$ time per query.*

Compared to Theorem 2, Theorem 3 trades the cost dependency on the number of chains from queries to updates, while also decreasing it by a factor k . Moreover, now the cost of both queries and updates is bounded by the cross-chain density of G (as opposed to only queries in Theorem 2).

Incremental CSSTs. Incremental CSSTs maintain $k(k-1)$ suffix minima arrays $A_{t_1}^{t_2}$, with $t_1 \neq t_2$, each implemented using an SST $T_{t_1}^{t_2}$. These arrays now store *transitive reachability*, as opposed to direct edges between chains. Algorithm 9 shows how each array $A_{t_1}^{t_2}$ is queried and updated, following the reachability queries and edge insertions in CSSTs. The functions $\text{successor}(\langle t_1, j_1 \rangle, t_2)$, $\text{predecessor}(\langle t_1, j_1 \rangle, t_2)$ and $\text{reachable}(\langle t_1, j_1 \rangle, \langle t_2, j_2 \rangle)$ handle successor, predecessor, and reachability queries in a straightforward manner: since each $A_{t_1}^{t_2}$ is transitively closed, these operations reduce to suffix-minima queries on $A_{t_1}^{t_2}$.

The function $\text{insertEdge}(\langle t_1, j_1 \rangle, \langle t_2, j_2 \rangle)$ is somewhat more complex, to guarantee that the arrays $A_{t_1}^{t_2}$ indeed store transitive reachability across chains. To achieve this, besides inserting the direct edge $\langle t_1, j_1 \rangle \rightarrow \langle t_2, j_2 \rangle$, this function performs a transitive closure computation by finding the predecessor $\langle t'_1, j'_1 \rangle$ of $\langle t_1, j_1 \rangle$ (Lines 10–11) and the successor $\langle t'_2, j'_2 \rangle$ of $\langle t_2, j_2 \rangle$ (Lines 12–13) in all pairs of chains t'_1, t'_2 . Then, an edge $\langle t'_1, j'_1 \rangle \rightarrow \langle t'_2, j'_2 \rangle$ is inserted unless a path between these nodes already exists.

Example 18. *Consider the chain DAG in Figure 6.9a and the operation $\text{insertEdge}(\langle 1, 1 \rangle, \langle 2, 0 \rangle)$. We obtain the transitive path $\langle 0, 1 \rangle \rightarrow^* \langle 3, 2 \rangle$ as follows. First, the predecessor of $\langle 1, 1 \rangle$ in thread 0 is identified via the query $\text{predecessor}(\langle 1, 1 \rangle, 0)$. This query performs an argleq operation on A_0^1 (Figure 6.9b) with value 1, returning node $\langle 0, 1 \rangle$. Then, the successor of $\langle 2, 0 \rangle$ in thread 3 is identified via the query $\text{successor}(\langle 2, 0 \rangle, 3)$. This query performs a min operation on A_2^3 (Figure 6.9c) with index 0, and returns the node $\langle 3, 2 \rangle$. Next, the update operation on A_0^3 adds the transitive edge, resulting in A_0^3 in Figure 6.9d.*

Analysis. The correctness of incremental CSSTs is based on the following invariants. The first concerns soundness, i.e., every entry in each array $A_{t_1}^{t_2}$ corresponds to a path between the respective nodes in chains t_1 and t_2 . The second concerns completeness, i.e., if there is a path $\langle t_1, j_1 \rangle \rightarrow^* \langle t_2, j_2 \rangle$ between two nodes, it is captured as a suffix minima in the array $A_{t_1}^{t_2}$. In conjunction, they imply the correctness of successor , predecessor , and reachable queries implemented as min and argleq operations on the corresponding array $A_{t_1}^{t_2}$.

Algorithm 9: Incremental CSSTs.

```

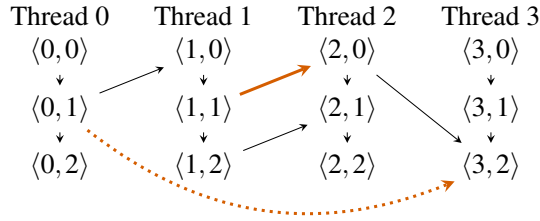
1 function successor( $\langle t_1, j_1 \rangle, t_2$ )
2   return  $\min(A_{t_1}^{t_2}, j_1)$ 

3 function predecessor( $\langle t_1, j_1 \rangle, t_2$ )
4   return  $\text{argleq}(A_{t_2}^{t_1}, j_1)$ 

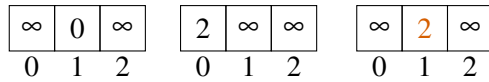
5 function reachable( $\langle t_1, j_1 \rangle, \langle t_2, j_2 \rangle$ )
6   if successor( $\langle t_1, j_1 \rangle, t_2$ )  $\leq j_2$  then return True else return False

7 function insertEdge( $\langle t_1, j_1 \rangle, \langle t_2, j_2 \rangle$ )
8   foreach  $t'_1 \in [k]$  do
9     foreach  $t'_2 \in [k] \setminus \{t'_1\}$  do
10      if  $t'_1 = t_1$  then Let  $j'_1 \leftarrow \langle t_1, j_1 \rangle$ 
11      else Let  $j'_1 \leftarrow \text{predecessor}(\langle t_1, j_1 \rangle, t'_1)$ 
12      if  $t'_2 = t_2$  then Let  $j'_2 \leftarrow \langle t_2, j_2 \rangle$ 
13      else Let  $j'_2 \leftarrow \text{successor}(\langle t_2, j_2 \rangle, t'_2)$ 
14      if successor( $\langle t'_1, j'_1 \rangle, t'_2$ )  $> j'_2$  then update( $A_{t'_1}^{t'_2}, j'_1, j'_2$ )

```



(a) A chain DAG G .



(b) Array A_0^1 . (c) Array A_2^3 . (d) Array A_0^3 .

Figure 6.9: Inserting an edge $\langle 1, 1 \rangle \rightarrow \langle 2, 0 \rangle$ and inferring the transitive path $\langle 0, 1 \rangle \rightarrow^* \langle 3, 2 \rangle$.

Lemma 9 (Soundness). $A_{t_1}^{t_2}[j_1] = j_2 \implies \langle t_1, j_1 \rangle \rightarrow^* \langle t_2, j_2 \rangle$.

Lemma 10 (Completeness). $\langle t_1, j_1 \rangle \rightarrow^* \langle t_2, j_2 \rangle$ and $t_1 \neq t_2 \implies \exists j'_1 \geq j_1. A_{t_1}^{t_2}[j'_1] \leq j_2$.

Recall that the key feature of SSTs is their ability to handle sparse arrays efficiently. As incremental CSSTs insert transitive edges in the arrays $A_{t_1}^{t_2}$, this might increase their density, adversely affecting the underlying SST, as we saw in Section 6.3. At

closer inspection, however, the density of each $A_{t_1}^{t_2}$ cannot grow beyond the cross-chain density of G .

Lemma 11 (Sparsity). *Let d be the cross-chain density of G . Then the density of each array $A_{t_1}^{t_2}$ is bounded by d .*

Combined with our bounds on the height of SSTs (Lemma 5 in Section 6.3), Lemma 11 implies that the height of the SST representing $A_{t_1}^{t_2}$ is bounded both by $\log n$ and by the cross-chain density of G . We thus arrive at Theorem 3.



Figure 6.10: The geometric mean of the ratio of time and memory used in each benchmark by VCs and STs over incremental CSSTs (left), and Graphs over fully dynamic CSSTs (right), in the dataset of the corresponding analysis.

Space usage. Due to Lemma 11, the total space used by the suffix minima arrays $A_{t_1}^{t_2}$ is $O(dk)$, by an analysis similar to that of fully dynamic CSSTs in Line 31. This time, CSSTs does not use heaps to store all edges, thus the total space used is $O(dk)$. Vector Clocks and STs [236], on the other hand, use $O(nk)$ space, which is larger in practice.

6.5 Experimental Evaluation

Here we incorporate CSSTs in a number of dynamic analyses from recent literature and evaluate their performance.

Experimental Setup

Data structures. Each analysis dynamically maintains a partial order P using various representations. We explore different standard ways for this purpose, as follows.

- **Graphs** is a standard graph representation of P , that is not transitively closed.
- **VCs** is a standard Vector Clock representation of P , that is (by design) transitively closed [205].
- **STs** is the data structure in [236] based on Segment Trees.
- **CSSTs**, as introduced in this paper.

Most analyses make only incremental updates, in which case Vector Clocks are preferred over plain graphs. Thus in this setting we compare CSSTs to VCs and ST. On the other hand, VCs and ST cannot handle decremental updates, hence in the fully dynamic setting we compare CSSTs to Graphs. The data structure used in the respective paper is marked by †. As some tools are not public, we implemented their analysis following the corresponding paper.

Optimizations. To set the threshold b for the size of the block nodes in CSSTs (Section 6.3), we perform a randomized stress test with varying sizes of b , and measure their performance. Based on this test, we set $b = 32$.

We also implement two optimizations in VCs. First, each new edge $e_1 \rightarrow e_2$ must be propagated to the successors of e_2 in its own chain. We stop this propagation as soon as we find some e'_2 with already $e_1 \rightarrow^* e'_2$. Second, for each chain, we do not create VCs for the suffix of its events that do not have any direct orderings from other chains, as ordering queries on such nodes can be inferred from their predecessors.

Setup. In each analysis, we use the dataset of the corresponding paper and relevant literature, and report on benchmarks on which some data structure runs in ≥ 1 s. We use an Ubuntu 22.04 machine with 2.4GHz CPU and 64GB of memory.

Experimental Results

We begin with a macroscopic view in Figure 6.10, showing the (geometric) average improvement in time and memory that CSSTs offer over VCs, STs and Graphs in each analysis. We see that in nearly all cases, these ratios are above 1, meaning that CSSTs indeed outperform the other data structures. The only exception concerns data races in C11; we will analyze this case in more detail later. The improvement is even more pronounced on the last analysis (linearizability), which is fully dynamic and thus the only baseline is plain Graphs.

Although the worst-case memory usage of CSSTs is $O(nk)$, and thus the same as VCs and STs, in practice it is less. This confirms our insights that typical analyses give rise to sparse chain DAGs, which is then exploited by CSSTs. We will take a closer look into the sparsity of each benchmark later.

We remark that the resource (time, memory) reported in each benchmark concerns the whole analysis, not just the corresponding data structure*. The actual resource improvement achieved by CSSTs is significantly larger.

*The analysis on data races for C11 is an exception, as we explain later.

1. Data race prediction. We start with the M2 data race detector [236]. This analysis observes traces σ which might be data-race free, and attempts to permute them to new traces σ' that are valid (called correct reorderings) and expose a data race. Internally, M2 utilizes STs for maintaining its partial order P . The reachability queries are incremental. We measure the total time taken in the analysis.

Table 6.1 shows the performance of M2 with each data structure. VCs become unscalable early, and are outperformed by STs, which were indeed developed to address this scalability issue. We observe that CSSTs are the most performant data structure on each individual benchmark. Their advantage over STs is primarily due to the efficient way that CSSTs handle sparsity (Section 6.3). Indeed, column q reports the mean density among each suffix minima array inside CSSTs when it obtained its densest form, and matches our expectations that is typically small. Our sparse treatment not only reduces the runtime of CSSTs but also their memory footprint, and thus support the analysis even when VCs and STs go out of memory (on xalan and batik).

As a side note, observe that the running time of each method is not always increasing when we move to larger traces (eg, all data structures take more time to process moldyn, with size $N = 200.3K$ than jigsaw, with size $3.1M$). This is because moldyn has more potential data races that the analysis needs to check, despite its smaller size. Similar, non-monotonic, patterns appear in other analyses as well.

Table 6.1: Race prediction results. The timeout is 5h.

benchmark	T	N	q	VCs (s)	STs [†] (s)	CSSTs (s)
clean	12	1.3K	0.28	2.4	2.6	1.6
bubblesort	29	4.7K	0.15	69.4	56.2	27.5
lang	10	6.3K	0.22	93.3	39.2	27.3
readerswriters	8	11.3K	0.32	54.4	23.5	18.5
raytracer	6	15.8K	0.17	10.5	10.2	9.3
bufwriter	9	22.3K	0.2	130	24.6	12.9
ftpserver	14	49.6K	0.06	41.2	14.1	9.0
moldyn	6	200.3K	0.12	T.O.	T.O.	12636
linkedlist	15	1.0M	0.06	T.O.	T.O.	8893
derby	7	1.4M	0.04	1436	215	197
jigsaw	15	3.1M	0.06	154	45.6	32.0
sunflow	17	11.7M	0.01	T.O.	780	505
xalan	9	122.5M	0.01	T.O.	O.O.M.	979
batik	8	157.9M	0.01	O.O.M.	O.O.M.	1956
Total	-	297.9M	-	>91992	>73211	25304

2. Deadlock prediction. Here we incorporate CSSTs in SeqCheck, a dynamic deadlock prediction analysis [54]. This analysis identifies potential deadlocks by analyzing lock-acquisition orders between threads, and then try to witness each deadlock by a valid reordering of the observed trace. For this purpose, it incrementally maintains a partial order which will eventually be linearized to this valid reordering.

Our results are shown in Table 6.2. VCs are again the slowest performer by far, though they are very competitive on a few benchmarks for which deadlocks can be detected fairly early in the analysis. Naturally, STs and CSSTs perform similarly on

these benchmarks as well. On the more demanding benchmarks, however, CSSTs is clearly the best performer. We also observe that the average density q is small.

Table 6.2: Deadlock results.

benchmark	T	N	q	VCs (s)	STs [†] (s)	CSSTs (s)
jigsaw	21	143.0K	0.03	356	14.8	7.8
elevator	5	245.9K	0.02	3.7	2.8	2.6
hedc	7	409.8K	0.04	23.2	22.0	23.2
JDBCMySQL	3	442.9K	0.01	3.7	4.1	3.5
cache4j	2	775.5K	0.01	5.5	6.6	5.6
Swing	8	3.8M	0.01	14.8	13.4	13.1
sunflow	15	21.5M	0.01	369	111	110
eclipse	15	64.2M	0.01	535	711	282
Total	-	91.4M	-	1311	886	448

3. Memory bug prediction. Here we incorporate CSSTs in ConVulPOE, a dynamic analysis targeting memory bugs due to concurrency [317], by reordering observed traces. The reachability queries are incremental. We measured the total time taken in the analysis.

Our results are shown in Table 6.3. We again observe that VCs are the slowest data structure to support this analysis. While STs offer a generous speedup, CSSTs are again the most performant, and even manage to handle benchmarks that makes VCs time out and STs run out of memory.

Table 6.3: Memory bug prediction results. The timeout is 5h.

benchmark	T	N	q	VCs (s)	STs [†] (s)	CSSTs (s)
pbzip2	7	243.5K	0.06	298	6.7	3.8
pigz	6	2.3M	0.01	T.O.	2087	1265
xz	2	3.0M	0.01	15.1	13.9	12.9
lbzip2	11	9.3M	0.04	273	96.6	46.9
x264	7	10.4M	0.03	474	165	62.0
libvpx	2	19.0M	0.01	440	83.6	77.6
libwebp	2	29.5M	0.1	324	118	115
x265	15	37.1M	0.02	T.O.	O.O.M.	627
Total	-	110.9M	-	>37824	>20570	2211

4. Consistency checking in x86-TSO. Here we report our results for consistency checking under the x86-TSO memory model. Though the problem is NP-complete, polynomial-time heuristics are known to be remarkably accurate. We experiment with the consistency analysis developed in [252]. In contrast to the previous analyses where the underlying chain DAG had one chain per thread, here we have two chains per thread, one for its program order and one for its store buffer. The reachability queries are incremental. We measure the total time taken in the analysis.

The results are shown in Table 6.4. VCs are very slow compared to the other two data structures. Indeed, this analysis is quite demanding, as it performs repeated updates between events that are in the middle of the partial order, which leads to deep edge propagations. Although STs perform considerably better than VCs, CSSTs are

decisively faster in this setting too. Finally, the average density q is larger here than in previous analyses, but still considerably below 1.

Table 6.4: Consistency checking results. The timeout is 1h.

benchmark	T	N	q	VCs (s)	STs (s)	CSSTs (s)
dekker	3	16.3K	0.41	6.9	0.3	0.25
peterson	3	19.0K	0.37	9.4	0.4	0.45
lamport	3	28.1K	0.39	18.3	0.6	0.6
dq	4	31.5K	0.24	11.5	0.5	0.44
chase-lev	5	32.2K	0.24	23.3	0.6	0.51
szymanski	3	77.9K	0.33	17.4	0.9	0.8
buf-ring	9	115.3K	0.39	322	7.3	6.27
mcs-lock	11	196.4K	0.17	298	28.3	20.71
spsc	3	243.6K	0.53	2548	2.1	1.74
linuxrwlocks	6	276.4K	0.28	T.O.	12.6	11.18
fib-bench	3	300.0K	0.4	T.O.	4.7	4.21
seqlock	17	318.1K	0.15	1227	46.1	28.16
spinlock	11	482.5K	0.39	T.O.	103	75.11
ttaslock	11	491.1K	0.41	T.O.	111	81.19
exp-bug	4	498.5K	0.38	2591	3.6	2.76
mutex	11	519.7K	0.56	T.O.	122	86.09
ticketlock	6	569.6K	0.4	T.O.	20.8	17.17
gcd	3	750.1K	0.28	T.O.	6.0	4.52
indexer	17	800.0K	0.51	7.8	5.8	3.23
twalock	11	900.0K	0.43	T.O.	174	118.39
treiber	6	1.0M	0.22	T.O.	20.7	16.73
mpmc	10	2.0M	0.17	T.O.	41.6	22.59
barrier	5	2.8M	0.6	T.O.	139	112.61
Total	-	12.5M	-	>46680	852	616

5. Use-after-free prediction. Here we incorporate CSSTs in the UFO analysis for use-after-free vulnerabilities due to concurrency [137]. This analysis is based on SMT, but relies on efficient partial-order reasoning to generate SMT queries. The reachability queries are incremental. We measure the time to generate the queries. Our results are shown in Table 6.5.

In alignment with our observations so far, CSSTs outperform VCs and STs on all benchmarks. However the speedup is not as large as in other analyses. Looking closer into the running times, we observe that the analysis itself spends considerable time in components other than maintaining reachability, which are common for all data structures. If we only focus on the time taken for maintaining the partial order, CSSTs indeed offer a significant speedup, e.g., $3.5\times$ and $1.7\times$ in `BoundedBuffer` over VCs and STs, respectively.

6. Data-race detection in C11. Here we incorporate CSSTs in `C11Tester`, a data-race detector for the C11 memory model [191]. The analysis constructs incrementally a trace σ , by iteratively mapping a read to a write to obtain its value from. To make these choices consistent with the C11 memory model, it maintains a partial order P using VCs. The reachability queries are incremental. To ensure that all data structures process the same trace, we run them alongside each other on a single execution, and measure the time taken to maintain the partial order in each.

Table 6.5: Use-after-free results. The timeout is 5h.

benchmark	T	N	q	$VCs^{\dagger}$ (s)	STs (s)	CSSTs (s)
bbuf	3	27.7K	0.02	426	459	415
BoundedBuffer	11	325.7K	0.02	835	736	679
DiningPhil	21	1.4M	0.01	1559	1191	1110
fanger01-ok	5	93.4K	0.06	366	194	190
qtsort	6	41.7M	0.01	104	48.2	31.9
pbzip	5	269.3K	0.01	T.O.	15894	14471
Total	-	43.8M	-	>21290	18521	16898

Our results are shown in Table 6.6. Interestingly, VCs are generally more performant than STs and CSSTs in this setting. Looking closer into each benchmark, we observe that new orderings lead to very little (typically none at all) propagation in P . Effectively, this makes VCs run in $O(1)$ time per update, and thus gain an advantage. In contrast, in readerswriters and atomicblocks C11Tester inserts non-trivial orderings in P (i.e., orderings that lead to propagation), and CSSTs are considerably more performant than VCs. Finally, the average density q is high in a few benchmarks, reaching even the value 1, which explains why CSSTs and STs have comparable performance, though CSSTs remain more performant.

Table 6.6: Race detection on C11.

benchmark	T	N	q	$VCs^{\dagger}$ (s)	STs (s)	CSSTs (s)
dq	5	72.8K	0.48	2.5	2.8	3.1
mabain	7	101.6K	0.33	0.6	1.2	1.1
seqlock	18	693.9K	0.2	4.5	6.7	6.3
iris-1	13	1.1M	0.2	54.8	66.5	60.3
qu	11	1.3M	0.43	5.9	7.5	7.2
indexer	18	1.6M	1	1.0	1.1	1.1
exp-bug	5	2.5M	0.25	2.6	2.5	2.5
twalock	12	4.5M	0.34	29.6	53.3	49.0
gcd	4	4.5M	1	2.6	2.8	2.7
spinlock	12	4.8M	0.2	23.0	37.9	34.0
ttaslock	12	4.9M	0.25	26.7	48.0	42.7
silo	5	5.6M	0.01	5.9	6.2	6.1
fib-bench	4	6.0M	1	6.5	6.3	6.5
linuxrwlocks	7	6.9M	0.39	30.8	40.6	37.7
barrier	6	8.3M	0.51	19.6	24.5	23.7
mpmc	11	9.2M	0.24	37.6	44.9	42.6
spsc	4	9.7M	0.55	12.2	12.2	12.1
mcs-lock	12	9.9M	0.37	55.2	86.8	80.1
treiber	7	10.1M	0.11	31.3	36.6	35.7
iris-2	4	11.5M	0.2	15.5	14.7	14.7
gdax	8	13.5M	0.97	8.5	7.6	7.6
ticketlock	7	14.2M	0.48	45.1	70.4	64.2
mutex	12	15.6M	0.39	48.5	68.5	63.7
readerswriters	13	10.1M	0.33	16.3	7.6	7.5
atomicblocks	33	15.5M	0.5	9.7	1.7	1.6
Total	-	172.4M	-	496	659	614

7. Root-causing linearizability violations. Here we incorporate CSSTs in a recent

root-cause analysis for linearizability violations [78]. The reachability queries are both incremental and decremental, hence the only baseline for maintaining the partial order are plain Graphs, as indeed used in [78]. We measure the total time taken in the analysis.

Our results are shown in Table 6.7. The experimental setup of [78] consists of three data structures, accessed an increasing number of times. CSSTs are typically orders of magnitude faster than plain Graphs used in [78], which strongly supports CSSTs as an efficient data structure also for analyses utilizing decremental partial-order updates.

Table 6.7: Root causing linearizability violation results.

benchmark	T	N	q	Graphs [†] (s)	CSSTs (s)
LogicalOrderingAVL	3	615	0.1	1.2	0.4
	3	1.6K	0.07	6.1	1.6
	3	2.6K	0.06	13.5	3.8
	3	5.0K	0.05	49.5	10.8
OptimisticListSorted-SetWaitFreeContains	3	219	0.19	1.4	0.2
	3	399	0.18	9.8	0.4
	3	759	0.17	114	0.9
	3	1.1K	0.17	512	1.7
RWLockCoarse-GrainedListIntSet	3	978	0.04	3.6	0.8
	3	1.6K	0.03	11.4	1.7
	3	3.2K	0.03	73.6	5.3
	3	4.8K	0.03	249	10.1
Total	-	22.9K	-	1045	38

Scalability Analysis

Here we perform controlled scalability experiments on the performance of CSSTs for edge insertions and queries. We consider partial orders P in two settings, consisting of $k = 10, 20$ chains, and varying number of events ℓ in each chain. Initially, P has no cross-chain edges. First, to measure the performance of edge insertions, we repeatedly insert random cross-chain edges $\langle t, i \rangle \mapsto \langle t', j \rangle$ such that the two endpoints are unordered and $|i - j| \leq b$. The last condition captures the fact that cross-chain orderings are typically between events that execute within the same time-window. We report on window $b = 10^4$, though we have observed similar results with other values of b . This condition also prevents P from becoming close to total. We attempt to insert 20ℓ edges in each case, and report the average time of edge insertions. Second, to measure the performance of edge queries, we use the partial order P at the end of the above process. We make 10^6 random queries, and report the average time per query.

The results are shown in Figure 6.11. As expected by theory, VCs suffer linear complexity for insertions but achieve queries in constant time. On the other hand, CSSTs and STs achieve logarithmic complexity in both insertions and deletions. CSSTs are far more performant than VCs in edge insertions, but also achieve query times that are similar to VCs, though, naturally, VCs are slightly faster, as each query is a simple lookup. This is achieved by our minima indexing and sparse representation,

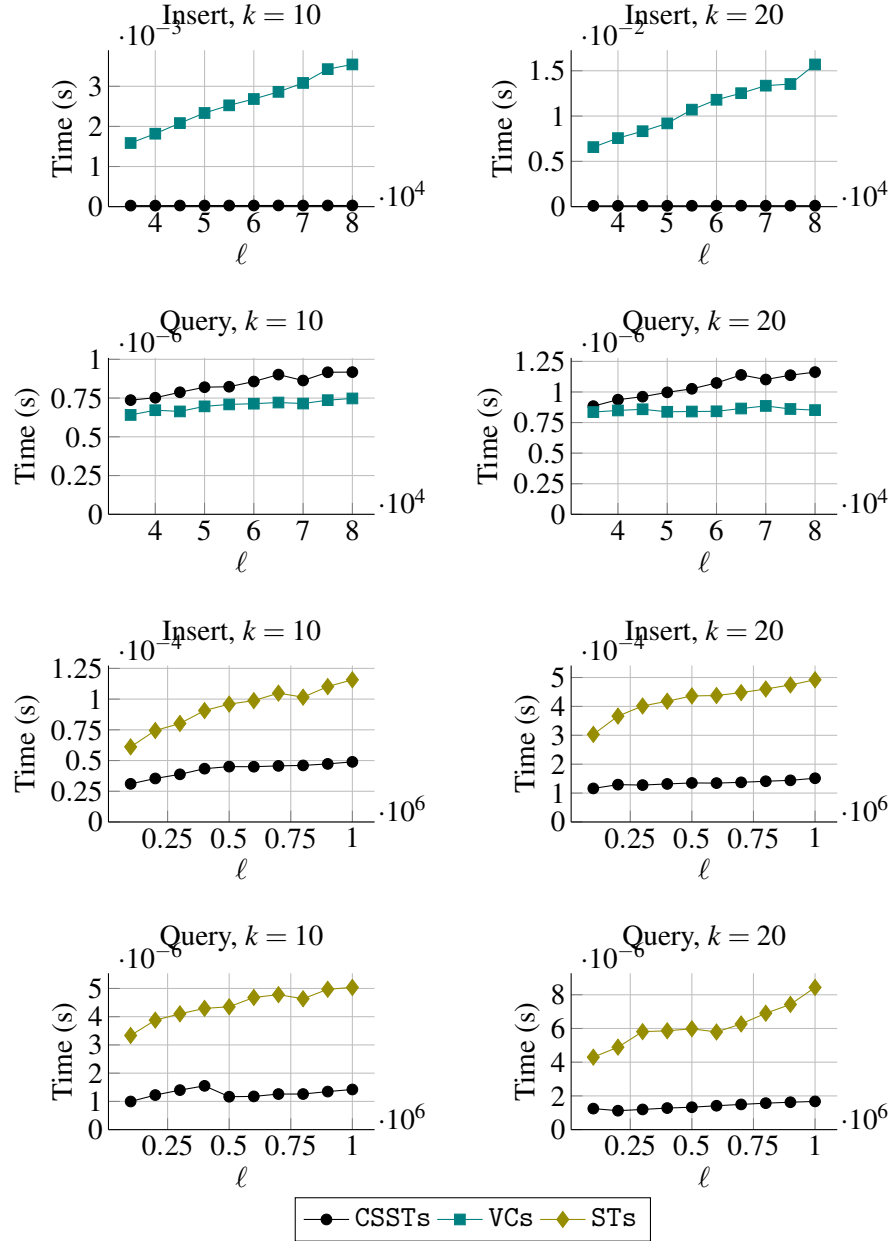


Figure 6.11: Scalability experiments on CSSTs, STs and VCs. Each partial order contains k chains and $\ell = n/k$ events per chain.

which makes queries run with minimal overhead in practice. Finally, CSSTs are considerably faster than STs in both insertions and queries, an improvement stemming from the Sparse Segment Tree (Section 6.3).

6.6 Related Work

The efficient maintenance of partial orders is a key component in concurrent program analysis. Vector Clocks [205] were originally proposed as an efficient mechanism for causality tracking in distributed systems [171], and have been used extensively in program analysis. Their efficient implementation has been approached in various ways, e.g., using varying-size arrays [77], AVL trees [179], Chain Clocks [11], and Tree Clocks [203]. These variations offer performance improvements over vanilla Vector Clocks, but only when the partial order is built incrementally and in a streaming fashion.

For non-streaming updates, partial orders are normally represented as plain graphs [9, 36, 78, 226, 247, 321] and Vector Clocks [121, 191]. The closest data structure to CSSTs has been STs, developed in [236] for data-race detection, and subsequently used in other dynamic analyses [54, 273, 317]. The development of CSSTs lies on a few technical novelties. For example, our minima indexing and sparse tree representation (Section 6.3) are novel, and lead to tighter complexity bounds. Moreover, in order to handle the fully dynamic setting, CSSTs do not store transitive reachability. This requires overcoming the challenge of discovering transitive reachability during queries in an efficient manner (Line 31).

6.7 Conclusion

We have introduced CSSTs, a data structure for the fully-dynamic maintenance of partial orders with small width, supporting a plethora of dynamic analyses for concurrency. Our experimental results indicate that CSSTs are an efficient, drop-in replacement of existing approaches to maintaining partial orders in various application domains of recent literature. Our aim is for CSSTs to become a standard reference for driving future scalable dynamic analyses.

Chapter 7

Sound Dynamic Deadlock Prediction in Linear Time

By Hünkar Can Tunç, Umang Mathur, Andreas Pavlogiannis, Mahesh Viswanathan.
Proceedings of the ACM on Programming Languages, Volume 7, Issue PLDI, June, 2023.

Abstract

Deadlocks are one of the most notorious concurrency bugs, and significant research has focused on detecting them efficiently. *Dynamic predictive analyses* work by observing concurrent executions, and reason about alternative interleavings that can witness concurrency bugs. Such techniques offer scalability and sound bug reports, and have emerged as an effective approach for concurrency bug detection, such as data races. Effective dynamic deadlock prediction, however, has proven a challenging task, as no deadlock predictor currently meets the requirements of *soundness*, *high-precision*, and *efficiency*.

In this paper, we first formally establish that this tradeoff is unavoidable, by showing that (a) sound and complete deadlock prediction is intractable, in general, and (b) even the seemingly simpler task of determining the presence of *potential* deadlocks, which often serve as unsound witnesses for actual predictable deadlocks, is intractable. The main contribution of this work is a new class of predictable deadlocks, called *sync(hronization)-preserving* deadlocks. Informally, these are deadlocks that can be predicted by reordering the observed execution while preserving the relative order of conflicting critical sections. We present two algorithms for *sound* deadlock prediction based on this notion. Our first algorithm SPD0ffline detects all sync-preserving deadlocks, with running time that is linear per *abstract deadlock pattern*, a novel notion also introduced in this work. Our second algorithm SPD0nline predicts all sync-preserving deadlocks that involve two threads in a *strictly online* fashion, runs in overall linear time, and is better suited for a runtime monitoring setting.

We implemented both our algorithms and evaluated their ability to perform offline and online deadlock-prediction on a large dataset of standard benchmarks. Our results indicate that our new notion of sync-preserving deadlocks is highly

effective, as (i) it can characterize the vast majority of deadlocks and (ii) it can be detected using an online, sound, complete and highly efficient algorithm.

7.1 Introduction

The verification of concurrent programs is a major challenge due to the non-deterministic behavior intrinsic to them. Certain scheduling patterns may be unanticipated by the programmers, which may then lead to introducing concurrency bugs. Such bugs are easy to introduce during development but can be very hard to reproduce during in-house testing, and have been notoriously called *heisenbugs* [218]. Among the most notorious concurrency bugs are deadlocks, occurring when the system blocks its execution because each thread is waiting for another thread to finish a task in a circular fashion. Deadlocks account for a large fraction of concurrency bugs in the wild across various programming languages [189, 289] while they are often introduced accidentally when fixing other concurrency bugs [315].

Deadlock-detection techniques can be broadly classified into static and dynamic techniques. As usual, static techniques analyze source code and have the potential to prove the absence of deadlocks [183, 219, 224]. However, as static analyses face simultaneously two dimensions of non-determinism, namely in inputs and scheduling, they lead to poor performance in terms of scalability and false positives, and are less suitable when the task at hand is to help software developers proactively find bugs. Dynamic analyses, on the other hand, have the more modest goal of discovering deadlocks by analyzing program executions, allowing for better scalability and few (or no) false positives. Although dynamic analyses cannot prove the absence of bugs, they offer *statistical* and *coverage* guarantees. These advantages have rendered dynamic techniques a standard practice in principled testing for various bugs, such as data races, atomicity violations, deadlocks, and others [32, 36, 111, 117, 198, 237, 259, 267]. A recent trend in this direction advocates for *predictive analysis* [117, 122, 137, 138, 154, 157, 276], where the goal is to enhance coverage by additionally reasoning about alternative reorderings of the observed execution trace that *could* have taken place and also manifest the bug.

Due to the difficulty of the problem, many dynamic deadlock analyses focus on detecting *deadlock patterns*, broadly defined as cyclic lock-acquisition patterns in the observed execution trace. One of the earliest works in this direction is the Goodlock algorithm [130]. As deadlock patterns are necessary but insufficient conditions for the presence of deadlocks, subsequent work has focused on refining this notion in order to reduce false-positives [13, 32]. Further techniques reduce the size of the lock graph to improve scalability [51, 53]. To further address the unsoundness (false positives) problem, various works propose controlled-scheduling techniques that attempt to realize deadlock warnings via program re-execution [33, 151, 257, 258, 280] and exhaustive exploration of all reorderings [152, 264].

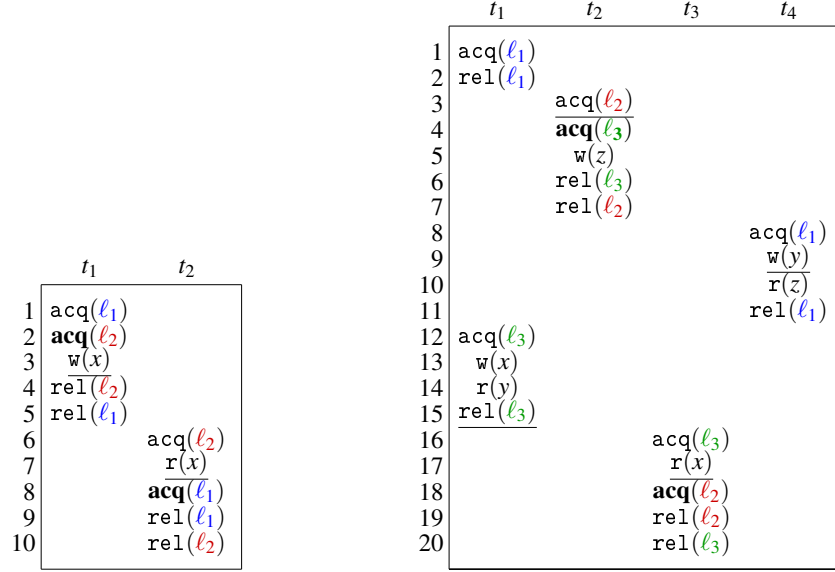
Fully sound deadlock prediction has traditionally relied on explicitly [152, 264] or symbolically (SMT-based) [103, 154] producing all sound witness reorderings.

The heavyweight nature of such techniques limits their applicability to executions of realistic size, which is often in the order of millions of events. The first steps for sound, polynomial-time deadlock prediction were made recently with SeqCheck [54], an extension of M2 [236] that targets data races.

This line of work highlights the need for a most-efficient sound deadlock predictor, approaching the golden standard of *linear time*. Moreover, dynamic analyses are often employed as runtime monitors, and must thus operate *online*, reporting bugs as soon as they occur. Unfortunately, most existing online algorithms only report *deadlock patterns*, thus suffering false positives. The lack of such a deadlock predictor is even more pronounced when contrasted to the problem of dynamic race prediction, which has seen a recent surge of sound, online, *linear-time* predictors (e.g., [157, 248]), and highlights the bigger challenges that deadlocks entail. We address these challenges in this work, by presenting the first high-precision, sound dynamic deadlock-prediction algorithm that works online and in linear time.

The task of checking if a potential deadlock is a real predictable deadlock, in general, involves searching for the reordering of the original execution that witnesses the deadlock. The first ingredient towards our technique is the notion of *synchronization-preserving reorderings* [202] that help systematize this search space. *Synchronization-preserving deadlocks* are then those predictable deadlocks that can be witnessed in some synchronization-preserving reordering. We will illustrate synchronization-preserving deadlocks using an example in Section 7.1.

This notion of synchronization-preservation, by itself, is not sufficient when it comes to deadlock detection as the prerequisite step towards predicting deadlocks also involves identifying *potential deadlock patterns*. Unlike data races, where *potential races* can be identified in polynomial-time, the identification of deadlock patterns is in general, intractable; we prove this in Section 7.3. As a result, an approach that works by explicitly enumerating cycles in a *lock graph* and then checking if any of these cycles is realizable to a deadlock is likely to be not scalable. To tackle this, we propose the novel notion of *abstract deadlock patterns* which, informally, represent clusters of deadlock patterns of the same signature. Intuitively, a set of deadlock patterns have the same signature if the threads and locks that participate in the patterns are the same. Our next *key observation* is that a single abstract deadlock pattern can be checked for sync-preserving deadlocks in *linear total time* in the length of the execution, *regardless* of how many concrete deadlock patterns it represents. Our first deadlock prediction algorithm SPDOffline builds upon this — it enumerates all abstract deadlock patterns in a first phase and then checks their realizability in a second phase, while running in linear time per abstract deadlock pattern. Since the number of abstract deadlock patterns is typically *far smaller* than the number of (concrete) deadlock patterns (see Table 7.1 in Section 7.6), this approach achieves high scalability. Our second algorithm SPDOnline works in a single streaming pass — it computes abstract deadlock patterns that involve only two threads and checks their realizability *on-the-fly* simultaneously in overall linear time in the length of the execution.



(a) A trace σ_1 with no predictable deadlock.

(b) A trace σ_2 with a sync-preserving deadlock, stalling t_2 on e_4 and t_3 on e_{18} .

Figure 7.1: Traces with no predictable deadlock (a), and with a sync-preserving deadlock (b).

Synchronization-Preserving Deadlocks

Consider the trace σ_1 in Figure 7.1a consisting of 10 events and two threads. We use e_i to denote the i -th event of σ_1 . The events e_2 and e_8 form a *deadlock pattern*: they respectively acquire the locks ℓ_2 and ℓ_1 while holding the locks ℓ_1 and ℓ_2 , and no common lock protects these operations.

A deadlock pattern is a necessary but insufficient condition for an actual deadlock: a sound algorithm must examine whether it can be realized to a deadlock via a witness. A witness is a reordering ρ of (a slice of) σ_1 that is also a valid trace, and such that e_2 and e_8 are locally enabled in their respective threads at the end of ρ . In general, the problem of checking if a deadlock pattern can be realized is intractable (Theorem 6). In this work we focus on checking whether a given deadlock pattern forms a *sync-preserving deadlock*, which is a subclass of the class of all predictable deadlocks.

A deadlock pattern is said to be sync-preserving deadlock if it can be witnessed in a *sync-preserving reordering*. A reordering ρ^{SP} of a trace σ is said to be sync-preserving if it preserves the control flow taken by the original observed trace σ , and further it preserves the mutual order of any two critical sections (on the same lock) that appear in the reordering ρ^{SP} . Consider, for example, the sequence $\rho_1 = e_1..e_3e_6e_7$ where $e_i..e_j$ denote the contiguous sequence of events that starts from e_i and ends at e_j . We call ρ_1 a *correct reordering* of σ_1 , being a slice of σ_1 closed under the thread order and preserving the writer of each read in σ_1 ; the precise definition is

presented in Section 7.2. In this case, however, ρ_1 does not witness the deadlock as the event e_2 is not *enabled* in ρ_1 . In fact, due to the dependency between the events e_3 and e_7 , there are no correct reorderings of σ_1 which make both e_2 and e_8 enabled. This makes the deadlock pattern $\langle e_2, e_8 \rangle$ non-predictable. Consider now σ_2 in Figure 7.1b, and the sequence $\rho_2 = e_3..e_7 e_8..e_{11} e_1 e_2$. Observe that ρ_2 is also a correct reordering. However, ρ_2 is not sync-preserving as the order of the two critical sections on lock ℓ_1 in ρ_2 is different from their original order in σ_2 . On the other hand, $\rho_3 = e_1 e_2 e_3 e_8 e_9 e_{12}..e_{15} e_{16} e_{17}$ is a correct reordering that is also sync-preserving — all pairs of critical sections on the same lock appear in the same order in ρ_3 as they did in σ_2 . Further, ρ_3 also witnesses the deadlock as the events e_4 and e_{18} are both *enabled* in ρ_3 . This makes the deadlock pattern $\langle e_4, e_{18} \rangle$ a sync-preserving deadlock.

In this work we show that sync-preserving deadlocks enjoy two remarkable properties. First, all sync-preserving deadlocks of a given *abstract* deadlock pattern can be checked in linear time. Second, our extensive experimental evaluation on standard benchmarks indicates that sync-preservation captures a vast majority of deadlocks in practice. In combination, these two benefits suggest that sync-preservation is the right notion of deadlocks to be targeted by dynamic deadlock predictors.

Our Contributions

In detail, the contributions of this work are as follows.

- (1) **Complexity of Deadlock Prediction.** Perhaps surprisingly, the complexity of detecting deadlock patterns, as well as predicting deadlocks has remained elusive. Our first contribution resolves such questions. Given a trace σ of size n and k threads, we first show that detecting even one deadlock *pattern* of length k is $W[1]$ -hard in k . This establishes that the problem is NP-hard, and further rules out algorithms that are fixed-parameter-tractable in k , i.e., with running time of the form $f(k) \cdot \text{poly}(n)$, for some function f . We next show that even with just $k = 2$ threads, the problem of detecting a single deadlock *pattern* (of size $k = 2$) admits a quadratic lower bound, i.e., it cannot be solved in time $O(n^{2-\varepsilon})$, no matter what $\varepsilon > 0$ we choose. These two results shed light on the difficulty in identifying deadlock patterns — a task that might otherwise appear easier than the core task of prediction. These hardness results, in particular the fine-grained lower bound result, are based on novel constructions, and results from fine-grained complexity [310]. Our third result is about confirming predictable deadlocks — even for a deadlock pattern of size $k = 2$, checking whether it yields a *predictable deadlock* is $W[1]$ -hard in the number of threads k (and thus again NP-hard), and is inspired from an analogous result in the context of data race prediction [200]. These results capture the intractability of deadlock prediction in general, even for the class of parametrized algorithms.
- (2) **Sync-preserving Deadlock Prediction and Abstract Deadlock Patterns.** Given the above hardness of predicting arbitrary deadlocks, we define a novel notion of sync(hronization)-preserving deadlocks, illustrated in Section 7.1. We develop

SPDOnline, an *online, sound* deadlock predictor that takes as input a trace and reports *all* sync-preserving deadlocks of size 2 in *linear time* $\tilde{O}(n)^*$. As most deadlocks in practice involve only two threads [189], restricting SPDOnline to size 2 deadlocks leads to linear-time deadlock prediction with small impact on its coverage. We also develop our more general algorithm, SPDOffline, that detects *all* sync-preserving deadlocks of all sizes. SPDOffline operates in two phases. In the first phase, it detects all *abstract deadlock patterns*. An abstract deadlock pattern is a novel notion that serves as a succinct representation of the class of deadlock patterns having the same signature. In the second phase, SPDOffline executes SPDOnline on each abstract pattern to decide whether a deadlock is formed. The running time of SPDOffline remains linear in n , but increases by a factor proportional to the number of abstract deadlock patterns in the lock graph.

- (3) **Implementation and Evaluation.** We have evaluated SPDOnline and SPDOffline in terms of performance and predictive power on a large dataset of standard benchmarks. In the offline setting, SPDOffline finds the same number of deadlocks as the recently introduced SeqCheck, while achieving a speedup of $> 200\times$ on the most demanding benchmarks, and $21\times$ overall. In the online setting, SPDOnline achieved a significant improvement in deadlock discovery and deadlock-hit-rate compared to the random scheduling based controlled concurrency testing technique of DeadlockFuzzer [151]. Our experiments thus support that the notion of sync-preserving deadlocks is suitable: (i) it captures the vast majority of the deadlocks in practice, and (ii) sync-preserving deadlocks can be detected online and optimally — that is, soundly, completely and in linear time, (iii) it can enhance the deadlock detection capability of controlled concurrency testing techniques, (iv) with reasonable runtime overhead.

7.2 Preliminaries

Here we set up our model and develop relevant notation, following related work in predictive analyses of concurrent programs [157, 248, 276].

Execution traces. A dynamic analysis observes traces generated by a concurrent program, and analyzes them to determine the presence of a bug. Each such trace σ is a linear arrangement of events Events_σ . An event $e \in \text{Events}_\sigma$ is tuple $e = \langle i, t, o \rangle$, where i is a unique identifier of e , t is the unique identifier of the thread performing e , and o is either a read or write ($o = \text{r}(x)$ or $o = \text{w}(x)$) operation to some variable x , or an acquire or release ($o = \text{acq}(\ell)$ or $o = \text{rel}(\ell)$) operation on some lock ℓ . For the sake of simplicity, we often omit i when referring to an event. We use $\text{tid}(e)$ and $\text{op}(e)$ to respectively denote the thread identifier and the operation performed in the

*We use $\tilde{O}$ to ignore *polynomial* appearance of trace parameters typically much smaller than n (e.g., number of threads).

event e . We use Threads_σ , Vars_σ and Locks_σ to denote the set of thread, variable and lock identifiers in σ .

We restrict our attention to *well-formed* traces σ , that abide to shared-memory semantics. That is, if a lock ℓ is acquired at an event e by thread t , then any later acquisition event e' of the same lock ℓ must be preceded by an event e'' that releases lock ℓ in thread t in between the occurrence of e and e' . Taking e'' to be the earliest such release event, we say that e and e'' are matching acquire and release events, and denote this by $e = \text{match}_\sigma(e'')$ and $e'' = \text{match}_\sigma(e)$. Moreover, every read event has at least one preceding write event on the same location, that it reads its value from.

Functions and relations on traces. A trace σ implicitly defines some relations. The *trace-order* $\leq_{\text{tr}}^\sigma \subseteq \text{Events}_\sigma \times \text{Events}_\sigma$ orders the events of σ in a total order based on their order of occurrence in the sequence σ . The *thread-order* $\leq_{\text{TO}}^\sigma$ is the unique partial order over Events_σ such that $e \leq_{\text{TO}}^\sigma e'$ iff $\text{tid}(e) = \text{tid}(e')$ and $e \leq_{\text{tr}}^\sigma e'$. We say $e <_{\text{TO}}^\sigma e'$ if $e \leq_{\text{TO}}^\sigma e'$ but $e \neq e'$. The *reads-from* function rf_σ is a map from the read events to the write events in σ . Under sequential consistency, for a read event e on variable x , we have that $e' = \text{rf}_\sigma(e)$ be the latest write event on the same variable x such that $e' \leq_{\text{tr}}^\sigma e$. We say that a lock $\ell \in \text{Locks}_\sigma$ is held at an event $e \in \text{Events}_\sigma$ if there is an event e' such that (i) $\text{op}(e') = \text{acq}(\ell)$, (ii) $e' <_{\text{TO}}^\sigma e$, and (iii) either $\text{match}_\sigma(e')$ does not exist in σ , or $e \leq_{\text{TO}}^\sigma \text{match}_\sigma(e')$. We use $\text{HeldLks}_\sigma(e)$ to denote the set of all the locks that are held by $\text{tid}(e)$ right before e . The lock nesting depth of σ is $\max_{e \in \text{Events}_\sigma} |\text{HeldLks}_\sigma(e)| + 1$ where $\text{op}(e) = \text{acq}(\ell)$.

Deadlock patterns. A deadlock pattern[†] of size k in a trace σ is a sequence $D = \langle e_0, e_1, \dots, e_{k-1} \rangle$, with k distinct threads $t_0, \dots, t_{k-1}$ and k distinct locks $\ell_0, \dots, \ell_{k-1}$ such that $\text{tid}(e_i) = t_i$, $\text{op}(e_i) = \text{acq}(\ell_i)$, $\ell_i \in \text{HeldLks}_\sigma(e_{(i+1) \% k})$, and further, $\text{HeldLks}_\sigma(e_i) \cap \text{HeldLks}_\sigma(e_j) = \emptyset$ for every i, j such that $i \neq j$ and $0 \leq i, j < k$. A deadlock pattern is a necessary but insufficient condition of an actual deadlock, due to subtle synchronization or control and data flow in the underlying program.

Dynamic predictive analysis and correct reorderings. Dynamic analyses aim to expose bugs by observing traces σ of a concurrent program, often without accessing the source code. While such purely dynamic approaches enjoy the benefits of scalability, simply detecting bugs that manifest on σ offer poor coverage and are bound to miss bugs that appear in select thread interleavings [218]. Therefore, for better coverage, *predictive* dynamic techniques are developed. Such techniques predict the occurrence of bugs in alternate executions that can be *inferred* from σ , irrespective of the program that produced σ . The notion of such inferred executions is formalized by the notion of correct reorderings [264, 266, 276].

A trace ρ is a *correct reordering* of a trace σ if

- (1) $\text{Events}_\rho \subseteq \text{Events}_\sigma$,
- (2) for every $e, f \in \text{Events}_\sigma$ with $e \leq_{\text{TO}}^\sigma f$, if $f \in \text{Events}_\rho$, then $e \in \text{Events}_\rho$ and $e \leq_{\text{TO}}^\rho f$, and

[†] Similar notions have been used in the literature, sometimes under the term *deadlock potential* [130].

- (3) for every read event $r \in \text{Events}_\rho$, we have $\text{rf}_\sigma(r) \in \text{Events}_\rho$ and $\text{rf}_\rho(r) = \text{rf}_\sigma(r)$.

Intuitively, a correct reordering ρ of σ is a permutation of σ that respects the thread order and preserves the values of each read and write that occur in ρ . This ensures a key property — every program that generated σ is also capable of generating ρ (possibly under a different thread schedule), and thus ρ serves as a true witness of a bug.

Predictable deadlocks. We say that an event e is σ -enabled in a correct reordering ρ of σ if $e \in \text{Events}_\sigma$, $e \notin \text{Events}_\rho$ and for every $f \in \text{Events}_\sigma$ if $f <_{\text{TO}}^\sigma e$, then $f \in \text{Events}_\rho$. A deadlock pattern $D = \langle e_0, e_1 \dots e_{k-1} \rangle$ of size k in trace σ is said to be a predictable deadlock if there is a correct reordering ρ of σ such that each of $e_0, \dots, e_{k-1}$ are σ -enabled in ρ . This notion guarantees that the witness ρ is a valid execution of *any* concurrent program that produced σ . Analogous definitions have also been widely used for other predictable bugs [138, 276]. We call a deadlock-prediction algorithm *sound* if for every input trace σ , all deadlock reports on σ are predictable deadlocks of σ (i.e., no false positives), and *complete* if all predictable deadlocks of σ are reported by the algorithm (i.e., no false negatives). This is in line with the previous works on the topic of predictive analyses [54, 154, 202, 236]. We remark that other domains sometimes use this terminology reversed.

Example 19. Let us illustrate these definitions on the trace σ_2 in Fig. 7.1b, with e_i denoting the i^{th} event in the figure. The set of events, threads, variables and locks of σ_2 are respectively $\text{Events}_{\sigma_2} = \{e_i\}_{i=1}^{20}$, $\text{Threads}_{\sigma_2} = \{t_1, t_2, t_3, t_4\}$, $\text{Vars}_{\sigma_2} = \{x, y, z\}$ and $\text{Locks}_{\sigma_2} = \{\ell_1, \ell_2, \ell_3\}$. The trace order yields $e_i \leq_{\text{tr}}^{\sigma_2} e_j$ iff $i \leq j$, and some examples of thread-ordered events are $e_1 <_{\text{TO}}^{\sigma_2} e_2 <_{\text{TO}}^{\sigma_2} e_{15}$ and $e_{16} <_{\text{TO}}^{\sigma_2} e_{18} <_{\text{TO}}^{\sigma_2} e_{20}$. The reads-from function is as follows: $\text{rf}_{\sigma_2}(e_{10}) = e_5$, $\text{rf}_{\sigma_2}(e_{14}) = e_9$ and $\text{rf}_{\sigma_2}(e_{17}) = e_{13}$. The lock nesting depth of σ_2 is 2. The sequence $D = \langle e_4, e_{18} \rangle$ forms a deadlock pattern because of the cyclic acquisition of locks ℓ_2 and ℓ_3 without simultaneously holding a common lock. The trace $\rho_4 = e_3..e_7e_8..e_{11}e_1e_2e_{12}..e_{15}e_{16}e_{17}$ is a correct reordering of σ_2 ; even though it differs from σ_2 in the relative order of the critical sections of lock ℓ_1 , and contains only a prefix of thread t_3 , it is consistent with rf_{σ_2} and $<_{\text{TO}}^{\sigma_2}$. However, ρ_4 does not witness $\langle e_4, e_{18} \rangle$ as a deadlock, as only e_{18} is σ_2 -enabled in ρ_4 . On the other hand, the trace $\rho_3 = e_1e_2e_3e_8e_9e_{12}..e_{15}e_{16}e_{17}$ is a correct reordering of σ_2 in which e_4 and e_{18} are σ_2 -enabled, witnessing D as a predictable deadlock of σ_2 .

7.3 The Complexity of Dynamic Deadlock Prediction

Detecting deadlock patterns and predictable deadlocks is clearly a problem in NP, as any witness for either problem can be verified in polynomial time. However, little has been known about the hardness of the problem in terms of rigorous lower bounds. Here we settle these questions, by proving strong intractability results. Due to space constraints, we state and explain the main results here, and refer to our technical report [296] for the full proofs.

Parametrized hardness for detecting deadlock patterns. We show that the basic problem of checking the existence of a deadlock *pattern* is itself hard parameterized by the size k of the pattern.

Theorem 4. *Checking if a trace σ contains a deadlock pattern of size k is $W[1]$ -hard in the parameter k . Moreover, the problem remains NP-hard even when the lock-nesting depth of σ is constant.*

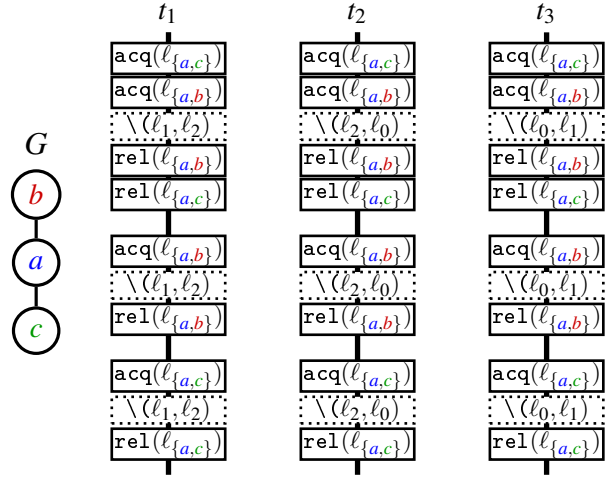
Proof. We show that there is a polynomial-time fixed parameter tractable reduction from INDEPENDENT-SET(c) to the problem of checking the existence of deadlock-patterns of size c . Our reduction takes as input an undirected graph G and outputs a trace σ such that G has an independent set of size c iff σ has a deadlock pattern of size c .

Construction. Let $V = \{v_1, v_2, \dots, v_n\}$. We assume a total ordering $<_E$ on the set of edges E . The trace σ we construct is a concatenation of c sub-traces: $\sigma = \sigma^{(1)} \cdot \sigma^{(2)} \dots \sigma^{(c)}$ and uses c threads $\{t_1, t_2, \dots, t_c\}$ and $|E| + c$ locks $\{\ell_{\{u,v\}}\}_{\{u,v\} \in E} \uplus \{\ell_0, \ell_1, \dots, \ell_{c-1}\}$. The i^{th} sub-trace $\sigma^{(i)}$ is a sequence of events performed by thread t_i , and is obtained by concatenation of $n = |V|$ sub-traces: $\sigma^{(i)} = \sigma_1^{(i)} \cdot \sigma_2^{(i)} \dots \sigma_n^{(i)}$. Each sub-trace $\sigma_j^{(i)}$ with $(i \leq c, j \leq n)$ comprises of nested critical sections over locks of the form $\ell_{\{v_j, u\}}$, where u is a neighbor of v_j . Inside the nested block we have critical sections on locks $\ell_{i \% c}$ and $\ell_{(i+1) \% c}$. Formally, let $\{v_j, v_{k_1}\}, \dots, \{v_j, v_{k_d}\}$ be the neighboring edges of v_j (ordered according to $<_E$). Then, $\sigma_j^{(i)}$ is the unique string generated by the grammar having $d + 1$ non-terminals $S_0, S_1, \dots, S_d$, start symbol S_d and the following production rules:

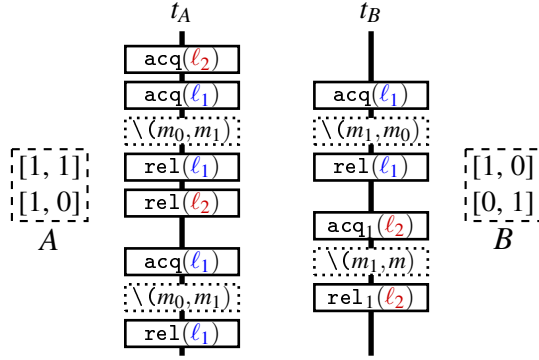
- $S_0 \rightarrow \langle t_i, \text{acq}(\ell_{i \% c}) \rangle \cdot \langle t_i, \text{acq}(\ell_{(i+1) \% c}) \rangle \cdot \langle t_i, \text{rel}(\ell_{(i+1) \% c}) \rangle \cdot \langle t_i, \text{rel}(\ell_{i \% c}) \rangle$.
- for each $1 \leq r \leq d$, $S_r \rightarrow \langle t_i, \text{acq}(\ell_{\{v_j, v_{k_r}\}}) \rangle \cdot S_{r-1} \cdot \langle t_i, \text{rel}(\ell_{\{v_j, v_{k_r}\}}) \rangle$.

Fig. 7.2a illustrates this construction for a graph with 3 nodes and parameter $c = 3$. Finally, observe that the lock-nesting depth in σ is bounded by $2 + \text{degree of } G$. $\square$

Theorem 4 implies that the problem is not only NP-hard, but also unlikely to be *fixed parameter tractable* in the size k of the deadlock pattern. In fact, under the well-believed Exponential Time Hypothesis (ETH), the parametrized problem INDEPENDENT-SET(c) cannot be solved in time $f(c) \cdot n^{o(c)}$ [68]. The above reduction preserves the parameter $k = c$, thus under ETH, detecting deadlock patterns of size k is unlikely to be solvable in time complexity $f(k) \cdot n^{g(k)}$, where $g(k)$ is $o(k)$ (such as $g(k) = \sqrt{k}$ or even $g(k) = k / \log(k)$). The problem of checking the existence of deadlock patterns is, intuitively, a precursor to the deadlock prediction problem. Thus, an approach for deadlock prediction that first identifies the existence of arbitrary deadlock patterns and then verifying their feasibility is unlikely to be tractable. In practice, the synchronization patterns corresponding to the hard instances are uncommon in executions, and our proposed algorithms (Section 7.4 and Section 7.5) can effectively expose predictable deadlocks (Section 7.6).



(a) Reduction of INDEPENDENT-SET(3) on G to detecting a deadlock pattern of size 3.



(b) Reduction for OV-hardness proof from an instance of size $n = 2$ and $d = 2$.

Figure 7.2: Construction of $W[1]$ -hardness (a) and OV-hardness (b) results. We use the shortcut $\langle \ell, \ell' \rangle$ to denote two nested critical sections on ℓ and ℓ' . That is, $\langle \ell, \ell' \rangle = \text{acq}(\ell) \cdot \text{acq}(\ell') \cdot \text{rel}(\ell') \cdot \text{rel}(\ell)$.

Fine-grained hardness for deadlock pattern detection. We next consider the problem of detecting deadlock patterns of size 2, as these form the most common case in practice [189]. Observe that Theorem 4 has no implications on this case, as here k is fixed. The problem admits a folklore $O(n^2)$ time algorithm, by iterating over all pairs of lock-acquisition events of the input trace, and checking whether any such pair forms a deadlock pattern. Perhaps surprisingly, here we show that, despite its simplicity, this algorithm is optimal, i.e., we cannot hope to improve over this quadratic bound. This result is based on a reduction from the popular Orthogonal Vectors (OV) problem.

Given two sets of d -dimensional vectors $A, B \subseteq \{0, 1\}^d$ of cardinality $|A| = |B| = n$, the OV problem asks if there are $a \in A, b \in B$ such that $a \cdot b = \sum_i a[i] \cdot b[i] = 0$. The OV hypothesis states that for any $\varepsilon > 0$, there is no $O(n^{2-\varepsilon} \cdot \text{poly}(d))$ algorithm for solving OV. This is also a consequence of the famous Strong Exponential Time Hypothesis (SETH) [309]. We next show that detecting deadlock patterns of size 2 is at least as hard as solving OV.

Theorem 5. *Given a trace σ of size n and $\mathcal{L}$ locks, for any $\varepsilon > 0$, there is no algorithm that determines in $O(n^{2-\varepsilon} \cdot \text{poly}(\mathcal{L}))$ time whether σ has a deadlock pattern of size 2, under the OV hypothesis.*

Proof. We show a fine-grained reduction from the Orthogonal Vectors Problem to the problem of checking for deadlock patterns of size 2. For this, we start with two sets $A, B \subseteq \{0, 1\}^d$ of d -dimensional vectors with $|A| = |B| = n$. We write the i^{th} vector in A as A_i and that in B as B_i .

Construction. We will construct a trace σ such that σ has a deadlock pattern of length 2 iff (A, B) is a positive OV instance. The trace σ is of the form $\sigma = \sigma^A \cdot \sigma^B$ and uses 2 threads $\{t_A, t_B\}$ and $d + 2$ distinct locks $\ell_1, \dots, \ell_d, m_0, m_1$. Intuitively, σ^A and σ^B encode the given sets of vectors A and B . The sub-traces $\sigma^A = \sigma_1^A \cdot \sigma_2^A \dots \sigma_n^A$ and $\sigma^B = \sigma_1^B \cdot \sigma_2^B \dots \sigma_n^B$ are defined as follows. For each $i \in \{1, 2, \dots, n\}$ and $Z \in \{A, B\}$, the sub-trace σ_i^Z is the unique string generated by the grammar having $d + 1$ non-terminals $S_0, S_1, \dots, S_d$, start symbol S_d and the following production rules:

- $S_0 \rightarrow \langle t_Z, \text{acq}(m) \rangle \cdot \langle t_Z, \text{acq}(m') \rangle \cdot \langle t_Z, \text{rel}(m') \rangle \cdot \langle t_Z, \text{rel}(m) \rangle$, where $(m, m') = (m_0, m_1)$ if $Z = A$, and $(m, m') = (m_1, m_0)$.
- for each $1 \leq j \leq d$, $S_j \rightarrow S_{j-1}$ if $Z_i[j] = 0$. Otherwise (if $Z_i[j] = 1$), $S_j \rightarrow \langle t_Z, \text{acq}(\ell_j) \rangle \cdot S_{j-1} \cdot \langle t_Z, \text{rel}(\ell_j) \rangle$.

In words, all events of σ^A are performed by thread t_A and those in σ^B are performed by t_B . Next, the i^{th} sub-trace of σ^A , denoted σ_i^A corresponds to the vector A_i as follows — σ_i^A is a nested block of critical sections, with the innermost critical section being on lock ℓ' , which is immediately enclosed in a critical section on lock ℓ . Further, in the sub-trace σ_i^A , the lock ℓ_j occurs iff $A_i[j] = 1$. The sub-traces σ_i^B is similarly constructed, except that the order of the two innermost critical sections is inverted. Fig. 7.2b illustrates the construction for an OV-instance with $n = 2$ and $d = 2$. $\square$

The complexity of deadlock prediction. Finally, we settle the complexity of the prediction problem for deadlocks, and show that, even for deadlock patterns of size 2, the problem is $W[1]$ -hard parameterized by the number of threads. In contrast, recall that the $W[1]$ -hardness of Theorem 4 concerns deadlock patterns of arbitrary size. Our result is based on a similar hardness that was established recently for predicting data races [200].

Theorem 6. *The problem of checking if a trace σ has a predictable deadlock of size 2 is $W[1]$ -hard in the number of threads k appearing in σ , and thus is also NP-hard.*

7.4 Synchronization-Preserving Deadlocks and their Prediction

Having established the intractability of general deadlock prediction in Section 7.3, we now define the subclass of predictable deadlocks called synchronization-preserving (*sync-preserving*, for short) in Section 7.4. The key benefit of sync-preserving deadlocks is that, unlike arbitrary deadlocks, they can be detected efficiently; we develop our algorithm `SPDOffline` for this task in Sections 4.2-4.5. Our experiments later indicate that most predictable deadlocks are actually sync-preserving, hence the benefit of fast detection comes at the cost of little-to-no precision loss in practice.

Overview of the algorithm. There are several insights behind our algorithm. First, given a deadlock pattern, one can verify if it is a sync-preserving deadlock in linear time (Section 7.4); this is based on our sound and complete characterization of sync-preserving deadlocks (Section 7.4). Next, instead of verifying single deadlock patterns one-by-one, we consider *abstract deadlock patterns*, which are essentially collections of deadlock patterns that share the same signature; the formal definition is given in Section 11. We show that our basic algorithm can be extended to *incrementally* verify *all* the concretizations of an abstract deadlock pattern in linear time (Section 11), in a single pass (Lemma 14). Finally, we feed this algorithm all the abstract deadlock patterns of the input trace, by constructing an *abstract lock graph* and enumerating cycles in it (Section 11).

Synchronization-Preserving Deadlocks

Our notion of sync-preserving deadlocks builds on the recently introduced concept of sync-preserving correct reorderings [202].

Definition 2 (Sync-preserving Correct Reordering). A correct reordering ρ of a trace σ is *sync-preserving* if for every lock $\ell \in \text{Locks}_\rho$ and every two acquire events $e_1 \neq e_2 \in \text{Events}_\rho$ with $\text{op}(e_1) = \text{op}(e_2) = \text{acq}(\ell)$, the order of e_1 and e_2 is the same in σ and ρ , i.e., $e_1 \leq_{\text{tr}}^\rho e_2$ iff $e_1 \leq_{\text{tr}}^\sigma e_2$.

A sync-preserving correct reordering preserves the order of those critical sections (on the same lock) that actually appear in the reordering, but allows intermediate critical sections to be dropped completely. This style of reasoning is more permissive than the space of reorderings induced by the Happens-Before partial order [171], that implicitly enforces that all intermediate critical sections on a lock be present. Sync-preserving deadlocks can now be defined naturally.

Definition 3 (Sync-preserving Deadlocks). Let σ be a trace and $D = \langle e_0, e_1, \dots, e_{k-1} \rangle$ be a deadlock pattern. We say that D is a sync-preserving deadlock of σ if there is a sync-preserving correct reordering ρ of σ such that each of $e_0, \dots, e_{k-1}$ is σ -enabled in ρ .

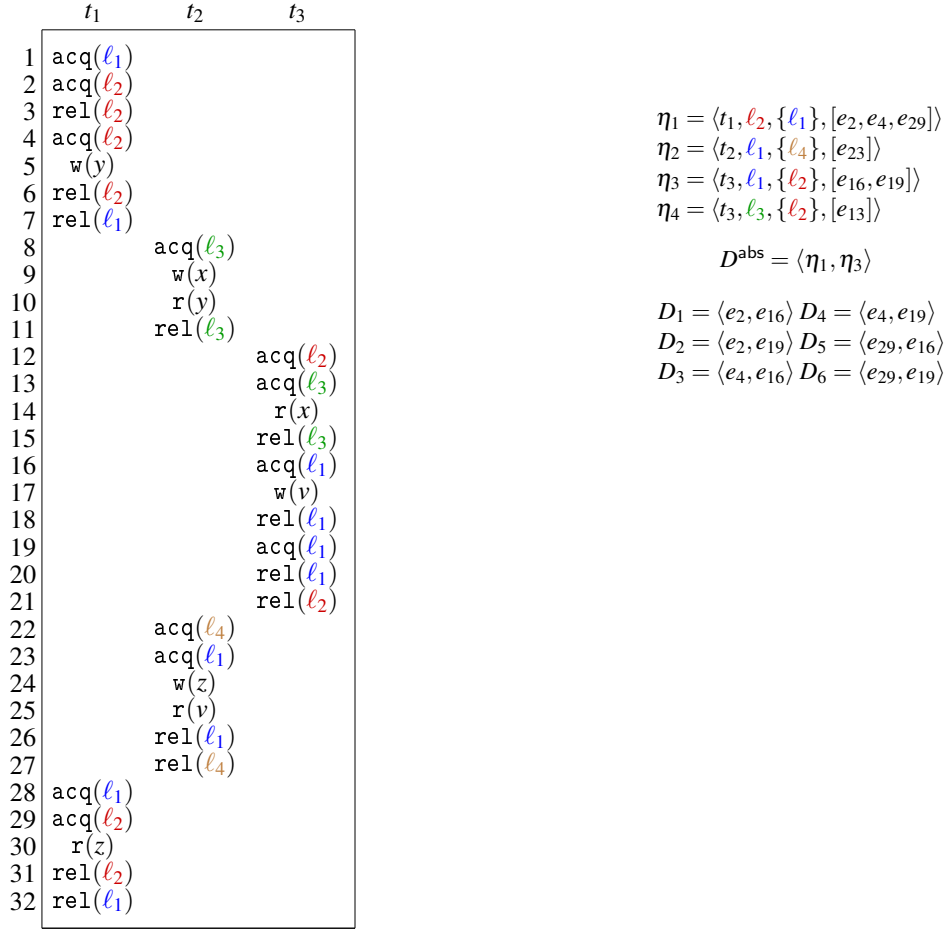


Figure 7.3: A trace σ_3 , its abstract acquires η_i , unique abstract deadlock pattern D^{abs} , concrete patterns $D_i \in D^{\text{abs}}$.

Example 20. Consider the trace σ_2 in Fig. 7.1b. The deadlock pattern $D = \langle e_4, e_{18} \rangle$ is a sync-preserving deadlock, witnessed by the sync-preserving correct reordering $\rho_3 = e_1 e_2 e_3 e_8 e_9 e_{12} \dots e_{15} e_{16} e_{17}$. Now consider the trace σ_3 from Fig. 7.3 and the deadlock pattern $D_5 = \langle e_{29}, e_{16} \rangle$. This is a predictable deadlock, witnessed by the correct reordering $\rho_5 = e_1 \dots e_7 e_8 \dots e_{11} e_{12} \dots e_{15} e_{28}$. Observe that ρ_5 is a sync-preserving reordering, which makes D_5 a sync-preserving deadlock. A key aspect in ρ_5 is that the events $e_{22} \dots e_{27}$ are dropped, as otherwise e_{16} cannot be σ_3 -enabled. A similar reasoning applies for the deadlock pattern D_6 , and it is also a sync-preserving deadlock. The other deadlock patterns (D_1, D_2, D_3, D_4) are not predictable deadlocks. Intuitively, the reason for this is that realizing these deadlock patterns require executing the read event e_{14} , which then enforces to execute the events $e_8 \dots e_{11}$ and $e_1 \dots e_6$. This prevents the deadlocks from becoming realizable as the events e_2 or e_4 that appear in these deadlock patterns are no longer σ_3 -enabled. This point is detailed in Example 21.

Characterizing Sync-Preserving Deadlocks

There are two fundamental tasks in searching for a correct reordering that witnesses a deadlock — (i) determining the set of events in the correct reordering, and (ii) identifying a total order on such events — both of which are intractable [200]. On the contrary, for sync-preserving deadlocks, we show that (a) the search for a correct reordering can be reduced to the problem of checking if some well-defined set of events (Definition 4) does not contain the events appearing in the deadlock pattern (Lemma 13), and that (b) this set can be constructed efficiently.

Definition 4 (Sync-Preserving Closure). Let σ be a trace and $S \subseteq \text{Events}_\sigma$. The sync-preserving closure of S , denoted $\text{SPClosure}_\sigma(S)$ is the smallest set S' such that

- (a) $S \subseteq S'$,
- (b) for every $e, e' \in \text{Events}_\sigma$ such that $e <_{\text{TO}}^\sigma e'$ or $e = \text{rf}_\sigma(e')$, if $e' \in S'$, then $e \in S'$, and
- (c) for every lock ℓ and every two distinct events $e, e' \in S'$ with $\text{op}(e) = \text{op}(e') = \text{acq}(\ell)$, if $e \leq_{\text{tr}}^\sigma e'$ then $\text{match}_\sigma(e) \in S'$.

Definition 4 resembles the notion of correct reorderings (Definition 2). Indeed, Lemma 12 justifies using this set — it is both a necessary and a sufficient set for sync-preserving correct reorderings.

Lemma 12. *Let σ be a trace and let $S \subseteq \text{Events}_\sigma$. For any sync-preserving correct reordering ρ of σ , if $S \subseteq \text{Events}_\rho$, then $\text{SPClosure}_\sigma(S) \subseteq \text{Events}_\rho$. Further, there is a sync-preserving correct reordering ρ of σ such that $\text{Events}_\rho = \text{SPClosure}_\sigma(S)$.*

For an intuition, consider again Figure 7.3 and the sync-preserving correct reordering $\rho_5 = e_1..e_7e_8..e_{11}e_{12}..e_{15}e_{28}$ computed in Example 20. According to Lemma 12, $\text{SPClosure}_{\sigma_3}(S) \subseteq \text{Events}_{\rho_5}$ holds for all S such that $S \subseteq \text{Events}_{\rho_5}$. For example, if we take $S = \{e_1, e_{15}\}$ then observe that $S \subseteq \text{Events}_{\rho_5}$ and $\text{SPClosure}_{\sigma_3}(S) = \{e_1, \dots, e_6, e_8, \dots, e_{15}\} \subseteq \text{Events}_{\rho_5}$ holds.

Based on Lemma 12, we present a sound and complete characterization of sync-preserving deadlocks (Lemma 13). For a set $S \subseteq \text{Events}_\sigma$, we let $\text{pred}_\sigma(S)$ denote the set of immediate thread predecessors of events in S . That is, $\text{pred}_\sigma(S) = \{e \in \text{Events}_\sigma \mid \exists f \in S, e <_{\text{tr}}^\sigma f \text{ and } \forall e' <_{\text{tr}}^\sigma f, e' \leq_{\text{TO}}^\sigma e\}$.

Lemma 13. *Let σ be a trace and let $D = \langle e_0, \dots, e_{k-1} \rangle$ be a deadlock pattern of size k in σ . D is a sync-preserving deadlock of σ iff $\text{SPClosure}_\sigma(\text{pred}_\sigma(S)) \cap S = \emptyset$, where $S = \{e_0, \dots, e_{k-1}\}$.*

Example 21. *Consider the trace σ_2 in Fig. 7.1b, and the deadlock pattern $D = \langle e_4, e_{18} \rangle$. We have $\text{SPClosure}_{\sigma_2}(\text{pred}_{\sigma_2}(\{e_4, e_{18}\})) = \{e_1, e_2, e_3, e_8, e_9, e_{12}, \dots, e_{17}\}$. Since we have that $e_4, e_{18} \notin \text{SPClosure}_{\sigma_2}(\text{pred}_{\sigma_2}(\{e_4, e_{18}\}))$, D is a sync-preserving deadlock. Now consider the trace σ_3 in Fig. 7.3, and the deadlock patterns $D_1 =$*

$\langle e_2, e_{16} \rangle$, $D_5 = \langle e_{29}, e_{16} \rangle$, and $D_6 = \langle e_{29}, e_{19} \rangle$. We have $\text{SPClosure}_{\sigma_3}(\text{pred}_{\sigma_3}(\{e_2, e_{16}\})) = \{e_1, \dots, e_6, e_8, \dots, e_{15}\}$, $\text{SPClosure}_{\sigma_3}(\text{pred}_{\sigma_3}(\{e_{29}, e_{16}\})) = \{e_1, \dots, e_{15}, e_{28}\}$, and $\text{SPClosure}_{\sigma_3}(\text{pred}_{\sigma_3}(\{e_{29}, e_{19}\})) = \{e_1, \dots, e_{18}, e_{28}\}$. Since $e_2 \in \text{SPClosure}_{\sigma_3}(\text{pred}_{\sigma_3}(\{e_2, e_{16}\}))$, D_1 is not a sync-preserving deadlock. However, $e_{29}, e_{16} \notin \text{SPClosure}_{\sigma_3}(\text{pred}_{\sigma_3}(\{e_{29}, e_{16}\}))$, and $e_{29}, e_{19} \notin \text{SPClosure}_{\sigma_3}(\text{pred}_{\sigma_3}(\{e_{29}, e_{19}\}))$, thus D_5 and D_6 are sync-preserving deadlocks (as we also concluded in Example 20).

Verifying Deadlock Patterns

Given a deadlock pattern, we check if it constitutes a sync-preserving deadlock by constructing the sync-preserving closure (Lemma 13) in linear time. Based on Definition 4, this can be done in an iterative manner. We (i) start with the set of $\leq_{\text{TO}}$ predecessors of the events in the deadlock pattern, and (ii) iteratively add $\leq_{\text{TO}}$ and rf predecessors of the current set of events. Additionally, we identify and add the release events that must be included in the set. We utilize *timestamps* to ensure that the *entire* fixpoint computation is performed in linear time.

Thread-read-from timestamps. Given a set *Threads* of threads, a timestamp is simply a mapping $T : \text{Threads} \rightarrow \mathbb{N}$. Given timestamps T_1, T_2 , we use the notations $T_1 \sqsubseteq T_2$ and $T_1 \sqcup T_2$ for pointwise comparison and pointwise maximum, respectively. For a set U of timestamps, we write $\bigsqcup U$ to denote the pointwise maximum over all elements of U . Let $\leq_{\text{TRF}}^\sigma$ be the reflexive transitive closure of the relation $(\leq_{\text{TO}}^\sigma \cup \{(\text{rf}_\sigma(e), e) \mid \exists x \in \text{Vars}_\sigma, \text{op}(e) = \text{r}(x)\})$; observe that $\leq_{\text{TRF}}^\sigma$ is a partial order. We define the timestamp TS_σ^e of an event e in σ to be a Threads_σ -indexed timestamp as follows: $\text{TS}_\sigma^e(t) = |\{f \mid f \leq_{\text{TRF}}^\sigma e\}|$. This ensures that for two events $e, e' \in \text{Events}_\sigma$, $e \leq_{\text{TRF}}^\sigma e'$ iff $\text{TS}_\sigma^e \sqsubseteq \text{TS}_\sigma^{e'}$. For a set $S \subseteq \text{Events}_\sigma$, we overload the notation and say the timestamp of S is $\text{TS}_\sigma^S = \bigsqcup \{\text{TS}_\sigma^e\}_{e \in S}$. Given a trace σ with n events and k threads we can compute these timestamps for all the events in $O(n \cdot k)$ time, using a simple vector clock algorithm [108, 206].

Computing sync-preserving closures. Recall the basic template of the fixpoint computation. In each iteration, we identify the set of release events that must be included in the set, together with their $\leq_{\text{TRF}}^\sigma$ -closure. In order to identify such events efficiently, for every thread t and lock ℓ , we maintain a FIFO queue $\text{CSHist}_{t,\ell}$ (*critical section history* of t and ℓ) to store the sequence of events that acquire ℓ in thread t . In each iteration, we traverse each list to determine the last acquire event that belongs to the current set. For a given lock, we need to add the matching release events of all thus identified events to the closure, except possibly the matching release event of the latest acquire event (see Definition 4). This computation is performed using timestamps, as shown in Algorithm 10. Starting with a set S , the algorithm runs in time $O(|S| \cdot k + k \cdot \mathcal{A})$, where k and $\mathcal{A}$ are respectively the number of threads and acquire events in σ .

Checking a deadlock pattern. After computing the timestamp T of the closure (output of Algorithm 10, starting with the set of events in the given deadlock pattern), determining whether a given deadlock pattern $D = e_0, \dots, e_{k-1}$ is a sync-preserving deadlock can be performed in time $O(k \cdot k)$ — simply check if $\forall i, \text{TS}_\sigma(e_i) \not\sqsubseteq T$. This

Algorithm 10: CompSPClosure: Computing sync-preserving closure.

Input: Trace σ , Timestamp T_0

```

1 let  $\{\text{CSHist}_{t,\ell}\}_{\ell \in \text{Locks}_\sigma, t \in \text{Threads}_\sigma}$  be the lock-acquisition histories in  $\sigma$ 
2  $T \leftarrow T_0$ 
3 repeat
4   for  $\ell \in \text{Locks}$  do
5     foreach  $t \in \text{Threads}$  do
6       let  $e_t$  be the last event in  $\text{CSHist}_{t,\ell}$  with  $\text{TS}_\sigma^{e_t} \sqsubseteq T$ 
7       Remove all earlier events in  $\text{CSHist}_{t,\ell}$ 
8       let  $e_{t*}$  be the last event in  $\{e_t\}_{t \in \text{Threads}_\sigma}$  according to  $\leq_{\text{tr}}^\sigma$ 
9        $T := T \sqcup \sqcup \{\text{TS}_\sigma^{\text{match}_\sigma(e_t)} \mid e_t \neq e_{t*}\}$ 
10 until  $T$  does not change
11 return  $T$ 

```

Algorithm 11: CheckAbsDdlck: Checking an abstract deadlock pattern.

Input: Trace σ , D^{abs} of length k

```

1 let  $F_0, \dots, F_{k-1}$  be the sequences of acquires in  $D^{\text{abs}}$ 
2 let  $n_0, \dots, n_{k-1}$  be the lengths of  $F_0, \dots, F_{k-1}$ 
3 foreach  $j \in \{0, \dots, k-1\}$  do  $i_j \leftarrow 1$ 
4  $T \leftarrow \lambda t, 0$  while  $\bigwedge_{j=0}^{k-1} i_j < n_j$  do
5   let  $e_0 = F_0[i_0], \dots, e_{k-1} = F_{k-1}[i_{k-1}]$ 
6    $S \leftarrow \text{pred}_\sigma\{e_0, \dots, e_{k-1}\}$ 
7    $T \leftarrow \text{CompSPClosure}(\sigma, T \sqcup \text{TS}_\sigma^S)$ 
8   if  $\forall j < k, \text{TS}_\sigma^{e_j} \sqsubseteq T$  then
9     report pattern  $D = e_0, \dots, e_{k-1}$  and exit
10  foreach  $j \in \{0, \dots, k-1\}$  do
11     $i_j = \min\{l \leq n_j \mid \text{TS}_\sigma^{F_j[l]} \not\sqsubseteq T\}$ 

```

gives an algorithm for checking if a deadlock pattern of length k is sync-preserving that runs in time $O(k \cdot n + k \cdot k + k \cdot \mathcal{A}) = O(n \cdot k)$.

Verifying Abstract Deadlock Patterns

Abstract acquires and abstract deadlock patterns. Given a thread t , a lock ℓ and a set of locks $L \subseteq \text{Locks}_\sigma \neq \emptyset$ with $\ell \notin L$, we define the *abstract acquire* $\eta = \langle t, \ell, L, F \rangle$, where $F = [e_1, \dots, e_n]$ is the sequence of all events $e_i \in \text{Events}_\sigma$ (in trace-order) such that for each i , we have (i) $\text{tid}(e_i) = t$, (ii) $\text{op}(e_i) = \text{acq}(\ell)$, and (iii) $\text{HeldLks}_\sigma(e_i) = L$. In words, the abstract acquire η contains the sequence of all acquire events of a specific thread, that access a specific lock and hold the same set of locks when executed, ordered as per thread order. An *abstract deadlock pattern* of size k in a trace

σ is a sequence $D^{\text{abs}} = \eta_0, \dots, \eta_{k-1}$ of abstract acquires $\eta_i = \langle t_i, \ell_i, L_i, F_i \rangle$ such that $t_0, \dots, t_{k-1}$ are distinct threads, $\ell_0, \dots, \ell_{k-1}$ are distinct locks, and $L_0, L_1, \dots, L_{k-1} \subseteq \text{Locks}_\sigma$ are such that $\ell_i \notin L_i$, $\ell_i \in L_{(i+1) \% k}$ for every i , and $L_i \cap L_j = \emptyset$ for every $i \neq j$. Thus, an abstract deadlock pattern D^{abs} succinctly encodes all concrete deadlock patterns $F_0 \times F_1 \times \dots \times F_{k-1}$, called *instantiations* of D^{abs} . We also write $D \in D^{\text{abs}}$ to denote that $D \in F_0 \times F_1 \times \dots \times F_{k-1}$. We say that D^{abs} contains a sync-preserving deadlock if there exists some instantiation $D \in D^{\text{abs}}$ that is a sync-preserving deadlock. See Fig. 7.3 for an example. Our next result is stated below, followed by its proof idea.

Lemma 14. *Consider a trace σ with n events and k threads, and an abstract deadlock pattern D^{abs} of σ . We can determine if D^{abs} contains a sync-preserving deadlock in $O(k \cdot n)$ time.*

An abstract deadlock pattern of length $k \geq 2$ can have n^k instantiations, giving a naive enumerate-and-check algorithm running in time $O(k \cdot n^{k+1})$, which is prohibitively large. Instead, we exploit (i) the monotonicity properties of the sync-preserving closure (Proposition 1) and (ii) instantiations of an abstract pattern (Corollary 1) that allow for an *incremental* algorithm that iteratively checks successive instantiations of a given abstract deadlock pattern, while spending total $O(n \cdot k)$ time. The first observation allows us to re-use a prior computation when checking later deadlock patterns.

Proposition 1. For a trace σ and sets $S, S' \subseteq \text{Events}_\sigma$. If for every event $e \in S$, there is an event $e' \in S'$ such that $e \leq_{\text{TO}}^\sigma e'$, then $\text{SPClosure}_\sigma(S) \subseteq \text{SPClosure}_\sigma(S')$.

Consider σ_3 in Figure 7.3 and let $S = \text{pred}_{\sigma_3}(\{e_{29}, e_{16}\})$, and $S' = \text{pred}_{\sigma_3}(\{e_{29}, e_{19}\})$. The sets S, S' satisfy the conditions of Proposition 1, hence $\text{SPClosure}_{\sigma_3}(S) \subseteq \text{SPClosure}_{\sigma_3}(S')$, as computed in Example 21. Next, we extend Proposition 1 to avoid redundant computations when a sync-preserving deadlock is not found and later deadlock patterns must be checked. Given two deadlock patterns $D_1 = e_0, \dots, e_{k-1}$ and $D_2 = f_0, \dots, f_{k-1}$ of the same length k , we say $D_1 \prec D_2$ if they are instantiations of a common abstract pattern D^{abs} (i.e., $D_1, D_2 \in D^{\text{abs}}$) and for every $i < k$, $e_i \leq_{\text{TO}}^\sigma f_i$.

Corollary 1. Let σ be a trace and let $D_1 = e_0, \dots, e_{k-1}$ and $D_2 = f_0, \dots, f_{k-1}$ be deadlock patterns of size k in σ such that $D_1 \prec D_2$. Let $S_1 = \{e_0, \dots, e_{k-1}\}$ and $S_2 = \{f_0, \dots, f_{k-1}\}$. If $\text{SPClosure}_\sigma(\text{pred}_\sigma(S_1)) \cap S_2 \neq \emptyset$, then $\text{SPClosure}_\sigma(\text{pred}_\sigma(S_2)) \cap S_2 \neq \emptyset$.

We now describe how Proposition 1 and Corollary 1 are used in our algorithms, and illustrate them later in Example 22. Algorithm 11 checks if an abstract deadlock pattern contains a sync-preserving deadlock. The algorithm iterates over the sequences $F_0, \dots, F_{k-1}$ of acquires (one for each abstract acquire) in trace order. For this, it maintains indices $i_0, \dots, i_{k-1}$ that point to entries in $F_0, \dots, F_{k-1}$. At each step, it determines whether the current deadlock pattern $D = e_0, \dots, e_{k-1}$ constitutes a sync-preserving deadlock by computing the sync-preserving closure of the thread-local

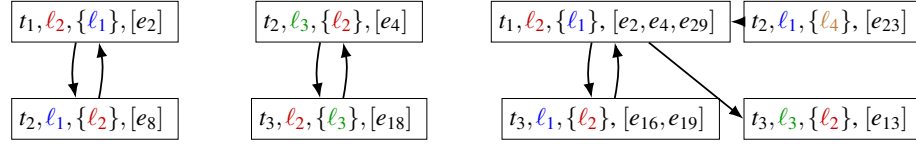


Figure 7.4: Abstract lock graphs of the traces from Fig. 7.1a (left), Fig. 7.1b (middle) and Fig. 7.3 (right).

predecessors of the events of the deadlock pattern. The algorithm reports a deadlock if the sync-preserving closure does not contain any of $e_0, \dots, e_{k-1}$. Otherwise, it looks for the next eligible deadlock pattern, which it determines based on Corollary 1. In particular, it advances the pointer i_j all the way until an entry which is outside of the closure computed so far. Observe that the timestamp T of the closure computed in an iteration is being used in later iterations; this is a consequence of Proposition 1. Furthermore, in the call to the Algorithm 10 at Line 7, we ensure that the list of acquires $\text{CSHist}_{t,\ell}$, used in the function CompSPCClosure is reused across iterations, and not re-assigned to the original list of all acquire events. The correctness of this optimization follows from Proposition 1. Let us now calculate the running time of Algorithm 11. Each of the $\text{CSHist}_{t,\ell}$ in CompSPCClosure is traversed at most once. Next, each element of the sequences $F_0, \dots, F_{k-1}$ is also traversed at most once. For each of these acquires, the algorithm spends $O(k)$ time for vector clock updates. The total time required is thus $O(n \cdot k)$. This concludes the proof of Lemma 14.

The Algorithm SPDOffline

We now present the final ingredients of SPDOffline. We construct the *abstract lock graph*, enumerate cycles in it, check whether any cycle is an abstract deadlock pattern, and if so, whether it contains sync-preserving deadlocks.

Abstract lock graph. The abstract lock graph of σ is a directed graph $\text{ALG}_\sigma = (V_\sigma, E_\sigma)$, where

- $V_\sigma = \{\langle t_1, \ell_1, L_1, F_1 \rangle, \dots, \langle t_k, \ell_k, L_k, F_k \rangle\}$ is the set of abstract acquires of σ , and
- for every $\eta_1 = \langle t_1, \ell_1, L_1, F_1 \rangle, \eta_2 = \langle t_2, \ell_2, L_2, F_2 \rangle \in V_\sigma$, we have $(\eta_1, \eta_2) \in E_\sigma$ iff $t_1 \neq t_2, \ell_1 \in L_2$, and $L_1 \cap L_2 = \emptyset$.

A node $\langle t_1, \ell_1, L_1, F_1 \rangle$ signifies that there is an event $\text{acq}_1(\ell_1)$ performed by thread t_1 while holding the locks in L_1 . The last component F_1 is a list which contains all such events acq_1 in order of appearance in σ . An edge (η_1, η_2) signifies that the lock ℓ_1 acquired by each of the events $\text{acq}_1 \in F_1$ was held by t_2 when it executed each of $\text{acq}_2 \in F_2$ while not holding a common lock. The abstract lock graph can be constructed incrementally as new events appear in σ . For n events, $\mathcal{L}$ locks and nesting depth d , the graph has $|V_\sigma| = O(k \cdot \mathcal{L}^d)$ vertices, $|E_\sigma| = O(|V_\sigma| \cdot \mathcal{L}^{d-1})$ edges and can be constructed in $O(n \cdot d)$ time. See Fig. 7.4 for examples. In the left graph, the cycle marks an abstract deadlock pattern and its single concrete deadlock pattern $D^{\text{abs}} = \{e_2\} \times \{e_8\}$, and similarly for the middle graph where $D^{\text{abs}} = \{e_4\} \times \{e_{18}\}$.

In the right graph, there is a unique cycle which marks an abstract deadlock pattern of 6 concrete deadlock patterns $D^{\text{abs}} = \{e_2, e_4, e_{29}\} \times \{e_{16}, e_{19}\}$.

Algorithm 12: Algorithm SPDOffline.

Input: A trace σ .

Output: All abstract deadlock patterns of σ that contain a sync-preserving deadlock.

```

1 Construct the abstract lock graph  $\text{ALG}_\sigma$ 
2 foreach cycle  $C = \langle \eta_0, \dots, \eta_{k-1} \rangle$  in  $G$  do
3   Let  $\eta_i = \langle t_i, \ell_i, L_i, F_i \rangle$ 
4   if  $\forall i \neq j$  we have  $t_i \neq t_j$  and  $\ell_i \neq \ell_j$  and  $L_i \cap L_j = \emptyset$  then //  $C$  is an abstract
      deadlock pattern
5   | if CheckAbsDdlck( $C$ ) then Report that  $C$  contains a sync-preserving deadlock
    
```

Algorithm SPDOffline. It is straightforward to verify that every abstract deadlock pattern of σ appears as a (simple) cycle in ALG_σ . However, the opposite is not true. A cycle $C = \eta_0, \eta_1, \dots, \eta_{k-1}$ of ALG_σ , where $\eta_i = \langle t_i, \ell_i, L_i, F_i \rangle$ defines an abstract deadlock pattern if additionally every thread t_i is distinct, all every lock ℓ_i is distinct, and all sets L_i are pairwise disjoint. This gives us a simple recipe for enumerating all abstract deadlock patterns, by using Johnson's algorithm [148] to enumerate every simple cycle C in ALG_σ , and check whether C is an abstract deadlock pattern. We thus arrived at our offline algorithm SPDOffline (Algorithm 12). The running time depends linearly on the length of σ and the number of cycles in ALG_σ .

Theorem 7. Consider a trace σ of n events, k threads and Cyc_σ cycles in ALG_σ . The algorithm SPDOffline reports all sync-preserving deadlocks of σ in time $O(n \cdot k \cdot \text{Cyc}_\sigma)$.

Although, in principle, we can have exponentially many cycles in ALG_σ , because the nodes of ALG_σ are *abstract* acquire events (as opposed to *concrete*), we expect that the number of cycles (and thus abstract deadlock patterns) in ALG_σ remains small, even though the number of *concrete* deadlock patterns can grow exponentially. Since SPDOffline spends linear time per abstract deadlock pattern, we have an efficient procedure overall for constant k and $\mathcal{L}$. We evaluate Cyc_σ experimentally in Section 7.6, and confirm that it is very small compared to the number of concrete deadlock patterns in σ . Nevertheless, Cyc_σ can become exponential when k and $\mathcal{L}$ are large, making Algorithm 12 run in exponential time. Note that this barrier is unavoidable in general, as proven in Theorem 4.

Example 22. We illustrate how the lock graph is integrated inside SPDOffline. Consider the trace σ_3 in Fig. 7.3. It contains 6 concrete deadlock patterns $D_1 \dots D_6$. A naive algorithm would enumerate each pattern explicitly until it finds a deadlock. However, the tight interplay between the abstract lock graph and sync-preservation enables a more efficient procedure. SPDOffline starts by computing the sync-preserving closure of D_1 , $\text{SPClosure}_{\sigma_3}(\text{pred}_{\sigma_3}(\{e_2, e_{16}\})) = \{e_1, \dots, e_6, e_8, \dots, e_{15}\}$. As $e_2 \in \text{SPClosure}_{\sigma_3}(\text{pred}_{\sigma_3}(\{e_2, e_{16}\}))$, we conclude that D_1 is not a sync-preserving

deadlock. The algorithm further deduces that the deadlock patterns D_2 , D_3 and D_4 are also not sync-preserving deadlocks, as follows. $D_2 = \langle e_2, e_{19} \rangle$ shares a common event e_2 with D_1 but contains the event e_{19} instead of e_{16} , while $e_5 \in \text{SPClosure}_{\sigma_3}(\text{pred}_{\sigma_3}(\{e_2, e_{16}\}))$. Since $e_{16} \leq_{TO}^{\sigma_3} e_{19}$, and the sync-preserving closure grows monotonically (Proposition 1), the sync-preserving closure of e_2 and e_{19} will also contain e_5 (and thus e_2). Therefore, D_2 cannot be a sync-preserving deadlock. This reasoning is formalized in Corollary 1, and also applies to D_3 and D_4 . Next, the algorithm proceeds with D_5 . The above reasoning does not hold for D_5 as $\text{SPClosure}_{\sigma_3}(\text{pred}_{\sigma_3}(\{e_2, e_{16}\})) \cap S_5 = \emptyset$ where $S_5 = \{e_{29}, e_{16}\}$. The algorithm then computes the sync-preserving closure of D_5 , reports a deadlock (Example 21) and stops analyzing this abstract deadlock pattern. In the end, we have only explicitly enumerated the deadlock patterns D_1 and D_5 .

Remark 3. Although the concept of lock graphs exists in the literature [32, 52, 53, 130], our notion of abstract lock graphs is novel and tailored to sync-preserving deadlocks. The closest concept to abstract lock graphs is that of equivalent cycles [52]. However, equivalent cycles unify all the concrete patterns of a given abstract pattern and lead to unsound deadlock detection, which was indeed their use.

7.5 On-the-fly Deadlock Prediction

Although SPDOffline is efficient, both theoretically (Theorem 7) and in practice (Table 7.1), it runs in two passes, akin to other predictive deadlock-detection methods [54, 154]. In a runtime monitoring setting, it is desirable to operate in an *online* fashion. Recall that CheckAbsDdlck($\cdot$) indeed operates online (Section 11), while the offline nature of SPDOffline is tied to the offline construction of the abstract lock graph ALG_σ . To achieve the golden standard of online, linear-time, sound deadlock prediction, we focus on deadlocks of size 2. This focus is barely restrictive as most deadlocks in the wild have size 2 [189]. Further, deadlocks of size 2 enjoy the following computational benefits:

- (a) cycles of length 2 can be detected instantaneously without performing graph traversals, and
- (b) every cycle of length 2 is an abstract deadlock pattern.

Algorithm SPDOnline. The algorithm SPDOnline maintains all abstract acquires of the form $\eta = \langle t, \ell_1, \{\ell_2\}, F \rangle$, i.e., we only focus on one lock ℓ_2 that is protecting each such acquire. When a new acquire event $e = \langle t, \text{acq}(\ell_1) \rangle$ is encountered, the algorithm iterates over all the locks $\ell_2 \in \text{HeldLks}_\sigma(e)$ that are held in e , and append the event e to the sequence F of the corresponding abstract acquire $\eta = \langle t, \ell_1, \{\ell_2\}, F \rangle$; F is maintained as FIFO queue. Recall that we use timestamps on the acquire events in F to determine membership in our closure computation. Our online algorithm computes these timestamps on-the-fly and stores them in these queues together with the events. Then the algorithm calls CheckAbsDdlck(C) on the abstract deadlock

Algorithm 13: SPDOnline.

```

1 function checkDeadlock( $Lst, I, \langle t_1, \ell_1, t_2, \ell_2 \rangle$ )
2   while not  $Lst \cdot \text{isEmpty}()$  do
3      $(C_{\text{pred}}, C) := Lst \cdot \text{first}()$ 
4      $I := \text{CompSPClosure}(I \sqcup C_{\text{pred}}, \langle t_1, \ell_1, t_2, \ell_2 \rangle)$ 
5     if  $C \not\sqsubseteq I$  then declare ‘Deadlock’ and break
6      $Lst \cdot \text{removeFirst}()$ 
7   return  $I$ 

8 handler writeHandler( $t, x$ )
9    $\mathbb{LW}_x := \mathbb{C}_t$ 
10   $\mathbb{C}_t := \mathbb{C}_t[t \mapsto \mathbb{C}_t(t) + 1]$ 

11 handler release( $t, \ell$ )
12   $\mathbb{C}_t := \mathbb{C}_t[t \mapsto \mathbb{C}_t(t) + 1]$ 
13  foreach  $t_1, t_2 \in \text{Threads}, \ell_1, \ell_2 \in \text{Locks}$  do
14     $\text{CSHist}_{t, \ell}^{\langle t_1, \ell_1, t_2, \ell_2 \rangle} \cdot \text{last}() \cdot \text{updateRelease}(\mathbb{C}_t)$ 

15 handler acquire( $t, \ell$ )
16   $C_{\text{pred}} := \mathbb{C}_t$ 
17   $\mathbb{C}_t := \mathbb{C}_t[t \mapsto \mathbb{C}_t(t) + 1]$ 
18   $g_\ell := g_\ell + 1$ 
19  foreach  $t_1, t_2 \in \text{Threads}, \ell_1, \ell_2 \in \text{Locks}$  do
20     $\text{CSHist}_{t, \ell}^{\langle t_1, \ell_1, t_2, \ell_2 \rangle} \cdot \text{addLast}((g_\ell, \mathbb{C}_t, \perp))$ 
21  foreach  $u \in \text{Threads}, \ell' \in \text{HeldLks}$  do
22     $\text{AcqHist}_{t, \ell, \ell'}^{\langle u \rangle} \cdot \text{addLast}((C_{\text{pred}}, \mathbb{C}_t))$ 
23  foreach  $u \neq t \in \text{Threads}, \ell' \in \text{HeldLks}$  do
24     $I := \mathbb{I}^{\langle u, \ell', \ell \rangle} \sqcup C_{\text{pred}}$ 
25     $\mathbb{I}^{\langle u, \ell', \ell \rangle} := \text{checkDeadlock}(\text{AcqHist}_{u, \ell', \ell}^{\langle t \rangle}, I, \langle u, \ell', \ell \rangle)$ 

26 handler readHandler( $t, x$ )
27   $\mathbb{C}_t := \mathbb{C}_t \sqcup \mathbb{LW}_x$ 

```

pattern C formed between η and $\eta' = \langle t' \neq t, \ell_2, \{\ell_1\}, F' \rangle$, in order to check for sync-preserving deadlocks between the deadlock patterns in $F \times F'$. If $\text{CheckAbsDdlck}(C)$ reports no deadlock, the contents of F' are emptied as we are guaranteed that F' will not cause a sync-preserving deadlock with any further acquire of thread t on lock ℓ_1 . For a trace with n events, k threads and $\mathcal{L}$ locks. The algorithm calls CheckAbsDdlck for each of the $O(k^2 \cdot \mathcal{L}^2)$ abstract deadlock patterns of size 2, each call taking $O(n \cdot k)$ time.

SPDOnline is shown in detail in Algorithm 13. The pseudocode contains handlers for processing the different events of σ in a streaming fashion, as well as a helper

function for checking deadlocks. The main data structures of the algorithm are (i) vector clocks $\mathbb{C}_t$, $\mathbb{LW}_x$, and $\mathbb{I}^{(t_1, \ell_1, t_2, \ell_2)}$, (ii) scalar g_ℓ , and (iii) FIFO queues of vector clocks $\text{CSHist}_{t, \ell}^{(t_1, \ell_1, t_2, \ell_2)}$ and $\text{AcqHist}_{t, \ell_1, \ell_2}^{(u)}$, where t, t_1, t_2, u range over threads, x ranges over variables, and ℓ, ℓ_1, ℓ_2 range over locks. $\mathbb{C}_t$ stores the timestamp TS_σ^e where e is the last event in thread t . $\mathbb{LW}_x$ keeps track of the TS_σ^e where e is the last event such that $\text{op}(e) = \text{w}(x)$. $\mathbb{I}^{(t_1, \ell_1, t_2, \ell_2)}$ stores the computed sync-preserving closures for every tuple $\langle t_1, \ell_1, t_2, \ell_2 \rangle$. The scalar variable g_ℓ keeps track of the index of the last acquire event on lock ℓ . Similar to Algorithm 10, the FIFO queue $\text{CSHist}_{t, \ell}^{(t_1, \ell_1, t_2, \ell_2)}$ is maintained to keep track of the critical section history of thread t and lock ℓ . Lastly, for an acquire event e , $\text{AcqHist}_{t, \ell_1, \ell_2}^{(u)}$ maintains a queue of tuples of the form $\langle C_{\text{pred}}, \mathbb{C}_t \rangle$ where C_{pred} and $\mathbb{C}_t$ are the timestamps of $\text{pred}_\sigma(e)$ and e , respectively. These tuples are utilized when checking for deadlocks (Line 25).

Theorem 8. *Consider a trace σ of n events, k threads and $\mathcal{L}$ locks. The online $\text{SPD}_{\text{online}}$ algorithm reports all sync-preserving deadlocks of size 2 of σ in $O(n \cdot k^3 \cdot \mathcal{L}^2)$ time.*

7.6 Experimental Evaluation

We first evaluated our algorithms in an offline setting (Section 7.6), where we record execution traces and evaluate different approaches on the *same* input. This eliminates biases due to non-deterministic thread scheduling. Next, we consider an online setting (Section 7.6), where we instrument programs and perform the analyses during runtime. We conducted all our experiments on a standard laptop with 1.8 GHz Intel Core i7 processor and 16 GB RAM.

Offline Experiments

Experimental setup The goal of the first set of experiments is to evaluate $\text{SPD}_{\text{offline}}$, and compare it against prior algorithms for dynamic deadlock prediction. In order for our evaluation to be precise we evaluate all algorithms on the *same* execution trace. We implemented $\text{SPD}_{\text{offline}}$ in Java inside the RAPID analysis tool [197], following closely the pseudocode in Algorithm 12. RAPID takes as input execution traces, as defined in Section 7.2. These also include fork, join, and lock-request events. We compare $\text{SPD}_{\text{offline}}$ with two state-of-the-art, theoretically-sound albeit computationally more expensive, deadlock predictors, SeqCheck [54] and Dirk [154], both of which also work on execution traces.

On the theoretical side, the complexity of SeqCheck is $\tilde{O}(n^4)$, as opposed to the $\tilde{O}(n)$ complexity of $\text{SPD}_{\text{offline}}$. Moreover, SeqCheck only predicts deadlocks of size 2, and though it could be extended to handle deadlocks of any size, this would degrade performance further. SeqCheck may miss sync-preserving deadlocks even of size 2, but can detect deadlocks that are not sync-preserving. Thus SeqCheck and $\text{SPD}_{\text{offline}}$ are theoretically incomparable in their detection capability. We refer to [296] for examples. We noticed that SeqCheck fails on traces with non-well-nested

locks — we encountered one such case in our dataset. *Dirk*’s algorithm is theoretically complete, i.e., it can find all predictable deadlocks in a trace. In addition, it can find deadlocks beyond the predictable ones, by reasoning about event values. However, *Dirk* relies on heavyweight SMT-solving and employs windowing techniques to scale to large traces. Due to windowing, it can miss deadlocks between events that are outside the given window. As with previous works [54, 154], we set a window size of 10K for *Dirk*.

Our dataset consists of several benchmarks from standard benchmark suites — IBM Contest suite [104], Java Grande suite [277], DaCapo [37], and SIR [91] — and recent literature [54, 151, 153, 154]. Each benchmark was instrumented with RV-Predict [251] or Wiretap [154] and executed in order to log a single execution trace.

Evaluation Table 7.1 presents our results. A bug identifies a unique tuple of source code locations corresponding to events participating in the deadlock. Trace lengths vary vastly from 39 to about 241M, while the number of threads ranges from 3 to about 800, which are representative features of real-world settings. *Hsqldb* contains critical sections that are not well nested, and *SeqCheck* was not able to handle this benchmark; our algorithm does not have such a restriction.

Abstract vs Concrete Patterns Columns 7-9 present statistics on the abstract lock graph ALG_σ of each trace σ . Many traces have a large number of concrete deadlock patterns but much fewer abstract deadlock patterns; a single abstract deadlock pattern can comprise up to an order of 10^4 more concrete patterns (Column 8 v/s Column 9). Unlike all prior sound techniques, our algorithms analyze abstract deadlock patterns, instead of concrete ones. We thus expect our algorithms to be much more scalable in practice.

Deadlock-detection capability In total, both *SeqCheck* and *SPDOffline* reported 40 deadlocks. *SeqCheck* misses a deadlock of size 5 in *DiningPhil*, which is detected by *SPDOffline*, and *SPDOffline* misses a deadlock in *jigsaw* which is detected by *SeqCheck*. As *SPDOffline* is complete for sync-preserving deadlocks, we conclude that there are no more such deadlocks in our dataset. Overall, *SPDOffline* and *SeqCheck* miss only three deadlocks reported by *Dirk*. On closer inspection, we found that these deadlocks are not witnessed by correct reorderings, and require reasoning about event values. On the other hand, *Dirk* struggles to analyze even moderately-sized benchmarks and times out in 3 of them. This results in *Dirk* failing to report 5 deadlocks after 9 hours, all of which are reported by *SPDOffline* in under a minute. Similar conclusions were recently made in [54]. Overall, our results strongly indicate that the notion of sync-preservation characterizes most deadlocks that other tools are able to predict.

Unsoundness of Dirk In our evaluation, we discovered that the soundness guarantee underlying *Dirk* [154] is broken, resulting in it reporting false positives. First, its constraint formulation [154] does not rule out deadlock patterns when acquire events in the pattern hold common locks, in which case mutual exclusion disallows such a pattern to be a real predictable deadlock. Second, *Dirk* also models conditional statements, allowing it to reason about witnesses beyond correct reorderings. While

Table 7.1: Trace characteristics, abstract lock graph statistics and performance comparison. Columns 2-6 show the number of events, threads, variables, locks and total number of lock acquire and lock request events. Columns 7-9 show the number of cycles, abstract and concrete deadlock patterns in the abstract lock graph. Columns 10 - 15 show the number of deadlocks reported and the times (in seconds) taken. by Dirk, SeqCheck, and SPDOffline. Time out (T.O) was set to 3h. F stands for technical failure.

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Benchmark	$\mathcal{N}$	$\mathcal{T}$	$\mathcal{V}$	$\mathcal{L}$	$\mathcal{A}/\mathcal{R}$	A. Lock Graph			Dirk		SeqCheck		SPDOffline	
						Cyc	A. P.	C. P.	Dlk	Time	Dlk	Time	Dlk	Time
Deadlock	39	3	4	3	8	1	1	1	1	0.02	0	0.09	0	0.16
NotADeadlock	60	3	4	5	16	1	1	1	0	0.02	0	0.09	0	0.16
Picklock	66	3	6	6	20	2	2	2	1	0.02	1	0.10	1	0.18
Bensalem	68	4	5	5	22	2	2	2	1	0.02	1	0.12	1	0.16
Transfer	72	3	11	4	12	1	1	1	1	0.02	0	0.09	0	0.15
Test-Dimminux	73	3	9	7	26	2	2	2	2	0.02	2	0.10	2	0.17
StringBuffer	74	3	14	4	16	1	3	6	2	0.02	2	0.12	2	0.19
Test-Calfuzzer	168	5	16	6	48	2	1	1	1	0.02	1	0.12	1	0.17
DiningPhil	277	6	21	6	100	1	1	3K	1	1.60	0	0.09	1	0.17
HashTable	318	3	5	3	174	1	2	43	2	0.19	2	0.12	2	0.19
Account	706	6	47	7	134	3	1	12	0	0.19	0	0.09	0	0.18
Log4j2	1K	4	334	11	43	1	1	1	1	0.65	1	0.11	1	0.20
Dbcp1	2K	3	768	5	56	2	2	3	-	F	2	0.11	2	0.19
Dbcp2	2K	3	592	10	76	1	2	4	-	F	0	0.10	0	0.18
Derby2	3K	3	1K	4	16	1	1	1	1	0.23	1	0.10	1	0.17
RayTracer	31K	5	5K	15	976	0	0	0	-	F	0	0.15	0	0.19
jigsaw	143K	21	8K	2K	67K	172	12	70	-	F	2	0.36	1	1.55
elevator	246K	5	727	52	48K	0	0	0	0	1.65	0	0.33	0	0.27
hedc	410K	7	109K	8	32	0	0	0	0	2.09	0	0.50	0	0.24
JDBCMySQL-1	442K	3	73K	11	13K	2	4	6	2	28.45	2	0.24	2	0.48
JDBCMySQL-2	442K	3	73K	11	13K	4	4	9	1	3.37	1	0.22	1	0.33
JDBCMySQL-3	443K	3	73K	13	13K	5	8	16	1	31.23	1	0.25	1	0.45
JDBCMySQL-4	443K	3	73K	14	13K	5	10	18	2	5.51	2	0.28	2	0.49
cache4j	775K	2	46K	20	35K	0	0	0	0	5.86	0	0.46	0	0.39
ArrayList	3M	801	121K	802	176K	9	3	672	3	8.7K	3	21.98	3	1.68
IdentityHashMap	3M	801	496K	802	162K	1	3	4	1	443.93	1	8.51	1	1.45
Stack	3M	801	118K	2K	405K	9	3	481	1	T.O	3	25.34	3	2.94
Sor	3M	301	2K	3	719K	0	0	0	0	15.89	0	44.12	0	0.61
LinkedList	3M	801	290K	802	176K	9	3	10K	3	4.7K	3	48.02	3	2.06
HashMap	3M	801	555K	802	169K	1	3	10K	3	4.4K	2	504.36	2	1.65
WeakHashMap	3M	801	540K	802	169K	1	3	10K	-	T.O	2	499.68	2	1.70
Swing	4M	8	31K	739	2M	0	0	0	-	F	0	0.72	0	0.88
Vector	4M	3	15	4	800K	1	1	1B	-	T.O	1	1.52	1	1.90
LinkedHashMap	4M	801	617K	802	169K	1	3	10K	2	40.74	2	492.87	2	1.69
montecarlo	8M	3	850K	3	26	0	0	0	0	2.6K	0	1.81	0	0.79
TreeMap	9M	801	493K	802	169K	1	3	10K	2	105.45	2	480.11	2	1.92
hsqldb	20M	46	945K	403	419K	0	0	0	-	F	-	-	0	2.38
sunflow	21M	16	2M	12	1K	0	0	0	-	F	0	8.35	0	1.62
jspider	22M	11	5M	15	10K	0	0	0	-	F	0	8.49	0	1.95
tradesoap	42M	236	3M	6K	245K	2	1	4	-	F	0	108.16	0	7.06
tradebeans	42M	236	3M	6K	245K	2	1	4	-	F	0	116.23	0	7.26
eclipse	64M	15	10M	5K	377K	9	5	280	-	F	0	26.67	0	9.90
TestPerf	80M	50	599	9	197K	0	0	0	0	795.04	0	47.56	0	4.30
Groovy2	120M	13	13M	10K	69K	0	0	0	0	1.7K	0	38.06	0	8.92
Tsp	200M	6	24K	3	882	0	0	0	0	7.6K	0	72.62	0	12.70
lusearch	203M	7	3M	98	273K	0	0	0	0	1.3K	0	75.88	0	14.44
biojava	221M	6	121K	79	16K	0	0	0	-	F	0	63.79	0	12.65
graphchi	241M	20	25M	61	1K	0	0	0	-	F	0	102.05	0	25.25
Totals	1B	7K	70M	37K	8M	256	93	1B	35	>18h	40	2801	40	135

this relaxation allows *Dirk* to predict additional deadlocks in *Transfer*, *Deadlock* and *HashMap*, its formalization is not precise and its implementation is erroneous. We elaborate these aspects further in [296].

Running time Our experimental results indicate that *Dirk*, backed by SMT solving, is the least efficient technique in terms of running time — it takes considerably longer or times out on large benchmark instances. *SPDOffline* analyzed the entire set of traces $\sim 21\times$ faster than *SeqCheck*. On the most demanding benchmarks, such as *HashMap* and *TreeMap*, *SPDOffline* is more than $200\times$ faster than *SeqCheck*. Although *SeqCheck* employs a polynomial-time algorithm for deadlock prediction, and thus significantly faster than the SMT-based *Dirk*, the large polynomial complexity in its running time hinders scalability on execution traces coming from benchmarks that are more representative of realistic workloads. In contrast, the linear time guarantees of *SPDOffline* are realized in practice, allowing it to scale on even the most challenging inputs. More importantly, the improved performance comes while preserving essentially the same precision.

False negatives Our benchmark set contains 93 abstract deadlock patterns, 40 of which are confirmed sync-preserving deadlocks. We inspected the remaining 53 abstract patterns to see if any of them are predictable deadlocks missed by our sync-preserving criterion, independently of the compared tools. 48 of these 53 patterns are in fact not predictable deadlocks — for every such pattern D , the set S_D of events in the downward-closure of $\text{pred}(D)$ with respect to $\leq_{\text{TO}}$ and rf , already contains an event from D , disallowing any correct reordering (sync-preserving or not) in which D can be enabled. Of the remaining, 4 deadlock patterns obey the following scheme: there are two acquire events $\text{acq}_1, \text{acq}_2$ participating in the deadlock pattern, each acq_i is preceded by a critical section on a lock that appears in $\text{HeldLks}(\text{acq}_{3-i})$, again disallowing a correct reordering that witnesses the pattern. Thus, *only one* predictable deadlock is not sync-preserving in our whole dataset. This analysis supports that the notion of sync-preservation is not overly conservative in practice.

The above analysis concerns false negatives wrt. predictable deadlocks. Some deadlocks are beyond the common notion of predictability we have adopted here, as they can only be exposed by reasoning about event values and control-flow dependencies, a problem that is NP-hard even for 3 threads [124]. We noticed 3 such deadlocks in our dataset, found by *Dirk*, though, as mentioned above, *Dirk*'s reasoning for capturing such deadlocks is unsound in practice.

Online Experiments

Experimental setup The objective of our second set of experiments is to evaluate the performance of our proposed algorithms in an *online* setting. For this, we implemented our *SPDOnline* algorithm inside the framework of *DeadlockFuzzer* [151] following closely the pseudocode in Algorithm 13. This framework instruments a concurrent program so that it can perform analysis on-the-fly while executing it. If a deadlock occurs during execution, it is reported and the execution halts. However, if a deadlock is predicted in an alternate interleaving, then this deadlock is reported

and the execution continues to search further deadlocks. We used the same dataset as in Section 7.6, after discarding some benchmarks that could not be instrumented by DeadlockFuzzer.

To the best of our knowledge, all prior deadlock prediction techniques work offline. For this reason, we only compared our online tool with the randomized scheduling technique of [151] already implemented inside the same DeadlockFuzzer framework. At a high level, this random scheduling technique works as follows. Initially, it (i) executes the input program with a random scheduler, (ii) constructs a *lock dependency relation*, and (iii) runs a cycle detection algorithm to discover deadlock patterns. For each deadlock pattern thus found, it spawns new executions that attempt to realize it as an actual deadlock. To increase the likelihood of hitting the deadlock, DeadlockFuzzer biases the random scheduler by pausing threads at specific locations.

The second, confirmation phase of [151] acts as a best-effort proxy for sound deadlock prediction. On the other hand, SPDOnline is already sound and predictive, and thus does not require additional confirmation runs, making it more efficient. Towards effective prediction, we also implemented a simple bias to the scheduler. If a thread t attempts to write on a shared variable x while holding a lock, then our procedure randomly decides to pause this operation for a short duration. This effectively explores race conditions in different orders. Overall, implementing SPDOnline inside DeadlockFuzzer provided the added advantage of supplementing a powerful prediction technique with a biased randomized scheduler. To our knowledge, our work is the first to effectively combine these two orthogonal techniques. We also remark that such a bias is of no benefit to DeadlockFuzzer itself since it does not employ any predictive reasoning.

For this experiment, we run DeadlockFuzzer on each benchmark, and for each deadlock pattern found in the initial execution, we let it spawn 3 new executions trying to realize the deadlock, as per standard (<https://github.com/ksen007/calFUZZER>). We repeated this process 50 times and recorded the total time taken. Then, we allocated the same time for SPDOnline to repeatedly execute the same program and perform deadlock prediction. We measured all deadlocks found by each technique.

Evaluation Table 7.2 presents our experimental results. Columns 2-3 of the table display the total number of bug hits, which is the total number of times a bug was predicted by SPDOnline in the entire duration, or was confirmed in any trial of DeadlockFuzzer. Columns 4-6 display the unique bugs (i.e., unique tuples of source code locations leading to a deadlock) found by the techniques. The employed techniques are able to find a maximum of 3 unique bugs for each benchmark in our benchmark set. Respectively, columns 7-12 display the detailed information on the number of times a particular bug was found by each technique. Runtime overheads are displayed in the columns 13-16, with $-I$ denoting the instrumentation phase only. Deadlock-detection capability DeadlockFuzzer had 2076 bug reports in total, and it found 42 unique bugs. In contrast, SPDOnline flagged 7633 bug reports, corresponding to 49 unique bugs. In more detail, DeadlockFuzzer missed 9 bugs reported by SPDOnline whereas SPDOnline missed 2 bugs reported by DeadlockFuzzer. Also,

Table 7.2: Performance comparison of SPDonline (SPD) and DeadlockFuzzer (DF). Columns 2-3 show the total number of bug reports. Columns 4-6 show the total number of unique bugs found by each tool, and their union. Columns 7-12 show the hit rate on each bug. Columns 13-16 show the runtime overhead of the tools.

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Benchmark	Bug	Hits	Unique Bugs			Bug 1		Bug 2		Bug 3		Runtime Overhead			
	SPD	DF	SPD	DF	All	SPD	DF	SPD	DF	SPD	DF	SPD-I	SPD	DF-I	DF
Deadlock	50	50	1	1	1	50	50	-	-	-	-	2×	3×	2×	4×
Picklock	227	97	2	1	2	226	97	1	0	-	-	2×	2×	2×	5×
Bensalem	355	32	2	1	2	8	0	347	32	-	-	2×	2×	2×	6×
Transfer	54	50	1	1	1	54	50	-	-	-	-	2×	2×	1×	4×
Test-Dimmunix	702	0	2	0	2	351	0	351	0	-	-	2×	2×	2×	4×
StringBuffer	153	131	2	2	2	128	118	25	13	-	-	-	-	-	-
Test-Calfuzzer	177	44	1	1	1	177	44	-	-	-	-	2×	2×	2×	4×
DiningPhil	162	100	1	1	1	162	100	-	-	-	-	-	-	-	-
HashTable	169	120	2	2	2	82	21	87	99	-	-	-	-	-	-
Account	19	188	1	1	1	19	188	-	-	-	-	2×	8×	2×	16×
Log4j2	290	100	2	1	2	145	100	145	0	-	-	-	-	-	-
Dbcp1	265	138	2	2	2	264	61	1	77	-	-	-	-	-	-
Dbcp2	129	126	2	2	2	86	99	43	27	-	-	-	-	-	-
RayTracer	0	0	0	0	0	-	-	-	-	-	-	122×	124×	109×	111×
Tsp	0	0	0	0	0	-	-	-	-	-	-	47×	60×	37×	40×
jigsaw	1189	1	1	1	2	1189	0	0	1	-	-	-	-	-	-
elevator	0	0	0	0	0	-	-	-	-	-	-	2×	2×	2×	2×
JDBCMySQL-1	349	117	2	3	3	1	21	0	4	348	92	3×	4×	2×	13×
JDBCMySQL-2	559	73	1	1	1	559	73	-	-	-	-	2×	4×	2×	18×
JDBCMySQL-3	560	224	1	1	1	560	224	-	-	-	-	2×	5×	2×	24×
JDBCMySQL-4	1717	101	3	1	3	95	0	834	0	788	101	3×	5×	2×	31×
hedc	0	0	0	0	0	-	-	-	-	-	-	2×	2×	1×	2×
cache4j	0	0	0	0	0	-	-	-	-	-	-	2×	2×	2×	2×
lusearch	0	0	0	0	0	-	-	-	-	-	-	16×	17×	13×	16×
ArrayList	47	45	3	3	3	20	22	3	5	24	18	50×	69×	18×	79×
Stack	44	27	3	3	3	18	13	8	4	18	10	69×	91×	64×	86×
IdentityHashMap	68	62	2	2	2	13	47	55	15	-	-	4×	8×	3×	10×
LinkedList	48	26	3	2	3	21	17	7	0	20	9	16×	28×	14×	32×
Swing	0	0	0	0	0	-	-	-	-	-	-	5×	6×	4×	6×
Sor	0	0	0	0	0	-	-	-	-	-	-	2×	7×	2×	2×
HashMap	46	44	2	2	2	18	11	28	33	-	-	7×	11×	4×	8×
Vector	126	50	1	1	1	126	50	-	-	-	-	2×	2×	2×	3×
LinkedHashMap	57	43	2	2	2	22	10	35	33	-	-	10×	10×	4×	8×
WeakHashMap	29	40	2	2	2	6	11	23	29	-	-	7×	12×	4×	8×
montecarlo	0	0	0	0	0	-	-	-	-	-	-	16×	100×	13×	126×
TreeMap	42	47	2	2	2	16	15	26	32	-	-	9×	12×	5×	9×
eclipse	0	0	0	0	0	-	-	-	-	-	-	2×	2×	2×	2×
TestPerf	0	0	0	0	0	-	-	-	-	-	-	2×	2×	2×	2×
Total	7633	2076	49	42	51	-	-	-	-	-	-	-	-	-	-

SPDOnline significantly outperformed DeadlockFuzzer in total number of bugs hits. Our experiments again support that the notion of sync-preservation captures most deadlocks that occur in practice, to the extent that other state-of-the-art techniques can capture. A further observation is that in the offline experiments, SPDOffline was not able to find deadlocks in Transfer and Deadlock. However, the random scheduling procedure allowed SPDOnline to navigate to executions from which deadlocks can be predicted. This demonstrates the potential of combining predictive dynamic techniques with controlled concurrency testing.

Runtime overhead We have also measured the runtime overhead of both SPDOnline and DeadlockFuzzer, both as incurred by instrumentation, as well as by the deadlock analysis. The latter is the time taken by Algorithm 13 for the case of SPDOnline, and the overhead introduced due to the new executions in the second confirmation phase for the case of DeadlockFuzzer. Our results show that the instrumentation overhead of SPDOnline is, in fact, comparable to that of DeadlockFuzzer, though somewhat larger. This is expected, as SPDOnline needs to also instrument memory access events, while DeadlockFuzzer only instruments lock events, but at the same time surprising because the number of memory access events is typically much larger than the number of lock events. On the other hand, the analysis overhead is often larger for DeadlockFuzzer, even though it reports fewer bugs. It was not possible to measure the runtime overhead in certain benchmarks as either they were always deadlocking or the computation was running indefinitely.

7.7 Related Work

Dynamic techniques for detecting deadlock patterns, like the GoodLock algorithm [130] have been improved in performance [53, 323] and precision [32], sometimes using re-executions to verify potential deadlocks [33, 151, 257, 258, 280]. Predictive analyses directly infer concurrency bugs in alternate executions [266] and are typically *sound* (no false positives). This approach has been successfully applied for detecting bugs such as data races [138, 157, 202, 236, 248, 256, 276], use-after-free vulnerabilities [137], and more recently for deadlocks [54, 103, 154].

The notion of sync-preserving deadlocks has been inspired by a similar notion pertaining to data races [202]. However, sync-preserving deadlock prediction rests on some further novelties. First, unlike data races, deadlocks can involve more than 2 events. Generalizing sync-preserving ideals of sets of events of arbitrary size, as well as establishing the monotonicity properties (Proposition 1 and Corollary 1) for arbitrarily many events is non-trivial. Second, our notions of abstract deadlock patterns (Section 11) and abstract lock graphs (Section 11) are novel and carefully crafted to leverage these monotonicity properties in the deadlock setting. Indeed, the linear-time sync-preserving verification of each abstract deadlock pattern is the cornerstone of our approach, for the first linear-time, sound and precise deadlock predictor.

Although the basic principles of data-race and deadlock prediction are similar, there are notable differences. First, identifying potential deadlocks is theoretically

intractable, whereas, potential races are identified easily. Second, popular partial-order based techniques [111, 157] for data races are likely to require non-trivial modifications for deadlocks, as they typically order critical sections, which may hide a deadlock. Nevertheless, bridging prediction techniques between data races and deadlocks is an interesting and relatively open direction.

Predicting deadlocks is an intractable problem, the complexity of which we have characterized in this work. Prior works have also focused on the complexity of predicting data races [164, 200] and atomicity violations [106].

7.8 Conclusion

We have studied the complexity of deadlock prediction and introduced the new tractable notion of sync-preserving deadlocks, along with sound, complete and efficient algorithms for detecting them. Our experiments show that the majority of deadlocks occurring in practice are indeed sync-preserving, and our algorithm `SPDOffline` is the first deadlock predictor that achieves sound and high coverage, while also spending only linear time to process its input. Our online algorithm `SPDOnline` enhances the bug detection capability of controlled concurrency testing techniques like [151], at close runtime overheads. Interesting future work includes incorporating static checks [244] and exploring ways for deeper integration of controlled concurrency testing with predictive techniques. Another step is to extend the coverage of sync-preserving deadlocks while maintaining efficiency, for example, by reasoning about program control flow.

Chapter 8

Optimal Reads-From Consistency Checking for C11-Style Memory Models

By Hünkar Can Tunç, Parosh Aziz Abdulla, Soham Chakraborty, Shankaranarayanan Krishna, Umang Mathur, Andreas Pavlogiannis. Proceedings of the ACM on Programming Languages, Volume 7, Issue PLDI, June, 2023.

Abstract

Over the years, several memory models have been proposed to capture the subtle concurrency semantics of C/C++. One of the most fundamental problems associated with a memory model $\mathcal{M}$ is *consistency checking*: given an execution X , is X consistent with $\mathcal{M}$? This problem lies at the heart of numerous applications, including specification testing and litmus tests, stateless model checking, and dynamic analyses. As such, it has been explored extensively and its complexity is well-understood for traditional models like SC and TSO. However, less is known for the numerous model variants of C/C++, for which the problem becomes challenging due to the intricacies of their concurrency primitives. In this work we study the problem of consistency checking for popular variants of the C11 memory model, in particular, the RC20 model, its release-acquire (RA) fragment, the strong and weak variants of RA (SRA and WRA), as well as the Relaxed fragment of RC20.

Motivated by applications in testing and model checking, we focus on *reads-from* consistency checking. The input is an execution X specifying a set of events, their program order and their reads-from relation, and the task is to decide the existence of a modification order on the writes of X that makes X consistent in a memory model. We draw a rich complexity landscape for this problem; our results include (i) nearly-linear-time algorithms for certain variants, which improve over prior results, (ii) fine-grained optimality results, as well as (iii) matching upper and lower bounds (NP-hardness) for other variants. To our knowledge, this is the first work to characterize the complexity of consistency checking for C11 memory models. We have implemented our

algorithms inside the TruSt model checker and the C11Tester testing tool. Experiments on standard benchmarks show that our new algorithms improve consistency checking, often by a significant margin.

8.1 Introduction

Modern programming languages such as C/C++ [142, 143] have introduced first-class platform-independent concurrency primitives to gain performance from weak memory architectures. The programming model is popularly known as C11 [28, 42]. The formal semantics of C11 has been an active area of research [28, 29, 59, 169, 176, 196, 301] and other programming languages such as Java [31] and Rust [82] have also adopted similar concurrency primitives.

One of the most fundamental computational problems associated with a memory model, particularly in testing and verification, is that of *consistency checking* [120, 123, 163]. Here, one focuses on a fixed memory model $\mathcal{M}$, typically described using constraints or axioms. The input to the consistency problem pertaining to the memory model $\mathcal{M}$ is then a partial execution X , typically described using a set of events E together with a set of relations on E . The consistency problem then asks to determine if X is *consistent* with $\mathcal{M}$. Here, by *partial* execution, we mean that the set of relations is not fully described in the input, in which case the problem asks whether X can be extended to a complete execution that is consistent with $\mathcal{M}$. The focus of this paper is *reads-from* consistency checking; in the rest of the paper we refer to this simply as consistency checking.

The problem of consistency checking has numerous applications in both software and hardware verification. First, viewing memory models as contracts between the system designer and the software developers, consistency checking is a common approach to testing memory subsystems, cache-coherence protocols and compiler transformations against the desired contract [69, 193, 238, 308, 312]. Second, since public documentations of memory architectures are typically not entirely formal, litmus tests can reveal or dismiss behaviors that are not covered in the documentation [18–20]. Consistency checking for litmus tests makes testing more efficient (and thus also more scalable), by avoiding the enumeration of behaviors that are impossible under the given model. Third, in the area of model checking, (partial) executions typically serve the role of abstraction mechanisms. Consistency checking, thus, ensures that model checkers indeed explore valid system behavior. As such, it has been instrumental in guiding recent research in partial-order reduction techniques and stateless model checking of concurrent software [8, 9, 12, 49, 61, 65, 162, 226]. Focusing on partial executions allows such algorithms to consider coarser equivalences such as the *reads-from* equivalence, allowing for more proactive state-space reductions and better performance as a result. These advances have also propelled the use of formal methods in the industry [45, 177, 227]. Consistency checking of partial executions also forms the foundation of dynamic *predictive analyses* by characterizing the space of perturbations that can be applied to an observed execution in an attempt

to expose a bug [138, 154, 157, 191, 199, 200, 202, 236].

The ubiquitous relevance of consistency checking has led to a systematic study of its computational complexity under various memory models. Under sequential consistency (SC), most variants of the problem were shown to be NP-hard in the seminal work of Gibbons and Korach [123]. Subsequently, more fine-grained investigations have characterized how input parameters such as the number of threads, memory locations, write accesses and communication topology affect the complexity of consistency checking [9, 12, 73, 200]. As the consistency problems remain intractable under most common weak memory models (such as SPARC/X86-TSO, PSO, RMO, PRAM) [120], parametric results have also been established for these models [49, 73]. Given its applications in analysis of concurrent programs, clever heuristics have been proposed to enhance the efficiency of checking consistency in practice [320].

The C11 memory model provides the flexibility to derive different weak memory model paradigms based on different subsets and combinations of the concurrency primitives, their memory orders, and their respective semantics. For instance, the release and acquire memory orders allow programmers to derive release-acquire (RA) as well as its weak (WRA) and strong (SRA) variants [165]. The RA model is weaker than SC and provides a rigorous foundation in defining synchronization and locking primitives [168]. The WRA and SRA are equivalent to variants of causal consistency [165], a well studied consistency model in the distributed systems literature. C11 also provides ‘relaxed’ memory access modes which constitutes the weaker memory model fragment Relaxed. Relaxed memory accesses can reorder freely and are the most performant compared to accesses with stronger memory orders. In our work, we focus on the recently proposed declarative RC20 memory model [196] capturing a rich fragment of C11, consisting of release, acquire and relaxed memory accesses as well as memory fence operations. This memory model is a natural fragment of the C11 model, given that “*there are only a few (practical) algorithms that actually employ SC accesses and become wrong when release/acquire accesses are used instead*” [196]. Further, focusing on the non-SC fragment allows us to reap the benefits of polynomial time consistency checking, which otherwise quickly becomes intractable [123].

The intricacies of C11 and the abundance of its variants give rise to a plethora of consistency-checking instances. Some first results show that consistency checking for RA admits a polynomial bound [8, 167], a stark difference to SC for which this problem is NP-hard and is not even well-parameterizable [123, 200]. These positive results have facilitated efficient model checking and testing techniques [8, 161, 191]. However, beyond these recent developments, little is known about the complexity of consistency checking for C11-style memories, and, to our knowledge, the setting remains poorly understood. Our work fills this gap.

Our contributions. In this paper we study the reads-from consistency checking for the RA, SRA, WRA, Relaxed and RC20 memory models, with results that are optimal or nearly-optimal. In all cases, the input is a partial execution $\bar{X} = \langle E, \text{po}, \text{rf} \rangle$ with $n = |E|$ events and k threads, where po and rf are the program order and reads-from relation, respectively (see Section 8.2), and the task is to determine if there is a

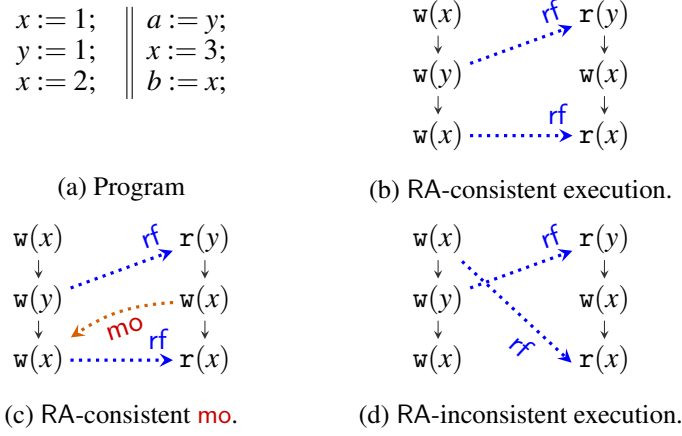


Figure 8.1: A program (a) and a partial execution $\bar{X}$ specifying the writer $rf^{-1}(r)$ of each read r (b). $\bar{X}$ is RA-consistent, as witnessed by the modification order **mo** that abides to RA semantics (c). The partial execution in (d) is RA-inconsistent, as there is no modification order **mo** that abides to RA semantics.

modification order **mo** such that the extension $X = \langle E, po, rf, mo \rangle$ is consistent with the memory model in consideration. Figure 8.1 illustrates an example for RA-consistency.

Our first result concerns RC20. Consistency checking is known to be in polynomial time [8, 167, 191], though of degree 3 (i.e., $O(n^3)$). This cubic complexity has been identified as a challenge for efficient model checking (e.g., [161, 162]). Here we show that the full RC20 model admits an algorithm that is nearly linear-time; i.e., a bound that becomes linear when the number of threads is bounded.

Theorem 9. *Consistency checking for RC20 can be solved in $O(n \cdot k)$ time.*

A key step towards Theorem 9 is our notion of *minimal coherence*, which is a novel characterization that serves as a witness of consistency. Our consistency-checking algorithm proves consistency by constructing a minimally coherent (partial) modification order. Although similar witnesses have been used in the past (e.g., the writes-before order [167], saturated traces [8], or C11Tester’s framework [191]), the simplicity of minimal coherence allows, for the first time to our knowledge, for a nearly linear-time algorithm.

Next we turn our attention to SRA. Perhaps surprisingly, although the model is conceptually close to RA, it turns out that checking consistency for SRA is intractable.

Theorem 10. *Consistency checking for SRA is NP-complete, and $\mathcal{W}[1]$ -hard in the parameter k .*

Here $\mathcal{W}[1]$ is a parameterized complexity class [67]. This result states that, not only is the problem NP-complete, but it is also unlikely to be *fixed parameter tractable* in k , i.e., solvable in time $O(n^c \cdot f(k))$, where $c > 0$ and f are independent of n . Nevertheless, our next result shows that this problem admits an upper bound

that is polynomial when $k = O(1)$. Given the $\mathcal{W}[1]$ -hardness, the next result is thus optimal, in the sense that k has to appear in the exponent of n .

Theorem 11. *Consistency checking for SRA can be solved in $O(k \cdot n^{k+1})$ time.*

Taking a closer look into the model, we identify RMWs as the source of intractability. Indeed, the RMW-free fragment of SRA admits a nearly linear bound, much like RC20. This fragment is relevant, as it coincides with the causal convergence model [47].

Theorem 12. *Consistency checking for the RMW-free fragment of SRA can be solved in $O(n \cdot k)$ time.*

Next, we show that the problem can be solved just as efficiently for WRA.

Theorem 13. *Consistency checking for WRA can be solved in $O(n \cdot k)$ time.*

Turning our attention to the Relaxed fragment of RC20, we show that the problem admits a truly linear bound (i.e., regardless of the number of threads).

Theorem 14. *Consistency checking for Relaxed can be solved in $O(n)$ time.*

Finally, observe that, in contrast to Theorem 14, the bounds in Theorem 9, Theorem 12 and Theorem 13 can become super-linear in the presence of many threads. It is thus tempting to search for a truly linear-time algorithm for any of RA, WRA and (RMW-free) SRA. Unfortunately, our final result shows that this is unlikely, in all models.

Theorem 15. *There is no consistency-checking algorithm for the RMW-free fragments of any of RA, WRA, and SRA that runs in time $O(n^{\omega/2-\varepsilon})$, for any fixed $\varepsilon > 0$. Moreover, there is no combinatorial algorithm for the problem that runs in time $O(n^{3/2-\varepsilon})$, under the combinatorial BMM hypothesis.*

Here ω is the matrix multiplication exponent, with currently $\omega \simeq 2.37$. Theorem 15 states that a truly linear-time algorithm for any of these models would bring matrix multiplication in $n^{2+o(1)}$ time, a major breakthrough. Focusing on combinatorial algorithms (i.e., excluding algebraic fast-matrix multiplication, which appears natural in our setting), consistency checking for any of these models requires at least $n^{3/2}$ time unless (boolean) matrix multiplication (BMM) is improved below the classic cubic bound (which is considered unlikely, aka the BMM hypothesis [310]).

Due to space restrictions, we relegate all proofs to our technical report [294].

Experiments. We have implemented our algorithms inside the TruSt model checker and the C11Tester testing tool, and evaluated their performance on consistency checking for benchmarks utilizing instructions in the RA model. Our results report consistent and often significant speedups that reach $162\times$ for TruSt and $104.2\times$ for C11Tester.

Overall, our efficiency results enable practitioners to perform model checking and testing for RC20, RMW-free SRA, WRA, and Relaxed more efficiently, and apply these techniques to larger systems. On the other hand, our hardness result for SRA indicates that, akin to SC, performing consistency checking for SRA efficiently requires developing practically oriented heuristics that work well in the common cases. Finally, our super-linear lower bound for all models except Relaxed indicates that further improvements over our $O(n \cdot k)$ bounds will likely be highly non-trivial.

8.2 Axiomatic Concurrency Semantics

In this section we introduce the C/C++ concurrency semantics we consider in this work, along with the RC20 model and its variants [165, 196].

Syntax. C/C++ defines a shared memory concurrency model using different kinds of concurrency primitives. In addition to plain (or non-atomic) load and store accesses, C/C++ provides atomic accesses for load, store, atomic read-modify-write (RMW – such as atomic increment), and fence operations. We only consider atomic accesses here. An atomic access is parameterized by a memory mode, among relaxed (RLX), acquire (ACQ), release (REL), acquire-release (ACQ-REL), and sequential-consistency (SC). The memory order for a read, write, RMW, and fence access is one of $\{\text{RLX}, \text{ACQ}, \text{SC}\}$, $\{\text{RLX}, \text{REL}, \text{SC}\}$, $\{\text{RLX}, \text{ACQ}, \text{REL}, \text{ACQ-REL}, \text{SC}\}$, and $\{\text{ACQ}, \text{REL}, \text{ACQ-REL}, \text{SC}\}$, respectively. These accesses result in different types of events during execution. In this paper we consider the models which are based on non-SC primitives. Nevertheless, RC20 defines SC fences using the release-acquire primitives [165]. The memory modes are partially ordered on increasing strength of synchronization according to the lattice $\text{RLX} \sqsubset \{\text{ACQ}, \text{REL}\} \sqsubset \text{ACQ-REL}$. An access is *acquire* (*release*) if its order is ACQ (REL), or stronger.

Executions

The axiomatic concurrency models are defined with respect to the executions they allow. Hence, a program can be represented as a set of executions. In turn, an execution is defined by a set of events that are generated from shared memory accesses or fences, and relations between these events.

Events. An event is a tuple $\langle \text{id}, \text{tid}, \text{lab} \rangle$ where *id*, *tid*, *lab* denote a unique identifier, thread identifier, and label of the event. The label $\text{lab} = \langle \text{op}, \text{ord}, \text{loc} \rangle$ is a tuple where *op* denotes the corresponding memory access or fence operation and *ord* denotes the memory mode. For memory accesses, *loc* denotes its memory location, while in the case of fences, we have $\text{loc} = \perp$. For the purpose of the reads-from consistency problem we consider in this paper, we omit the *values* read or written in memory access events. We write $w(t, x)/r(t, x)/\text{rmw}(t, x)$ to denote a write/read/read-modify-write event in thread *t* on location *x*, and simply write $e(x)$ to denote an event for which the thread is implied or not relevant. As a matter of convention, we omit mentioning the memory order throughout the paper, as it will either be clear from the context or not relevant.

The set of read, write, atomic update, and fence events are R, W, RMW and F respectively, and are generated from the executions of load, store, atomic load store, fence accesses respectively. As we only deal with executions (as opposed to program source), we use RMW to denote a successful read-modify-write operation. Failed read-modify-write operations simply result in read accesses. We refine the set of events in various ways. For instance, $E^{\sqsupset_{\text{REL}}}$ denotes the set of events with memory order that is at least as strong as REL. For a set of events E, we write $E.\text{locs}$, E_x , and $E.\text{tids}$ to denote the set of distinct locations accessed by events in E, the subset of events in E that access memory location x , and the different threads participating in E.

Notation on relations. Consider a binary relation S over a set of events E. The reflexive, transitive, reflexive-transitive closures, and inverse relations of S are denoted as $S^?$, S^+ , S^* , S^{-1} respectively. The relation S is acyclic if S^+ is irreflexive. We write $\text{irr}(S)$ and $\text{acy}(S)$ to denote that relation S irreflexive and acyclic respectively. Given two relations S_1 and S_2 , we denote their composition by $S_1; S_2$. $[A]$ denotes the identity relation on a set A , i.e. $[A](x, y) \triangleq x = y \wedge x \in A$. For a given memory location x , we let $S_x = [E_x]; S; [E_x]$ be the restriction of S to all events of E on x .

Candidate executions and relations. An execution (also referred to as candidate execution [28] or execution graph [169]) is a tuple $X = \langle E, \text{po}, \text{rf}, \text{mo} \rangle$ where $X.E$ is a set of events and $X.\text{po}$, $X.\text{rf}$, $X.\text{mo}$ are binary relations over $X.E$. In particular, the *program order* po is a partial order that enforces a total order on events of the same thread. The *reads-from relation* $\text{rf} \subseteq (W \cup \text{RMW}) \times (R \cup \text{RMW})$ associates write/RMW events e_1 to read/RMW events e_2 reading from e_1 . Naturally, we require that $e_1.\text{loc} = e_2.\text{loc}$, and that rf^{-1} is a function, i.e., every read/RMW event has a unique writer. The *modification order* $\text{mo} \subseteq (W \cup \text{RMW}) \times (W \cup \text{RMW})$ is the union of modification orders mo_x , where each mo_x is a total order over $(W_x \cup \text{RMW}_x)$.

We frequently also use some derived relations (see Figure 8.2). The *from-read relation* $\text{fr} \subseteq (R \cup \text{RMW}) \times (W \cup \text{RMW})$ relates every read or RMW event to all the write or RMW events that are mo -after its own writer. The *synchronizes-with relation* $\text{sw} \subseteq E^{\sqsupset_{\text{REL}}} \times E^{\sqsupset_{\text{ACQ}}}$ relates release and acquire events, for instance, when an acquire read event reads from a release write. Fence instructions combined with relaxed accesses also participate in sw . The *happens-before relation* hb is the transitive closure of po and sw . We also project hb to individual locations, denoted as hblloc .

Consistency Axioms

Consistency axioms characterize different aspects or constraints of a memory model. We broadly classify these constraints as coherence, atomicity, global ordering, and causality cycles. Different interpretations of these constraints give rise to different consistency models as shown in Figure 8.2.

Coherence. In an execution, *coherence* enforces ‘SC-per-location’: the memory accesses per memory locations are totally ordered. *Write-coherence* enforces that mo agrees with hb . A stronger variant is *strong-write-coherence*, which requires that this condition holds transitively. *Read coherence* enforces that a read $r(x)$ cannot read from a write $w(x)$ if there is an *intermediate* write $w'(x)$ that happens-before $r(x)$,

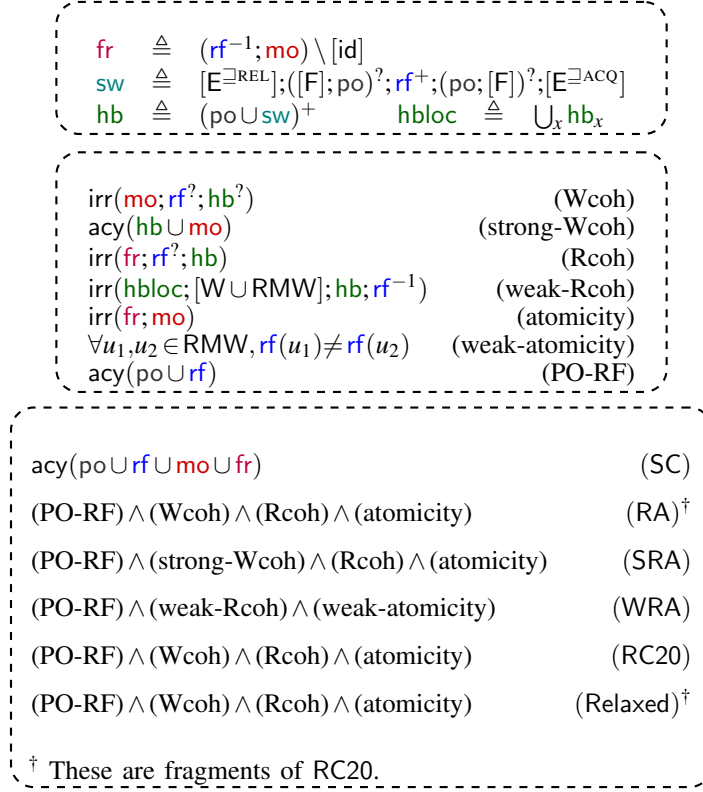


Figure 8.2: Relations, Axioms, and Consistency Models on C11 Concurrency.

i.e. $hb(w'(x), r(x))$ holds. In the vanilla read-coherence, the notion of ‘intermediate’ relates to **mo**, i.e., we have $(w(x), w'(x)) \in mo$, while in *weak-read-coherence* [165], ‘intermediate’ relates to **hb**, i.e., we have $(w(x), w'(x)) \in hb$.

Atomicity. The property ensures that (successful) RMW accesses indeed update the memory locations atomically. Following [165], we consider two variants. *Atomicity* ensures that no intermediate write event on the same location takes place between an RMW and its writer. *Weak-atomicity* simply prohibits two RMW events to have the same writer.

Causality cycles. A causality cycle arises in the presence of primitives that have weaker behaviors than release-acquire accesses. Such a cycle consists of **po** and **rf** orderings and may result in ‘out-of-thin-air’ behavior in certain programs. To avoid such ‘out-of-thin-air’ behavior, many consistency models explicitly disallow such cycles [165, 191]. In the absence of RLX accesses, the PO-RF axiom coincides with the requirement for **hb** acyclicity.

Axiomatic Consistency Models and Consistency Checking

We can now present the memory models we consider in this work. See Figure 8.2 for a summary.

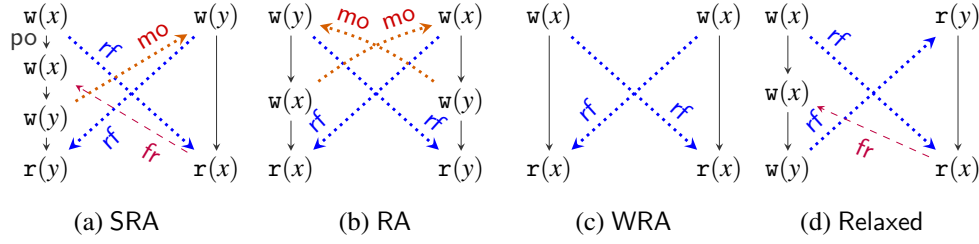


Figure 8.3: Executions consistent in various memory models. **mo** edges that go along $(po \cup rf)$ are not shown.

Sequential consistency. Sequential consistency (SC) enforces a global order on all memory accesses. This is a stronger constraint than coherence, which orders same-location memory accesses. In addition, SC also guarantees atomicity.

Release-Acquire and variants. The release-acquire (RA) memory model is weaker than SC, and is arguably the most well-understood fragment of C11. At the same time, RA enables high-performance implementations of fundamental concurrency algorithms [86, 168]. Broadly, under the release-acquire semantics (including other related variants), each read-from ordering establishes a synchronization. In this case hb reduces to $hb \triangleq (po \cup rf)^+$. Following [165], we consider three variants of release-acquire models: RA, strong RA (SRA), and weak RA (WRA). These models coincide with standard variants of causal consistency [47, 50, 187], which is a ubiquitous consistency model relevant also in other domains such as distributed systems.

SRA enforces a stronger coherence guarantee (namely strong-write-coherence) on write accesses compared to RA. WRA does not place any restrictions on the **mo** ordering between same-location writes. Instead, the only orderings considered between same-location writes are through the $[W]; hbloc; [W]$ relation. Thus, WRA provides weaker constraints for coherence and atomicity.

RC20. The recently introduced RC20 model [196] defines a rich fragment of the C11 model consisting of acquire/release and relaxed accesses. Despite the absence of SC accesses, RC20 can express many practical synchronization algorithms, and can simulate SC fences.

Relaxed. Finally, the relaxed fragment of RC20 contains only RLX accesses, resulting in $hb = po$.

Comparison between memory models. The above models can be partially ordered according to the behaviors (executions) they allow as $SC \preceq SRA \preceq RA \preceq \{WRA, \{RC20 \preceq Relaxed\}\}$, with models towards the right allowing more behaviors. All models are weaker than SC. Relaxed is weaker than RA but incomparable with WRA. In particular, the lack of synchronization across **rf** in Relaxed makes hb weaker in Relaxed compared to WRA. On the other hand, WRA allows extra behaviors over Relaxed because it does not enforce write-coherence. Finally, RC20 can be viewed as a combination of RA and Relaxed, where fences may add synchronization between relaxed accesses. See Figure 8.3 for an illustration.

Extensions of the models with non-atomics. For ease of presentation, we do not

explicitly handle non-atomic accesses. The above memory models can be straightforwardly extended to include non-atomics with “catch-fire” semantics, similarly to previous works [165]. Intuitively, non-atomic accesses on any given location must always be **hb**-ordered, as otherwise this implies a data race, leading to undefined behavior [142, 143].

The reads-from consistency problem. An execution $X = \langle E, \text{po}, \text{rf}, \text{mo} \rangle$ is *consistent* in a memory model $\mathcal{M}$, written $X \models \mathcal{M}$, if it satisfies the axioms of the model. A *partial execution* $\bar{X} = \langle E, \text{po}, \text{rf} \rangle$ is an abstraction of executions without the modification order. We call $\bar{X}$ *consistent* in $\mathcal{M}$, written similarly as $\bar{X} \models \mathcal{M}$, if there exists an **mo** such that $X \models \mathcal{M}$, where $X = \langle E, \text{po}, \text{rf}, \text{mo} \rangle$. Thus the problem of *reads-from consistency checking* (or simply consistency checking, from now on) is to find an **mo** that turns $\bar{X}$ consistent*.

8.3 Auxiliary Functions, Data Structures and Observations

Our consistency checking algorithms rely on some common notation and computations. To avoid repetition, we present these here as auxiliary functions, while we refer to Figure 8.4 for examples.

Happens-before computation. One common component in most of our algorithms is the computation of the **hb**-timestamp $\text{HIB}_e : E.\text{tids} \rightarrow \mathbb{N}_{\geq 0}$ of each event e , declaratively defined as

$$\text{HIB}_e(t) = |\{f \mid f.\text{tid} = t \wedge (f, e) \in \text{hb}^?\}|$$

That is, $\text{HIB}_e(t)$ points to the last event of thread t that is **hb**-ordered before (and including) e . The computation of all HIB_e can be computed by a relatively straightforward algorithm (see e.g., [191]). We will thus take the following proposition for granted in this work.

Proposition 2. The happens-before relation can be computed in $O(n \cdot k)$ time.

Last write and last read. Given a thread t , location x , and index $c \in \mathbb{N}_{\geq 0}$, we define

$$\text{lastWriteBefore}(t, x, c) = \begin{cases} e & \text{if } e \text{ is the last event such that, } e \in W_x \cup \text{RMW}_x, \\ & e.\text{tid} = t \text{ and } |\{g \mid (g, e) \in \text{po}^?\}| \leq c \\ \perp & \text{if no such event exists} \end{cases}$$

$$\text{lastReadBefore}(t, x, c) = \begin{cases} e & \text{if } e \text{ is the last event such that, } e \in R_x \cup \text{RMW}_x, \\ & e.\text{tid} = t \text{ and } |\{g \mid (g, e) \in \text{po}^?\}| \leq c \\ \perp & \text{if no such event exists} \end{cases}$$

*Except for WRA, the axioms of which do not involve **mo**.

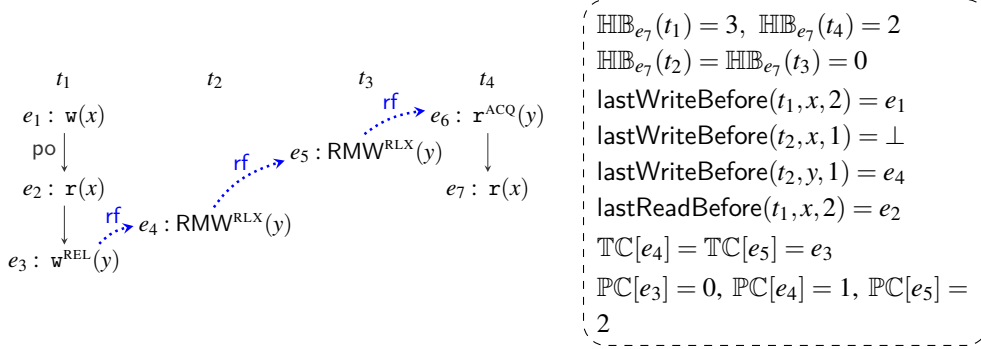


Figure 8.4: Example of a partial execution (left) and its auxiliary functions (right).

In words, $\text{lastWriteBefore}(t, x, c)$ returns the latest po -predecessor $w(t, x)$ or $rmw(t, x)$ of the c -th event of thread t (similarly for $\text{lastReadBefore}(t, x, c)$). When our consistency algorithms process a read/RMW event $e(u, x)$, they query for $\text{lastWriteBefore}(t, x, c)$ and $\text{lastReadBefore}(t, x, c)$ on each thread t , where $c = \mathbb{H}\mathbb{B}_e(t)$. We call u the *observer thread*. Our efficient handling of such queries is based on the insight that, along subsequent queries from the same observer thread, c is monotonically increasing ($\mathbb{H}\mathbb{B}$ timestamps are monotonic along po -ordered events). We develop a simple data structure for handling such queries as follows. For each thread t , memory location x , and observer thread u , we maintain lists $\mathbb{W}\mathbb{L}_{t,x}^u$ and $\mathbb{R}\mathbb{L}_{t,x}^u$, each containing the sequence of write/RMW events and read/RMW events performed by t on x , together with their thread-local indices. The parameterization by u ensures that u observes its own local version of this list. Answering a query $\text{lastWriteBefore}(t, x, c)$ consists of iterating over $\mathbb{W}\mathbb{L}_{t,x}^u$ until the correct event is identified. Subsequent queries continue the iteration from the last returned position. The total cost of traversing all these lists is $O(n \cdot k)$ (each event appears in k lists, one per observer thread). In pseudocode descriptions, we will call $\mathbb{W}\mathbb{L}_{t,x}^u \cdot \text{get}(c)$ (resp., $\mathbb{R}\mathbb{L}_{t,x}^u \cdot \text{get}(c)$) to access the event $e = \text{lastWriteBefore}(t, x, c)$ (resp., $e = \text{lastReadBefore}(t, x, c)$). This implementation of lastWriteBefore and lastReadBefore is novel compared to prior works (e.g., [191]), and crucial for obtaining the linear-time bounds developed in our work.

Top of, and position in rf -chain. All memory models we consider satisfy weak-atomicity (atomicity implies weak atomicity), i.e., no two RMW events can have the same writer. This implies that all write and RMW events are arranged in disjoint rf -chains, where a chain is a maximal sequence of events $e_0, e_1, \dots, e_\ell$ ($\ell \geq 0$), such that (i) $e_0 \in W$ and $e_1, \dots, e_\ell \in RMW$, and (ii) $rf^{-1}(e_i) = e_{i-1}$ for each $i \geq 1$. In words, we have a chain of events connected by rf , starting with the top write event e_0 , and (optionally) continuing with a maximal sequence of RMW events that read from this chain. Given an event $e \in (W \cup RMW)$, we often refer to the rf -chain that contains e . Specifically, the top of the chain $\text{TC}[e]$ is the unique event f such that $(f, e) \in rf^*$ and $f.op = w$. The position $\text{PC}[e]$ of e in its rf -chain is $|\{f \mid (f, e) \in rf^+\}|$. Both TC and PC can be computed in $O(n)$ time for all events.

Conflicting triplets. A *conflicting triplet* (or just *triplet*, for short) is a triplet of distinct events (e_1, e_2, e_3) such that (i) all events access the same location x , (ii) $e_1, e_3 \in (W_x \cup \text{RMW}_x)$ and $e_2 \in (R_x \cup \text{RMW}_x)$, and (iii) $\text{rf}^{-1}(e_2) = e_1$.

Finally, we state a simple lemma that is instrumental throughout the paper. This lemma identifies certain **mo** orderings implied by the basic axioms of read-coherence, write-coherence and atomicity, and thus applies to all models except WRA. Figure 8.5 provides an illustration.

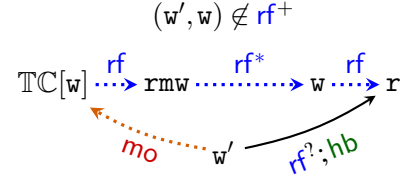


Figure 8.5: An **mo** ordering implied by read-coherence, write-coherence and atomicity.

Lemma 15. Let $X = \langle E, \text{po}, \text{rf}, \text{mo} \rangle$ be an execution that satisfies read-coherence, write-coherence and atomicity. Let (w, r, w') be a triplet. If $(w', r) \in \text{rf}^?; \text{hb}$ and $(w', w) \notin \text{rf}^+$, then $(w', \text{TC}[w]) \in \text{mo}$.

8.4 Consistency Checking

This section presents the main results of the paper, as outlined in Section 8.1. We start with an algorithm for checking consistency under WRA in Section 8.4. For SRA, we show in Line 11 that the problem is NP-complete in general, but has a polynomial time algorithm for the RMW-free programs as shown in Line 11. Line 12 and Line 17 show that consistency checking is polynomial time for RC20 and linear-time for Relaxed along with the respective algorithms. Finally, we study the lower bound of consistency checking for RMW-free RA, WRA, and SRA in Line 16.

Consistency Checking for WRA

We start with the WRA model, which is conceptually simpler as there is no **mo** involved in the consistency axioms. Algorithm 14 checks for consistency in $O(n \cdot k)$, towards Theorem 13.

Given a partial execution $\bar{X} = (E, \text{po}, \text{rf})$, the algorithm first verifies that there are no $(\text{po} \cup \text{rf})$ -cycles and every write/RMW event is read by at most one RMW event (Line 1). Afterwards, the algorithm streams the events of E in an order consistent with po and verifies weak-read-coherence, i.e., there is no triplet (w, r, w') such that $\{(w, w'), (w', r)\} \subseteq \text{hb}$. In order to check this in linear time, the algorithm first computes the array of **hb**-timestamps (Line 2) and the last write events $\text{lastWriteBefore}(t, x, c)$ for each thread t , location x and index c (Line 3), as defined in Section 8.3.

In order to check that weak-read-coherence is not violated, at a read/RMW event e with $e.\text{tid} = t$ and $e.\text{loc} = x$, Algorithm 14 checks if there is an event $e' \in W_x \cup \text{RMW}_x$ such that e' is **hb**-sandwiched between $\text{rf}^{-1}(e)$ and e . Since $\text{po} \subseteq \text{hb}$, it suffices to check if for any thread u , the event $\text{lastWriteBefore}(u, x, \mathbb{H}\mathbb{B}_e[u])$ can play the role of e' above (Line 10).

Algorithm 14: Checking Consistency for WRA.

Input: A partial execution $\bar{X} = (E, \text{po}, \text{rf})$

- 1 **if** $(\text{po} \cup \text{rf})$ is cyclic or rf violates weak-atomicity **then declare** ‘Inconsistent’
- 2 **let** HIB be an E-indexed array storing the hb-timestamps of events
- 3 **let** $\{\text{WL}_{t,x}^u\}_{t,x,u}$ be data structures implementing lastWriteBefore($\cdot, \cdot, \cdot$)
- 4 **foreach** $e \in E$ in po-order **do**
- 5 **case** $e = \text{r}(t, x)$ or $e = \text{rmw}(t, x)$ **do**
- 6 **let** $w = \text{rf}^{-1}(e)$, $t' = w.\text{tid}$ and $c = \text{HIB}[w][t']$
- 7 **foreach** $u \in E.\text{tids}$ **do**
- 8 **let** $c_u = \text{if } (e.\text{op} = \text{rmw} \wedge u = t) \text{ then } \text{HIB}[e][u] - 1 \text{ else } \text{HIB}[e][u]$
- 9 **let** $w_u = \text{WL}_{t,x}^u.\text{get}(c_u)$
- 10 **if** $(\text{HIB}[w_u][t'] \geq c) \wedge ((u = t') \Rightarrow \text{HIB}[w_u][t'] > c)$ **then declare**
- 11 ‘Inconsistent’
- 11 **declare** ‘Consistent’

The total running time on an input partial execution $\bar{X}$ with n events and k threads can be computed as follows. The initialization of HIB and the lists $\{\text{WL}_{t,x}^u\}_{t,x,u}$, and the total cost of all calls to $\text{WL}_{t,x}^u.\text{get}(c_u)$ takes $O(n \cdot k)$ time (Section 8.3). Afterwards, the algorithm spends $O(k)$ time at each event. This gives a total running time of $O(n \cdot k)$. We thus have the following theorem.

Theorem 13. *Consistency checking for WRA can be solved in $O(n \cdot k)$ time.*

Consistency Checking for SRA

We now turn our attention to SRA, and prove the bounds of Theorem 10 and Theorem 11.

The hardness of consistency checking for SRA. First, note that consistency checking is a problem in NP. Indeed, given a partial execution $\bar{X} = (E, \text{po}, \text{rf})$, one can guess a candidate mo and verify that $X \models \text{SRA}$, where $X = (E, \text{po}, \text{rf}, \text{mo})$ is a complete execution, by checking against the axioms of SRA. Each axiom can be verified in polynomial time, giving us membership in NP. Now we turn our attention to $\mathcal{W}[1]$ -hardness (which will also imply NP-hardness). Our reduction is from the consistency problem for SC, which is known to be NP-hard [123] and more recently shown to be $\mathcal{W}[1]$ -hard [200]. We obtain hardness in two steps.

First, we observe that the consistency problem for SC is $\mathcal{W}[1]$ -hard even over instances in which every write event is observed at most once. This can be obtained from the proof of $\mathcal{W}[1]$ -hardness in [200]. Towards our $\mathcal{W}[1]$ -hardness proof for SRA, we can substitute in such instances every read access by an RMW access without affecting the SC consistency of the execution. Intuitively, as any write observed by a read does not have any other readers, the write of the substituting RMW has no effect. Formally, we have the following lemma.

Lemma 16. *Consistency checking for SC with only write and RMW events is $\mathcal{W}[1]$ -hard in the parameter k .*

Given Lemma 16, we can now prove Theorem 10. The key observation is that the strong-write-coherence of SRA implies a total order on all write/RMW events. Thus, over instances where every event is either a write or an RMW, strong-write-coherence yields a total order on all events, which, in turn, implies an SC-consistent execution. We arrive at the following theorem.

Theorem 10. *Consistency checking for SRA is NP-complete, and $\mathcal{W}[1]$ -hard in the parameter k .*

A parameterized upper bound. We now turn our attention to Theorem 11, i.e., we solve consistency checking for SRA in time $O(k \cdot n^{k+1})$. Recall that our goal is to construct an **mo** that witnesses the consistency of $\bar{X} = (E, \text{po}, \text{rf})$. One natural approach is to enumerate all possible **mo**'s and check whether any of them leads to a consistent X . However, this leads to an exponential algorithm regardless of the number of threads (there are exponentially many possible **mo**'s even with two threads) which is beyond the bound of Theorem 11. We instead follow a different approach.

Algorithm. Given the poset (E, hb) , a set $Y \subseteq E$ is said to be *downward-closed* if for all $e_1 \in Y, e_2 \in E$, if $(e_2, e_1) \in \text{hb}$ then $e_2 \in Y$. We define a (directed) *downward graph* $\mathcal{G}_{\bar{X}}$ induced by (E, hb) , and show that the question of $\bar{X} \models \text{SRA}$ reduces to checking reachability in $\mathcal{G}_{\bar{X}}$. The node set of $\mathcal{G}_{\bar{X}}$ consists of all downward closed subsets S of E , with $\emptyset$ being the *root node* and E being the *terminal node*. Given a node S in $\mathcal{G}_{\bar{X}}$, we insert edges $S \rightarrow S'$ where S' is obtained by extending S with an event which is *executable* in S . An event e executable in S if the following conditions hold.

1. All events e' such that $(e', e) \in \text{hb}$ are in S .
2. If $e \in (W \cup \text{RMW})$ is a write/RMW event, then it must also be *enabled*. We say that a write/RMW event w is enabled if the following hold.
 - (a) For every triplet (w, r, w') , if $(w', r) \in \text{hb}$, then $w' \in S$. Intuitively, executing w while $w' \notin S$ represents a guess that $(w, w') \in \text{mo}$, which would violate read-coherence as $(w', r) \in \text{hb}$.
 - (b) For every RMW event rmw and triplet (w', rmw, w) , if $\text{rmw} \notin S$ then $w' \notin S$. Intuitively, executing w while $\text{rmw} \notin S$ but $w' = \text{rf}^{-1}(\text{rmw}) \in S$ represents a guess that $(w', w) \in \text{mo}$ and $(w, \text{rmw}) \in \text{mo}$, which would violate atomicity as it would imply $(\text{rmw}, w) \in \text{fr}$ and thus $(\text{rmw}, \text{rmw}) \in \text{fr}; \text{mo}$.

Figure 8.6 illustrates the above notions. Conceptually, every path from the root $\emptyset$ to a node S in $\mathcal{G}_{\bar{X}}$ represents an **mo** on the write/RMW events of S . Although there can be exponentially many such paths, the node S “forgets” their corresponding **mo**'s. Instead, S represents a partial **mo**, which orders every write/RMW event on a location x of S before every write/RMW event on x outside

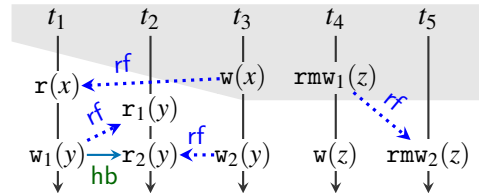


Figure 8.6: Enabledness and executability of a set S , marked in gray. Only $r(x)$ and $\text{rmw}_2(z)$ are executable events. $w_1(y)$ is not executable as $r(x) \notin S$. $r_1(y)$ is not executable as $w_1(y) \notin S$. $w_2(y)$ is not enabled as $(w_1(y), r_2(y)) \in \text{hb}$ but $w_1(y) \notin S$. $w(z)$ is not enabled as $\text{rmw}_2(z) \notin S$.

5. The algorithm terminates and returns that $\bar{X} \models \text{SRA}$ iff the node E is reachable from the root node $\emptyset$ in $\mathcal{G}_{\bar{X}}$. Observe that $\mathcal{G}_{\bar{X}}$ contains $O(n^k)$ nodes, while each node has $\leq k$ successors. Deciding whether a node has a transition to another node can be easily done in $O(n)$ time. Hence, the total time of the algorithm is $O(k \cdot n^{k+1})$. We thus arrive at the following theorem.

Theorem 11. *Consistency checking for SRA can be solved in $O(k \cdot n^{k+1})$ time.*

Consistency Checking for the RMW-Free Fragment of SRA

On close inspection, RMW events played a central role in the NP-hardness of Theorem 10. A natural question thus arises: does the hardness persist in the absence of RMWs? Here we show that the RMW-free fragment of SRA can be handled efficiently (Theorem 12). Observe that Lemma 15 applies to SRA, as strong-write-coherence implies write-coherence. However, the lemma admits a simplification under RMW-free SRA. In particular, as X does not contain RLX accesses, $(w', r) \in \text{rf}^?; \text{hb}$ reduces to $(w', r) \in \text{hb}$. Moreover, as X is RMW-free, we have $\text{TC}[w] = w$, and hence $(w', w) \in \text{mo}$. Thus, Lemma 15 reduces to the following corollary.

Corollary 2. Consider any execution $X = \langle E, \text{po}, \text{rf}, \text{mo} \rangle$ that satisfies read-coherence and strong-write-coherence. Consider any triplet (w, r, w') . If $(w', r) \in \text{hb}$ then $(w', w) \in \text{mo}$.

Minimal coherence under SRA. Corollary 2 identifies necessary orderings in any modification order that witnesses the consistency of $\bar{X}$. Towards an algorithm, we must also determine if there are non-trivial conditions *sufficient* to conclude consistency. We answer this in the positive, by capturing these conditions in the notion of *minimal coherence*. Consider a partial modification order $\bar{\text{mo}} = \bigcup_x \bar{\text{mo}}_x$, where each $\bar{\text{mo}}_x$ is a partial order. We call $\bar{\text{mo}}$ *minimally coherent for $\bar{X}$ under SRA* if the following conditions hold.

1. For every triplet (w, r, w') with $(w', r) \in \text{hb}$, we have $(w', w) \in (\text{hb} \cup \bar{\text{mo}})^+$.
2. $(\text{hb} \cup \bar{\text{mo}})$ is acyclic.

Figure 8.7 illustrates the notion of minimal coherence under SRA. Observe that any mo witnessing the consistency of $\bar{X}$ satisfies these conditions. In the following, we show that minimally coherent modification orders are also sufficient for witnessing consistency. We note that for RMW-free executions, minimal coherence coincides with the previous notions of coherence that witnesses consistency under RA [8, 167, 191]. However, these notions also handle RMWs, and

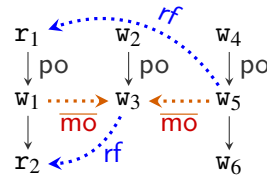


Figure 8.7: A minimally coherent $\bar{\text{mo}}$ under SRA. All events access the same location.

Algorithm 15: Checking consistency for the RMW-free fragment of SRA.

Input: Events E , program order po and reads-from relation rf

```

1 if  $(po \cup rf)$  is cyclic then declare ‘Inconsistent’
2 let  $\mathbb{HB}$  be an  $E$ -indexed array storing the  $hb$ -timestamps of events
3 let  $\{\mathbb{WL}_{t,x}^u\}_{t,x,u}$  be data structures implementing  $\text{lastWriteBefore}(\cdot, \cdot, \cdot)$ 
4 foreach  $x \in E.locs$  do  $\overline{mo}_x \leftarrow \emptyset$ 
5 foreach  $e \in E$  in po-order do
6   case  $e = r(t, x)$  do
7     let  $w_{rf} = rf^{-1}(e)$ 
8     foreach  $u \in E.tids$  do
9       let  $w_u = \mathbb{WL}_{u,x}^t \cdot \text{get}(\mathbb{HB}[e][u])$ 
10      if  $w_u \neq w_{rf}$  then  $\overline{mo}_x \leftarrow \overline{mo}_x \cup \{(w_u, w_{rf})\}$ 
11 if  $(hb \cup \bigcup_{x \in E.locs} \overline{mo}_x)$  is cyclic then declare ‘Inconsistent’
12 else declare ‘Consistent’

```

are not directly applicable in SRA, as the problem of consistency checking is NP-hard for SRA with RMWs (Theorem 10).

Lemma 17. *Consider any RMW-free, partial execution $\bar{X} = (E, po, rf)$. If there exists partial modification order $\overline{mo}$ that is minimally coherent for $\bar{X}$ under SRA, then $\bar{X} \models \text{SRA}$.*

Algorithm. Corollary 2 and Lemma 17 suggest a polynomial-time algorithm for deciding the SRA consistency of an RMW-free, partial execution $\bar{X} = (E, po, rf)$. Similarly to WRA, we first verify that $(po \cup rf)$ is acyclic. Then, we construct a partial modification order $\overline{mo}$ by identifying all conflicting triplets (w, r, w') such that $(w', r) \in hb$, and inserting an ordering $(w', w) \in \overline{mo}$. Finally, we report that $\bar{X} \models \text{SRA}$ iff $(hb \cup \overline{mo})$ is acyclic.

Although this process runs in polynomial time, it is still far from the nearly linear bound we aim for (Theorem 12). The key extra step towards this bound comes from a closer look at minimal coherence: based on Item 1., it suffices to only consider conflicting triplets (w, r, w') in which w' is po-maximal among all write events w'' forming a conflicting triplet (w, r, w'') such that $(w'', r) \in hb$. For each thread t , we thus only need to identify the po-maximal write w' in the scheme outlined above. This concept is illustrated in Figure 8.7. Consider the triplets (w_3, r_2, w_4) , (w_3, r_2, w_5) , and (w_3, r_2, w_6) . Only the first two satisfy the above definition (since $(w_4, r_2), (w_5, r_2) \in hb$ but $(w_6, r_2) \notin hb$). In this case, only identifying the event w_5 is sufficient as it is the po-maximal write among w_4 and w_5 .

This insight is precisely formulated in Algorithm 15. The algorithm uses the auxiliary functions from Section 8.3 to compute the $\mathbb{HB}$ -timestamp of each event. Also recall that, for threads t and u and location x , $\mathbb{WL}_{t,x}^u$ denotes (thread u ’s copy of) the po-ordered list of write accesses performed by t on location x . It then processes events in $\bar{X}$ in an order consistent with po and builds a partial modification order $\overline{mo}$. When processing a read event r , the algorithm identifies for every thread u , the

po-maximal write w' of u that forms a conflicting triplet (w, r, w') with $(w', r) \in \text{hb}$ (Line 9), and inserts $(w', w) \in \overline{\text{mo}}$ (Line 10). Finally, it checks whether $\overline{\text{mo}}$ violates strong-write-coherence.

Correctness and complexity. The completeness follows directly from Corollary 2: every ordering inserted in $\overline{\text{mo}}_x$ is present in any modification order mo that witnesses the consistency of $\overline{X}$, while the acyclicity check in Line 11 is necessary for strong-write-coherence. Hence, if the algorithm returns “Inconsistent”, we have $\overline{X} \not\models \text{SRA}$. The soundness comes from the fact that $\overline{\text{mo}}$ satisfies Item 1. of minimal coherence at the end of the loop of Line 5, while if the acyclicity check in Line 11 passes, $\overline{\text{mo}}$ also satisfies Item 2. of minimal coherence. Thus, by Lemma 17, $\overline{X} \models \text{SRA}$.

The time spent in computing HIB and initializing and accessing the lists $\text{WL}_{t,x}^u$ is $O(n \cdot k)$ (Section 8.3). The number of orderings added in $\overline{\text{mo}}$ is $O(n \cdot k)$, taking $O(n \cdot k)$ total time. Finally, the check in Line 11 is $O(n \cdot k)$ time as it corresponds to detecting a cycle on a graph with $|E| = n$ nodes and $\leq n \cdot (k + 1)$ edges. This gives a total running time of $O(n \cdot k)$. We thus arrive at Theorem 12.

Theorem 12. *Consistency checking for the RMW-free fragment of SRA can be solved in $O(n \cdot k)$ time.*

Consistency Checking for RC20

We now turn our attention to the full RC20 memory model, which comprises a mixture of REL, ACQ and RLX memory accesses. Similarly to the RMW-free SRA, we obtain a nearly linear bound (Theorem 9). Note, however, that here we also allow RMW events. As RC20 satisfies read-coherence, write-coherence and atomicity, Lemma 15 applies also in this setting. However, our earlier notion of minimal coherence under SRA is no longer applicable as is — Lemma 17 does not hold for RC20. Fortunately, we show this model enjoys a similar notion of coherence minimality.

Minimal coherence under RC20. Consider a partial modification order $\overline{\text{mo}} = \bigcup_x \overline{\text{mo}}_x$. We call $\overline{\text{mo}}$ *minimally coherent for $\overline{X}$ under RC20* if the following conditions hold.

1. For every triplet (w, r, w') accessing location x , if $(w', r) \in \text{rf}^?; \text{hb}$ and $(w', w) \notin \text{rf}^+$, then $(w', \text{TC}[w]) \in (\text{rf}_x \cup \text{hb}_x \cup \overline{\text{mo}}_x)^+$.
2. For every two write/RMW events w_1, w_2 accessing location x , if $(w_1, w_2) \notin \text{rf}^+$ and $(w_1, w_2) \in \overline{\text{mo}}_x$, then $(w_1, \text{TC}[w_2]) \in \overline{\text{mo}}_x$.
3. $(\text{rf}_x \cup \text{hb}_x \cup \overline{\text{mo}}_x)$ is acyclic, for each $x \in E.\text{locs}$.

Figure 8.8 illustrates the above definition. Observe that any mo witnessing the consistency of $\overline{X}$ satisfies minimal coherence. As before, minimal coherence is also a sufficient witness of consistency.

Lemma 18. *Consider any partial execution $\overline{X} = (E, \text{po}, \text{rf})$. If there exists partial modification order $\overline{\text{mo}}$ that is minimally coherent for $\overline{X}$ under RC20, then $\overline{X} \models \text{RC20}$.*

Our algorithm for consistency checking in RC20 relies on Lemma 18 to construct a minimally-coherent partial modification order that witnesses the consistency of $\bar{X}$. In particular, the algorithm employs the simple inference rule of $\overline{\text{mo}}$ edges illustrated earlier in Figure 8.5, and is a direct application of Item 1. of minimal coherence. At a glance, it might come as a surprise that such a simple rule suffices to deduce consistency. Indeed, analogous relations have been used in the past as consistency witnesses (e.g., the writes-before order [167], saturated traces [8], or C11Tester’s framework [191]). However, these witness relations are stronger than minimal coherence, while the algorithms for computing them (and thus checking consistency) have a higher polynomial complexity $O(n^3)$ (or $O(n^2 \cdot k)$) compared to our nearly linear bound.

On a more technical level, not every total extension of $(\text{rf}_x \cup \text{hb}_x \cup \overline{\text{mo}}_x)$ qualifies as the complete mo_x that witnesses consistency; in particular, some extensions might violate atomicity. This is also the case in prior witness relations [8, 167, 191]. However, a key difference between prior work and minimal coherence is the following. In prior witness relations, the events of an rf -chain are either totally ordered or unordered with respect to any event outside this chain. In contrast, minimal coherence allows only some events of the rf -chain being ordered with outside events. For example, in Figure 8.8 observe that $(\text{rmw}_1, w_2) \in \overline{\text{mo}}$. This implies that, due to atomicity, the pair (rmw_2, w_2) must be ordered in any valid total extension of $\overline{\text{mo}}$. However, minimal coherence does not force (rmw_2, w_2) in $\overline{\text{mo}}$. Nevertheless, our proof of Lemma 18 shows that, as long as $\overline{\text{mo}}_x$ is minimally coherent, there always exists an extension $\overline{\text{mo}}'_x \supseteq \overline{\text{mo}}_x$ that can serve as the witnessing modification order, in the spirit of the prior notions of witness relations. In Figure 8.8, for example, this extension would be $\overline{\text{mo}}'_x = \overline{\text{mo}}_x \cup (\text{rmw}_2, w_3)$.

Algorithm. The insights made above are turned into a consistency checking procedure in Algorithm 16. This algorithm first verifies the absence of $(\text{po} \cup \text{rf})$ cycles (which also implies that $\text{hb} \subseteq (\text{po} \cup \text{rf})^+$ is irreflexive), and that rf follows weak-atomicity (Line 1). Then, it computes auxiliary data discussed in Section 8.3 (Lines 2–4). The main computation is performed in Lines 6–14, where the algorithm constructs a minimally coherent partial modification order $\overline{\text{mo}}_x$ for each location x . The algorithm iterates over all read/RMW events e accessing some location x , and identifies $w_{\text{rf}} = \text{rf}^{-1}(e)$. Then, it iterates over all threads u and identifies the po-maximal write/RMW event w_u such that either $(w', e) \in \text{hb}$ (in which case w' is the event w_u^w in Line 11) or $(w', e) \in \text{rf}; \text{hb}$ (in which case w' is the event w_u^r in Line 11). It then checks whether $(w_u, w_{\text{rf}}) \notin \text{rf}^+$, by checking that either w_u and w_{rf} belong to different rf -chains ($(\text{TC}[w_{\text{rf}}] \neq \text{TC}[w_u])$), or w_{rf} appears earlier than w_u in the common rf -chain ($(\text{PC}[w_{\text{rf}}] < \text{PC}[w_u])$); see Line 13. If so, the algorithm inserts an ordering $(w_u, \text{TC}[w_{\text{rf}}])$ in $\overline{\text{mo}}_x$ (Line 14). Finally, Line 16 verifies that $\overline{\text{mo}}_x$ satisfies write-coherence. Figure 8.9a displays the resulting $\overline{\text{mo}}$ computed by Algorithm 16 on a partial execution.

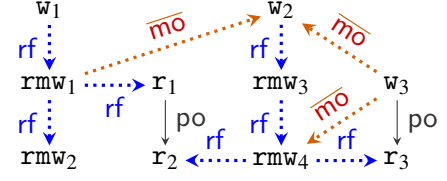


Figure 8.8: A minimally coherent $\overline{\text{mo}}$ under RC20. All events access the same location.

Algorithm 16: Checking consistency for RC20.

Input: Events E , program order po and reads-from relation rf

```

1 if  $(po \cup rf)$  is cyclic or  $rf$  violates weak-atomicity then declare ‘Inconsistent’

2 let  $HIB$  be an  $E$ -indexed array storing the  $hb$ -timestamps of events
3 let  $\{WL_{t,x}^u\}_{t,x,u}$  and  $\{RL_{t,x}^u\}_{t,x,u}$  be data structures implementing
   lastWriteBefore() and lastReadBefore()
4 let  $TC$  and  $PC$  be  $E$ -indexed arrays denoting the top and position of events in
   their  $rf$ -chains
5 foreach  $x \in E.locs$  do  $\overline{mo}_x \leftarrow \emptyset$ ;
6 foreach  $e \in E$  in  $po$ -order do
7   case  $e = r(t,x)$  or  $e = rmw(t,x)$  do
8     let  $w_{rf} = rf^{-1}(e)$ 
9     foreach  $u \in E.tids$  do
10      let  $c_u = \text{if } (e.op = rmw \wedge u = t) \text{ then } HIB[e][u] - 1 \text{ else } HIB[e][u]$ 
11      let  $w_u^w = WL_{u,x}^t \cdot get(c_u)$  and let  $w_u^r = rf^{-1}(RL_{u,x}^t \cdot get(c_u))$ 
12      for  $w_u \in \{w_u^w, w_u^r\}$  do
13        if  $(TC[w_{rf}] \neq TC[w_u])$  or  $(PC[w_{rf}] < PC[w_u])$  then
14           $\overline{mo}_x \leftarrow \overline{mo}_x \cup \{(w_u, TC[w_{rf}])\}$ 
15 foreach  $x \in E.locs$  do
16   if  $(rf_x \cup hb_x \cup \overline{mo}_x)$  is cyclic then declare ‘Inconsistent’
17 declare ‘Consistent’

```

Correctness and complexity. Completeness follows from Lemma 15: every ordering inserted in $\overline{mo}_x$ is present in any modification order mo that witnesses the consistency of $\overline{X}$, while the acyclicity check in Line 16 is necessary for write-coherence. Hence, if the algorithm returns “Inconsistent”, $\overline{X} \not\models RC20$. The soundness comes from the fact that $\overline{mo}$ satisfies Item 1. of minimal coherence at the end of the loop of Line 6. Since all orderings inserted in $\overline{mo}$ are to the top of an rf -chain, Item 2. of minimal coherence is trivially satisfied at all times. Finally, if the acyclicity check in Line 16 passes, $\overline{mo}$ also satisfies Item 3. of minimal coherence. Thus, by Lemma 18, we have $\overline{X} \models RC20$.

The time spent in computing HIB , (TC, PC) and accessing the lists $\{WL_{t,x}^u\}_{t,x,u}$ and $\{RL_{t,x}^u\}_{t,x,u}$ is $O(n \cdot k)$ (Section 8.3). The number of orderings added in $\overline{mo}$ is $O(n \cdot k)$, taking $O(n \cdot k)$ total time. For each location $x \in E.locs$, the acyclicity check in Line 16 can be performed in $O(n_x \cdot k)$ time, where $n_x = |W_x \cup RMW_x|$. For this, we construct a graph G_x that consists of all events $(W_x \cup RMW_x)$ and $O(n_x \cdot k)$ edges. Given two events $e_1 = (t_1, x)$, $e_2 = (t_2, x)$ we have an edge $e_1 \rightarrow e_2$ in G_x iff $(e_1, e_2) \in (rf \cup \overline{mo}_x)$ or $e_1 = \text{lastWriteBefore}(t_1, x, HIB[e_2][t_1] - 1)$. We then check for a cycle in G_x . Repeating this for all locations x , we obtain $O(n \cdot k)$ total time. We thus arrive at Theorem 9.

Theorem 9. Consistency checking for RC20 can be solved in $O(n \cdot k)$ time.

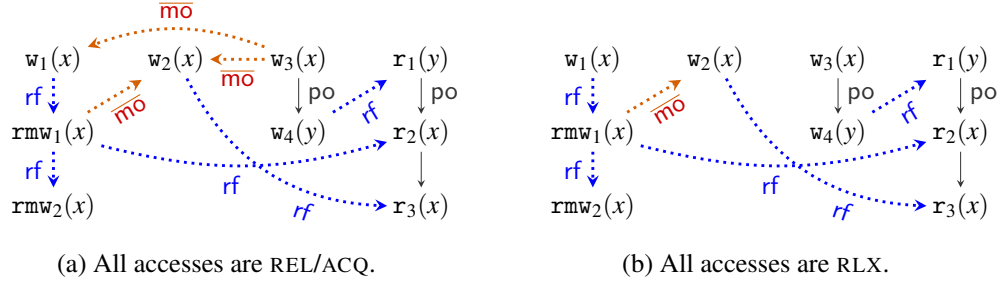


Figure 8.9: A minimally coherent $\overline{mo}$ computed by Algorithm 16 (a) and Algorithm 17 (b).

Consistency Checking for Relaxed

We now turn our attention to the Relaxed fragment. As a strict subset of RC20 (where $hb = po$), consistency checking for this model can be performed in $O(n \cdot k)$ time by Theorem 9. Although this bound is nearly linear time, here we show that the Relaxed fragment enjoys a *truly* linear time consistency checking, independent of k (Theorem 14). This improvement is based on two insights.

As rf edges do not induce any synchronization in this fragment, our first insight is that the input partial execution $\bar{X} = (E, po, rf)$ can be partitioned into separate executions $\bar{X}_x = (E_x, po_x, rf_x)$, one for each location $x \in E.locs$. Indeed, we have $\bar{X} \models \text{Relaxed}$ iff $acy(po \cup rf)$ and $\bar{X}_x \models \text{Relaxed}$ for each $x \in E.locs$. Thus, without loss of generality, we may assume that $\bar{X}$ consists of a single location. Our second insight comes from the simplified formulation of minimal coherence under Relaxed.

Minimal coherence under Relaxed. Let us revisit the concept of minimal coherence under RC20. Focusing on the Relaxed fragment, we have $hb = po$. Thus, the first and third conditions of minimal coherence are reduced to the following.

- (1') For every triplet (w, r, w') accessing location x , if $(w', r) \in rf^?$; po and $(w', w) \notin rf^+$, we have $(w', \mathbb{TC}[w]) \in (rf_x \cup po_x \cup \overline{mo}_x)^+$.
- (3') $(rf_x \cup po_x \cup \overline{mo}_x)$ is acyclic, for each $x \in E.locs$.

Similarly to RC20, Figure 8.8 also serves as an illustration of the above definition. The key insight towards a truly linear-time algorithm is as follows. Consider the execution of Algorithm 16 on a partial execution $\bar{X}$ under Relaxed semantics. Further, consider a read/RMW event e processed by the algorithm with $w_{rf} = rf^{-1}(e)$. Among all events w' forming a triplet (w_{rf}, e, w') and such that $(w', e) \in rf^?; po$, there exists one that is $(rf \cup po \cup \overline{mo}_x)^+$ -maximal. In particular, if the immediate po -predecessor of e is a read event r , then the event $rf^{-1}(r)$ is this maximal w' . Otherwise, the immediate po -predecessor of e is a write/RMW event w'' , which is also the maximal w' . Thus, it suffices to keep track of this information on-the-fly, and only insert $(w', \mathbb{TC}[w_{rf}])$ in $\overline{mo}_x$, if necessary, to make $\overline{mo}_x$ minimally-coherent. As we now do not have to compute $\mathbb{HIB}$ -timestamps or iterate over all threads during the processing of e , we have a truly linear-time algorithm.

Algorithm. The above insights are turned into an algorithm in Algorithm 17. The algorithm first verifies that $(po \cup rf)$ is acyclic and rf satisfies weak-atomicity (Line 1).

Algorithm 17: Checking consistency for Relaxed.**Input:** Events E , program order po and reads-from relation rf

```

1 if  $(po \cup rf)$  is cyclic or  $rf$  violates weak-atomicity then declare ‘Inconsistent’

2 let  $\mathbb{TC}$  and  $\mathbb{PC}$  be  $E$ -indexed arrays denoting the top and position of events in
   their  $rf$ -chains
3 foreach  $x \in E.locs$  do
4   foreach  $t \in E_x.tids$  do  $LW_{t,x} \leftarrow \text{NIL}$ 
5    $\overline{mo}_x \leftarrow \emptyset$ 
6   foreach  $e \in E_x$  in  $(po_x \cup rf_x)$ -order do
7     case  $e = w(t, x)$  do
8        $LW_{t,x} \leftarrow e$ 
9     case  $e = r(t, x)$  do
10      if  $(\mathbb{TC}[rf_x^{-1}(e)] \neq \mathbb{TC}[LW_{t,x}])$  or  $(\mathbb{PC}[rf_x^{-1}(e)] < \mathbb{PC}[LW_{t,x}])$  then
11         $\overline{mo}_x \leftarrow \overline{mo}_x \cup \{(LW_{t,x}, \mathbb{TC}[rf_x^{-1}(e)])\}$ 
12         $LW_{t,x} \leftarrow rf^{-1}(e)$ 
13      case  $e = rmw(t, x)$  do
14        Execute Lines 10–11 followed by Line 8
15      if  $(po_x \cup rf_x \cup \overline{mo}_x)$  is cyclic then declare ‘Inconsistent’
16 declare ‘Consistent’

```

Then, it performs a separate pass for each location x and constructs the minimally coherent $\overline{mo}_x$. To this end, it keeps track in $LW_{t,x}$ the unique $(rf_x \cup po_x \cup \overline{mo}_x)^+$ -maximal write/RMW event that has an rf^2 ; po path to the current event of thread t . When a read/RMW event e is processed, the algorithm potentially updates $\overline{mo}_x$ with an ordering $(LW_{t,x}, \mathbb{TC}[w_{rf}])$ (Line 10), using the same condition as in Algorithm 16. Figure 8.9 contrasts the $\overline{mo}$ computed by Algorithm 17 to the $\overline{mo}$ computed by Algorithm 16 on the same partial execution but with different access levels. We arrive at the following theorem.

Theorem 14. *Consistency checking for Relaxed can be solved in $O(n)$ time.*

A Super-Linear Lower Bound for RMW-Free RA, WRA, and SRA

Finally, we address the existence of a truly linear-time algorithm for any model other than Relaxed. We show that this is unlikely, by proving the two lower bounds of Theorem 15. The proof is via a *fine-grained* reduction from the problem of checking triangle freeness in undirected graphs, which suffers the same lower bounds. That is, there is no algorithm (resp. combinatorial algorithm, under the BMM hypothesis) for checking triangle-freeness in time $O(n^{\omega/2-\varepsilon})$ (resp. $O(n^{3/2-\varepsilon})$) for any fixed $\varepsilon > 0$ [311], where n is the number of nodes in the graph.

Reduction. Given a graph $G = (V_G, E_G)$ of n vertices, we construct an RMW-free partial execution $\bar{X} = \langle E, po, rf \rangle$ with $|E| = O(n)$ such that $\bar{X}$ is consistent with any of RA, WRA and SRA iff G is triangle-free. For simplicity, we let $V_G = \{1, \dots, n\}$.

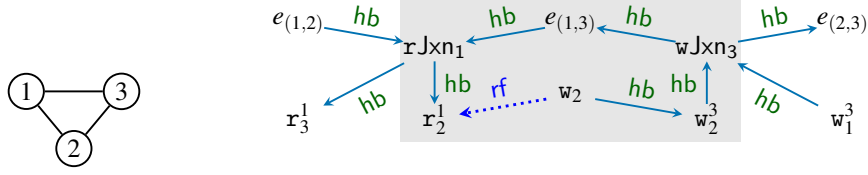


Figure 8.10: *Left*: A graph G with three nodes $V_G = \{1, 2, 3\}$ containing a triangle. *Right*: A slice of the partial execution $\bar{X}$ for G . We have $(w_2, w_2^3) \in \text{hb}$ and $(w_2^3, r_2^1) \in \text{hb}$, thus violating weak read coherence in $\bar{X}$.

Events and memory locations. We start with the event set E . For the moment, all events belong to different threads, while we only define the memory location of an event when relevant. For every node $\alpha \in V_G$, $\bar{X}$ contains (i) a location y_α and a write event $w_\alpha(y_\alpha)$ and (ii) auxiliary “junction” events $rJxn_\alpha$ and $wJxn_\alpha$ on fresh locations. For every edge $(\alpha, \beta) \in E_G$ with $\alpha < \beta$, $\bar{X}$ contains (i) an event $e_{(\alpha, \beta)}$ that accesses a fresh location, (ii) a read event $r_\beta^\alpha(y_\beta)$, and (iii) a write event $w_\alpha^\beta(y_\alpha)$.

Relations rf and hb. We now define the rf relation. Our construction also makes certain events hb ordered. This can be done trivially by introducing auxiliary events with an rf relation between them, while $\bar{X}$ remains of size $O(n)$. In particular, every $(e_1, e_2) \in \text{hb}$ edge can be simulated using fresh events r, w such that (i) $(w, r) \in \text{rf}$ (ii) $(e_1, w) \in \text{po}$, and (iii) $(r, e_2) \in \text{po}$. For simplicity of presentation, we do not mention these events explicitly, but rather directly the hb relation they result in. For every edge $(\alpha, \beta) \in E_G$ with $\alpha < \beta$, we have the following relations:

$$\begin{aligned} (w_\beta, r_\beta^\alpha) &\in \text{rf} & (w_\alpha, w_\alpha^\beta) &\in \text{hb} & (w_\alpha^\beta, wJxn_\beta) &\in \text{hb} \\ (wJxn_\beta, e_{(\alpha, \beta)}) &\in \text{hb} & (e_{(\alpha, \beta)}, rJxn_\alpha) &\in \text{hb} & (rJxn_\alpha, r_\beta^\alpha) &\in \text{hb} \end{aligned}$$

Figure 8.10 illustrates the above construction for a slice of the constructed partial execution $\bar{X}$. We conclude with a proof sketch of Theorem 15, and refer to [294] for the full proof.

Correctness and time complexity. If there is a triangle (α, β, γ) in G with $\alpha < \beta, \gamma$, then $(w_\beta^\gamma, r_\beta^\alpha) \in \text{hb}$ because of the sequence of hb edges: $(w_\beta^\gamma, wJxn_\gamma)$, $(wJxn_\gamma, e_{(\alpha, \gamma)})$, $(e_{(\alpha, \gamma)}, rJxn_\alpha)$, $(rJxn_\alpha, r_\beta^\alpha)$. Together with $(w_\beta, r_\beta^\alpha) \in \text{rf}$ and $(w_\beta, w_\beta^\gamma) \in \text{hb}$ by construction, we obtain a weak-read-coherence violation. In the other direction, if there are no triangles in G , then the modification order $\text{mo} = \bigcup_{\alpha \in V_G} \text{mo}_{y_\alpha}$ where mo_{y_α} orders w_α before every other write w_α^β on y_α , makes $X = \langle \bar{X}.E, \bar{X}.\text{po}, \bar{X}.\text{rf}, \text{mo} \rangle$ SRA- (and thus also RA- and WRA-) consistent. Such an mo ensures that $(\text{hb} \cup \text{mo})$ is acyclic. Triangle-freeness ensures read-coherence — a violation of read coherence implies that there are three events $e_1 = w_\beta, e_2 = w_\beta^\gamma, e_3 = r_\beta^\alpha$ such that $(e_1, e_3) \in \text{rf}$, $(e_1, e_2) \in \text{mo}$ and $(e_2, e_3) \in \text{hb}$, implying a triangle (α, β, γ) in G . The total time to construct $\bar{X}$ is $O(|V_G| + |E_G|)$. Further, our reduction is completely combinatorial. We thus arrive at the following theorem.

Theorem 15. *There is no consistency-checking algorithm for the RMW-free fragments of any of RA, WRA, and SRA that runs in time $O(n^{\omega/2-\varepsilon})$, for any fixed $\varepsilon > 0$. Moreover, there is no combinatorial algorithm for the problem that runs in time $O(n^{3/2-\varepsilon})$, under the combinatorial BMM hypothesis.*

8.5 Experimental Evaluation

We implemented our consistency-checking algorithms for RA/RC20 and evaluated their performance on two standard settings of program analysis, namely, (i) *stateless model checking*, using TruSt [162], and (ii) *online testing*, using C11Tester [191]. These tools are designed to handle variants of C11 including SC accesses, and performing race-detection, which are beyond the scope of this work. Here, we focus on the consistency-checking component for the RA/RC20 fragment, which is common in these tools and our work. We conducted our experiments on a machine running Ubuntu 22.04 with 2.4GHz CPU and 64GB of memory.

Benchmarks. We used standard benchmark programs from prior state-of-the-art verification and testing papers [8, 159, 191, 226], as well as the applications Silo, GDAX, Mabain, and Iris [191] for online testing. These benchmarks use C11 concurrency primitives extensively. For thorough evaluation, we have scaled up some of them, when their baseline versions were too small, by increasing the number of threads or loop counters. Some benchmarks also contain accesses outside our scope; we converted those accesses to access modes applicable for our experiments, in line with the evaluation in prior works [8, 166].

Stateless Model Checking

The TruSt model checker explores all behaviors of a bounded program by enumerating executions, making use of different strategies to avoid redundant exploration. One such strategy is to enumerate partial executions $\bar{X}$ and perform a consistency check for the maximal ones, to verify that they represent valid program behavior. As the number of explored executions is typically large, it is imperative that consistency checks are performed as fast as possible.

Consistency checking inside TruSt. The algorithm for consistency checking in TruSt constructs a writes-before order **wb** [167], which is a partial modification order that serves as a witness of consistency. The time taken to construct **wb** is $O(n^3)$, which has been identified as a bottleneck in the model checking task [161, 162]. We replaced TruSt’s **wb** algorithm for consistency checking with the **m̄o** computation of Algorithm 16, and measured (i) the speedup realized for consistency checking, and (ii) the effect of this speedup on the overall model-checking task. We executed TruSt on several benchmarks, each with a time budget of 2 hours, measuring the average time for consistency checking (for evaluating (i)) as well as the total number of executions explored (for evaluating (ii)). Finally, we note that TruSt employs a number of simpler consistency checks during the exploration. Although we expect that our new algorithm can improve those as well, we have left them intact as it was

unclear to us how they interact with the rest of the tool, and in order to maintain soundness of the obtained results.

Experimental results. Our results are shown in Table 8.1. We mark with † benchmarks on which the model checker found an error and halted early. We observe that Algorithm 16 is always faster, typically by a significant margin. The maximum speedup for consistency checking is $162\times$, and the geometric mean of speedups is $36\times$. Regarding the number of executions, the model checker explores $4.3\times$ more on (geometric) average, and as high as $71.6\times$ more, when using Algorithm 16. In some benchmarks, the two approaches observe a similar number of executions. This is due to consistency checking being only part of the overall model-checking procedure, which also consists of other computationally intensive tasks such as backtracking. As consistency checking appears now to not be a bottleneck, it is meaningful to focus further optimization efforts on these other tasks. For ttaslock, we noticed a livelock that blocks the model checker. Overall, our experiments highlight that the benefit of the new, nearly linear time property of consistency checking leads to a measurable speedup that positively impacts the overall efficiency of model checking. We refer to [294] for experiments on RC20, which lead to the same qualitative conclusions.

Table 8.1: Impact on model checking. Columns 2 and 3 denote the average time (in seconds) spent in consistency checking by resp. TruSt and our algorithm. Columns 5 and 6 denote the total number of executions explored by resp. TruSt and our algorithm. Columns 4 and 7 denote the respective speedups and ratios.

1	2	3	4	5	6	7	1	2	3	4	5	6	7
Benchmark	Avg. Time			Executions			Benchmark	Avg. Time			Executions		
	TruSt	Our Alg.	S	TruSt	Our Alg.	R		TruSt	Our Alg.	S	TruSt	Our Alg.	R
barrier	0.5	0.01	45.0	14K	81K	5.98	ms-queue†	0.07	0.01	7.0	208	208	1
buf-ring	1.6	0.02	79.5	3K	6K	2.52	mutex	0.8	0.01	78.0	7K	218K	29.67
chase-lev†	0.0	0.0	-	2	2	1	peterson	0.3	0.01	26.0	4K	5K	1.16
control-flow	0.08	0.01	8.0	88K	1M	16.56	qu	0.1	0.01	12.0	2K	2K	1.04
dekker	0.2	0.01	24.0	26K	64K	2.45	seqlock	0.2	0.01	24.0	24K	171K	7.02
dq	0.1	0.01	13.0	40K	163K	4.11	sigma	0.2	0.01	22.0	4K	62K	5.97
exp-bug	0.1	0.01	13.0	55K	2M	29.98	spinlock	1.0	0.01	98.0	5K	31K	6.04
fib-bench	1.6	0.01	162.0	4K	315K	71.63	szymanski	0.4	0.01	44.0	2K	2K	1.16
gcd	0.3	0.01	31.0	18K	80K	4.46	ticketlock	0.9	0.02	45.5	7K	89K	12.47
lambport	0.4	0.01	36.0	17K	93K	5.54	treiber	0.6	0.01	58.0	403	403	1
linuxrwlocks	0.4	0.01	40.0	12K	32K	2.75	ttaslock	1.0	0.01	97.0	401	401	1
mcs-spinlock	1.1	0.01	108.0	5K	38K	7.19	twalock	0.5	0.01	52.0	8K	30K	3.58
mpmc	0.7	0.01	66.0	10K	77K	8.01							
Totals	-	-	-	-	-	-	-	14.5	0.26	-	356K	4.6M	-

Online Testing

We now turn our attention to the online testing setting using C11Tester’s framework. In C11Tester’s setting, a partial execution $\bar{X}$ is constructed incrementally, by iteratively (i) revealing a randomly chosen new read/RMW event r , (ii) choosing a valid writer $rf(r)$, and (iii) continuing the execution of the program until the next read/RMW events. Hence, every iteration requires a consistency check. Although we could use Algorithm 16 from scratch at each step, this would result in unnecessary re-

computations of $\overline{\text{mo}}$. Instead, we follow a different approach here — we maintain $\overline{\text{mo}}$ on-the-fly, in a way that incremental consistency checks can be done more efficiently.

Incremental consistency checking. Our incremental algorithm constructs a similar minimally coherent partial modification order $\overline{\text{mo}}$ as our offline algorithm (Algorithm 16). However, unlike the offline setting, we need efficient incremental consistency checks. For this, we maintain a per-location order hbmo_x on write/RMW events that satisfies following invariants: (i) $\text{hbmo}_x \subseteq (\text{hb}_x \cup \overline{\text{mo}}_x)^+$ and (ii) $(\text{hb}_x \cup (\overline{\text{mo}}_x; \text{po}_x^?)) \subseteq \text{hbmo}_x$. In order to decide whether a new read/RMW event $r(x)$ can observe a write/RMW event $w(x)$, we must determine if there exists another write/RMW event $w'(x)$ such that $(w', r) \in \text{hb}_x$ and $(w', w) \in (\text{hb}_x \cup \overline{\text{mo}}_x)^+$, as this would lead to a consistency violation. Using hbmo_x , this check is performed as follows: (a) for each thread u , we identify the po-maximal write/RMW event $w'(u, x)$ for which $(w', r) \in \text{hb}$, and (b) we update $\text{hbmo}_x \leftarrow \text{hbmo}_x \cup \{(w'', w') \mid (w'', w') \in \text{hbmo}_x^+\}$. Due to invariant (ii), at this point we are guaranteed that, for all write events w'' , we have $(w'', w') \in \text{hbmo}_x$ iff $(w'', w') \in (\text{hb}_x \cup \overline{\text{mo}}_x)^+$. We can now test whether w is a valid writer for r by checking whether $(w, w') \in \text{hbmo}_x$, for one of the aforementioned write/RMW events w' . We refer to [294] for implementation details.

Main differences with C11Tester. The consistency-checking algorithm implemented inside C11Tester also infers mo orderings as implied by read and write coherence. The two key differences between that approach and our incremental algorithm described above are the following: (i) C11Tester's mo is stronger than our minimally coherent $\overline{\text{mo}}$ that is contained in hbmo , and (ii) this mo is always maintained transitively-closed. These two differences are expected to make hbmo computationally cheaper to maintain than C11Tester's mo . Although we also have to compute transitive paths hbmo^+ when encountering read/RMW operations (step (b) above), in our experience, these paths typically touch a small part of the input, leading to an efficient computation.

Table 8.2: Impact on online testing. Columns 2, 3 and 4 give the average number of events, threads and locations in each benchmark. Columns 5 and 6 denote the average times in seconds to check for consistency by resp. C11Tester and our algorithm. Column 6 denotes the speedup.

1	2	3	4	5	6	7	1	2	3	4	5	6	7
Benchmark	n	k	d	C11Test.	Our Alg.	SpeedUp	Benchmark	n	k	d	C11Test.	Our Alg.	SpeedUp
control-flow	52K	25	3	4.2	0.04	104.25	mutex	15M	11	11	20.9	16.8	1.24
sigma	36K	10	9	2.4	0.03	80	gdax	11M	5	46K	7.0	5.7	1.23
dq	599K	4	2	22.1	0.5	46.78	spinlock	5M	11	10	7.0	5.9	1.18
iris-1	1M	12	45	28.4	1.8	16.25	ticketlock	14M	6	20	15.7	13.2	1.18
seqlock	478K	17	20	14.4	1.4	10.38	ttaslock	5M	11	10	7.8	6.6	1.18
exp-bug	2M	4	2	1.6	0.7	2.35	fib-bench	6M	3	2	3.3	2.8	1.18
chase-lev	7M	5	2	12.4	5.4	2.3	qu	1M	10	29	1.6	1.3	1.18
linuxrwlocks	7M	6	10	10.9	5.9	1.84	treiber	1M	6	11	1.1	1.0	1.12
mabain	5M	6	18	7.3	4.0	1.82	silo	8M	4	4K	5.3	4.8	1.1
iris-2	12M	3	12	9.9	5.7	1.72	barrier	8M	5	20	7.8	7.4	1.04
mcs-lock	10M	11	30	17.8	12.2	1.45	mpmc	9M	10	3	12.9	12.5	1.03
lamport	6M	3	5	3.4	2.5	1.36	indexer	2M	17	128	1.6	1.5	1.03
peterson	5M	3	4	2.9	2.2	1.29	buf-ring	5M	9	12	7.0	6.8	1.02
spsc	10M	3	699	6.4	5.0	1.28	ms-queue	4M	11	13	8.2	8.2	0.99
dekker	16M	3	3	9.2	7.2	1.27	gcd	5M	3	2	2.2	2.5	0.89
twalock	4M	11	4K	8.9	7.0	1.26	szymanski	4M	3	3	1.8	2.3	0.81
Totals	-	-	-	-	-	-	-	196M	-	-	286.4	170.8	-

Experimental results. Our results are shown in Table 8.2. For robust measurements, we report averages over 10 executions per benchmark, focusing on benchmarks for which at least one algorithm took ≥ 1 s. Our approach achieves a maximum speedup of $104.2\times$ and a geometric speed-up of $2\times$. In more detail, we observe significant improvement in the first 5 benchmarks, and consistent speedups of at least $1.2\times$ on 18 benchmarks. We have encountered only 2 benchmarks, gcd and szymanski, on which the new algorithm is arguably slower. These benchmarks contain no RMWs and only a small number of write events. This results in the computation of very small modification orders, diminishing the benefit of our algorithm and results in a marginal slowdown.

8.6 Conclusion

Checking the reads-from consistency of concurrent executions is a fundamental computational task in the development of formal concurrency semantics, program verification and testing. In this paper we have addressed this problem in the context of C11-style weak memory models, for which this problem is both highly meaningful, and intricate. We have developed a collection of algorithms and complexity results that are either optimal or nearly-optimal, and thus accurately characterize the complexity of the problem in this setting. Further, our experimental evaluation indicates that the new algorithms have a measurable, and often significant, impact on the consistency-checking tasks that arise in practice. Thus our algorithms enable the development of more performant and scalable program analysis tools in this domain. This work is focused on non-SC fragments of C11, as otherwise, consistency checking inherits the NP-hardness of SC consistency checking. For applications having an abundance

of SC accesses, however, a meaningful direction for future work is to combine our techniques with heuristics developed for checking SC consistency (e.g., [8, 236]), and apply them on programs that mix all types of C11 accesses.

Bibliography

- [1] CORAL Benchmarks. <https://asc.llnl.gov/coral-benchmarks>, . Accessed: 2021-08-01. 72
- [2] CORAL-2 Benchmarks. <https://asc.llnl.gov/coral-2-benchmarks>, . Accessed: 2021-08-01. 72
- [3] ECP Proxy Applications. <https://proxyapps.exascaleproject.org>. Accessed: 2021-08-01. 72
- [4] Mantevo Project. <https://mantevo.org>. Accessed: 2021-08-01. 72
- [5] Rust Programming Language. URL <https://www.rust-lang.org>. 17
- [6] Swift Programming Language. URL <https://www.swift.org>. 17
- [7] Concurrentqueue, 2021. URL <https://github.com/cameron314/concurrentqueue>. 14
- [8] Parosh Aziz Abdulla, Mohamed Faouzi Atig, Bengt Jonsson, and Tuan Phong Ngo. Optimal Stateless Model Checking under the Release-Acquire Semantics. *Proc. ACM Program. Lang.*, 2(OOPSLA), 2018. doi: 10.1145/3276505. 47, 138, 139, 140, 151, 154, 159, 163
- [9] Parosh Aziz Abdulla, Mohamed Faouzi Atig, Bengt Jonsson, Magnus Lång, Tuan Phong Ngo, and Konstantinos Sagonas. Optimal Stateless Model Checking for Reads-from Equivalence under Sequential Consistency. *Proc. ACM Program. Lang.*, 3(OOPSLA), 2019. doi: 10.1145/3360576. 20, 32, 48, 82, 105, 138, 139
- [10] S.V. Adve and K. Gharachorloo. Shared memory consistency models: a tutorial. *Computer*, 29(12):66–76, 1996. doi: 10.1109/2.546611. 14
- [11] Anurag Agarwal and Vijay K. Garg. Efficient Dependency Tracking for Relevant Events in Shared-Memory Systems. In *Proceedings of the Twenty-Fourth Annual ACM Symposium on Principles of Distributed Computing*, PODC ’05, page 19–28, New York, NY, USA, 2005. Association for Computing Machinery. doi: 10.1145/1073814.1073818. 37, 43, 105

- [12] Pratyush Agarwal, Krishnendu Chatterjee, Shreya Pathak, Andreas Pavlogianis, and Viktor Toman. Stateless Model Checking Under a Reads-Value-From Equivalence. In Alexandra Silva and K. Rustan M. Leino, editors, *Computer Aided Verification*, volume 12759, pages 341–366. Springer International Publishing, Cham, 2021. doi: 10.1007/978-3-030-81685-8_16. 20, 32, 81, 138, 139
- [13] Rahul Agarwal, Liqiang Wang, and Scott D. Stoller. Detecting Potential Deadlocks with Static Analysis and Runtime Monitoring. In *Proceedings of the Parallel and Distributed Systems: Testing and Debugging (PADTAD) Track of the 2005 Haifa Verification Conference*, volume 3875 of *Lecture Notes in Computer Science*, pages 191–207. Springer-Verlag, 2006. doi: 10.1007/11678779_14. 108
- [14] T. Agerwala and Y.-C. Choed-Amphai. A Synthesis Rule for Concurrent Systems. In *15th Design Automation Conference*, pages 305–311, 1978. doi: 10.1109/DAC.1978.1585190. 18
- [15] Hiralal Agrawal, Richard A. Demillo, and Eugene H. Spafford. Debugging with dynamic slicing and backtracking. *Software: Practice and Experience*, 23 (6):589–616, 1993. doi: <https://doi.org/10.1002/spe.4380230603>. 21
- [16] Kunal Agrawal, Joseph Devietti, Jeremy T. Fineman, I-Ting Angelina Lee, Robert Utterback, and Changming Xu. Race Detection and Reachability in Nearly Series-Parallel DAGs. In *Proceedings of the Twenty-Ninth Annual ACM-SIAM Symposium on Discrete Algorithms*, SODA ’18, page 156–171, USA, 2018. Society for Industrial and Applied Mathematics. doi: 10.1137/1.9781611975031.11. 43, 77
- [17] Adil Ahmad, Sangho Lee, Pedro Fonseca, and Byoungyoung Lee. Kard: lightweight data race detection with per-thread memory protection. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS ’21, page 647–660, New York, NY, USA, 2021. Association for Computing Machinery. doi: 10.1145/3445814.3446727. 22, 35, 49
- [18] Jade Alglave. *A Shared Memory Poetics*. PhD thesis, l’Université Paris 7 - Denis Diderot, 2010. 138
- [19] Jade Alglave, Luc Maranget, Susmit Sarkar, and Peter Sewell. Litmus: Running Tests against Hardware. In Parosh Aziz Abdulla and K. Rustan M. Leino, editors, *Tools and Algorithms for the Construction and Analysis of Systems*, pages 41–44, Berlin, Heidelberg, 2011. Springer Berlin Heidelberg. doi: 10.1007/978-3-642-19835-9_5.

- [20] Jade Alglave, Luc Maranget, and Michael Tautschnig. Herding Cats: Modelling, Simulation, Testing, and Data Mining for Weak Memory. *ACM Trans. Program. Lang. Syst.*, 36(2), 2014. doi: 10.1145/2627752. 138
- [21] Jade Alglave, Will Deacon, Richard Grisenthwaite, Antoine Hacquard, and Luc Maranget. Armed Cats: Formal Concurrency Modelling at Arm. *ACM Trans. Program. Lang. Syst.*, 43(2), 2021. doi: 10.1145/3458926. 18
- [22] Lars Arge, Johannes Fischer, Peter Sanders, and Nodari Sitchinava. On (Dynamic) Range Minimum Queries in External Memory. In Frank Dehne, Roberto Solis-Oba, and Jörg-Rüdiger Sack, editors, *Algorithms and Data Structures*, pages 37–48, Berlin, Heidelberg, 2013. Springer Berlin Heidelberg. doi: 10.1007/978-3-642-40104-6_4. 85
- [23] Simone Atzeni, Ganesh Gopalakrishnan, Zvonimir Rakamaric, Dong H. Ahn, Ignacio Laguna, Martin Schulz, Gregory L. Lee, Joachim Protze, and Matthias S. Müller. ARCHER: Effectively Spotting Data Races in Large OpenMP Applications. In *2016 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 53–62, 2016. doi: 10.1109/IPDPS.2016.68. 4, 5
- [24] Christel Baier and Joost-Pieter Katoen. *Principles of model checking*. MIT press, 2008. 3, 4
- [25] D. H. Bailey, E. Barszcz, J. T. Barton, D. S. Browning, R. L. Carter, L. Dagum, R. A. Fatoohi, P. O. Frederickson, T. A. Lasinski, R. S. Schreiber, H. D. Simon, V. Venkatakrisnan, and S. K. Weeratunga. The NAS Parallel Benchmarks—Summary and Preliminary Results. In *Proceedings of the 1991 ACM/IEEE Conference on Supercomputing*, Supercomputing '91, page 158–165, New York, NY, USA, 1991. Association for Computing Machinery. doi: 10.1145/125826.125925. 72
- [26] T. Ball and J.R. Larus. Efficient path profiling. In *Proceedings of the 29th Annual IEEE/ACM International Symposium on Microarchitecture. MICRO 29*, pages 46–57, 1996. doi: 10.1109/MICRO.1996.566449. 21
- [27] Thoms Ball. The concept of dynamic analysis. In *Proceedings of the 7th European Software Engineering Conference Held Jointly with the 7th ACM SIGSOFT International Symposium on Foundations of Software Engineering, ESEC/FSE-7*, page 216–234, Berlin, Heidelberg, 1999. Springer-Verlag. 5, 21, 22, 23
- [28] Mark Batty, Scott Owens, Susmit Sarkar, Peter Sewell, and Tjark Weber. Mathematizing C++ concurrency. In *Proceedings of the 38th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL '11*, page 55–66, New York, NY, USA, 2011. Association for Computing Machinery. doi: 10.1145/1926385.1926394. 36, 138, 143

- [29] Mark Batty, Alastair F. Donaldson, and John Wickerson. Overhauling SC Atomics in C11 and OpenCL. In *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '16, page 634–648, New York, NY, USA, 2016. Association for Computing Machinery. doi: 10.1145/2837614.2837637. 138
- [30] Martin Beck, Koustubha Bhat, Lazar Stričević, Geng Chen, Diogo Behrens, Ming Fu, Viktor Vafeiadis, Haibo Chen, and Hermann Härtig. Atomig: Automatically migrating millions lines of code from tso to wmm. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ASPLOS 2023, page 61–73, New York, NY, USA, 2023. Association for Computing Machinery. doi: 10.1145/3575693.3579849. 15
- [31] John Bender and Jens Palsberg. A Formalization of Java's Concurrent Access Modes. *Proc. ACM Program. Lang.*, 3(OOPSLA), 2019. doi: 10.1145/3360568. 138
- [32] Saddek Bensalem and Klaus Havelund. Dynamic Deadlock Analysis of Multi-Threaded Programs. In *Proceedings of the First Haifa International Conference on Hardware and Software Verification and Testing*, HVC'05, page 208–223, Berlin, Heidelberg, 2005. Springer-Verlag. doi: 10.1007/11678779_15. 37, 108, 126, 134
- [33] Saddek Bensalem, Jean-Claude Fernandez, Klaus Havelund, and Laurent Mounier. Confirmation of Deadlock Potentials Detected by Runtime Analysis. In *Proceedings of the 2006 Workshop on Parallel and Distributed Systems: Testing and Debugging*, PADTAD '06, page 41–50, New York, NY, USA, 2006. Association for Computing Machinery. doi: 10.1145/1147403.1147412. 108, 134
- [34] A. Bertoni, G. Mauri, and N. Sabadini. Membership problems for regular and context-free trace languages. *Information and Computation*, 82(2):135–150, 1989. doi: 10.1016/0890-5401(89)90051-5. 56, 71
- [35] Ranadeep Biswas and Constantin Enea. On the Complexity of Checking Transactional Consistency. *Proc. ACM Program. Lang.*, 3(OOPSLA), 2019. doi: 10.1145/3360591. 6
- [36] Swarnendu Biswas, Jipeng Huang, Aritra Sengupta, and Michael D. Bond. DoubleChecker: Efficient Sound and Precise Atomicity Checking. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '14, page 28–39, New York, NY, USA, 2014. Association for Computing Machinery. doi: 10.1145/2594291.2594323. 56, 80, 105, 108

- [37] Stephen M. Blackburn, Robin Garner, Chris Hoffmann, Asjad M. Khang, Kathryn S. McKinley, Rotem Bentzur, Amer Diwan, Daniel Feinberg, Daniel Frampton, Samuel Z. Guyer, Martin Hirzel, Antony Hosking, Maria Jump, Han Lee, J. Eliot B. Moss, Aashish Phansalkar, Darko Stefanović, Thomas VanDrunen, Daniel von Dincklage, and Ben Wiedermann. The DaCapo Benchmarks: Java Benchmarking Development and Analysis. In *Proceedings of the 21st Annual ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages, and Applications*, OOPSLA '06, page 169–190, New York, NY, USA, 2006. Association for Computing Machinery. doi: 10.1145/1167473.1167488. 71, 129
- [38] Sam Blackshear, Nikos Gorogiannis, Peter W. O'Hearn, and Ilya Sergey. RacerD: compositional static race detection. *Proc. ACM Program. Lang.*, 2(OOPSLA), oct 2018. doi: 10.1145/3276514. 4, 35
- [39] Tim Blechmann. Boost. lockfree. *Boost C++ Libraries*, 2013. 14
- [40] Hans-J. Boehm. How to Miscompile Programs with “Benign” Data Races. In *Proceedings of the 3rd USENIX Conference on Hot Topic in Parallelism*, HotPar'11, page 3, USA, 2011. USENIX Association. 35, 56
- [41] Hans-J. Boehm. Position paper: nondeterminism is unavoidable, but data races are pure evil. In *Proceedings of the 2012 ACM Workshop on Relaxing Synchronization for Multicore and Manycore Scalability*, RACES '12, page 9–14, New York, NY, USA, 2012. Association for Computing Machinery. doi: 10.1145/2414729.2414732. 4, 15
- [42] Hans-J. Boehm and Sarita V. Adve. Foundations of the C++ concurrency memory model. In *Proceedings of the 29th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '08, page 68–78, New York, NY, USA, 2008. Association for Computing Machinery. doi: 10.1145/1375581.1375591. 34, 35, 138
- [43] Michael D. Bond, Katherine E. Coons, and Kathryn S. McKinley. PACER: proportional detection of data races. In *Proceedings of the 31st ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '10, page 255–268, New York, NY, USA, 2010. Association for Computing Machinery. doi: 10.1145/1806596.1806626. 16
- [44] Michael D. Bond, Milind Kulkarni, Man Cao, Minjia Zhang, Meisam Fathi Salmi, Swarnendu Biswas, Aritra Sengupta, and Jipeng Huang. OCTET: Capturing and Controlling Cross-Thread Dependences Efficiently. In *Proceedings of the 2013 ACM SIGPLAN International Conference on Object Oriented Programming Systems Languages & Applications*, OOPSLA '13, page 693–712, New York, NY, USA, 2013. Association for Computing Machinery. doi: 10.1145/2509136.2509519. 77

- [45] James Bornholt, Rajeev Joshi, Vytautas Astrauskas, Brendan Cully, Bernhard Kragl, Seth Markle, Kyle Sauri, Drew Schleit, Grant Slatton, Serdar Tasiran, Jacob Van Geffen, and Andrew Warfield. Using Lightweight Formal Methods to Validate a Key-Value Storage Node in Amazon S3. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles, SOSP '21*, page 836–850, New York, NY, USA, 2021. Association for Computing Machinery. doi: 10.1145/3477132.3483540. 4, 138
- [46] Ahmed Bouajjani, Javier Esparza, and Tayssir Touili. A generic approach to the static analysis of concurrent programs with procedures. In *Proceedings of the 30th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL '03*, page 62–73, New York, NY, USA, 2003. Association for Computing Machinery. doi: 10.1145/604131.604137. 4
- [47] Ahmed Bouajjani, Constantin Enea, Rachid Guerraoui, and Jad Hamza. On Verifying Causal Consistency. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages, POPL '17*, page 626–638, New York, NY, USA, 2017. Association for Computing Machinery. doi: 10.1145/3009837.3009888. 141, 145
- [48] Chandrasekhar Boyapati, Robert Lee, and Martin Rinard. Ownership types for safe programming: preventing data races and deadlocks. In *Proceedings of the 17th ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications, OOPSLA '02*, page 211–230, New York, NY, USA, 2002. Association for Computing Machinery. doi: 10.1145/582419.582440. 17
- [49] Truc Lam Bui, Krishnendu Chatterjee, Tushar Gautam, Andreas Pavlogiannis, and Viktor Toman. The Reads-from Equivalence for the TSO and PSO Memory Models. *Proceedings of the ACM on Programming Languages*, 5(OOPSLA): 1–30, 2021. ISSN 2475-1421. doi: 10.1145/3485541. 32, 81, 138, 139
- [50] Sebastian Burckhardt. Principles of Eventual Consistency. *Foundations and Trends® in Programming Languages*, 1(1-2):1–150, 2014. doi: 10.1561/25000000011. 145
- [51] Yan Cai and W. K. Chan. MagicFuzzer: Scalable Deadlock Detection for Large-Scale Applications. In *Proceedings of the 34th International Conference on Software Engineering, ICSE '12*, page 606–616. IEEE Press, 2012. doi: 10.1109/ICSE.2012.6227156. 37, 108
- [52] Yan Cai and W.K. Chan. Magiclock: Scalable Detection of Potential Deadlocks in Large-Scale Multithreaded Programs . *IEEE Transactions on Software Engineering*, 40(3):266–281, 2014. doi: 10.1109/TSE.2014.2301725. 126
- [53] Yan Cai, Ruijie Meng, and Jens Palsberg. Low-Overhead Deadlock Prediction. In *Proceedings of the ACM/IEEE 42nd International Conference on Software*

- Engineering*, ICSE '20, page 1298–1309, New York, NY, USA, 2020. Association for Computing Machinery. doi: 10.1145/3377811.3380367. 37, 108, 126, 134
- [54] Yan Cai, Hao Yun, Jinqiu Wang, Lei Qiao, and Jens Palsberg. Sound and Efficient Concurrency Bug Prediction. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE 2021, page 255–267, New York, NY, USA, 2021. Association for Computing Machinery. doi: 10.1145/3468264.3468549. 5, 9, 35, 38, 45, 46, 80, 82, 99, 105, 109, 114, 126, 128, 129, 134
- [55] Cristiano Calcagno, Dino Distefano, Jérémy Dubreil, Dominik Gabi, Pieter Hooimeijer, Martino Luca, Peter O’Hearn, Irene Papakonstantinou, Jim Purbrick, and Dulma Rodriguez. Moving Fast with Software Verification. pages 3–11, 04 2015. doi: 10.1007/978-3-319-17524-9_1. 4
- [56] Calin Cascaval, Colin Blundell, Maged Michael, Harold W. Cain, Peng Wu, Stefanie Chiras, and Siddhartha Chatterjee. Software transactional memory: why is it only a research toy? *Commun. ACM*, 51(11):40–46, November 2008. ISSN 0001-0782. doi: 10.1145/1400214.1400228. 18
- [57] Pavol Černý, Krishnendu Chatterjee, Thomas A. Henzinger, Arjun Radhakrishna, and Rohit Singh. Quantitative Synthesis for Concurrent Programs. In Ganesh Gopalakrishnan and Shaz Qadeer, editors, *Computer Aided Verification*, pages 243–259, Berlin, Heidelberg, 2011. Springer Berlin Heidelberg. 18
- [58] Milind Chabbi and Murali Krishna Ramanathan. A study of real-world data races in Golang. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, PLDI 2022, page 474–489, New York, NY, USA, 2022. Association for Computing Machinery. doi: 10.1145/3519939.3523720. 4
- [59] Soham Chakraborty and Viktor Vafeiadis. Grounding Thin-Air Reads with Event Structures. *Proc. ACM Program. Lang.*, 3(POPL), 2019. doi: 10.1145/3290383. 138
- [60] Soham Chakraborty, Shankara Narayanan Krishna, Umang Mathur, and Andreas Pavlogiannis. How Hard Is Weak-Memory Testing? *Proceedings of the ACM on Programming Languages*, 8(POPL):66:1978–66:2009, 2024. doi: 10.1145/3632908. 32, 48, 81
- [61] Marek Chalupa, Krishnendu Chatterjee, Andreas Pavlogiannis, Nishant Sinha, and Kapil Vaidya. Data-Centric Dynamic Partial Order Reduction. *Proceedings of the ACM on Programming Languages*, 2(POPL):1–30, 2018. ISSN 2475-1421. doi: 10.1145/3158119. 32, 81, 138

- [62] W. Chan, R.J. Anderson, P. Beame, S. Burns, F. Modugno, D. Notkin, and J.D. Reese. Model checking large software specifications. *IEEE Transactions on Software Engineering*, 24(7):498–520, 1998. doi: 10.1109/32.708566. 16
- [63] Robert N Charette. Why software fails. *IEEE spectrum*, 42(9), 2005. 3
- [64] Bernadette Charron-Bost. Concerning the size of logical clocks in distributed systems. *Information Processing Letters*, 39(1):11–16, 1991. ISSN 0020-0190. doi: 10.1016/0020-0190(91)90055-M. 77
- [65] Krishnendu Chatterjee, Andreas Pavlogiannis, and Viktor Toman. Value-Centric Dynamic Partial Order Reduction. *Proc. ACM Program. Lang.*, 3 (OOPSLA), 2019. doi: 10.1145/3360550. 138
- [66] Hongxu Chen, Shengjian Guo, Yinxing Xue, Yulei Sui, Cen Zhang, Yuekang Li, Haijun Wang, and Yang Liu. MUZZ: thread-aware grey-box fuzzing for effective bug hunting in multithreaded programs. In *Proceedings of the 29th USENIX Conference on Security Symposium, SEC’20, USA, 2020*. USENIX Association. 50
- [67] Jianer Chen, Xiuzhen Huang, Iyad A. Kanj, and Ge Xia. Linear FPT Reductions and Computational Lower Bounds. In *Proceedings of the Thirty-Sixth Annual ACM Symposium on Theory of Computing, STOC ’04*, page 212–221, New York, NY, USA, 2004. Association for Computing Machinery. doi: 10.1145/1007352.1007391. 140
- [68] Jianer Chen, Xiuzhen Huang, Iyad A. Kanj, and Ge Xia. Strong computational lower bounds via parameterized complexity. *Journal of Computer and System Sciences*, 72(8):1346–1367, 2006. doi: <https://doi.org/10.1016/j.jcss.2006.04.007>. 115
- [69] Yunji Chen, Yi Lv, Weiwu Hu, Tianshi Chen, Haihua Shen, Pengyu Wang, and Hong Pan. Fast complete memory consistency verification. In *2009 IEEE 15th International Symposium on High Performance Computer Architecture*, pages 381–392, 2009. doi: 10.1109/HPCA.2009.4798276. 138
- [70] Yunji Chen, Shijin Zhang, Qi Guo, Ling Li, Ruiyang Wu, and Tianshi Chen. Deterministic replay: A survey. *ACM Comput. Surv.*, 48(2), September 2015. doi: 10.1145/2790077. 15
- [71] Guang-Ien Cheng, Mingdong Feng, Charles E. Leiserson, Keith H. Randall, and Andrew F. Stark. Detecting Data Races in Cilk Programs That Use Locks. In *Proceedings of the Tenth Annual ACM Symposium on Parallel Algorithms and Architectures, SPAA ’98*, pages 298–309, New York, NY, USA, 1998. ACM. doi: 10.1145/277651.277696. 43, 77

- [72] John Joseph Chilenski and Steven P. Miller. Applicability of modified condition/decision coverage to software testing. *Software Engineering Journal*, 9: 193–200, 1994. doi: 10.1049/sej.1994.0025. 22
- [73] Peter Chini and Prakash Saivasan. A Framework for Consistency Algorithms. In Nitin Saxena and Sunil Simon, editors, *40th IARCS Annual Conference on Foundations of Software Technology and Theoretical Computer Science (FSTTCS 2020)*, volume 182 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 42:1–42:17, Dagstuhl, Germany, 2020. Schloss Dagstuhl–Leibniz-Zentrum für Informatik. doi: 10.4230/LIPIcs.FSTTCS.2020.42. 139
- [74] Andy Chou, Junfeng Yang, Benjamin Chelf, Seth Hallem, and Dawson Engler. An empirical study of operating systems errors. In *Proceedings of the Eighteenth ACM Symposium on Operating Systems Principles, SOSP '01*, page 73–88, New York, NY, USA, 2001. Association for Computing Machinery. doi: 10.1145/502034.502042. 3
- [75] Shaubik Roy Choudhary, Alessandra Gorla, and Alessandro Orso. Automated Test Input Generation for Android: Are We There Yet? (E). In *2015 30th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 429–440, 2015. doi: 10.1109/ASE.2015.89. 22
- [76] Maria Christakis and Christian Bird. What developers want and need from program analysis: an empirical study. In *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering, ASE '16*, page 332–343, New York, NY, USA, 2016. Association for Computing Machinery. ISBN 9781450338455. doi: 10.1145/2970276.2970347. 6, 49
- [77] Mark Christiaens and Koenraad De Bosschere. Accordion Clocks: Logical Clocks for Data Race Detection. In *Proceedings of the 7th International Euro-Par Conference Manchester on Parallel Processing, Euro-Par '01*, page 494–503, Berlin, Heidelberg, 2001. Springer-Verlag. doi: 10.1007/3-540-44681-8_73. 43, 105
- [78] Berk Çirisci, Constantin Enea, Azadeh Farzan, and Suha Orhun Mutluergil. Root Causing Linearizability Violations. In Shuvendu K. Lahiri and Chao Wang, editors, *Computer Aided Verification*, pages 350–375, Cham, 2020. Springer International Publishing. doi: 10.1007/978-3-030-53288-8_17. 45, 80, 82, 103, 105
- [79] Edmund Clarke, Orna Grumberg, Somesh Jha, Yuan Lu, and Helmut Veith. Counterexample-Guided Abstraction Refinement . In E. Allen Emerson and Aravinda Prasad Sistla, editors, *Computer Aided Verification*, pages 154–169, Berlin, Heidelberg, 2000. Springer Berlin Heidelberg. 17

- [80] Edmund Clarke, Orna Grumberg, Somesh Jha, Yuan Lu, and Helmut Veith. *Progress on the State Explosion Problem in Model Checking*, pages 176–194. Springer Berlin Heidelberg, Berlin, Heidelberg, 2001. doi: 10.1007/3-540-44577-3_12. 17
- [81] Jonathan E. Cook and Alexander L. Wolf. Automating process discovery through event-data analysis. In *Proceedings of the 17th International Conference on Software Engineering, ICSE '95*, page 73–82, New York, NY, USA, 1995. Association for Computing Machinery. doi: 10.1145/225014.225021. 21
- [82] Hoang-Hai Dang, Jacques-Henri Jourdan, Jan-Oliver Kaiser, and Derek Dreyer. RustBelt Meets Relaxed Memory. *Proc. ACM Program. Lang.*, 4(POPL), 2019. doi: 10.1145/3371102. 138
- [83] Clive Davidson. A dark Knight for algos. *Risk*, 25(9):32–34, 09 2012. Name - Knight Capital Group Inc; Copyright - Copyright Incisive Media Plc Sep 2012; Document feature - Photographs; Last updated - 2023-12-04; Subject-sTermNotLitGenreText - United Kingdom–UK. 3
- [84] David Delmas and Jean Souyris. Astrée: From Research to Industry. In Hanne Riis Nielson and Gilberto Filé, editors, *Static Analysis*, pages 437–451, Berlin, Heidelberg, 2007. Springer Berlin Heidelberg. 4
- [85] Dongdong Deng, Wei Zhang, and Shan Lu. Efficient concurrency-bug detection across inputs. In *Proceedings of the 2013 ACM SIGPLAN International Conference on Object Oriented Programming Systems Languages & Applications, OOPSLA '13*, page 785–802, New York, NY, USA, 2013. Association for Computing Machinery. doi: 10.1145/2509136.2509539. 16
- [86] Mathieu Desnoyers, Paul E McKenney, Alan S Stern, Michel R Dagenais, and Jonathan Walpole. User-level implementations of read-copy update. *IEEE Transactions on Parallel and Distributed Systems*, 23(2):375–382, 2011. doi: 10.1109/TPDS.2011.159. 145
- [87] Joseph Devietti, Benjamin P. Wood, Karin Strauss, Luis Ceze, Dan Grossman, and Shaz Qadeer. RADISH: Always-on Sound and Complete Race Detection in Software and Hardware. In *Proceedings of the 39th Annual International Symposium on Computer Architecture, ISCA '12*, page 201–212, USA, 2012. IEEE Computer Society. doi: 10.1109/ISCA.2012.6237018. 77
- [88] Dimitar Dimitrov, Martin Vechev, and Vivek Sarkar. Race Detection in Two Dimensions. In *Proceedings of the 27th ACM Symposium on Parallelism in Algorithms and Architectures, SPAA '15*, page 101–110, New York, NY, USA, 2015. Association for Computing Machinery. doi: 10.1145/2755573.2755601. 43, 77

- [89] Anne Dinning and Edith Schonberg. Detecting Access Anomalies in Programs with Critical Sections. In *Proceedings of the 1991 ACM/ONR Workshop on Parallel and Distributed Debugging*, PADD '91, pages 85–96, New York, NY, USA, 1991. ACM. doi: 10.1145/122759.122767. 35
- [90] Dino Distefano, Manuel Fähndrich, Francesco Logozzo, and Peter W. O'Hearn. Scaling static analyses at Facebook. *Commun. ACM*, 62(8):62–70, July 2019. ISSN 0001-0782. doi: 10.1145/3338112. 6, 49
- [91] Hyunsook Do, Sebastian G. Elbaum, and Gregg Rothermel. Supporting Controlled Experimentation with Testing Techniques: An Infrastructure and its Potential Impact. *Empirical Software Engineering: An International Journal*, 10(4):405–435, 2005. doi: 10.1007/s10664-005-3861-2. 71, 129
- [92] A.J. Dorta, C. Rodriguez, and F. de Sande. The OpenMP source code repository. In *13th Euromicro Conference on Parallel, Distributed and Network-Based Processing*, pages 244–250, 2005. doi: 10.1109/EMPDP.2005.41. 72
- [93] Zakir Durumeric, Frank Li, James Kasten, Johanna Amann, Jethro Beekman, Mathias Payer, Nicolas Weaver, David Adrian, Vern Paxson, Michael Bailey, and J. Alex Halderman. The Matter of Heartbleed. In *Proceedings of the 2014 Conference on Internet Measurement Conference*, IMC '14, page 475–488, New York, NY, USA, 2014. Association for Computing Machinery. doi: 10.1145/2663716.2663755. 3
- [94] Ariel Eizenberg, Yuanfeng Peng, Toma Pigli, William Mansky, and Joseph Devietti. BARRACUDA: Binary-Level Analysis of Runtime RAces in CUDA Programs. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI 2017, pages 126–140, New York, NY, USA, 2017. Association for Computing Machinery. doi: 10.1145/3062341.3062342. 80
- [95] Tayfun Elmas, Shaz Qadeer, and Serdar Tasiran. Goldilocks: A Race and Transaction-Aware Java Runtime. In *Proceedings of the 28th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '07, page 245–255, New York, NY, USA, 2007. Association for Computing Machinery. doi: 10.1145/1250734.1250762. 35, 56, 77
- [96] Pär Emanuelsson and Ulf Nilsson. A Comparative Study of Industrial Static Analysis Tools. *Electronic Notes in Theoretical Computer Science*, 217:5–21, 2008. ISSN 1571-0661. doi: <https://doi.org/10.1016/j.entcs.2008.06.039>. Proceedings of the 3rd International Workshop on Systems Software Verification (SSV 2008). 4
- [97] Michael Emmi and Constantin Enea. Weak-Consistency Specification via Visibility Relaxation. *Proc. ACM Program. Lang.*, 3(POPL), 2019. doi: 10.1145/3290373. 32, 81

- [98] Michael Emmi, Rupak Majumdar, and Koushik Sen. Dynamic test input generation for database applications. In *Proceedings of the 2007 International Symposium on Software Testing and Analysis*, ISSTA '07, page 151–162, New York, NY, USA, 2007. Association for Computing Machinery. doi: 10.1145/1273463.1273484. 22
- [99] Dawson Engler and Ken Ashcraft. RacerX: effective, static detection of race conditions and deadlocks. In *Proceedings of the Nineteenth ACM Symposium on Operating Systems Principles*, SOSP '03, page 237–252, New York, NY, USA, 2003. Association for Computing Machinery. doi: 10.1145/945445.945468. 4
- [100] Dawson Engler and Madanlal Musuvathi. Static Analysis versus Software Model Checking for Bug Finding. In Bernhard Steffen and Giorgio Levi, editors, *Verification, Model Checking, and Abstract Interpretation*, pages 191–210, Berlin, Heidelberg, 2004. Springer Berlin Heidelberg. 5
- [101] Michael D. Ernst. Static and dynamic analysis: Synergy and duality. In *WODA 2003: Workshop on Dynamic Analysis*, pages 24–27, Portland, OR, USA, May 2003. 5, 23
- [102] Hamed Esfahani, Jonas Fietz, Qi Ke, Alexei Kolomiets, Erica Lan, Erik Mavrinac, Wolfram Schulte, Newton Sanches, and Srikanth Kandula. Cloud-Build: Microsoft’s distributed and caching build service. In *Proceedings of the 38th International Conference on Software Engineering Companion*, ICSE '16, page 11–20, New York, NY, USA, 2016. Association for Computing Machinery. doi: 10.1145/2889160.2889222. 4
- [103] Mahdi Eslamimehr and Jens Palsberg. Sherlock: Scalable Deadlock Detection for Concurrent Programs. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*, FSE 2014, page 353–365, New York, NY, USA, 2014. Association for Computing Machinery. doi: 10.1145/2635868.2635918. 108, 134
- [104] Eitan Farchi, Yarden Nir, and Shmuel Ur. Concurrent Bug Patterns and How to Test Them. In *Proceedings of the 17th International Symposium on Parallel and Distributed Processing*, IPDPS '03, pages 286.2–, Washington, DC, USA, 2003. IEEE Computer Society. 56, 71, 129
- [105] Azadeh Farzan and P. Madhusudan. Monitoring Atomicity in Concurrent Programs. In Aarti Gupta and Sharad Malik, editors, *Computer Aided Verification*, pages 52–65, Berlin, Heidelberg, 2008. Springer Berlin Heidelberg. 50
- [106] Azadeh Farzan and Parthasarathy Madhusudan. The Complexity of Predicting Atomicity Violations. In Stefan Kowalewski and Anna Philippou, editors, *Tools and Algorithms for the Construction and Analysis of Systems*, pages 155–169, Berlin, Heidelberg, 2009. Springer Berlin Heidelberg. doi: 10.1007/978-3-642-00768-2_14. 46, 135

- [107] Mingdong Feng and Charles E. Leiserson. Efficient Detection of Determinacy Races in Cilk Programs. In *Proceedings of the Ninth Annual ACM Symposium on Parallel Algorithms and Architectures*, SPAA '97, page 1–11, New York, NY, USA, 1997. Association for Computing Machinery. doi: 10.1145/258492.258493. 43, 77
- [108] Colin Fidge. Logical Time in Distributed Computing Systems. *Computer*, 24 (8):28–33, August 1991. ISSN 0018-9162. doi: 10.1109/2.84874. 56, 60, 121
- [109] Colin J. Fidge. Timestamps in Message-Passing Systems That Preserve the Partial Ordering. In *Proc. 11th Australian Comput. Science Conf.*, pages 56–66, 1988. 8, 29, 56, 60
- [110] Cormac Flanagan and Stephen N. Freund. Type-based race detection for Java. In *Proceedings of the ACM SIGPLAN 2000 Conference on Programming Language Design and Implementation*, PLDI '00, page 219–232, New York, NY, USA, 2000. Association for Computing Machinery. doi: 10.1145/349299.349328. 17
- [111] Cormac Flanagan and Stephen N. Freund. FastTrack: Efficient and Precise Dynamic Race Detection. In *Proceedings of the 30th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '09, pages 121–133, New York, NY, USA, 2009. ACM. doi: 10.1145/1542476.1542490. 4, 8, 29, 43, 56, 57, 59, 68, 72, 77, 80, 81, 108, 135
- [112] Cormac Flanagan and Stephen N. Freund. The RoadRunner Dynamic Analysis Framework for Concurrent Programs. In *Proceedings of the 9th ACM SIGPLAN-SIGSOFT Workshop on Program Analysis for Software Tools and Engineering*, PASTE '10, page 1–8, New York, NY, USA, 2010. Association for Computing Machinery. doi: 10.1145/1806672.1806674. 4
- [113] Cormac Flanagan and Stephen N. Freund. RedCard: Redundant Check Elimination for Dynamic Race Detectors. In Giuseppe Castagna, editor, *ECOOP 2013 – Object-Oriented Programming*, pages 255–280, Berlin, Heidelberg, 2013. Springer Berlin Heidelberg. doi: 10.1007/978-3-642-39038-8_11. 77
- [114] Cormac Flanagan and Stephen N. Freund. Mover Logic: A Concurrent Program Logic for Reduction and Rely-Guarantee Reasoning. In Jonathan Aldrich and Guido Salvaneschi, editors, *38th European Conference on Object-Oriented Programming (ECOOP 2024)*, volume 313 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 16:1–16:29, Dagstuhl, Germany, 2024. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. doi: 10.4230/LIPIcs.ECOOP.2024.16. 17
- [115] Cormac Flanagan and Patrice Godefroid. Dynamic Partial-Order Reduction for Model Checking Software. In *Proceedings of the 32nd ACM SIGPLAN-SIGACT*

- Symposium on Principles of Programming Languages*, POPL '05, pages 110–121, New York, NY, USA, 2005. Association for Computing Machinery. doi: 10.1145/1040305.1040315. 4, 71
- [116] Cormac Flanagan and Shaz Qadeer. A type and effect system for atomicity. In *Proceedings of the ACM SIGPLAN 2003 Conference on Programming Language Design and Implementation*, PLDI '03, page 338–349, New York, NY, USA, 2003. Association for Computing Machinery. doi: 10.1145/781131.781169. 17
- [117] Cormac Flanagan, Stephen N. Freund, and Jaeheon Yi. Velodrome: A Sound and Complete Dynamic Atomicity Checker for Multithreaded Programs. In *Proceedings of the 29th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '08, page 293–303, New York, NY, USA, 2008. Association for Computing Machinery. doi: 10.1145/1375581.1375618. 9, 56, 71, 80, 108
- [118] P.G. Frankl and E.J. Weyuker. An applicable family of data flow testing criteria. *IEEE Transactions on Software Engineering*, 14(10):1483–1498, 1988. doi: 10.1109/32.6194. 22
- [119] Kelsey R. Fulton, Anna Chan, Daniel Votipka, Michael Hicks, and Michelle L. Mazurek. Benefits and drawbacks of adopting a secure programming language: rust as a case study. In *Proceedings of the Seventeenth USENIX Conference on Usable Privacy and Security*, SOUPS'21, USA, 2021. USENIX Association. 17
- [120] Florian Furbach, Roland Meyer, Klaus Schneider, and Maximilian Senftleben. Memory-Model-Aware Testing: A Unified Complexity Analysis. *ACM Trans. Embed. Comput. Syst.*, 14(4), 2015. doi: 10.1145/2753761. 138, 139
- [121] Mingyu Gao, Soham Chakraborty, and Burcu Kulahcioglu Ozkan. Probabilistic Concurrency Testing for Weak Memory Programs. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ASPLOS 2023, page 603–616, New York, NY, USA, 2023. Association for Computing Machinery. doi: 10.1145/3575693.3575729. 80, 105
- [122] Kaan Genç, Jake Roemer, Yufan Xu, and Michael D. Bond. Dependence-Aware, Unbounded Sound Predictive Race Detection. *Proc. ACM Program. Lang.*, 3(OOPSLA), October 2019. doi: 10.1145/3360605. 5, 9, 44, 46, 56, 77, 108
- [123] Phillip B. Gibbons and Ephraim Korach. Testing Shared Memories. *SIAM Journal on Computing*, 26(4):1208–1244, 1997. doi: 10.1137/S0097539794279614. 6, 19, 32, 33, 81, 82, 138, 139, 149

- [124] Phillip B. Gibbons and Ephraim Korach. Testing Shared Memories. *SIAM Journal on Computing*, 26(4):1208–1244, 1997. doi: 10.1137/S0097539794279614. 131
- [125] Patrice Godefroid and Nachiappan Nagappan. Concurrency at Microsoft: An exploratory survey. In *CAV workshop on exploiting concurrency efficiently and correctly*. Princeton, USA, 2008. 4
- [126] Patrice Godefroid, J. van Leeuwen, J. Hartmanis, G. Goos, and Pierre Wolper. *Partial-Order Methods for the Verification of Concurrent Systems: An Approach to the State-Explosion Problem*. Springer-Verlag, Berlin, Heidelberg, 1996. 71
- [127] Jim Gray. Why Do Computers Stop And What Can Be Done About It? *Büroautomation*, pages 128–145, 1985. 4, 14, 15
- [128] Sumit Gulwani, Oleksandr Polozov, and Rishabh Singh. Program synthesis. *Foundations and Trends® in Programming Languages*, 4(1-2):1–119, 2017. doi: 10.1561/25000000010. 18
- [129] Tim Harris, James Larus, and Ravi Rajwar. *Transactional Memory, 2nd Edition*. Morgan and Claypool Publishers, 2nd edition, 2010. 17
- [130] Klaus Havelund. Using Runtime Analysis to Guide Model Checking of Java Programs. In *Proceedings of the 7th International SPIN Workshop on SPIN Model Checking and Software Verification*, page 245–264, Berlin, Heidelberg, 2000. Springer-Verlag. doi: 10.1007/10722468_15. 108, 113, 126, 134
- [131] Klaus Havelund and Thomas Pressburger. Model checking JAVA programs using JAVA PathFinder. *International Journal on Software Tools for Technology Transfer*, 2(4):366–381, Mar 2000. ISSN 1433-2779. doi: 10.1007/s100090050043. 4, 37
- [132] Matthew Hennessy and Robin Milner. Algebraic laws for nondeterminism and concurrency. *J. ACM*, 32(1):137–161, January 1985. doi: 10.1145/2455.2460. 15
- [133] Gerard J. Holzmann and Margaret H. Smith. Software model checking: extracting verification models from source code. *Software Testing, Verification and Reliability*, 11(2):65–79, 2001. doi: <https://doi.org/10.1002/stvr.228>. 17
- [134] G.J. Holzmann. The model checker SPIN. *IEEE Transactions on Software Engineering*, 23(5):279–295, 1997. doi: 10.1109/32.588521. 16
- [135] Susan Horwitz. Precise flow-insensitive may-alias analysis is np-hard. *ACM Trans. Program. Lang. Syst.*, 19(1):1–6, January 1997. doi: 10.1145/239912.239913. 6

- [136] Jeff Huang. Scalable thread sharing analysis. In *Proceedings of the 38th International Conference on Software Engineering, ICSE '16*, page 1097–1108, New York, NY, USA, 2016. Association for Computing Machinery. doi: 10.1145/2884781.2884811. 23
- [137] Jeff Huang. UFO: Predictive Concurrency Use-after-Free Detection. In *Proceedings of the 40th International Conference on Software Engineering, ICSE '18*, page 609–619, New York, NY, USA, 2018. Association for Computing Machinery. doi: 10.1145/3180155.3180225. 5, 9, 21, 32, 38, 45, 80, 81, 82, 101, 108, 134
- [138] Jeff Huang, Patrick O’Neil Meredith, and Grigore Rosu. Maximal sound predictive race detection with control flow abstraction. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI '14*, page 337–348, New York, NY, USA, 2014. Association for Computing Machinery. doi: 10.1145/2594291.2594315. 4, 9, 35, 108, 114, 134, 139
- [139] V Ignat. Digitalization and the global technology trends. *IOP Conference Series: Materials Science and Engineering*, 227(1):012062, aug 2017. doi: 10.1088/1757-899X/227/1/012062. 3
- [140] Marian V. Iordache and Panos J. Antsaklis. Concurrent program synthesis based on supervisory control. In *Proceedings of the 2010 American Control Conference*, pages 3378–3383, 2010. doi: 10.1109/ACC.2010.5530904. 18
- [141] Jim Isaak and Mina J. Hanna. User Data Privacy: Facebook, Cambridge Analytica, and Privacy Protection. *Computer*, 51(8):56–59, 2018. doi: 10.1109/MC.2018.3191268. 3
- [142] ISO/IEC 14882. Programming Language C++, 2011. 18, 138, 146
- [143] ISO/IEC 9899. Programming Language C, 2011. 18, 138, 146
- [144] Ayal Itzkovitz, Assaf Schuster, and Oren Zeev-Ben-Mordehai. Toward Integration of Data Race Detection in DSM Systems. *J. Parallel Distrib. Comput.*, 59(2):180–203, November 1999. ISSN 0743-7315. doi: 10.1006/jpdc.1999.1574. 56, 57
- [145] Dae R. Jeong, Kyungtae Kim, Basavesh Shivakumar, Byoungyoung Lee, and Insik Shin. Razzer: Finding Kernel Race Bugs through Fuzzing. In *2019 IEEE Symposium on Security and Privacy (SP)*, pages 754–768, 2019. doi: 10.1109/SP.2019.00017. 50
- [146] Zu-Ming Jiang, Jia-Ju Bai, Kangjie Lu, and Shi-Min Hu. Context-sensitive and directional concurrency fuzzing for data-race detection. In *Network and Distributed Systems Security (NDSS) Symposium 2022*, 2022. 50

- [147] Guoliang Jin, Linhai Song, Wei Zhang, Shan Lu, and Ben Liblit. Automated atomicity-violation fixing. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '11, page 389–400, New York, NY, USA, 2011. Association for Computing Machinery. doi: 10.1145/1993498.1993544. 16
- [148] Donald B. Johnson. Finding All the Elementary Circuits of a Directed Graph. *SIAM Journal on Computing*, 4(1):77–84, 1975. doi: 10.1137/0204007. 125
- [149] C. B. Jones. Tentative steps toward a development method for interfering programs. *ACM Trans. Program. Lang. Syst.*, 5(4):596–619, October 1983. ISSN 0164-0925. doi: 10.1145/69575.69577. 17
- [150] Pallavi Joshi, Mayur Naik, Chang-Seo Park, and Koushik Sen. CalFuzzer: An Extensible Active Testing Framework for Concurrent Programs. In Ahmed Bouajjani and Oded Maler, editors, *Computer Aided Verification*, pages 675–681, Berlin, Heidelberg, 2009. Springer Berlin Heidelberg. 4, 47
- [151] Pallavi Joshi, Chang-Seo Park, Koushik Sen, and Mayur Naik. A Randomized Dynamic Program Analysis Technique for Detecting Real Deadlocks . In *Proceedings of the 30th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '09, page 110–120, New York, NY, USA, 2009. Association for Computing Machinery. doi: 10.1145/1542476.1542489. 37, 108, 112, 129, 131, 132, 134, 135
- [152] Pallavi Joshi, Mayur Naik, Koushik Sen, and David Gay. An Effective Dynamic Analysis for Detecting Generalized Deadlocks. In *Proceedings of the Eighteenth ACM SIGSOFT International Symposium on Foundations of Software Engineering*, FSE '10, page 327–336, New York, NY, USA, 2010. Association for Computing Machinery. doi: 10.1145/1882291.1882339. 108
- [153] Horatiu Julia, Daniel Tralamazza, Cristian Zamfir, and George Candea. Deadlock Immunity: Enabling Systems to Defend against Deadlocks. In *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation*, OSDI'08, page 295–308, USA, 2008. USENIX Association. 129
- [154] Christian Gram Kalhauge and Jens Palsberg. Sound Deadlock Prediction. *Proc. ACM Program. Lang.*, 2(OOPSLA), 2018. doi: 10.1145/3276516. 9, 16, 32, 37, 38, 46, 80, 81, 108, 114, 126, 128, 129, 134, 139
- [155] Baris Kasikci, Cristian Zamfir, and George Candea. Data races vs. data race bugs: telling the difference with portend. In *Proceedings of the Seventeenth International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS XVII, page 185–198, New York, NY, USA, 2012. Association for Computing Machinery. doi: 10.1145/2150976.2150997. 50

- [156] Baris Kasikci, Weidong Cui, Xinyang Ge, and Ben Niu. Lazy Diagnosis of In-Production Concurrency Bugs. In *Proceedings of the 26th Symposium on Operating Systems Principles, SOSP '17*, page 582–598, New York, NY, USA, 2017. Association for Computing Machinery. doi: 10.1145/3132747.3132767. 23, 51
- [157] Dileep Kini, Umang Mathur, and Mahesh Viswanathan. Dynamic Race Prediction in Linear Time. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2017*, pages 157–170, New York, NY, USA, 2017. ACM. doi: 10.1145/3062341.3062374. 5, 9, 44, 56, 57, 59, 77, 108, 109, 112, 134, 135, 139
- [158] Naoki Kobayashi. A New Type System for Deadlock-Free Processes. In Christel Baier and Holger Hermanns, editors, *CONCUR 2006 – Concurrency Theory*, pages 233–247, Berlin, Heidelberg, 2006. Springer Berlin Heidelberg. 17
- [159] Michalis Kokologiannakis and Viktor Vafeiadis. GenMC: A Model Checker for Weak Memory Models. In *Computer Aided Verification*, page 427–440, Berlin, Heidelberg, 2021. Springer-Verlag. doi: 10.1007/978-3-030-81685-8_20. 159
- [160] Michalis Kokologiannakis, Ori Lahav, Konstantinos Sagonas, and Viktor Vafeiadis. Effective Stateless Model Checking for C/C++ Concurrency. *Proc. ACM Program. Lang.*, 2(POPL), 2017. doi: 10.1145/3158105. 20
- [161] Michalis Kokologiannakis, Azalea Raad, and Viktor Vafeiadis. Model Checking for Weakly Consistent Libraries. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2019*, page 96–110, New York, NY, USA, 2019. Association for Computing Machinery. doi: 10.1145/3314221.3314609. 20, 139, 140, 159
- [162] Michalis Kokologiannakis, Iason Marmanis, Vladimir Gladstein, and Viktor Vafeiadis. Truly Stateless, Optimal Dynamic Partial Order Reduction. *Proc. ACM Program. Lang.*, 6(POPL), 2022. doi: 10.1145/3498711. 138, 140, 159
- [163] Michalis Kokologiannakis, Ori Lahav, and Viktor Vafeiadis. Kater: Automating Weak Memory Model Metatheory and Consistency Checking. *Proc. ACM Program. Lang.*, 7(POPL), 2023. doi: 10.1145/3571212. 138
- [164] Rucha Kulkarni, Umang Mathur, and Andreas Pavlogiannis. Dynamic Data-Race Detection Through the Fine-Grained Lens. In Serge Haddad and Daniele Varacca, editors, *32nd International Conference on Concurrency Theory (CONCUR 2021)*, volume 203 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 16:1–16:23, Dagstuhl, Germany, 2021. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. doi: 10.4230/LIPIcs.CONCUR.2021.16. 8, 29, 57, 68, 135

- [165] Ori Lahav and Udi Boker. What’s Decidable About Causally Consistent Shared Memory? *ACM Trans. Program. Lang. Syst.*, 44(2), 2022. doi: 10.1145/3505273. 139, 142, 144, 145, 146
- [166] Ori Lahav and Roy Margalit. Robustness against Release/Acquire Semantics. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI 2019, page 126–141, New York, NY, USA, 2019. Association for Computing Machinery. doi: 10.1145/3314221.3314604. 20, 159
- [167] Ori Lahav and Viktor Vafeiadis. Owicki-Gries Reasoning for Weak Memory Models. In Magnús M. Halldórsson, Kazuo Iwama, Naoki Kobayashi, and Bettina Speckmann, editors, *Automata, Languages, and Programming*, pages 311–323, Berlin, Heidelberg, 2015. Springer Berlin Heidelberg. doi: 10.1007/978-3-662-47666-6_25. 47, 139, 140, 151, 154, 159
- [168] Ori Lahav, Nick Giannarakis, and Viktor Vafeiadis. Taming Release-Acquire Consistency. In *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL ’16, page 649–662, New York, NY, USA, 2016. Association for Computing Machinery. doi: 10.1145/2837614.2837643. 139, 145
- [169] Ori Lahav, Viktor Vafeiadis, Jeehoon Kang, Chung-Kil Hur, and Derek Dreyer. Repairing Sequential Consistency in C/C++11. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI 2017, page 618–632, New York, NY, USA, 2017. Association for Computing Machinery. doi: 10.1145/3062341.3062352. 138, 143
- [170] Lamport. How to Make a Multiprocessor Computer That Correctly Executes Multiprocess Programs. *IEEE Transactions on Computers*, C-28(9):690–691, 1979. doi: 10.1109/TC.1979.1675439. 18
- [171] Leslie Lamport. Time, Clocks, and the Ordering of Events in a Distributed System. *Commun. ACM*, 21(7):558–565, 1978. ISSN 0001-0782. doi: 10.1145/359545.359563. 56, 61, 80, 105, 118
- [172] Leslie Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, June 2002. 16
- [173] William Landi and Barbara G. Ryder. Pointer-induced aliasing: a problem classification. In *Proceedings of the 18th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL ’91, page 93–103, New York, NY, USA, 1991. Association for Computing Machinery. doi: 10.1145/99583.99599. 6
- [174] Carl E. Landwehr, Alan R. Bull, John P. McDermott, and William S. Choi. A taxonomy of computer program security flaws. *ACM Comput. Surv.*, 26(3): 211–254, sep 1994. ISSN 0360-0300. doi: 10.1145/185403.185412. 3

- [175] Dongyoon Lee, Peter M. Chen, Jason Flinn, and Satish Narayanasamy. Chimera: hybrid program analysis for determinism. In *Proceedings of the 33rd ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '12, page 463–474, New York, NY, USA, 2012. Association for Computing Machinery. doi: 10.1145/2254064.2254119. 5
- [176] Sung-Hwan Lee, Minki Cho, Anton Podkopaev, Soham Chakraborty, Chung-Kil Hur, Ori Lahav, and Viktor Vafeiadis. Promising 2.0: Global Optimizations in Relaxed Memory Concurrency. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI 2020, page 362–376, New York, NY, USA, 2020. Association for Computing Machinery. doi: 10.1145/3385412.3386010. 138
- [177] Carl Lerche. Loom, 2020. Available at <https://github.com/tokio-rs/loom>. 138
- [178] Nancy G Leveson and Clark S Turner. An investigation of the Therac-25 accidents. *Computer*, 26(7), 1993. 3
- [179] Guangpu Li, Shan Lu, Madanlal Musuvathi, Suman Nath, and Rohan Padhye. Efficient scalable thread-safety-violation detection: finding thousands of concurrency bugs during testing. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, SOSP '19, page 162–180, New York, NY, USA, 2019. Association for Computing Machinery. doi: 10.1145/3341301.3359638. 4, 43, 80, 105
- [180] Yuanbo Li, Qirun Zhang, and Thomas Reps. On the complexity of bidirected interleaved dyck-reachability. *Proc. ACM Program. Lang.*, 5(POPL), January 2021. doi: 10.1145/3434340. 6
- [181] Zhenmin Li, Lin Tan, Xuanhui Wang, Shan Lu, Yuanyuan Zhou, and Chengxiang Zhai. Have things changed now? an empirical study of bug characteristics in modern open source software. In *Proceedings of the 1st Workshop on Architectural and System Support for Improving Software Dependability*, ASID '06, page 25–33, New York, NY, USA, 2006. Association for Computing Machinery. doi: 10.1145/1181309.1181314. 3
- [182] Chunhua Liao, Pei-Hung Lin, Joshua Asplund, Markus Schordan, and Ian Karlin. DataRaceBench: A Benchmark Suite for Systematic Evaluation of Data Race Detection Tools. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, SC '17, New York, NY, USA, 2017. Association for Computing Machinery. doi: 10.1145/3126908.3126958. 72
- [183] Bozhen Liu, Peiming Liu, Yanze Li, Chia-Che Tsai, Dilma Da Silva, and Jeff Huang. When Threads Meet Events: Efficient and Precise Static Race Detection with Origins. In *Proceedings of the 42nd ACM SIGPLAN International*

- Conference on Programming Language Design and Implementation*, PLDI 2021, page 725–739, New York, NY, USA, 2021. Association for Computing Machinery. doi: 10.1145/3453483.3454073. 108
- [184] Peng Liu, Omer Tripp, and Charles Zhang. Grail: context-aware fixing of concurrency bugs. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*, FSE 2014, page 318–329, New York, NY, USA, 2014. Association for Computing Machinery. doi: 10.1145/2635868.2635881. 16
- [185] Si Liu, Long Gu, Hengfeng Wei, and David Basin. Plume: Efficient and Complete Black-Box Checking of Weak Isolation Levels. *Proc. ACM Program. Lang.*, 8(OOPSLA2), October 2024. doi: 10.1145/3689742. 44
- [186] Ziheng Liu, Shuofei Zhu, Boqin Qin, Hao Chen, and Linhai Song. Automatically detecting and fixing concurrency bugs in go software systems. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS ’21, page 616–629, New York, NY, USA, 2021. Association for Computing Machinery. doi: 10.1145/3445814.3446756. 16
- [187] Wyatt Lloyd, Michael J. Freedman, Michael Kaminsky, and David G. Andersen. Don’t Settle for Eventual: Scalable Causal Consistency for Wide-Area Storage with COPS. In *Proceedings of the Twenty-Third ACM Symposium on Operating Systems Principles*, SOSP ’11, page 401–416, New York, NY, USA, 2011. Association for Computing Machinery. doi: 10.1145/2043556.2043593. 145
- [188] Shan Lu, Joseph Tucek, Feng Qin, and Yuanyuan Zhou. AVIO: detecting atomicity violations via access interleaving invariants. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS XII, page 37–48, New York, NY, USA, 2006. Association for Computing Machinery. doi: 10.1145/1168857.1168864. 4, 50
- [189] Shan Lu, Soyeon Park, Eunsoo Seo, and Yuanyuan Zhou. Learning from mistakes: a comprehensive study on real world concurrency bug characteristics. In *Proceedings of the 13th International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS XIII, page 329–339, New York, NY, USA, 2008. Association for Computing Machinery. doi: 10.1145/1346281.1346323. 3, 4, 36, 56, 108, 112, 116, 126
- [190] Chi-Keung Luk, Robert Cohn, Robert Muth, Harish Patil, Artur Klauser, Geoff Lowney, Steven Wallace, Vijay Janapa Reddi, and Kim Hazelwood. Pin: building customized program analysis tools with dynamic instrumentation. In *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI ’05, page 190–200, New York,

- NY, USA, 2005. Association for Computing Machinery. doi: 10.1145/1065010.1065034. 21, 22
- [191] Weiyu Luo and Brian Demsky. C11Tester: A Race Detector for C/C++ Atomics. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '21, page 630–646, New York, NY, USA, 2021. Association for Computing Machinery. doi: 10.1145/3445814.3446711. 20, 32, 35, 45, 47, 80, 81, 82, 101, 105, 139, 140, 144, 146, 147, 151, 154, 159
- [192] Nuno Machado, Brandon Lucia, and Luís Rodrigues. Concurrency debugging with differential schedule projections. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '15, page 586–595, New York, NY, USA, 2015. Association for Computing Machinery. doi: 10.1145/2737924.2737973. 16, 35
- [193] C. Manovit and S. Hangal. Completely verifying memory consistency of test program executions. In *The Twelfth International Symposium on High-Performance Computer Architecture*, 2006., pages 166–175, 2006. doi: 10.1109/HPCA.2006.1598123. 138
- [194] Jeremy Manson, William Pugh, and Sarita V. Adve. The Java memory model. In *Proceedings of the 32nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '05, page 378–391, New York, NY, USA, 2005. Association for Computing Machinery. doi: 10.1145/1040305.1040336. 34
- [195] Jeremy Manson, William Pugh, and Sarita V. Adve. The Java Memory Model. In *POPL '05*. ACM, 2005. 18
- [196] Roy Margalit and Ori Lahav. Verifying Observational Robustness against a C11-Style Memory Model. *Proc. ACM Program. Lang.*, 5(POPL), 2021. doi: 10.1145/3434285. 138, 139, 142, 145
- [197] Umang Mathur. RAPID. <https://github.com/umangm/rapid>, 2019. Accessed: 2021-08-01. 128
- [198] Umang Mathur and Mahesh Viswanathan. Atomicity Checking in Linear Time Using Vector Clocks. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '20, page 183–199, New York, NY, USA, 2020. Association for Computing Machinery. doi: 10.1145/3373376.3378475. 8, 29, 56, 57, 71, 81, 108
- [199] Umang Mathur, Dileep Kini, and Mahesh Viswanathan. What Happens-after the First Race? Enhancing the Predictive Power of Happens-before Based Dynamic Race Detection. *Proc. ACM Program. Lang.*, 2(OOPSLA), 2018. doi: 10.1145/3276515. 5, 9, 35, 44, 56, 57, 70, 80, 81, 139

- [200] Umang Mathur, Andreas Pavlogiannis, and Mahesh Viswanathan. The Complexity of Dynamic Data Race Prediction. In *Proceedings of the 35th Annual ACM/IEEE Symposium on Logic in Computer Science*, pages 713–727, Saarbrücken Germany, 2020. ACM. doi: 10.1145/3373718.3394783. 6, 46, 56, 80, 111, 117, 120, 135, 139, 149
- [201] Umang Mathur, Andreas Pavlogiannis, Hünkar Can Tunç, and Mahesh Viswanathan. A Tree Clock Data Structure for Causal Orderings in Concurrent Executions. December 2021. doi: 10.5281/zenodo.5749092. Artifact. 10
- [202] Umang Mathur, Andreas Pavlogiannis, and Mahesh Viswanathan. Optimal Prediction of Synchronization-Preserving Races. *Proc. ACM Program. Lang.*, 5(POPL), 2021. doi: 10.1145/3434317. 5, 8, 21, 29, 35, 38, 40, 46, 56, 109, 114, 118, 134, 139
- [203] Umang Mathur, Andreas Pavlogiannis, Hünkar Can Tunç, and Mahesh Viswanathan. A tree clock data structure for causal orderings in concurrent executions. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '22, page 710–725, New York, NY, USA, 2022. Association for Computing Machinery. doi: 10.1145/3503222.3507734. 8, 10, 105
- [204] Umang Mathur, Andreas Pavlogiannis, Hünkar Can Tunç, and Mahesh Viswanathan. A Tree Clock Data Structure for Causal Orderings in Concurrent Executions, 2022. URL <https://arxiv.org/abs/2201.06325>. Technical Report. 58, 72
- [205] Friedemann Mattern. Virtual Time and Global States of Distributed Systems. In M. Cosnard et. al., editor, *Parallel and Distributed Algorithms: proceedings of the International Workshop on Parallel & Distributed Algorithms*, pages 215–226. Elsevier Science Publishers B. V., 1989. 8, 29, 43, 81, 98, 105
- [206] Friedemann Mattern. Virtual Time and Global States of Distributed Systems. In M. Cosnard et. al., editor, *Parallel and Distributed Algorithms: proceedings of the International Workshop on Parallel & Distributed Algorithms*, pages 215–226. Elsevier Science Publishers B. V., 1989. 56, 60, 121
- [207] Scott Matteson. Report: Software failure caused \$1.7 trillion in financial losses in 2017. 2018. 3
- [208] Antoni Mazurkiewicz. Trace theory. In W. Brauer, W. Reisig, and G. Rozenberg, editors, *Petri Nets: Applications and Relationships to Other Models of Concurrency*, pages 278–324, Berlin, Heidelberg, 1987. Springer Berlin Heidelberg. doi: 10.1007/3-540-17906-2_30. 44, 56, 70

- [209] Charles E. McDowell and David P. Helmbold. Debugging concurrent programs. *ACM Comput. Surv.*, 21(4):593–622, December 1989. ISSN 0360-0300. doi: 10.1145/76894.76897. 16
- [210] Maged M. Michael and Michael L. Scott. Simple, fast, and practical non-blocking and blocking concurrent queue algorithms. In *Proceedings of the Fifteenth Annual ACM Symposium on Principles of Distributed Computing*, PODC '96, page 267–275, New York, NY, USA, 1996. Association for Computing Machinery. ISBN 0897918002. doi: 10.1145/248052.248106. URL <https://doi.org/10.1145/248052.248106>. 14
- [211] Mae Milano, Joshua Turcotti, and Andrew C. Myers. A flexible type system for fearless concurrency. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, PLDI 2022, page 458–473, New York, NY, USA, 2022. Association for Computing Machinery. doi: 10.1145/3519939.3523443. 17
- [212] Barton P. Miller, Lars Fredriksen, and Bryan So. An empirical study of the reliability of UNIX utilities. *Commun. ACM*, 33(12):32–44, December 1990. ISSN 0001-0782. doi: 10.1145/96267.96279. 4
- [213] R. Milner. *Communication and Concurrency*. Prentice-Hall, Inc., USA, 1989. 14
- [214] Anders Møller and Michael I. Schwartzbach. Static Program Analysis, October 2018. Department of Computer Science, Aarhus University, <http://cs.au.dk/~amoeller/spa/>. 5, 22
- [215] A Muir and J Lopatto. Final report on the August 14, 2003 blackout in the United States and Canada: causes and recommendations, Apr 2004. 3
- [216] Madanlal Musuvathi, Shaz Qadeer, Thomas Ball, Gerard Basler, Piramanayagam Arumuga Nainar, and Iulian Neamtii. Finding and reproducing Heisenbugs in concurrent programs. In *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation*, OSDI'08, page 267–280, USA, 2008. USENIX Association. 4, 6, 14, 15
- [217] Madanlal Musuvathi, Shaz Qadeer, Thomas Ball, Gerard Basler, Piramanayagam Arumuga Nainar, and Iulian Neamtii. Finding and Reproducing Heisenbugs in Concurrent Programs. In *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation*, OSDI'08, pages 267–280, Berkeley, CA, USA, 2008. USENIX Association. 56
- [218] Madanlal Musuvathi, Shaz Qadeer, Thomas Ball, Gerard Basler, Piramanayagam Arumuga Nainar, and Iulian Neamtii. Finding and Reproducing Heisenbugs in Concurrent Programs. In *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation*, OSDI'08, page 267–280, USA, 2008. USENIX Association. 80, 108, 113

- [219] Mayur Naik, Chang-Seo Park, Koushik Sen, and David Gay. Effective Static Deadlock Detection. In *Proceedings of the 31st International Conference on Software Engineering, ICSE '09*, page 386–396, USA, 2009. IEEE Computer Society. doi: 10.1109/ICSE.2009.5070538. 108
- [220] Iqra Naseer. The crowdstrike incident: Analysis and unveiling the intricacies of modern cybersecurity breaches. *World Journal of Advanced Engineering Technology and Sciences*, 10, 2024. 3
- [221] Nicholas Nethercote and Julian Seward. Valgrind: a framework for heavyweight dynamic binary instrumentation. In *Proceedings of the 28th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI '07*, page 89–100, New York, NY, USA, 2007. Association for Computing Machinery. doi: 10.1145/1250734.1250746. 22
- [222] Robert H.B. Netzer and Barton P. Miller. On the Complexity of Event Ordering for Shared-Memory Parallel Program Executions. In *Proceedings of the 1990 International Conference on Parallel Processing*, pages 93–97, 1990. 71
- [223] Chris Newcombe, Tim Rath, Fan Zhang, Bogdan Munteanu, Marc Brooker, and Michael Deardeuff. How Amazon web services uses formal methods. *Commun. ACM*, 58(4):66–73, March 2015. ISSN 0001-0782. doi: 10.1145/2699417. 16, 17
- [224] Nicholas Ng and Nobuko Yoshida. Static Deadlock Detection for Concurrent Go by Global Session Graph Synthesis . In *Proceedings of the 25th International Conference on Compiler Construction, CC 2016*, page 174–184, New York, NY, USA, 2016. Association for Computing Machinery. doi: 10.1145/2892208.2892232. 108
- [225] Flemming Nielson, Hanne R. Nielson, and Chris Hankin. *Principles of Program Analysis*. Springer Publishing Company, Incorporated, 2010. ISBN 3642084745. 20
- [226] Brian Norris and Brian Demsky. CDSchecker: Checking Concurrent Data Structures Written with C/C++ Atomics. In *Proceedings of the 2013 ACM SIGPLAN International Conference on Object Oriented Programming Systems Languages & Applications, OOPSLA '13*, pages 131–150, New York, NY, USA, 2013. Association for Computing Machinery. doi: 10.1145/2509136.2509514. 14, 105, 138, 159
- [227] Jonas Oberhauser, Rafael Lourenco de Lima Chehab, Diogo Behrens, Ming Fu, Antonio Paolillo, Lilith Oberhauser, Koustubha Bhat, Yuzhong Wen, Haibo Chen, Jaeho Kim, and Viktor Vafeiadis. VSync: Push-Button Verification and Optimization for Synchronization Primitives on Weak Memory Models. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS '21*, page

- 530–545, New York, NY, USA, 2021. Association for Computing Machinery. doi: 10.1145/3445814.3446748. 138
- [228] Robert O’Callahan and Jong-Deok Choi. Hybrid dynamic data race detection. In *Proceedings of the Ninth ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, PPOPP ’03, page 167–178, New York, NY, USA, 2003. Association for Computing Machinery. doi: 10.1145/781498.781528. 5
- [229] Robert O’Callahan and Jong-Deok Choi. Hybrid Dynamic Data Race Detection. *SIGPLAN Not.*, 38(10):167–178, June 2003. ISSN 0362-1340. doi: 10.1145/966049.781528. 77
- [230] Peter O’Hearn, John Reynolds, and Hongseok Yang. Local Reasoning about Programs that Alter Data Structures. In Laurent Fribourg, editor, *Computer Science Logic*, pages 1–19, Berlin, Heidelberg, 2001. Springer Berlin Heidelberg. 17
- [231] David Oppenheimer, Archana Ganapathi, et al. Why do Internet services fail, and what can be done about it? In *4th Usenix Symposium on Internet Technologies and Systems (USITS 03)*, 2003. 3
- [232] Victor Pankratiy and Ali-Reza Adl-Tabatabai. A study of transactional memory vs. locks in practice. In *Proceedings of the Twenty-Third Annual ACM Symposium on Parallelism in Algorithms and Architectures*, SPAA ’11, page 43–52, New York, NY, USA, 2011. Association for Computing Machinery. doi: 10.1145/1989493.1989500. 18
- [233] Chang-Seo Park and Koushik Sen. Randomized active atomicity violation detection in concurrent programs. In *Proceedings of the 16th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, SIGSOFT ’08/FSE-16, page 135–145, New York, NY, USA, 2008. Association for Computing Machinery. doi: 10.1145/1453101.1453121. 50
- [234] Soyeon Park, Shan Lu, and Yuanyuan Zhou. CTrigger: exposing atomicity violation bugs from their hiding places. In *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS XIV, page 25–36, New York, NY, USA, 2009. Association for Computing Machinery. doi: 10.1145/1508244.1508249. 15, 50
- [235] Pavel Parízek and Ondřej Lhoták. Model checking of concurrent programs with static analysis of field accesses. *Science of Computer Programming*, 98: 735–763, 2015. doi: <https://doi.org/10.1016/j.scico.2014.10.008>. 23, 51
- [236] Andreas Pavlogiannis. Fast, Sound, and Effectively Complete Dynamic Race Prediction. *Proc. ACM Program. Lang.*, 4(POPL), 2019. doi: 10.1145/3371085. 5, 9, 29, 32, 35, 38, 40, 45, 46, 48, 56, 80, 82, 83, 85, 97, 98, 99, 105, 109, 114, 134, 139, 163

- [237] Eli Pozniansky and Assaf Schuster. Efficient On-the-fly Data Race Detection in Multithreaded C++ Programs. *SIGPLAN Not.*, 38(10):179–190, June 2003. ISSN 0362-1340. doi: 10.1145/966049.781529. 56, 108
- [238] S. Qadeer. Verifying sequential consistency on shared-memory multiprocessors by model checking. *IEEE Transactions on Parallel and Distributed Systems*, 14(8):730–741, 2003. doi: 10.1109/TPDS.2003.1225053. 138
- [239] Boqin Qin, Yilun Chen, Zeming Yu, Linhai Song, and Yiyang Zhang. Understanding memory and thread safety practices and issues in real-world Rust programs. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI 2020, page 763–779, New York, NY, USA, 2020. Association for Computing Machinery. doi: 10.1145/3385412.3386036. 17
- [240] Raghavan Raman, Jisheng Zhao, Vivek Sarkar, Martin Vechev, and Eran Yahav. Efficient data race detection for async-finish parallelism. *Formal Methods in System Design*, 41(3):321–347, Dec 2012. ISSN 1572-8102. doi: 10.1007/s10703-012-0143-7. 43, 77
- [241] Veselin Raychev, Martin Vechev, and Manu Sridharan. Effective Race Detection for Event-Driven Programs. In *Proceedings of the 2013 ACM SIGPLAN International Conference on Object Oriented Programming Systems Languages & Applications*, OOPSLA ’13, page 151–166, New York, NY, USA, 2013. Association for Computing Machinery. doi: 10.1145/2509136.2509538. 43, 77
- [242] Giles Reger and Klaus Havelund. What Is a Trace? A Runtime Verification Perspective. In Tiziana Margaria and Bernhard Steffen, editors, *Leveraging Applications of Formal Methods, Verification and Validation: Discussion, Dissemination, Applications*, pages 339–355, Cham, 2016. Springer International Publishing. 21
- [243] Thomas Reps, Thomas Ball, Manuvir Das, and James Larus. The use of program profiling for software maintenance with applications to the year 2000 problem. In *Proceedings of the 6th European SOFTWARE ENGINEERING Conference Held Jointly with the 5th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, ESEC ’97/FSE-5, page 432–449, Berlin, Heidelberg, 1997. Springer-Verlag. doi: 10.1145/267895.267925. 21
- [244] Dustin Rhodes, Cormac Flanagan, and Stephen N. Freund. BigFoot: Static Check Placement for Dynamic Race Detection. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI 2017, page 141–156, New York, NY, USA, 2017. Association for Computing Machinery. doi: 10.1145/3062341.3062350. 77, 135
- [245] H. G. Rice. Classes of recursively enumerable sets and their decision problems. *Transactions of the American Mathematical Society*, 74(2):358–366, 1953. 3

- [246] Erik Rigtorp. Concurrentqueue, 2017. URL <https://github.com/rigtorp/MPMCQueue>. Accessed: 2024-11-12. 14
- [247] Jake Roemer, Kaan Genç, and Michael D. Bond. High-Coverage, Unbounded Sound Predictive Race Detection. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI 2018, page 374–389, New York, NY, USA, 2018. Association for Computing Machinery. doi: 10.1145/3192366.3192385. 5, 8, 9, 21, 29, 32, 44, 46, 56, 57, 77, 80, 81, 82, 105
- [248] Jake Roemer, Kaan Genç, and Michael D. Bond. SmartTrack: Efficient Predictive Race Detection. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI 2020, pages 747–762, New York, NY, USA, 2020. Association for Computing Machinery. doi: 10.1145/3385412.3385993. 5, 35, 68, 77, 80, 109, 112, 134
- [249] Anthony Roscoe. The theory and practice of concurrency. 1998. 13
- [250] Christopher J. Rossbach, Owen S. Hofmann, and Emmett Witchel. Is transactional programming actually easier? In *Proceedings of the 15th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, PPoPP ’10, page 47–56, New York, NY, USA, 2010. Association for Computing Machinery. doi: 10.1145/1693453.1693462. 18
- [251] Grigore Rosu. RV-Predict, Runtime Verification. <https://runtimeverification.com/predict/>, 2024. Accessed: 2024-10-27. 5, 72, 129
- [252] Amitabha Roy, Stephan Zeisset, Charles J. Fleckenstein, and John C. Huang. Fast and Generalized Polynomial Time Memory Consistency Verification. In Thomas Ball and Robert B. Jones, editors, *Computer Aided Verification*, pages 503–516, Berlin, Heidelberg, 2006. Springer Berlin Heidelberg. doi: 10.1007/11817963_46. 32, 81, 82, 100
- [253] Caitlin Sadowski and Jaeheon Yi. How Developers Use Data Race Detection Tools. In *Proceedings of the 5th Workshop on Evaluation and Usability of Programming Languages and Tools*, PLATEAU ’14, page 43–51, New York, NY, USA, 2014. Association for Computing Machinery. doi: 10.1145/2688204.2688205. 77
- [254] Caitlin Sadowski, Jeffrey Van Gogh, Ciera Jaspan, Emma Soderberg, and Collin Winter. Tricorder: Building a Program Analysis Ecosystem. In *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, volume 1, pages 598–608, 2015. doi: 10.1109/ICSE.2015.76. 4
- [255] Caitlin Sadowski, Edward Aftandilian, Alex Eagle, Liam Miller-Cushon, and Ciera Jaspan. Lessons from building static analysis tools at Google. *Commun.*

- ACM, 61(4):58–66, mar 2018. ISSN 0001-0782. doi: 10.1145/3188720. 4, 6, 49
- [256] Mahmoud Said, Chao Wang, Zijiang Yang, and Karem Sakallah. Generating Data Race Witnesses by an SMT-based Analysis. In *Proceedings of the Third International Conference on NASA Formal Methods*, NFM’11, pages 313–327, Berlin, Heidelberg, 2011. Springer-Verlag. 134
- [257] Malavika Samak and Murali Krishna Ramanathan. Trace Driven Dynamic Deadlock Detection and Reproduction. In *Proceedings of the 19th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, PPOPP ’14, page 29–42, New York, NY, USA, 2014. Association for Computing Machinery. doi: 10.1145/2555243.2555262. 56, 57, 108, 134
- [258] Malavika Samak and Murali Krishna Ramanathan. Multithreaded Test Synthesis for Deadlock Detection. In *Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages & Applications*, OOPSLA ’14, page 473–489, New York, NY, USA, 2014. Association for Computing Machinery. doi: 10.1145/2660193.2660238. 108, 134
- [259] Stefan Savage, Michael Burrows, Greg Nelson, Patrick Sobalvarro, and Thomas Anderson. Eraser: a dynamic data race detector for multithreaded programs. *ACM Trans. Comput. Syst.*, 15(4):391–411, nov 1997. ISSN 0734-2071. doi: 10.1145/265924.265927. 4, 35, 108
- [260] Adrian Schmitz, Joachim Protze, Lechen Yu, Simon Schwitanski, and Matthias S. Müller. DataRaceOnAccelerator – A Micro-benchmark Suite for Evaluating Correctness Tools Targeting Accelerators. In Ulrich Schwardmann, Christian Boehme, Dora B. Heras, Valeria Cardellini, Emmanuel Jeannot, Antonio Salis, Claudio Schifanella, Ravi Reddy Manumachu, Dieter Schwamborn, Laura Ricci, Oh Sangyoon, Thomas Gruber, Laura Antonelli, and Stephen L. Scott, editors, *Euro-Par 2019: Parallel Processing Workshops*, pages 245–257, Cham, 2020. Springer International Publishing. doi: 10.1007/978-3-030-48340-1_19. 72
- [261] Fred B. Schneider. On concurrent programming. *Commun. ACM*, 41(4):128, April 1998. doi: 10.1145/273035.273072. 4
- [262] Reinhard Schwarz and Friedemann Mattern. Detecting causal relationships in distributed computations: In search of the holy grail. *Distributed computing*, 7(3):149–174, 1994. doi: 10.1007/BF02277859. 56
- [263] Koushik Sen. Race Directed Random Testing of Concurrent Programs. In *Proceedings of the 29th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI ’08, pages 11–21, New York, NY, USA, 2008. ACM. doi: 10.1145/1375581.1375584. 35

- [264] Koushik Sen, Grigore Roşu, and Gul Agha. Detecting Errors in Multithreaded Programs by Generalized Predictive Analysis of Executions. In Martin Steffen and Gianluigi Zavattaro, editors, *Formal Methods for Open Object-Based Distributed Systems*, pages 211–226, Berlin, Heidelberg, 2005. Springer Berlin Heidelberg. 108, 113
- [265] Aritra Sengupta, Swarnendu Biswas, Minjia Zhang, Michael D. Bond, and Milind Kulkarni. Hybrid Static–Dynamic Analysis for Statically Bounded Region Serializability. In *Proceedings of the Twentieth International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS ’15, page 561–575, New York, NY, USA, 2015. Association for Computing Machinery. doi: 10.1145/2694344.2694379. 5
- [266] Traian Florin Şerbănuţă, Feng Chen, and Grigore Roşu. Maximal Causal Models for Sequentially Consistent Systems. In Shaz Qadeer and Serdar Tasiran, editors, *Runtime Verification*, pages 136–150, Berlin, Heidelberg, 2013. Springer Berlin Heidelberg. doi: 10.1007/978-3-642-35632-2_16. 6, 41, 113, 134
- [267] Konstantin Serebryany and Timur Iskhodzhanov. ThreadSanitizer: Data Race Detection in Practice. In *Proceedings of the Workshop on Binary Instrumentation and Applications*, WBIA ’09, page 62–71, New York, NY, USA, 2009. Association for Computing Machinery. doi: 10.1145/1791194.1791203. 4, 5, 21, 34, 35, 56, 72, 78, 108
- [268] Konstantin Serebryany, Derek Bruening, Alexander Potapenko, and Dmitriy Vyukov. AddressSanitizer: A fast address sanity checker. In *2012 USENIX Annual Technical Conference (USENIX ATC 12)*, pages 309–318, Boston, MA, June 2012. USENIX Association. 21
- [269] Kostya Serebryany, Evgenii Stepanov, Aleksey Shlyapnikov, Vlad Tsyrklevich, and Dmitry Vyukov. Memory Tagging and how it improves C/C++ memory safety. *CoRR*, abs/1802.09517, 2018. 4, 22
- [270] Peter Sewell, Susmit Sarkar, Scott Owens, Francesco Zappa Nardelli, and Magnus O. Myreen. x86-tso: a rigorous and usable programmer’s model for x86 multiprocessors. *Commun. ACM*, 53(7):89–97, July 2010. ISSN 0001-0782. doi: 10.1145/1785414.1785443. URL <https://doi.org/10.1145/1785414.1785443>. 18
- [271] Dennis Shasha and Marc Snir. Efficient and Correct Execution of Parallel Programs That Share Memory. *ACM Trans. Program. Lang. Syst.*, 10(2): 282–312, April 1988. ISSN 0164-0925. doi: 10.1145/42190.42277. 56, 71
- [272] Yao Shi, Soyeon Park, Zuoning Yin, Shan Lu, Yuanyuan Zhou, Wenguang Chen, and Weimin Zheng. Do I Use the Wrong Definition?: DeFuse: Definition-use Invariants for Detecting Concurrency and Sequential Bugs. In *Proceedings*

- of the ACM International Conference on Object Oriented Programming Systems Languages and Applications, OOPSLA '10*, pages 160–174, New York, NY, USA, 2010. ACM. doi: 10.1145/1869459.1869474. 56
- [273] Zheng Shi, Umang Mathur, and Andreas Pavlogiannis. Optimistic Prediction of Synchronization-Reversal Data Races, 2024. 105
- [274] Denini Silva, Leopoldo Teixeira, and Marcelo d’Amorim. Shake it! detecting flaky tests caused by concurrency with shaker. In *2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 301–311, 2020. doi: 10.1109/ICSME46990.2020.00037. 15
- [275] Arnab Sinha, Sharad Malik, Chao Wang, and Aarti Gupta. Predictive analysis for detecting serializability violations through Trace Segmentation. In *Ninth ACM/IEEE International Conference on Formal Methods and Models for Code-sign (MEMPCODE2011)*, pages 99–108, 2011. doi: 10.1109/MEMCOD.2011.5970516. 9, 38
- [276] Yannis Smaragdakis, Jacob Evans, Caitlin Sadowski, Jaeheon Yi, and Cormac Flanagan. Sound Predictive Race Detection in Polynomial Time. In *Proceedings of the 39th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL '12*, page 387–400, New York, NY, USA, 2012. Association for Computing Machinery. doi: 10.1145/2103656.2103702. 9, 29, 34, 35, 38, 40, 46, 56, 59, 108, 112, 113, 114, 134
- [277] L. A. Smith, J. M. Bull, and J. Obdržálek. A Parallel Java Grande Benchmark Suite. In *Proceedings of the 2001 ACM/IEEE Conference on Supercomputing, SC '01*, pages 8–8, New York, NY, USA, 2001. ACM. doi: 10.1145/582034.582042. 71, 129
- [278] Vincent Smyth. Software vulnerability management: how intelligence helps reduce the risk. *Network Security*, 2017(3):10–12, 2017. ISSN 1353-4858. doi: [https://doi.org/10.1016/S1353-4858\(17\)30027-2](https://doi.org/10.1016/S1353-4858(17)30027-2). 3
- [279] Daniel Sorin, Mark Hill, and David Wood. *A primer on memory consistency and cache coherence*. Springer Nature, 2022. 18
- [280] Francesco Sorrentino. PickLock: A Deadlock Prediction Approach under Nested Locking. In *Proceedings of the 22nd International Symposium on Model Checking Software - Volume 9232, SPIN 2015*, page 179–199, Berlin, Heidelberg, 2015. Springer-Verlag. doi: 10.1007/978-3-319-23404-5_13. 108, 134
- [281] J. Straunstrup, H.R. Andersen, H. Hulgaard, J. Lind-Nielsen, G. Behrmann, K. Kristoffersen, A. Skou, HH. Leerberg, and N.B. Theilgaard. Practical verification of embedded software. *Computer*, 33(5):68–75, 2000. doi: 10.1109/2.841786. 16

- [282] Bjarne Stroustrup. The c++ programming language, 2013. 14
- [283] R. Subramanyam and M.S. Krishnan. Empirical analysis of ck metrics for object-oriented design complexity: implications for software defects. *IEEE Transactions on Software Engineering*, 29(4):297–310, 2003. doi: 10.1109/TSE.2003.1191795. 3
- [284] Martin Sulzmann and Kai Stadtmüller. Two-Phase Dynamic Analysis of Message-Passing Go Programs Based on Vector Clocks. In *Proceedings of the 20th International Symposium on Principles and Practice of Declarative Programming*, PPDP ’18, New York, NY, USA, 2018. Association for Computing Machinery. doi: 10.1145/3236950.3236959. 8, 29, 56, 57
- [285] Martin Sulzmann and Kai Stadtmüller. *Efficient, near complete, and often sound hybrid dynamic data race prediction*, page 30–51. Association for Computing Machinery, New York, NY, USA, 2020. 5
- [286] Rishi Surendran and Vivek Sarkar. Dynamic Determinacy Race Detection for Task Parallelism with Futures. In Yliès Falcone and César Sánchez, editors, *Runtime Verification*, pages 368–385, Cham, 2016. Springer International Publishing. doi: 10.1007/978-3-319-46982-9_23. 43, 77
- [287] Lin Tan, Chen Liu, Zhenmin Li, Xuanhui Wang, Yuanyuan Zhou, and Chengxiang Zhai. Bug characteristics in open source software. *Empirical Software Engineering*, 19(6):1665–1705, Dec 2014. ISSN 1573-7616. doi: 10.1007/s10664-013-9258-8. 3
- [288] R.N. Taylor, D.L. Levine, and C.D. Kelly. Structural testing of concurrent programs. *IEEE Transactions on Software Engineering*, 18(3):206–215, 1992. doi: 10.1109/32.126769. 4
- [289] Tengfei Tu, Xiaoyu Liu, Linhai Song, and Yiying Zhang. Understanding Real-World Concurrency Bugs in Go. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS ’19, page 865–878, New York, NY, USA, 2019. Association for Computing Machinery. doi: 10.1145/3297858.3304069. 4, 36, 56, 108
- [290] Hünkar Can Tunç, Parosh Aziz Abdulla, Soham Chakraborty, Shankaranarayanan Krishna, Umang Mathur, and Andreas Pavlogiannis. Optimal Reads-From Consistency Checking for C11-Style Memory Models. *Proc. ACM Program. Lang.*, 7(PLDI), jun 2023. doi: 10.1145/3591251. 9, 10, 81
- [291] Hünkar Can Tunç, Umang Mathur, Andreas Pavlogiannis, and Mahesh Viswanathan. Sound Dynamic Deadlock Prediction in Linear Time. *Proc. ACM Program. Lang.*, 7(PLDI), jun 2023. doi: 10.1145/3591291. 8, 10, 32, 80, 81

- [292] Hünkar Can Tunç, Ameya Prashant Deshmukh, Berk Cirisci, Constantin Enea, and Andreas Pavlogiannis. CSSTs: A Dynamic Data Structure for Partial Orders in Concurrent Execution Analysis. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ASPLOS '24, page 223–238, New York, NY, USA, 2024. Association for Computing Machinery. doi: 10.1145/3620666.3651358. 8, 10, 32
- [293] Hünkar Can Tunç, Parosh Aziz Abdulla, Soham Chakraborty, Shankaranarayanan Krishna, Umang Mathur, and Andreas Pavlogiannis. Optimal Reads-From Consistency Checking for C11-Style Memory Models. March 2023. doi: 10.5281/zenodo.7816526. Artifact. 10
- [294] Hünkar Can Tunç, Parosh Aziz Abdulla, Soham Chakraborty, Shankaranarayanan Krishna, Umang Mathur, and Andreas Pavlogiannis. Optimal Reads-From Consistency Checking for C11-Style Memory Models, 2023. URL <https://arxiv.org/abs/2304.03714>. Technical Report. 141, 158, 160, 161
- [295] Hünkar Can Tunç, Umang Mathur, Andreas Pavlogiannis, and Mahesh Viswanathan. Sound Dynamic Deadlock Prediction in Linear Time. 2023. doi: 10.5281/zenodo.7809600. Artifact. 10
- [296] Hünkar Can Tunç, Umang Mathur, Andreas Pavlogiannis, and Mahesh Viswanathan. Sound Dynamic Deadlock Prediction in Linear Time, 2023. URL <https://arxiv.org/abs/2304.03692>. Technical Report. 114, 128, 131
- [297] Hünkar Can Tunç, Ameya Prashant Deshmukh, Berk Cirisci, Constantin Enea, and Andreas Pavlogiannis. CSSTs: A Dynamic Data Structure for Partial Orders in Concurrent Execution Analysis. 2024. doi: 10.5281/zenodo.10798906. Artifact. 10
- [298] Hünkar Can Tunç, Ameya Prashant Deshmukh, Berk Çirisci, Constantin Enea, and Andreas Pavlogiannis. CSSTs: A Dynamic Data Structure for Partial Orders in Concurrent Execution Analysis, 2024. URL <https://arxiv.org/abs/2403.17818>. Technical Report. 83
- [299] A. M. Turing. On computable numbers, with an application to the entscheidungsproblem. *Proceedings of the London Mathematical Society*, s2-42(1): 230–265, 1937. doi: <https://doi.org/10.1112/plms/s2-42.1.230>. 3
- [300] Rachel Tzoref, Shmuel Ur, and Elad Yom-Tov. Instrumenting where it hurts: an automatic concurrent debugging technique. In *Proceedings of the 2007 International Symposium on Software Testing and Analysis*, ISSTA '07, page 27–38, New York, NY, USA, 2007. Association for Computing Machinery. doi: 10.1145/1273463.1273469. 16

- [301] Viktor Vafeiadis, Thibaut Balabonski, Soham Chakraborty, Robin Morisset, and Francesco Zappa Nardelli. Common Compiler Optimisations Are Invalid in the C11 Memory Model and What We Can Do about It. In *Proceedings of the 42nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '15, page 209–220, New York, NY, USA, 2015. Association for Computing Machinery. doi: 10.1145/2676726.2676995. 138
- [302] Willem Visser, Klaus Havelund, Guillaume Brat, SeungJoon Park, and Flavio Lerda. Model Checking Programs. *Automated Software Engineering*, 10(2): 203–232, Apr 2003. ISSN 1573-7535. doi: 10.1023/A:1022920129859. 4
- [303] Willem Visser, Corina S. Pundefindsundefindreanu, and Sarfraz Khurshid. Test input generation with java PathFinder. In *Proceedings of the 2004 ACM SIGSOFT International Symposium on Software Testing and Analysis*, ISSTA '04, page 97–107, New York, NY, USA, 2004. Association for Computing Machinery. doi: 10.1145/1007512.1007526. 22
- [304] Reinhard von Hanxleden. Synccharts in c: a proposal for light-weight, deterministic concurrency. In *Proceedings of the Seventh ACM International Conference on Embedded Software*, EMSOFT '09, page 225–234, New York, NY, USA, 2009. Association for Computing Machinery. doi: 10.1145/1629335.1629366. 15
- [305] Chao Wang, Sudipta Kundu, Malay Ganai, and Aarti Gupta. Symbolic Predictive Analysis for Concurrent Programs. In *Proceedings of the 2Nd World Congress on Formal Methods*, FM '09, pages 256–272, Berlin, Heidelberg, 2009. Springer-Verlag. doi: 10.1007/978-3-642-05089-3_17. 51
- [306] Xinli Wang, J. Mayo, W. Gao, and J. Slusser. An Efficient Implementation of Vector Clocks in Dynamic Systems. In *PDPTA*, 2006. 43, 77
- [307] Dasarath Weeratunge, Xiangyu Zhang, William N. Sumner, and Suresh Jaganathan. Analyzing concurrency bugs using dual slicing. In *Proceedings of the 19th International Symposium on Software Testing and Analysis*, ISSTA '10, page 253–264, New York, NY, USA, 2010. Association for Computing Machinery. doi: 10.1145/1831708.1831740. 16
- [308] John Wickerson, Mark Batty, Tyler Sorensen, and George A. Constantinides. Automatically Comparing Memory Consistency Models. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages*, POPL '17, page 190–204, New York, NY, USA, 2017. Association for Computing Machinery. doi: 10.1145/3009837.3009838. 138
- [309] Ryan Williams. A new algorithm for optimal 2-constraint satisfaction and its implications. *Theoretical Computer Science*, 348(2):357–365, 2005. doi: <https://doi.org/10.1016/j.tcs.2005.09.023>. Automata, Languages and Programming: Algorithms and Complexity (ICALP-A 2004). 117

- [310] Virginia Vassilevska Williams. On some fine-grained questions in algorithms and complexity. In *Proceedings of the International Congress of Mathematicians: Rio de Janeiro 2018*, pages 3447–3487. World Scientific, 2018. doi: 10.1142/9789813272880_0188. 111, 141
- [311] Virginia Vassilevska Williams and R. Ryan Williams. Subcubic Equivalences Between Path, Matrix, and Triangle Problems. *J. ACM*, 65(5), 2018. doi: 10.1145/3186893. 157
- [312] Matt Windsor, Alastair F. Donaldson, and John Wickerson. High-coverage metamorphic testing of concurrency support in C compilers. *Software Testing, Verification and Reliability*, 32(4), 2022. doi: <https://doi.org/10.1002/stvr.1812>. 138
- [313] Benjamin P. Wood, Man Cao, Michael D. Bond, and Dan Grossman. Instrumentation bias for dynamic data race detection. *Proc. ACM Program. Lang.*, 1 (OOPSLA), October 2017. doi: 10.1145/3133893. 21, 77
- [314] Di Wu, Lin Chen, Yuming Zhou, and Baowen Xu. An extensive empirical study on c++ concurrency constructs. *Information and Software Technology*, 76:1–18, 2016. doi: <https://doi.org/10.1016/j.infsof.2016.04.004>. 14
- [315] Zuoning Yin, Ding Yuan, Yuanyuan Zhou, Shankar Pasupathy, and Lakshmi Bairavasundaram. How Do Fixes Become Bugs? In *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering*, ESEC/FSE ’11, page 26–36, New York, NY, USA, 2011. Association for Computing Machinery. doi: 10.1145/2025113.2025121. 108
- [316] M. Young and R.N. Taylor. Combining static concurrency analysis with symbolic execution. *IEEE Transactions on Software Engineering*, 14(10):1499–1511, 1988. doi: 10.1109/32.6195. 23, 51
- [317] Kunpeng Yu, Chenxu Wang, Yan Cai, Xiapu Luo, and Zijiang Yang. Detecting Concurrency Vulnerabilities Based on Partial Orders of Memory and Thread Events. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE 2021, page 280–291, New York, NY, USA, 2021. Association for Computing Machinery. doi: 10.1145/3468264.3468572. 9, 38, 45, 56, 80, 82, 100, 105
- [318] Yuan Yu, Tom Rodeheffer, and Wei Chen. RaceTrack: Efficient Detection of Data Race Conditions via Adaptive Tracking. *SIGOPS Oper. Syst. Rev.*, 39(5): 221–234, October 2005. ISSN 0163-5980. doi: 10.1145/1095809.1095832. 35, 77

- [319] Andreas Zeller. Specifications for Free. In Mihaela Bobaru, Klaus Havelund, Gerard J. Holzmann, and Rajeev Joshi, editors, *NASA Formal Methods*, pages 2–12, Berlin, Heidelberg, 2011. Springer Berlin Heidelberg. 5
- [320] Rachid Zennou, Ahmed Bouajjani, Constantin Enea, and Mohammed Erradi. Gradual Consistency Checking. In Isil Dillig and Serdar Tasiran, editors, *Computer Aided Verification*, pages 267–285, Cham, 2019. Springer International Publishing. doi: 10.1007/978-3-030-25543-5_16. 32, 82, 139
- [321] Rachid Zennou, Mohamed Faouzi Atig, Ranadeep Biswas, Ahmed Bouajjani, Constantin Enea, and Mohammed Erradi. Boosting Sequential Consistency Checking Using Saturation. In Dang Van Hung and Oleg Sokolsky, editors, *Automated Technology for Verification and Analysis*, pages 360–376, Cham, 2020. Springer International Publishing. doi: 10.1007/978-3-030-59152-6_20. 32, 82, 105
- [322] M. Zhivich and R. K. Cunningham. The Real Cost of Software Errors. *IEEE Security and Privacy*, 7(2):87–90, March 2009. ISSN 1540-7993. doi: 10.1109/MSP.2009.56. 4, 56
- [323] Jinpeng Zhou, Sam Silvestro, Hongyu Liu, Yan Cai, and Tongping Liu. UN-DEAD: Detecting and Preventing Deadlocks in Production Software. In *Proceedings of the 32nd IEEE/ACM International Conference on Automated Software Engineering, ASE '17*, page 729–740. IEEE Press, 2017. doi: 10.1109/ASE.2017.8115684. 134
- [324] P. Zhou, R. Teodorescu, and Y. Zhou. HARD: Hardware-Assisted Lockset-based Race Detection. In *2007 IEEE 13th International Symposium on High Performance Computer Architecture*, pages 121–132, 2007. doi: 10.1109/HPCA.2007.346191. 77
- [325] Hong Zhu, Patrick A. V. Hall, and John H. R. May. Software unit test coverage and adequacy. *ACM Comput. Surv.*, 29(4):366–427, December 1997. ISSN 0360-0300. doi: 10.1145/267580.267590. 22